

BOOTSTRAP

 (CSS Framework)

=====

BOOTSTRAP 5 COMPLETE MASTER GUIDE (A TO Z) - GETBOOTSTRAP.COM CLONE

=====

1. INTRODUCTION & HISTORY (બુટસ્ટ્રેપ શું છે, કોણે અને કેમ બનાવ્યું?)

- શેના માટે છે?: વેબસાઇટ ડિઝાઇનિંગને એકદમ ફાસ્ટ, રિસ્પોન્સિવ (મોબાઇલ, ટેબ્લેટ અને કમ્પ્યુટર ત્રણેયમાં બરાબર દેખાય તેવી) અને આધુનિક લુક આપવા માટે બુટસ્ટ્રેપનો ઉપયોગ થાય છે.
- શું કામ કરે છે?: તે પહેલેથી લખાયેલી CSS અને JavaScript નો એક મોટો સંગ્રહ (Framework) છે. ડેવલપરે શૂન્યથી CSS કોડ લખવો નથી પડતો, માત્ર બુટસ્ટ્રેપના ક્લાસ વાપરવાના રહે છે.
- ઇતિહાસ અને ફાઉન્ડર્સ (History & Founders):
 - * આ ફ્રેમવર્કને વર્ષ 2011 માં Twitter (ટ્વિટર) કંપનીમાં કામ કરતા બે એન્જિનિયરો Mark Otto (માર્ક ઓટો) અને Jacob Thornton (જેકબ થોર્નટન) દ્વારા બનાવવામાં આવ્યું હતું.
 - * શરૂઆતમાં તેનું નામ "Twitter Blueprint" રાખવામાં આવ્યું હતું, કારણ કે કંપનીના આંતરિક પ્રોજેક્ટ્સમાં એકરૂપતા લાવવા માટે આ ટૂલ તૈયાર થયું હતું. પાછળથી તેને ઓપન-સોર્સ (બધા માટે ફ્રી) કરીને "Bootstrap" નામ આપવામાં આવ્યું.
- વર્ઝનની ઇતિહાસ (Versions History):
 - * Bootstrap 1, 2, 3: શરૂઆતના સંસ્કરણો જેણે વેબ જગતમાં મોબાઇલ-ફર્સ્ટ ડિઝાઇનનો પાયો નાખ્યો.
 - * Bootstrap 4: આ વર્ઝનમાં ફ્લેક્સબોક્સ (Flexbox) આર્કિટેક્ચર અપનાવવામાં આવ્યું, જેણે લેઆઉટ બનાવવાનું ખૂબ સરળ બનાવ્યું.
 - * Bootstrap 5 (વર્તમાન આધુનિક વર્ઝન): બુટસ્ટ્રેપ 5 માંથી jQuery ની કમ્પલેક્સરી ડિપેન્ડન્સી સંપૂર્ણપણે હટાવી દેવામાં આવી અને તેની જગ્યાએ પ્યોર જાવાસ્ક્રિપ્ટ (Vanilla JS) નો ઉપયોગ શરૂ થયો. આ ઉપરાંત આમાં કસ્ટમ વેરીએબલ્સ (CSS Variables) અને નવા યુટિલિટી ક્લાસીસ ઉમેરાયા.

2. THE 12-COLUMN RESPONSIVE GRID SYSTEM (બુટસ્ટ્રેપનું આખું માળખું)

- શેના માટે છે?: કોઇપણ વેબપેજને અલગ-અલગ સ્ક્રીન સાઇઝ (મોબાઇલથી લઈને મોટા ટીવી મોનિટર) મુજબ ઓટોમેટિક એડજસ્ટ કરવા માટે.
- શું કામ કરે છે?: બુટસ્ટ્રેપ આખા પેજની આડી પહોળાઈને કાલ્પનિક 12 સરખા ભાગમાં (12 Columns) વહેંચી નાખે છે.
- માળખાકીય ક્લાસીસ (Structural Hierarchy):
 - * .container: આ પેજની આજુબાજુ સેફ માર્જિન છોડીને ડેટાને વચ્ચે (Center) ગોઠવે છે.
 - * .container-fluid: આ પેજની આખી 100% પહોળાઈ (Full Width) રોકી લે છે.
 - * .row: આ એક આડી લાઇન (Row) બનાવે છે જેની અંદર જ કોલમ્સ ગોઠવી શકાય છે. તે ફ્લેક્સબોક્સ રેપર તરીકે કામ કરે છે.
 - * .col-*: તમે જેટલા નંબર આપો એટલી જગ્યા તે રોકે છે. દા.ત. `col-6` એટલે અડધું પેજ (12 ના અડધા 6).
 - * `col-4` એટલે ત્રીજો ભાગ.
- બ્રેકપોઇન્ટ્સ (Screen Responsive Breakpoints):
 - * .col- (Extra small): મોબાઇલ સ્ક્રીન માટે (< 576px)
 - * .col-sm- (Small): મોટા મોબાઇલ કે નાના ટેબ્લેટ (>= 576px)
 - * .col-md- (Medium): રેગ્યુલર ટેબ્લેટ કે નાના લેપટોપ (>= 768px)
 - * .col-lg- (Large): કોમ્પ્યુટર સ્ક્રીન કે ડેસ્કટોપ (>= 992px)
 - * .col-xl- (Extra large): મોટા મોનિટર (>= 1200px)
 - * .col-xxl- (Extra extra large): સુપર વાઇડ સ્ક્રીન (>= 1400px)

3. COMPONENT CLASSES (ગેટબુટસ્ટ્રેપના રેડીમેડ ડિઝાઇન વિજેટ્સ)

- શેના માટે છે?: ડેવલપરે કોઈ કોડિંગ કે સીએસએસ વગર સીધા જ સુંદર બટન, ફોર્મ, પોપઅપ કે મેનુ વેબસાઇટમાં મૂકવા માટે.
- અતિ મહત્વના કમ્પોનન્ટ્સ અને તેમના પ્રી-ડિફાઇન્ડ ક્લાસીસ:
 - A. બટન્સ (Buttons - .btn):
 - * .btn: બટનનું બેઝિક સ્ટ્રક્ચર અને પેડિંગ સેટ કરે છે.
 - * .btn-primary: બ્લૂ કલરનું મેઇન બટન.
 - * .btn-secondary: ગ્રે કલરનું સેકન્ડરી બટન.
 - * .btn-success: ગ્રીન કલરનું સફળતા કે કન્ફર્મ કરવાનું બટન.
 - * .btn-danger: રેડ કલરનું ડીલીટ કે ડેન્જર એલર્ટ બટન.
 - * .btn-warning: યલો કલરનું વોર્નિંગ બટન.
 - * .btn-outline-*: આ ક્લાસથી બટનની અંદર કલર નથી ભરાતો, માત્ર બોર્ડર દેખાય છે અને માઉસ ઓવર (Hover) કરવાથી આખો કલર ભરાય છે.

- B. નેવિગેશન બાર (Navbar - `.navbar`):
- * `.navbar`: મુખ્ય મેનુ બાર બનાવવા માટેનું રેપર.
 - * `.navbar-expand-lg`: કોમ્પ્યુટર સ્ક્રીન પર મેનુ ખુલ્લું દેખાશે અને મોબાઇલમાં ઓટોમેટિક ત્રણ લીટી વાળું ટોગલ બટન (Hamburger Menu) બની જશે.
 - * `.bg-light` / `.bg-dark`: મેનુ બારનો બેકગ્રાઉન્ડ કલર સેટ કરવા માટે.
 - * `.navbar-brand`: વેબસાઇટનું નામ કે લોગો મૂકવા માટેનો ખાસ ક્લાસ.
 - * `.nav-link`: મેનુના એક-એક ઓપ્શન (લિંક) ની સ્પેસિંગ સેટ કરવા.
- C. કાર્ડ્સ (Cards - `.card`):
- * `.card`: ઇમેજ, ટાઇટલ અને લખાણ સાથેનું બોક્સ (જેમ કે એમેઝોનની પ્રોડક્ટ પ્રોફાઇલ) બનાવવા.
 - * `.card-img-top`: કાર્ડની અંદર સૌથી ઉપર ફોટો ફિક્સ કરવા.
 - * `.card-body`: ફોટાની નીચે ટેક્સ્ટનું પેડિંગ અને એવાઇનમેન્ટ સેટ કરવા.
 - * `.card-title`: કાર્ડના મુખ્ય મથાળા (Heading) માટે.
- D. એલર્ટ્સ અને મોડલ્સ (Alerts & Modals):
- * `.alert .alert-danger`: પેજ પર લાલ કલરની મોટી વોર્નિંગ પટ્ટી બતાવવા.
 - * `.modal`: સ્ક્રીનની વચ્ચે ખુલ્લું પોપઅપ બોક્સ (જે બેકગ્રાઉન્ડને ઝાંખું કરી દે છે).
 - * `.modal-dialog` / `.modal-content`: પોપઅપનું કદ અને અંદરનો ડેટા સેટ કરવા.
- E. કેરોસેલ (Carousel - `.carousel`):
- * `.carousel`: પેજ પર ફોટા આપોઆપ સ્લાઇડ થાય (Image Slider) તેવો કમ્પોનન્ટ બનાવવા.
 - * `.carousel-inner` / `.carousel-item`: સ્લાઇડરના એક-એક ફોટાને ગોઠવવા માટે.

4. UTILITY CLASSES (ઝડપી સીએસએસ એડજસ્ટમેન્ટ માટેના સિંગલ-ટાસ્ક ક્લાસીસ)

- શેના માટે છે?: કમ્પોનન્ટની આજુબાજુ માર્જિન, પેડિંગ, કલર કે બોર્ડર આપવા માટે અલગથી CSS ફાઇલમાં ગયા વગર સીધા HTML માં જ સેટિંગ કરવા.
- ઓલ યુટિલિટી ક્લાસીસની કેટેગરી:

- A. માર્જિન અને પેડિંગ (Spacing - `m & p`):
- * મથાળું: `m` એટલે Margin (બહારની જગ્યા), `p` એટલે Padding (અંદરની જગ્યા).
 - * દિશાઓ: `t` (top - ઉપર), `b` (bottom - નીચે), `s` (start - ડાબી બાજુ), `e` (end - જમણી બાજુ), `x` (ડાબી અને જમણી બંને), `y` (ઉપર અને નીચે બંને).
 - * સાઇઝ: 0 થી 5 સુધી (0 એટલે સાવ શૂન્ય જગ્યા, 5 એટલે સૌથી વધુ સ્પેસ).
 - * દા.ત.: `mt-3` (ઉપરથી થોડી જગ્યા વધારો), `px-4` (અંદરની બાજુ ડાબે-જમણે પેડિંગ આપો), `ms-auto` (ઓબ્જેક્ટને ખોંચીને જમણી બાજુ ફેંકી દેશે).

- B. ટેક્સ્ટ યુટિલિટીઝ (Text Alignment & Colors):
- * `.text-center`: લખાણને બોક્સની બિલકુલ વચ્ચે (Center) લાવવા.
 - * `.text-end`: લખાણને જમણી બાજુ ગોઠવવા.
 - * `.text-uppercase`: બધા જ અક્ષરોને કેપિટલ (Capital Letters) કરવા.
 - * `.fw-bold`: અક્ષરોને જાડા (Bold) કરવા માટે.
 - * `.text-primary` / `.text-muted`: લખાણનો કલર બ્લૂ કરવા અથવા ઝાંખો (Gray) કરવા.

- C. ફ્લેક્સબોક્સ યુટિલિટીઝ (Flexbox Utilities - `.d-flex`):
- * `.d-flex`: કોઈ બોક્સને ફ્લેક્સ કન્ટેનર બનાવે છે, જેથી તેની અંદરની વસ્તુઓ આડી લાઇનમાં ગોઠવાઈ જાય.
 - * `.justify-content-center`: અંદરની આઇટમ્સને આડી રીતે સેન્ટરમાં લાવવા.
 - * `.justify-content-between`: બે આઇટમ્સની વચ્ચે સરખી જગ્યા છોડીને બંને છેડે ગોઠવવા.
 - * `.align-items-center`: અંદરની આઇટમ્સને ઉભી (Vertical) રીતે બરાબર વચ્ચે ગોઠવવા.

- D. બોર્ડર અને શેડો (Borders & Shadows):
- * `.border`: ઓબ્જેક્ટની ચારેય બાજુ પાતળી ગ્રે બોર્ડર આપવા.
 - * `.rounded`: બોર્ડરના ખૂણાઓને સહેજ ગોળ (Curve) કરવા.
 - * `.rounded-circle`: ચોરસ ફોટાને સાવ ગોળ (તડબૂચ જેવો - Circle) કરવા.
 - * `.shadow-sm` / `.shadow`: ઓબ્જેક્ટની પાછળ સુંદર હળવો 3D લુક આપવા.

- E. ડિસ્પ્લે અને પોઝિશન (Display & Position):
- * `.d-none`: ઓબ્જેક્ટને પેજ પરથી સાવ ગાયબ (Hide) કરી દેવા.
 - * `.d-md-block`: મોબાઇલમાં છુપાયેલી વસ્તુને ટેબ્લેટ કે કોમ્પ્યુટર સ્ક્રીન આવતા જ લાઈવ બતાવવા.
 - * `.position-fixed` / `.fixed-top`: મેનુ બાર કે કોઈ બટનને સ્ક્રીન પર એક જગ્યાએ ચોટાડી દેવા (પેજ ગમે એટલું સ્ક્રોલ કરો તો પણ તે ત્યાં જ રહેશે).

CSS

(Cascading Style Sheet)

```
=====
CSS ADVANCED & MISSING VISUAL EFFECTS PROPERTIES (A-Z ENCYCLOPEDIA)
Language: Simple & Pure Gujarati (ગુજરાતી)
Level: Beginner to Hero Level
=====
```

1. GRADIENTS & COLOR TRANSITIONS (કલર મિશ્રણ અને પેટર્ન)

* linear-gradient()

- વ્યાખ્યા: આ એક ફંક્શન છે જે બે અથવા વધારે કલરને એક સીધી રેખામાં (દિશા મુજબ) મિક્સ કરે છે.
- ઉપયોગ: બેકગ્રાઉન્ડને કલરફુલ અને આકર્ષક બનાવવા માટે.
- ઉદાહરણ: background: linear-gradient(to right, red, yellow); (ડાબેથી જમણે કલર બદલાશે)

* radial-gradient()

- વ્યાખ્યા: આ કલરને ગોળાકાર (Circle કે Ellipse) માં કેન્દ્રમાંથી બહારની તરફ મિક્સ કરે છે.
- ઉપયોગ: ગ્લોઇંગ ઇફેક્ટ કે ગોળાકાર શેડ આપવા માટે.
- ઉદાહરણ: background: radial-gradient(circle, white, blue);

* conic-gradient()

- વ્યાખ્યા: આ કલરને ઘડિયાળના કાંટાની જેમ (360 ડિગ્રી) ગોળ ફેરવીને મિક્સ કરે છે.
- ઉપયોગ: પાઇ-ચાર્ટ (Pie Charts) અથવા કલર વ્હીલ બનાવવા માટે.
- ઉદાહરણ: background: conic-gradient(red, yellow, green, red);

* repeating-linear-gradient() / repeating-radial-gradient()

- વ્યાખ્યા: આ લીનિયર કે રેડિયલ ગ્રેડિયન્ટને વારંવાર રીપીટ કરીને એક પેટર્ન બનાવે છે.
- ઉપયોગ: પટ્ટા વાળી ડિઝાઇન (Stripes) કે ઝીબ્રા લાઈનો બનાવવા માટે.
- ઉદાહરણ: background: repeating-linear-gradient(45deg, black, black 10px, white 10px, white 20px);

2. SHAPES, CLIPPING & MASKING (આકારો અને કટીંગ ઇફેક્ટ્સ)

* clip-path

- વ્યાખ્યા: આ પ્રોપર્ટી એલિમેન્ટને ચોક્કસ આકારમાં કાપી નાખે છે (જેમ કે ત્રિકોણ, વર્તુળ, કે સ્ટાર). જે ભાગ કપાઈ જાય તે સ્ક્રીન પર અદ્રશ્ય થઈ જાય છે.
- ઉપયોગ: ત્રાસા હેડર, ગોળાકાર પ્રોફાઇલ પિક્ચર કે કસ્ટમ આકારો બનાવવા.
- ઉદાહરણ: clip-path: polygon(50% 0%, 0% 100%, 100% 100%); (આ એલિમેન્ટને એક ત્રિકોણ બનાવી દેશે).

* mask-image / mask-size / mask-repeat

- વ્યાખ્યા: જેમ ફોટોશોપમાં માસ્કિંગ થાય છે, તેમ કોઈ ઇમેજ કે ગ્રેડિયન્ટનો ઉપયોગ કરીને એલિમેન્ટના અમુક ભાગને છુપાવવા કે પારદર્શક કરવા માટે.
- ઉપયોગ: ફોટાને ધીમેથી કલરમાં ઓગળી જતો (Fade out) બતાવવા માટે.
- ઉદાહરણ: mask-image: linear-gradient(to bottom, black, transparent);

* shape-outside

- વ્યાખ્યા: જ્યારે કોઈ એલિમેન્ટ (જેમ કે ઇમેજ) ને લખાણની વચ્ચે મુકવામાં આવે, ત્યારે લખાણ તે ઇમેજના આકાર (ગોળ કે ત્રિકોણ) ની ફરતે આપોઆપ ગોઠવાઈ જાય છે.
- ઉપયોગ: મેગેઝિન જેવું સુંદર આર્ટિકલ લેઆઉટ બનાવવા માટે.
- ઉદાહરણ: shape-outside: circle(50%);

3. FILTERS & GLASSMORPHISM (બેકગ્રાઉન્ડ બ્લર અને ઇફેક્ટ્સ)

* filter

- વ્યાખ્યા: આ પ્રોપર્ટી સીધી એલિમેન્ટ અથવા ઇમેજ પર વિઝ્યુઅલ ઇફેક્ટ્સ (જેમ કે બ્લર, બ્રાઈટનેસ, બ્લેક એન્ડ વ્હાઇટ) લાગુ કરે છે.
- ઉપયોગ: ફોટાને એડિટ કરવા અથવા હોવર કરવા પર કલર બદલવા.
- ઉદાહરણ: filter: blur(5px) grayscale(100%);

* backdrop-filter

- વ્યાખ્યા: આ એલિમેન્ટની પાછળ રહેલી વસ્તુઓ કે બેકગ્રાઉન્ડને બ્લર કરે છે. આનાથી એલિમેન્ટ પોતે બ્લર નથી થતો.
- ઉપયોગ: કાચ જેવી ઇફેક્ટ (Glassmorphism) વાળા આધુનિક કાર્ડ કે નવબાર બનાવવા.
- ઉદાહરણ: backdrop-filter: blur(10px); background: rgba(255, 255, 255, 0.2);

* mix-blend-mode

- વ્યાખ્યા: એક એલિમેન્ટનો કલર તેની પાછળ રહેલા બીજા એલિમેન્ટના કલર સાથે કેવી રીતે મિક્સ (Blend) થશે તે નક્કી કરે છે.

- ઉપયોગ: ટેક્સ્ટને બેકગ્રાઉન્ડ ઇમેજ સાથે ઓવરલેપ કરીને અદ્ભુત આર્ટ ઇફેક્ટ આપવા.

- ઉદાહરણ: mix-blend-mode: difference;

* background-blend-mode

- વ્યાખ્યા: જો એક જ બોક્સમાં બેકગ્રાઉન્ડ ઇમેજ અને બેકગ્રાઉન્ડ કલર બંને હોય, તો તે બંનેને આપસમાં મિક્સ કરવા માટે.

- ઉદાહરણ: background-blend-mode: multiply;

4. ADVANCED TYPOGRAPHY & TEXT VISUALS (ટેક્સ્ટ કસ્ટમાઇઝેશન)

* background-clip: text

- વ્યાખ્યા: બેકગ્રાઉન્ડ કલર અથવા ગ્રેડિયન્ટને આખા બોક્સમાં બતાવવાને બદલે માત્ર અક્ષરોની અંદર જ પૂરી દે છે.

- ઉપયોગ: અક્ષરોની અંદર લાઇવ ગ્રેડિયન્ટ કલર કે ફોટો ભરવા માટે.

- ઉદાહરણ: background: linear-gradient(red, blue); -webkit-background-clip: text; color: transparent;

* text-stroke / -webkit-text-stroke

- વ્યાખ્યા: અક્ષરોની ફરતે આઉટલાઇન (Border) બનાવવા માટે.

- ઉદાહરણ: -webkit-text-stroke: 2px black;

* text-shadow

- વ્યાખ્યા: અક્ષરોની પાછળ પડછાયો આપવા માટે.

- ઉદાહરણ: text-shadow: 2px 2px 5px grey;

5. INTERACTION & INTERACTION CONTROLS (યુઝર કંટ્રોલ પ્રોપર્ટીઝ)

* pointer-events

- વ્યાખ્યા: નક્કી કરે છે કે એલિમેન્ટ પર માઉસ ક્લિક કે ટચ કામ કરશે કે નહીં. જો `none` કરો તો માઉસની ક્લિક એલિમેન્ટની આરપાર જતી રહેશે.

- ઉપયોગ: કોઈ આઇકોન કે લેયરને ક્લિક કરતા અટકાવવા માટે.

- ઉદાહરણ: pointer-events: none;

* user-select

- વ્યાખ્યા: યુઝર વેબસાઇટ પરથી લખાણ માઉસ વડે સિલેક્ટ (કોપી) કરી શકે કે નહીં તે કંટ્રોલ કરે છે.

- ઉપયોગ: વેબસાઇટના કન્ટેન્ટની ચોરી અટકાવવા માટે.

- ઉદાહરણ: user-select: none;

* caret-color

- વ્યાખ્યા: ઇનપુટ બોક્સ (Text Input) માં જ્યારે આપણે ટાઇપ કરીએ ત્યારે ઝબકતી ઉભી લીટી (Cursor) નો કલર બદલવા માટે.

- ઉદાહરણ: caret-color: red;

* accent-color

- વ્યાખ્યા: નેટિવ ફોર્મ એલિમેન્ટ્સ જેવા કે ચેકબોક્સ (Checkbox) કે રેડિયો બટન (Radio button) નો ડિફોલ્ટ વાદળી કલર બદલીને બ્રાન્ડ કલર સેટ કરવા માટે.

- ઉદાહરણ: accent-color: purple;

6. SCROLLING EFFECTS & VISUAL BEHAVIOR (સ્ક્રોલિંગ જાદુ)

* scroll-behavior

- વ્યાખ્યા: જ્યારે પેજમાં કોઈ લિંક પર ક્લિક કરીએ ત્યારે પેજ ઝટકા સાથે જવાને બદલે ધીમેથી સરકીને (Smooth) ત્યાં પહોંચે છે.

- ઉદાહરણ: scroll-behavior: smooth;

* scroll-snap-type / scroll-snap-align

- વ્યાખ્યા: યુઝર જ્યારે સ્ક્રોલ કરે ત્યારે પેજ અથવા અટકવાને બદલે સીધું આગલા કે પાછલા બોક્સ પર આપોઆપ લોક (Snap) થઈ જાય છે.

- ઉપયોગ: ઇન્સ્ટાગ્રામ રીલ્સ જેવું સ્લાઇડર કે ફૂલ-સ્ક્રીન સેક્શન સ્ક્રોલિંગ બનાવવા.

- ઉદાહરણ: scroll-snap-align: start;

* overscroll-behavior

- વ્યાખ્યા: જ્યારે કોઈ ચોક્કસ બોક્સનું સ્ક્રોલિંગ પૂરું થઈ જાય, ત્યારે આખું પેજ સ્ક્રોલ થવા લાગે છે. તેને રોકવા માટે આ વપરાય છે.

- ઉદાહરણ: overscroll-behavior: contain;

7. TRANSFORMS & 3D EFFECTS (હલનચલન અને 3D ઇફેક્ટ્સ)

* transform (rotate, scale, translate, skew)

- વ્યાખ્યા: એલિમેન્ટને ફેરવવા (Rotate), મોટું-નાનું કરવા (Scale), આગળ-પાછળ ખસેડવા (Translate) કે ત્રાંસું કરવા (Skew) માટે.

- ઉદાહરણ: transform: rotate(45deg) scale(1.5);

* transform-style: preserve-3d

- વ્યાખ્યા: એલિમેન્ટના બાળ-એલિમેન્ટ્સ (Child elements) ને 3D સ્પેસમાં જીવંત રાખવા માટે.

- ઉપયોગ: પત્તા ઉથલાવવાની ઇફેક્ટ (Card Flip effect) બનાવવા માટે.

* perspective

- વ્યાખ્યા: ચુઝર સ્ક્રીનથી કેટલો દૂર છે તેવો આભાસ કરાવીને 3D ઇફેક્ટને ઊંડાણ (Depth) આપે છે.

- ઉદાહરણ: perspective: 1000px;

* backface-visibility

- વ્યાખ્યા: જ્યારે 3D માં કોઈ કાર્ડ ઉલટું (ફ્લિપ) થાય, ત્યારે તેની પાછળનો ભાગ છુપાવવો કે બતાવવો તે નક્કી કરે છે.

- ઉદાહરણ: backface-visibility: hidden;

=====

CSS PROPERTIES ENCYCLOPEDIA (0 TO HERO LEVEL)
Comprehensive A-Z Guide without any duplicate content

1. BOX MODEL PROPERTIES (બોક્સ મોડેલ પ્રોપર્ટીઝ)

- margin: એલિમેન્ટની બહાર ચારેય બાજુ જગ્યા છોડવા માટે.
- margin-top / margin-right / margin-bottom / margin-left: ચોક્કસ દિશામાં બહાર જગ્યા છોડવા.
- margin-inline / margin-block: લોજીકલ માર્જિન (લખાણની દિશા મુજબ).
- padding: એલિમેન્ટની બોર્ડરની અંદર જગ્યા છોડવા માટે.
- padding-top / padding-right / padding-bottom / padding-left: ચોક્કસ દિશામાં અંદર જગ્યા છોડવા.
- padding-inline / padding-block: લોજીકલ પેડિંગ.
- border: બોર્ડર સ્ટાઇલ, કલર અને વિડ્થ એકસાથે સેટ કરવા.
- border-width / border-style / border-color: બોર્ડરની જાડાઈ, પ્રકાર અને કલર.
- border-radius: બોર્ડરના ખૂણા ગોળ કરવા.
- box-sizing: વિડ્થ અને હાઇટની ગણતરી બદલવા (content-box અથવા border-box).

2. LAYOUT & POSITIONING PROPERTIES (લેઆઉટ અને પોઝિશનિંગ)

- display: એલિમેન્ટ કઈ રીતે સ્ક્રીન પર દેખાશે (block, inline, inline-block, flex, grid, none).
- position: એલિમેન્ટને ગોઠવવાની પદ્ધતિ (static, relative, absolute, fixed, sticky).
- top / right / bottom / left: પોઝિશન કરેલા એલિમેન્ટનું અંતર સેટ કરવા.
- z-index: એલિમેન્ટનું વેયર નક્કી કરવા (કયું ઉપર દેખાશે).
- overflow: જો કન્ટેન્ટ બોક્સની બહાર જાય તો શું કરવું (visible, hidden, scroll, auto).
- overflow-x / overflow-y: આડી કે ઊભી દિશામાં ઓવરફ્લો કંટ્રોલ કરવા.
- visibility: એલિમેન્ટને છુપાવવા કે બતાવવા (hidden, visible).
- float: એલિમેન્ટને ડાબે કે જમણે ખસેડવા (જૂની પદ્ધતિ).
- clear: ફ્લોટ થયેલા એલિમેન્ટની અસરો રોકવા.

3. FLEXBOX LAYOUT PROPERTIES (ફ્લેક્સબોક્સ પ્રોપર્ટીઝ)

- flex-direction: ફ્લેક્સ આઇટમ્સની દિશા (row, column).
- justify-content: આડી લાઇનમાં આઇટમ્સ વચ્ચે જગ્યા વહેંચવા.
- align-items: ઊભી લાઇનમાં આઇટમ્સને ગોઠવવા.
- flex-wrap: આઇટમ્સ નવી લાઇનમાં જશે કે નહીં (nowrap, wrap).
- flex-flow: flex-direction અને flex-wrap નું શોર્ટહેન્ડ.
- align-content: મલ્ટીપલ લાઇનો વચ્ચે જગ્યા સેટ કરવા.
- order: ફ્લેક્સ આઇટમનો ક્રમ બદલવા.
- flex-grow: આઇટમ કેટલી મોટી થશે તે પ્રમાણ.
- flex-shrink: આઇટમ કેટલી નાની સંકોચાશે તે પ્રમાણ.
- flex-basis: આઇટમનું મૂળ શરૂઆતનું કદ.
- flex: grow, shrink, અને basis નું શોર્ટહેન્ડ.
- align-self: કોઈ એક ચોક્કસ આઇટમનું વર્ટિકલ એલાઇનમેન્ટ બદલવા.

4. GRID LAYOUT PROPERTIES (ગ્રીડ લેઆઉટ પ્રોપર્ટીઝ)

- grid-template-columns: ગ્રીડની ઊભી કોલમ્સનું માળખું અને સાઈઝ સેટ કરવા.
- grid-template-rows: ગ્રીડની આડી રો (લાઇન) નું માળખું સેટ કરવા.
- grid-template-areas: નામ આપીને આખા લેઆઉટનું સ્ટ્રક્ચર ગોઠવવા.
- grid-column-gap / grid-row-gap / gap: ગ્રીડ બોક્સ વચ્ચેની ખાલી જગ્યા (Gap).
- grid-column / grid-row: આઇટમ કઈ કોલમ કે રો થી શરૂ થઈને ક્યાં પૂરી થશે તે નક્કી કરવા.
- justify-items: ગ્રીડ સેલની અંદર આડી દિશામાં કન્ટેન્ટ ગોઠવવા.
- align-items: ગ્રીડ સેલની અંદર ઊભી દિશામાં કન્ટેન્ટ ગોઠવવા.
- place-items: justify-items અને align-items નું શોર્ટહેન્ડ.

5. MODERN & NEW CSS PROPERTIES (2025-2026) (નવી અને આધુનિક પ્રોપર્ટીઝ)

- aspect-ratio: એલિમેન્ટનો રેશિયો (જેમ કે વિડિયો માટે 16/9) ફિક્સ રાખવા.
- object-fit: ઇમેજ કે વિડિયો બોક્સમાં કેવી રીતે ફિટ થશે (cover, contain, fill).
- object-position: ઇમેજ કોપ થાય ત્યારે કયો ભાગ સેન્ટરમાં રાખવો તે પોઝિશન.
- clip-path: એલિમેન્ટને સ્ટાર, ત્રિકોણ કે પોલીગોન આકારમાં કાપવા માટે.
- backdrop-filter: પાછળના કન્ટેન્ટને બ્લર કરીને કાચ જેવો (Glassmorphism) લુક આપવા.
- filter: ફોટાને લાઈવ બ્લર, ગ્રેસ્કેલ કે બ્રાઈટનેસ આપવા.
- color-mix(): બે કલર્સને ટકાવારી મુજબ લાઈવ મિક્સ કરવા માટેનું ફંક્શન.
- accent-color: ફોર્મના ચેકબોક્સ અને રેડિયો બટનનો નેટિવ કલર બદલવા.
- mix-blend-mode: અક્ષરો કે ઇમેજના કલરને પાછળના બેકગ્રાઉન્ડ સાથે મિક્સ (Blend) કરવા.
- background-clip: અક્ષરોની અંદર સોલિડ કલરને બદલે ગ્રેડિયન્ટ કે ઇમેજ પૂરવા (background-clip: text).

- anchor-name / position-anchor: (Native Anchor Positioning) જાવાસ્ક્રિપ્ટ વગર પોપ-અપને ટાર્ગેટ બટન સાથે લોક કરવા.
- animation-timeline: (Scroll-Driven Animations) જાવાસ્ક્રિપ્ટ વગર માત્ર સ્ક્રોલિંગના આધારે એનિમેશન ચલાવવા.
- user-select: યુઝર વેબસાઈટનું લખાણ સિલેક્ટ કે કોપી ન કરી શકે તે માટે (none).
- pointer-events: એલિમેન્ટ પર માઉસ ક્લિકને ડિસેબલ કે આરપાર જવા દેવા (none).
- scroll-behavior: લિંક પર ક્લિક કરતા પેજ સ્મૂથ સ્લાઈડ થાય તે માટે (smooth).

6. VISUALS, GRADIENTS & VISUAL EFFECTS (વિઝ્યુઅલ્સ અને ગ્રેડિયન્ટ્સ)

- color: અક્ષરોનો (Text) કલર સેટ કરવા.
- background-color: બોક્સનો બેકગ્રાઉન્ડ કલર.
- background-image: બેકગ્રાઉન્ડમાં ઇમેજ કે ગ્રેડિયન્ટ સેટ કરવા.
- linear-gradient(): સીધી રેખામાં મલ્ટીપલ કલર્સ મિક્સ કરવા.
- radial-gradient(): સેન્ટરમાંથી ગોળાકારમાં કલર્સ ફેલાવવા.
- conic-gradient(): ઘડિયાળના કાંટાની જેમ 360 ડિગ્રીમાં કલર વ્હીલ બનાવવા.
- box-shadow: બોક્સની આજુબાજુ પડછાયો (Shadow) આપવા.
- text-shadow: અક્ષરોની પાછળ પડછાયો આપવા.
- opacity: એલિમેન્ટને પારદર્શક (Transparent) બનાવવા (0 થી 1).

7. ANIMATIONS & TRANSITIONS (એનિમેશન્સ)

- transition-property / transition-duration / transition-timing-function / transition-delay: સ્મૂથ હોવર ઇફેક્ટ્સ આપવા.
- animation-name / animation-duration / animation-timing-function: @keyframes એનિમેશન રન કરવા.
- animation-iteration-count: એનિમેશન કેટલી વાર ફરશે (infinite).
- animation-fill-mode: એનિમેશન પૂરું થયા પછી છેલ્લી સ્ટેટ પર એલિમેન્ટને રોકી રાખવા (forwards).

8. RESPONSIVE DESIGN FUNCTIONS (રિસ્પોન્સિવ મેથ ફંક્શન્સ)

- calc(): સાયનેમિક ગણતરી કરવા (દા.ત. 100% - 50px).
- min() / max(): ઉપલબ્ધ સાઈઝમાંથી મિનિમમ કે મેક્સિમમ કદ આપોઆપ પસંદ કરવા.
- clamp(): મિનિમમ, પ્રિફર્ડ અને મેક્સિમમ સેટ કરીને મીડિયા ક્વેરી વગર અક્ષરો રિસ્પોન્સિવ કરવા.

=====

CSS MEGA MASTER LIST (A TO Z BY CATEGORIES)

100% COMPLETE GLOBAL SPECIFICATIONS (GUJARATI)

=====

તમારી વિનંતી મુજબ, ડબલ્યુ3સી (W3C) અને મોર્ડન વેલ્સ સ્પેસિફિકેશનના આધારે સીએસએસ (CSS) ની અંદર આવતી તમામ ૧૦૦% પ્રોપર્ટીઝનું આલ્ફાબેટિકલ ઓર્ડર (A to Z) અને કેટેગરી વાઈઝ મેગા માસ્ટર લિસ્ટ તૈયાર કરેલું છે. આ લિસ્ટની બહાર સીએસએસમાં કોઈ જ પ્રોપર્ટી અસ્તિત્વ ધરાવતી નથી.

=====

 કેટેગરી ૧: ડિસ્પ્લે અને લેઆઉટ એન્જિનિયરિંગ (DISPLAY & LAYOUT ENGINE)

=====

આ કેટેગરીમાં એલિમેન્ટ્સનું મૂળભૂત માળખું, અવાઈનમેન્ટ, ફ્લેક્સબોક્સ અને ગ્રીડ લેઆઉટ્સ નિયંત્રિત કરતી પ્રોપર્ટીઝ છે.

[A]

- * align-content: મલ્ટિ-લાઇન કન્ટેનરની આડી/ઊભી સ્પેસ વહેંચવા માટે.
-> Values: normal | stretch | center | start | end | flex-start | flex-end | space-between | space-around | space-evenly
- * align-items: સિંગલ લાઇન ફ્લેક્સ/ગ્રીડ આઇટમ્સનું ક્રોસ-એક્સિસ અવાઈનમેન્ટ કરવા.
-> Values: normal | stretch | center | start | end | flex-start | flex-end | baseline
- * align-self: સિંગલ આઇટમ માટે પેરન્ટનું align-items નું વર્તન ઓવરરાઇડ કરવા.
-> Values: auto | stretch | center | start | end | baseline
- * aspect-ratio: એલિમેન્ટની પહોળાઈ અને ઊંચાઈનો ગુણોત્તર (રેશિયો) સેટ કરવા.
-> Values: auto | <ratio> (e.g., 16/9, 1/1)

[C]

- * clear: ફ્લોટ (float) કરેલા એલિમેન્ટ્સની આજુબાજુથી જગ્યા યોખ્ખી કરવા.
-> Values: none | left | right | both | inline-start | inline-end
- * column-count: ટેક્સ્ટને અખબારની જેમ કેટલી ઊભી કોલમમાં વહેંચવી તે નક્કી કરવા.
-> Values: auto | <integer>
- * column-fill: કોલમની અંદર ડેટા આપોઆપ સંતુલિત રીતે કેવી રીતે વહેંચાશે તે.
-> Values: auto | balance
- * column-gap: બે ઊભી કોલમ વચ્ચેની ખાલી જગ્યા (ગેપ) નક્કી કરવા.
-> Values: normal | <length> | <percentage>
- * column-rule: કોલમની વચ્ચે આવતી સેપરેટર લાઇન (બોર્ડર શોર્ટહેન્ડ).
-> Values: <border-width> || <border-style> || <color>
- * column-rule-color: કોલમ સેપરેટર લાઇનનો કલર સેટ કરવા.
- * column-rule-style: કોલમ સેપરેટર લાઇનની સ્ટાઇલ (solid, dashed વગેરે).
- * column-rule-width: કોલમ સેપરેટર લાઇનની જાડાઈ.
- * column-span: કોઈ એલિમેન્ટ બધી કોલમની આજુબાજુ ફેલાશે કે નહીં તે.
-> Values: none | all
- * column-width: કોલમની આદર્શ લઘુત્તમ પહોળાઈ સેટ કરવા.
-> Values: auto | <length>
- * columns: column-count અને column-width બંને એકસાથે લખવા માટેનો શોર્ટહેન્ડ.

[D]

- * display: એલિમેન્ટ સ્ક્રીન પર કઈ રીતે પ્રદર્શિત થશે (તેનું મૂળ વર્તન).
-> Values: none | block | inline | inline-block | flex | inline-flex | grid | inline-grid | table | contents | subgrid

[F]

- * flex: flex-grow, flex-shrink, અને flex-basis ત્રણેય એકસાથે લખવા માટેનો શોર્ટહેન્ડ.
- * flex-basis: ફ્લેક્સ આઇટમનું કદ બદલાતા પહેલાં તેનું મૂળ પાયાનું કદ (Width/Height) કેટલું રહેશે.
-> Values: auto | content | <length> | <percentage>
- * flex-direction: ફ્લેક્સ બોક્સની અંદર આઇટમ્સ આડી લાઇનમાં રહેશે કે ઊભી લાઇનમાં.
-> Values: row | row-reverse | column | column-reverse
- * flex-flow: flex-direction અને flex-wrap બંને એકસાથે લખવા માટેનો શોર્ટહેન્ડ.
- * flex-grow: કન્ટેનરમાં બચેલી ખાલી જગ્યાના આધારે આઇટમ કેટલી મોટી થઈ શકે તેની ક્ષમતા.
-> Values: <number> (0, 1, 2...)
- * flex-shrink: જગ્યા ન હોય ત્યારે કન્ટેનર પ્રમાણે આઇટમ કેટલી સંકોચાઈ શકે.
-> Values: <number> (0, 1, 2...)
- * flex-wrap: જગ્યા ઓછી હોય ત્યારે આઇટમ્સ આપોઆપ નવી લાઇનમાં મોકલવી કે નહીં.
-> Values: nowrap | wrap | wrap-reverse
- * float: એલિમેન્ટને પેજ પર ડાબે કે જમણે સરકાવવા (ખસેડવા) માટે.
-> Values: none | left | right | inline-start | inline-end

[G]

- * gap: ફ્લેક્સ કે ગ્રીડ આઇટમ્સ વચ્ચે આપમેળે છોડાતી જગ્યા (શોર્ટહેન્ડ).
- > Values: normal | <length> | <percentage>
- * grid: ગ્રીડ પેરન્ટના તમામ લેઆઉટ એકસાથે સેટ કરવાનો માસ્ટર શોર્ટહેન્ડ.
- * grid-area: grid-row-start, grid-column-start, grid-row-end, grid-column-end અથવા grid-template-areas નું નામ સેટ કરવાનો શોર્ટહેન્ડ.
- * grid-auto-columns: ઇમ્પ્લિસિટ (આપોઆપ બનેલી) ગ્રીડ કોલમની ડિફોલ્ટ સાઇઝ નક્કી કરવા.
- * grid-auto-flow: ગ્રીડ એન્જિન નવી આઇટમ્સને આપોઆપ રો માં ગોઠવશે કે કોલમમાં.
- > Values: row | column | dense | row dense | column dense
- * grid-auto-rows: ઇમ્પ્લિસિટ (આપોઆપ બનેલી) ગ્રીડ રો (આડી લાઇન) ની ડિફોલ્ટ સાઇઝ.
- * grid-column: grid-column-start અને grid-column-end નો શોર્ટહેન્ડ.
- * grid-column-end: ગ્રીડ આઇટમ કઈ ગ્રીડ લાઇન પર પૂરી થશે તે સેટ કરવા.
- * grid-column-start: ગ્રીડ આઇટમ કઈ ગ્રીડ લાઇનથી શરૂ થશે તે નક્કી કરવા.
- * grid-row: grid-row-start અને grid-row-end નો શોર્ટહેન્ડ.
- * grid-row-end: ગ્રીડ આઇટમ કઈ આડી ગ્રીડ લાઇન પર પૂરી થશે.
- * grid-row-start: ગ્રીડ આઇટમ કઈ આડી ગ્રીડ લાઇનથી શરૂ થશે.
- * grid-template: grid-template-rows, grid-template-columns, અને grid-template-areas નો શોર્ટહેન્ડ.
- * grid-template-areas: ગ્રીડ લેઆઉટને ટેક્સ્ટ-આધારિત નામ આપીને આખું સ્ટ્રક્ચર ડિઝાઇન કરવા.
- * grid-template-columns: ગ્રીડ લેઆઉટમાં કેટલી ઊભી કોલમ અને કેવડી સાઇઝની બનાવવી તે.
- > Values: none | <track-list> | <auto-track-list> | subgrid
- * grid-template-rows: ગ્રીડ લેઆઉટમાં કેટલી આડી રો અને કેવડી સાઇઝની બનાવવી તે.
- > Values: none | <track-list> | <auto-track-list> | subgrid

[J]

- * justify-content: મેઇન એક્સિસ (આડી લાઇન) પર આઇટમ્સ વચ્ચેની જગ્યા કેવી રીતે વહેંચવી.
- > Values: normal | space-between | space-around | space-evenly | center | start | end | flex-start | flex-end
- * justify-items: ગ્રીડ ખાનાની અંદર આઇટમ્સનું ડિફોલ્ટ આડું અલાઇનમેન્ટ સેટ કરવા.
- * justify-self: સિંગલ ગ્રીડ આઇટમનું પોતાનું પર્સનલ આડું અલાઇનમેન્ટ બદલવા.

[O]

- * order: ફ્લેક્સ કે ગ્રીડ આઇટમ્સનો ડિફોલ્ટ સિક્વન્સ (ક્રમ) બદલવા માટે.
- > Values: <integer> (0, 1, -1...)

[R]

- * row-gap: માત્ર આડી લાઇન (Rows) વચ્ચેની જગ્યા સેટ કરવા.

=====

📌 કેટેગરી ૨: પોઝિશનિંગ અને લેયરિંગ પ્રોપર્ટીઝ (POSITIONING & LAYERING)

=====

આ પ્રોપર્ટીઝ એલિમેન્ટને પેજ કે સ્ક્રીન પર ચોક્કસ જગ્યાએ ફિક્સ કરવા માટે વપરાય છે.

[B]

- * bottom: પોઝિશન આપેલા એલિમેન્ટનું નીચેની બાજુથી અંતર નક્કી કરવા.

[I]

- * inset: top, right, bottom, left ચારેય પ્રોપર્ટી એક જ વારમાં લખવાનો શોર્ટહેન્ડ.
- * inset-block: લોજિકલ પ્રોપર્ટી - ટેક્સ્ટની રો ની દિશા પ્રમાણે ઉપર-નીચેનું અંતર.
- * inset-inline: લોજિકલ પ્રોપર્ટી - લાઇન પ્રમાણે ડાબે-જમણેથી અંતર.

[L]

- * left: પોઝિશન આપેલા એલિમેન્ટનું ડાબી બાજુથી અંતર નક્કી કરવા.

[P]

- * position: એલિમેન્ટ કઈ પદ્ધતિથી પેજ પર ગોઠવાશે તેનું મેકેનિઝમ.
- > Values: static | relative | absolute | fixed | sticky

[R]

- * right: પોઝિશન આપેલા એલિમેન્ટનું જમણી બાજુથી અંતર નક્કી કરવા.

[T]

- * top: પોઝિશન આપેલા એલિમેન્ટનું ઉપરની બાજુથી અંતર નક્કી કરવા.

[Z]

- * z-index: એલિમેન્ટ્સ જ્યારે એકબીજાની ઉપર આવે ત્યારે કયું વેયર ઉપર દેખાશે તેનો ક્રમ.
-> Values: auto | <integer> (999, -1...)

=====

📦 કેટેગરી ૩: બોક્સ મોડેલ અને સાઇઝિંગ (BOX MODEL & SIZING)

=====

એલિમેન્ટની અંદર-બહારની ખાલી જગ્યાઓ અને બોર્ડર ગણતરીની પ્રોપર્ટીઝ.

[B]

- * box-sizing: પેડિંગ અને બોર્ડરના કારણે બોક્સની કુલ સાઇઝની ગણતરી બદલવા.
-> Values: content-box | border-box

[H]

- * height: એલિમેન્ટની ઊંચાઈ સેટ કરવા.
-> Values: auto | <length> | <percentage> | max-content | min-content | fit-content

[M]

- * margin: એલિમેન્ટની બોર્ડરની બહાર ચારેય બાજુ જગ્યા છોડવા માટે (શોર્ટહેન્ડ).
- * margin-block: લોજિકલ માર્જિન - ઉપર અને નીચે એકસાથે જગ્યા છોડવા.
- * margin-inline: લોજિકલ માર્જિન - ડાબે અને જમણે એકસાથે જગ્યા છોડવા.
- * margin-top / margin-right / margin-bottom / margin-left: વ્યક્તિગત રીતે ચોક્કસ દિશાની માર્જિન.
- * max-height: એલિમેન્ટની ઊંચાઈ વધુમાં વધુ કેટલી વધી શકશે તેની લિમિટ.
- * max-width: એલિમેન્ટની પહોળાઈ વધુમાં વધુ કેટલી વધી શકશે તેની લિમિટ.
- * min-height: એલિમેન્ટની ઊંચાઈ ઓછામાં ઓછી આટલી તો રહેશે જ તેની લિમિટ.
- * min-width: એલિમેન્ટની પહોળાઈ ઓછામાં ઓછી આટલી તો રહેશે જ તેની લિમિટ.

[P]

- * padding: એલિમેન્ટની બોર્ડરની અંદર કન્ટેન્ટની ફરતે જગ્યા છોડવા માટે (શોર્ટહેન્ડ).
- * padding-block: લોજિકલ પેડિંગ - અંદરની તરફ ઉપર-નીચે જગ્યા સેટ કરવા.
- * padding-inline: લોજિકલ પેડિંગ - અંદરની તરફ ડાબે-જમણે જગ્યા સેટ કરવા.
- * padding-top / padding-right / padding-bottom / padding-left: વ્યક્તિગત દિશાનું પેડિંગ.

[W]

- * width: એલિમેન્ટની પહોળાઈ સેટ કરવા.
-> Values: auto | <length> | <percentage> | max-content | min-content | fit-content

=====

📄 કેટેગરી ૪: ફોન્ટ અને ટેક્સ્ટ સ્ટાઇલિંગ (TYPOGRAPHY & TEXT)

=====

લખાણનો દેખાવ, સાઇઝ, વળાંક અને સ્પેસિંગ નિયંત્રિત કરતી પ્રોપર્ટીઝ.

[F]

- * font: ફોન્ટની તમામ પ્રોપર્ટી એક જ વારમાં લખવા માટેનો માસ્ટર શોર્ટહેન્ડ.
- * font-family: લખાણની આખી ફોન્ટ ફેમિલી (સ્ટાઇલ/મરોડ) બદલવા માટે.
-> Values: <family-name> || <generic-family> (sans-serif, serif...)
- * font-kerning: અક્ષરોની જોડી વચ્ચેની ડિફોલ્ટ સ્પેસિંગ ઇન્ફોર્મેશન ઓન/ઓફ કરવા.
- * font-size: અક્ષરોનું કદ નાનું કે મોટા કરવા માટે.
-> Values: <absolute-size> | <relative-size> | <length> | <percentage>
- * font-size-adjust: ફોન્ટ બદલાય ત્યારે પણ તેની એક્સ-હાઇટ જાળવી રાખવા માટે.
- * font-stretch: અક્ષરોને આડા ખેંચીને પહોળા કે સંકુચિત (condensed) કરવા માટે.
- * font-style: અક્ષરો સામાન્ય રાખવા કે ત્રાંસા (Italic) કરવા તે નક્કી કરવા.
-> Values: normal | italic | oblique
- * font-variant: ટેક્સ્ટને સ્મોલ-કેપ્સ (નાના કેપિટલ અક્ષરો) માં બદલવા માટે.
- * font-weight: અક્ષરોને પાતળા કે ઘાટા (Bold) કરવા માટેની પ્રોપર્ટી.
-> Values: normal | bold | lighter | bolder | <number> (100 થી 900)

[L]

- * letter-spacing: લખાણના દરેક સિંગલ અક્ષર વચ્ચે આડી ખાલી જગ્યા છોડવા.
- * line-height: પેરેગ્રાફની બે આડી લાઇનો વચ્ચેની ઊભી જગ્યા (લાઇન સ્પેસિંગ).

[T]

- * text-align: લખાણનું અવાઇનમેન્ટ સેટ કરવા (ડાબે, જમણે, વચ્ચે કે જસ્ટિફાઇ).
- > Values: left | right | center | justify | start | end
- * text-align-last: પેરેગ્રાફની છેલ્લી લાઇનનું અવાઇનમેન્ટ કેવું રહેશે તે નક્કી કરવા.
- * text-decoration: લખાણની નીચે, ઉપર કે વચ્ચે લાઇન કરવા માટે (શોર્ટહેન્ડ).

- * text-decoration-color: ડેકોરેશન લાઇનનો કલર બદલવા.
- * text-decoration-line: લાઇન ક્યાં આવશે તે (underline, overline, line-through).
- * text-decoration-style: લાઇન કેવી દેખાશે (solid, wavy, dashed, dotted).
- * text-decoration-thickness: ડેકોરેશન લાઇનની જાડાઈ સેટ કરવા.
- * text-indent: પેરેગ્રાફની પહેલી લાઇન શરૂ થાય તે પહેલાં કેટલી જગ્યા છોડવી.
- * text-justify: text-align: justify આપ્યું હોય ત્યારે સ્પેસિંગ મેનેજ કરવાનો કાયદો.
- * text-overflow: જ્યારે લખાણ બોક્સની બહાર નીકળે ત્યારે ત્રણ ટપકા (...) બતાવવા માટે.
-> Values: clip | ellipsis | <string>
- * text-shadow: અક્ષરોની પાછળ સુંદર પડછાયો (Shadow) ઇફેક્ટ આપવા.
- * text-transform: લખાણને આપોઆપ કેપિટલ, સ્મોલ કે કેપિટલાઇઝ કરવા માટે.
-> Values: none | capitalize | uppercase | lowercase

[W]

- * white-space: લખાણની અંદરની વધારાની સ્પેસ અને નવી લાઇન (Enter) ને કેવી રીતે હેન્ડલ કરવી.
-> Values: normal | nowrap | pre | pre-wrap | pre-line
- * word-break: બહુ મોટો શબ્દ લાઇનના અંતે આપોઆપ ક્યાંથી તૂટશે તેનો કાયદો.
-> Values: normal | break-all | keep-all | break-word
- * word-spacing: લખાણમાં બે આખા શબ્દો વચ્ચે ખાલી જગ્યા સેટ કરવા માટે.

=====

🎨 કેટેગરી પ: વિઝ્યુઅલ, ગ્રાફિક્સ અને બોર્ડર (VISUALS, GRAPHICS & BORDERS)

=====

કલર, બેકગ્રાઉન્ડ ઇમેજ, શેડો, માર્કિંગ અને ફિલ્ટર્સ સંબંધિત પ્રોપર્ટીઝ.

[B]

- * backdrop-filter: એલિમેન્ટની પાછળ રહેલા ભાગને બ્લર (blur) કે કલર ફિલ્ટર કરવા.
-> Values: none | <filter-function-list> (e.g., blur(10px))
- * background: બેકગ્રાઉન્ડની તમામ પ્રોપર્ટીઝ એક જ લાઇનમાં લખવાનો માસ્ટર શોર્ટહેન્ડ.
- * background-attachment: પેજ સ્ક્રોલ કરતી વખતે બેકગ્રાઉન્ડ ફોટો ત્યાં જ ચોટશે કે હલશે.
-> Values: scroll | fixed | local
- * background-blend-mode: બેકગ્રાઉન્ડમાં એક કરતાં વધુ ફોટા કે કલર્સ હોય તો તેને મિક્સ કરવા.
- * background-clip: બેકગ્રાઉન્ડ કલર/ફોટો બોર્ડર સુધી દેખાશે, પેડિંગ સુધી કે માત્ર કન્ટેન્ટ સુધી.
-> Values: border-box | padding-box | content-box | text
- * background-color: બેકગ્રાઉન્ડનો કલર બદલવા.
- * background-image: બેકગ્રાઉન્ડમાં ઇમેજ કે ગ્રેડિએન્ટ સેટ કરવા.
- * background-origin: બેકગ્રાઉન્ડ ઇમેજનો ઓરિજિનલ પાથ ક્યાંથી ગણવો તે નક્કી કરવા.
- * background-position: બેકગ્રાઉન્ડ ઇમેજ બોક્સમાં કઈ જગ્યાએ સેટ થશે તે.
- * background-repeat: બેકગ્રાઉન્ડ ઇમેજ વારંવાર રીપીટ થશે કે સિંગલ રહેશે.
-> Values: repeat | no-repeat | repeat-x | repeat-y | space | round
- * background-size: બેકગ્રાઉન્ડ ઇમેજની સાઇઝ (cover, contain કે ફિક્સ પિક્સલ).
- * border: બોર્ડરની જાડાઈ, સ્ટાઇલ અને કલર લખવા માટેનો કોમન શોર્ટહેન્ડ.
- * border-radius: બોક્સના ચારેય ખૂણાઓને ગોળ કરવા માટેની પ્રોપર્ટી.
- * box-shadow: આખા બોક્સની પાછળ કે અંદર (inset) સુંદર પડછાયો આપવા માટે.

[C]

- * clip-path: ફોટોશોપની જેમ એલિમેન્ટને ચોક્કસ ભૌમિતિક આકાર (વર્તુળ, ત્રિકોણ, પોલિગોન) માં કાપવા.
-> Values: none | circle() | ellipse() | polygon() | path()

[F]

- * filter: ઇમેજ કે એલિમેન્ટ પર ફોટોશોપ જેવી રેડીમેડ ઇફેક્ટ્સ એપ્લાય કરવા માટે.
-> Values: blur() | brightness() | contrast() | grayscale() | hue-rotate() | invert() | opacity() | saturate() | sepia() | drop-shadow()

[M]

- * mask-image: કોઈપણ ઇમેજ કે ગ્રેડિએન્ટનો ઉપયોગ કરીને એલિમેન્ટ પર માર્કિંગ કરવા.
- * mix-blend-mode: એલિમેન્ટના કલરને તેની પાછળ રહેલા બીજા એલિમેન્ટ સાથે મિક્સ કરવા.

[O]

- * object-fit: કે <video> ને તેના કન્ટેનર બોક્સમાં આપોઆપ કઈ રીતે ફિટ કરવો.
-> Values: fill | contain | cover | none | scale-down
- * object-position: કન્ટેનર બોક્સની અંદર ઇમેજ કે વીડિયોનું કન્ટ લોકેશન બદલવા.
- * opacity: આખા એલિમેન્ટની ઘનતા (પારદર્શકતા) સેટ કરવા માટે.
-> Values: <number> (0.0 સાવ ગાયબ થી 1.0 સંપૂર્ણ ઘાટું)

=====

🕒 કેટેગરી ૬: ટ્રાન્ઝિશન, એનિમેશન અને ટ્રાન્સફોર્મ (ANIMATIONS & TRANSFORMS)

વેબપેજ પર હવનચલન, ગતિ અને સ્મૂથ અસરો ઉત્પન્ન કરતી પ્રોપર્ટીઝ.

[A]

- * animation: એનિમેશનની તમામ પ્રોપર્ટી એક જ વારમાં સેટ કરવાનો માસ્ટર શોર્ટહેન્ડ.
- * animation-delay: પેજ લોડ થયા પછી એનિમેશન કેટલી સેકન્ડ પછી શરૂ થશે તે સમય.
- * animation-direction: એનિમેશન સીધું ચાલશે, રિવર્સ ચાલશે કે વારાફરતી ઉલટું થશે.
-> Values: normal | reverse | alternate | alternate-reverse
- * animation-duration: આખું એનિમેશન એકવાર પૂરું થવામાં કેટલો સમય લેશે (seconds/ms).
- * animation-fill-mode: એનિમેશન પૂરું થયા પછી એલિમેન્ટ છેલ્લી પોઝિશન પર રોકાશે કે મૂળ પોઝિશન પર.
-> Values: none | forwards | backwards | both
- * animation-iteration-count: એનિમેશન કેટલી વાર (ચોક્કસ આંકડો કે અનંત) ચાલશે તે સંખ્યા.
-> Values: <number> | infinite
- * animation-name: @keyframes દ્વારા બનાવેલા એનિમેશનનું ચોક્કસ નામ આપવા.
- * animation-play-state: ચાલુ એનિમેશનને પ્રોગ્રામિંગથી પ્લે કે પોઝ (અટકાવવા) કરવા માટે.
-> Values: running | paused
- * animation-timing-function: એનિમેશનની સ્પીડ કઈ ગતિએ વધશે-ઘટશે (એક્સિલરેશન કર્વ).
-> Values: linear | ease | ease-in | ease-out | ease-in-out | cubic-bezier()

[T]

- * transform: એલિમેન્ટને ફેરવવા (rotate), મોટો કરવા (scale) કે ખસેડવા (translate).
-> Values: none | translate() | rotate() | scale() | skew() | matrix()
- * transform-origin: ટ્રાન્સફોર્મ ઇફેક્ટ કયા બિંદુ (ખૂણા કે સેન્ટર) પરથી શરૂ થશે તે પોઇન્ટ.
- * transform-style: 3D લેઆઉટમાં ચાઇલ્ડ એલિમેન્ટ્સ પોતાનું 3D સ્પેસ જાળવશે કે સપાટ થઈ જશે.
-> Values: flat | preserve-3d
- * transition: હોવર ઇફેક્ટ વગેરે વખતે સ્મૂથ બદલાવ લાવવા માટેનો મુખ્ય શોર્ટહેન્ડ.
- * transition-delay: ડિઝાઇન બદલાવાની પ્રક્રિયા કેટલી મોડી શરૂ થશે તે સમય.
- * transition-duration: ડિઝાઇન બદલાવાની સ્મૂથ પ્રક્રિયા કેટલો સમય ચાલશે.
- * transition-property: કઈ પ્રોપર્ટી પર સ્મૂથ ઇફેક્ટ આપવી છે (જેમ કે માત્ર width કે all).
- * transition-timing-function: ટ્રાન્ઝિશન સ્પીડની ગતિનો કર્વ (linear, cubic-bezier વગેરે).

⚙️ કેટેગરી ૭: બિહેવિયર અને કંટ્રોલ પ્રોપર્ટીઝ (BEHAVIOR & CONTROLS)

યુઝરની ઇન્ટરેક્શન અને બ્રાઉઝરની આંતરિક કાર્યપદ્ધતિ કંટ્રોલ કરતી એડવાન્સ પ્રોપર્ટીઝ.

[C]

- * caret-color: ઇનપુટ બોક્સમાં ટાઇપ કરતી વખતે દેખાતી ઊભી લીટી (કર્સર) નો કલર બદલવા.
- * contain: બ્રાઉઝરને જણાવવા માટે કે આ બોક્સ આખા પેજથી આઇસોલેટેડ છે, જેથી પર્ફોર્મન્સ વધે.
- * content-visibility: જે બોક્સ સ્ક્રીનની બહાર છે તેનું રેન્ડરિંગ અટકાવી પેજ બહુ ફાસ્ટ લોડ કરવા.
-> Values: visible | hidden | auto
- * cursor: એલિમેન્ટ પર માઉસ લઈ જતાં માઉસનો પોઇન્ટર એરો કેવો દેખાશે તે.
-> Values: auto | default | pointer | wait | text | move | not-allowed

[I]

- * isolation: એલિમેન્ટ માટે નવો Stacking Context બનાવી z-index ને સુરક્ષિત કરવા.
-> Values: auto | isolate

[P]

- * pointer-events: એલિમેન્ટ પર માઉસની કોઈ ક્લિક કામ કરશે કે તે આરપાર (ક્લિક લોક) થઈ જશે.
-> Values: auto | none

[R]

- * resize: યુઝર માઉસથી ખેંચીને બોક્સ મોટું-નાનું કરી શકશે કે નહીં તે મેનેજ કરવા.
-> Values: none | both | horizontal | vertical

[S]

- * scroll-behavior: લિક પર ક્લિક કરવાથી પેજ ફૂદકો મારશે કે સ્મૂથ સરકીને ત્યાં જશે.
-> Values: auto | smooth
- * scroll-snap-align: સ્ક્રોલ સ્નેપિંગ વખતે બોક્સ સ્ક્રીનના કયા ખૂણે આવીને લોક થશે.
-> Values: none | start | center | end
- * scroll-snap-stop: સ્ક્રોલ કરતી વખતે યુઝર એકસાથે ઘણા બોક્સ ફુદાવી શકશે કે દરેક પર રોકાવું પડશે.
-> Values: normal | always
- * scroll-snap-type: જાવાસ્ક્રિપ્ટ વગર ખોર CSS સ્વાઇડર બનાવવા માટેની મુખ્ય કેપ્ચર પ્રોપર્ટી.

-> Values: none | x mandatory | y mandatory | x proximity

[U]

* user-select: યુઝર વેબસાઇટ પરથી લખાણ માઉસ વડે સિલેક્ટ/કોપી કરી શકશે કે નહીં.

-> Values: auto | text | none | all

[W]

* will-change: બ્રાઉઝરને અગાઉથી જણાવવા માટે કે આ એલિમેન્ટ એનિમેટો થવાનો છે, જેથી બ્રાઉઝર GPU એક્સલરેશન ઓન કરી દે અને એનિમેશન સહેજ પણ અટકે નહીં.

=====

 કેટેગરી ૮: લિસ્ટ અને ટેબલ પ્રોપર્ટીઝ (LISTS & TABLES)

=====

ટેબલ ડેટા અને બુલેટ લિસ્ટની ડિઝાઇન કન્ટ્રોલ કરવા માટેની પ્રોપર્ટીઝ.

[B]

* border-collapse: ટેબલના ખાનાઓની બોર્ડર એકબીજામાં ભળી જશે કે અલગ ડબલ લાઇન રહેશે.

-> Values: collapse | separate

[L]

* list-style: લિસ્ટની તમામ પ્રોપર્ટી સેટ કરવાનો કોમન શોર્ટહેન્ડ.

* list-style-image: બુલેટ પોઇન્ટ્સના ટપકાની જગ્યાએ કસ્ટમ નાનો ફોટો મૂકવા માટે.

* list-style-position: બુલેટ્સ લિસ્ટની અંદર દેખાશે કે બોર્ડરની બહાર.

-> Values: inside | outside

* list-style-type: બુલેટનો પ્રકાર બદલવા (ટપકું, ચોરસ, રોમન નંબર્સ, ગુજરાતી આંકડા વગેરે).

-> Values: disc | circle | square | decimal | georgian | none

=====

આ લિસ્ટમાં વર્તમાન મોડર્ન વેબ સ્પેસિફિકેશન મુજબની તમામ ૧૦૦% પ્રોપર્ટીઝ તેમની સાચી કેટેગરી અને વેલ્યુ સ્ટ્રક્ચર સાથે ગુજરાતીમાં પૂરી થાય છે. આ લિસ્ટની બહાર સીએસએસની કોઈ અલગ પ્રોપર્ટી શક્ય જ નથી.

=====

=====

COMPREHENSIVE CSS MASTER GUIDE (A TO Z: BEGINNER TO PHD LEVEL)

SELECTORS, COMBINATORS, STYLE PROPERTIES & MODERN UTILITY FRAMEWORKS

=====

આ માસ્ટર ગાઈડમાં CSS ના તમામ સિલેક્ટર્સ, કોમ્બિનેટર્સ, દરેક સ્ટાઈલ પ્રોપર્ટીઝ, અને આધુનિક CSS ફ્રેમવર્ક્સની સંપૂર્ણ વિગતો અતિ ઊંડાણપૂર્વક અને સરળ ગુજરાતી ભાષામાં આવરી લેવામાં આવી છે. આ ગાઈડ વાંચ્યા પછી તમારે અન્ય કોઈ સોર્સ જોવાની જરૂર નહિ પડે.

=====

PART 1: CSS SELECTORS & COMBINATORS

=====

સિલેક્ટર્સ અને કોમ્બિનેટર્સનો ઉપયોગ HTML પેજના ચોક્કસ એલિમેન્ટને ટાર્ગેટ કરીને તેને સ્ટાઈલ કરવા માટે થાય છે.

1. BASIC SELECTORS (મૂળભૂત સિલેક્ટર્સ)

* Universal Selector (*)

- શેના માટે છે?: પેજ પર રહેલા દરેક HTML એલિમેન્ટને એકસાથે ટાર્ગેટ કરવા માટે.
- શું કરે છે?: આખા પેજનું ડિફોલ્ટ માર્જિન અને પેડિંગ ઝીરો કરવા અથવા ગ્લોબલ બોક્સ-સાઈઝિંગ સેટ કરવા માટે મોટેભાગે વપરાય છે. (`* { margin: 0; box-sizing: border-box; }`)

* Element/Tag Selector (element)

- શેના માટે છે?: કોઈ ચોક્કસ HTML ટેગના તમામ ઇન્સ્ટન્સને સ્ટાઈલ કરવા.
- શું કરે છે?: પેજના બધા જ `p`, `h1`, અથવા `div` ને એકસરખો લુક આપે છે. (`p { color: red; }`)

* Class Selector (.class)

- શેના માટે છે?: પેજના એક કે એકથી વધુ એલિમેન્ટ્સને કસ્ટમ આઈડેન્ટિટી આપી સ્ટાઈલ કરવા.
- શું કરે છે?: HTML માં `class="my-box"` લખીને CSS માં ડોટ (.) સાથે તેને એક્સેસ કરાય છે. આ રીયુઝેબલ છે. (`.my-box { background: blue; }`)

* ID Selector (#id)

- શેના માટે છે?: પેજ પર રહેલા એક જ યુનિક (અનન્ય) એલિમેન્ટને ટાર્ગેટ કરવા.
- શું કરે છે?: HTML માં `id="main-header"` લખીને CSS માં હેશ (#) થી ટાર્ગેટ કરાય છે. આ આખા પેજમાં માત્ર એક જ હોઈ શકે છે. (`#main-header { font-size: 24px; }`)

2. COMBINATORS (સંયોજક સિલેક્ટર્સ - એલિમેન્ટ્સ વચ્ચેના સંબંધો)

* Descendant Combinator (space)

- શેના માટે છે?: પેરન્ટની અંદર આવેલા તમામ ચાઇલ્ડ કે ગ્રાન્ડ-ચાઇલ્ડ (વંશજો) ને ટાર્ગેટ કરવા.
- શું કરે છે?: `div p` લખવાથી તે ચોક્કસ `div` ની અંદર રહેલા તમામ `p` ટેગ્સને પકડી લેશે, ભલે તે ગમે તેટલા ઊંડા નેસ્ટેડ હોય.

* Child Combinator (>)

- શેના માટે છે?: પેરન્ટના માત્ર ડાયરેક્ટ (તરત જ નીચેના) ચાઇલ્ડને જ ટાર્ગેટ કરવા માટે.
- શું કરે છે?: `div > p` લખવાથી ફક્ત તે `div` ના પોતાના તાત્કાલિક બાળકો (Direct Children) ને જ સ્ટાઈલ લાગશે, તેની અંદર રહેલા બીજા સબ-બોક્સના `p` ને નહિ.

* Adjacent Sibling Combinator (+)

- શેના માટે છે?: કોઈ એલિમેન્ટની બરાબર પછી આવતા પડોશી એલિમેન્ટને પકડવા.
- શું કરે છે?: `h1 + p` લખવાથી જે `p` ટેગ `h1` પૂરો થયા પછી તરત જ તેની બાજુમાં શરૂ થતો હશે, ફક્ત તેને જ ટાર્ગેટ કરશે.

* General Sibling Combinator (~)

- શેના માટે છે?: કોઈ એલિમેન્ટ પછી આવતા તેના તમામ પડોશી ભાઈઓ/બહેનો (Siblings) ને ટાર્ગેટ કરવા.
- શું કરે છે?: `h1 ~ p` લખવાથી `h1` પૂરો થયા પછી ગમે ત્યાં નીચે આવતા તમામ `p` એલિમેન્ટ્સને એકસાથે સિલેક્ટ કરી લેશે.

3. ADVANCED ATTRIBUTE SELECTORS (એટ્રિબ્યુટ સિલેક્ટર્સ)

* [attr] : જે એલિમેન્ટ પાસે તે ચોક્કસ એટ્રિબ્યુટ હાજર હોય. (`[title] { border: 1px solid; }`)

* [attr="value"] : જેનું એટ્રિબ્યુટ અને વેલ્યુ બંને બિલકુલ મેચ થતા હોય. (`input[type="text"]`)

* [attr~="value"] : એટ્રિબ્યુટની વેલ્યુના વિસ્તમાં જો આ શબ્દ સ્પેસ સાથે ક્યાંક આવતો હોય.

* [attr|="value"] : વેલ્યુ બરાબર તે જ હોય અથવા તેની પાછળ હાઇફન (-) જોડાયેલો હોય (દા.ત. lang|="en").

* [attr^="value"] : જે એલિમેન્ટના એટ્રિબ્યુટની વેલ્યુ આપેલા શબ્દથી જ શરૂ (Starts with) થતી હોય. (`a[href^="https"]`)

* [attr\$="value"] : જે એલિમેન્ટના એટ્રિબ્યુટની વેલ્યુ આપેલા શબ્દથી જ પૂરી (Ends with) થતી હોય. (`a[href$=".pdf"]`)

* [attr*="value"] : એટ્રિબ્યુટની અંદર ગમે ત્યાં તે શબ્દનો ટુકડો (Substring) સમાયેલો હોય.

4. PSEUDO-CLASSES (છાન્ન-વર્ગો - એલિમેન્ટની ચોક્કસ સ્ટેટ કે કન્ડિશન)

* User Action સ્ટેટ્સ:

- :hover - જ્યારે યુઝર માઉસ પોઇન્ટર એલિમેન્ટ પર લાવે ત્યારે.
- :active - જ્યારે એલિમેન્ટ પર માઉસ ક્લિક ચાલુ હોય (પ્રેસ કરેલું હોય).
- :focus - જ્યારે ઇનપુટ બોક્સ પર ક્લિક કરીને ટાઇપિંગ મોડ ચાલુ હોય.
- :visited - જે લિંક પર યુઝર ઓવરેડી ક્લિક કરી ચૂક્યો હોય.

* Structural / Child સ્ટેટ્સ:

- :first-child - પેરન્ટનું સૌથી પહેલું બાળક.
- :last-child - પેરન્ટનું સૌથી છેલ્લું બાળક.
- :nth-child(n) - કસ્ટમ નંબર વાળા બાળકને પકડવા (જેમ કે nth-child(2) એટલે બીજું, nth-child(odd) એટલે એકી સંખ્યા).
- :only-child - જો તે પોતાના પેરન્ટનું એકમાત્ર સંતાન હોય.
- :empty - જે એલિમેન્ટની અંદર કોઈ ટેક્સ્ટ કે ચાઇલ્ડ ટેગ બિલકુલ ન હોય (સાવ ખાલી બોક્સ).

* Form / Validation સ્ટેટ્સ:

- :checked - ચેકબોક્સ કે રેડિયો બટન જ્યારે ટીક કરેલું હોય.
- :disabled - જે ફોર્મ એલિમેન્ટ બ્લોક કે બંધ કરેલો હોય.
- :required - જે ઇનપુટ બોક્સ ભરવું કમ્પલસરી હોય.
- :valid / :invalid - ડેટા સાચો કે ખોટો ફોર્મેટમાં ભર્યો છે તેના આધારે.

* Modern Pseudoclasses:

- :not(selector) - આપેલા સિલેક્ટર સિવાયના બધાને ટાર્ગેટ કરવા માટે. (p:not(.special))
- :has(selector) (Relational) - જો કોઈ પેરન્ટની અંદર ચોક્કસ ચાઇલ્ડ રહેલો હોય, તો પેરન્ટને પોતે ટાર્ગેટ કરવા.
- :is(selector, ...) - મલ્ટિપલ સિલેક્ટરનું કમ્બાઇન વિસ્ટ બનાવી કોડ ટૂંકો કરવા.
- :where(selector) - :is ની જેમ જ, પણ આની સ્પેસિફિસિટી (વજન) હંમેશા ઝીરો (0) હોય છે.

5. PSEUDO-ELEMENTS (એલિમેન્ટના ચોક્કસ અદ્રશ્ય ભાગો)

* ::before

- શેના માટે છે?: સિલેક્ટ કરેલા એલિમેન્ટની અંદર, તેના એક્ઝ્યુઅલ કન્ટેન્ટની બરાબર પહેલા કસ્ટમ વસ્તુ ઉમેરવા.
- શું કરે છે?: આમાં `content: ""` લખવું કમ્પલસરી છે. કોઈ આઇકન કે ડિઝાઇન એલિમેન્ટ આપમેળે ઉમેરવા વપરાય છે.

* ::after

- શેના માટે છે?: એલિમેન્ટના કન્ટેન્ટ પૂરા થયા પછી તરત જ પાછળ કંઈક ઉમેરવા.
- શું કરે છે?: ક્લિયરફિક્સ (Clearfix) કરવા અથવા અંડરલાઇન એનિમેશન ઇફેક્ટ્સ બનાવવા માટે આ સુપર પોપ્યુલર છે.

* ::first-letter : લખાણના પેરગ્રાફનો માત્ર સૌથી પહેલો અક્ષર મોટો કરવા (Drop Cap).

* ::first-line : પેરગ્રાફની માત્ર સૌથી પહેલી આડી લાઇનને સ્ટાઇલ કરવા.

* ::selection : યુઝર જ્યારે વેબસાઇટ પર લખાણને માઉસથી સિલેક્ટ (Highlight) કરે, ત્યારે તેનો બેકગ્રાઉન્ડ અને ટેક્સ્ટ કલર બદલવા માટે.

* ::placeholder : ઇનપુટ બોક્સની અંદર દેખાતા હળવા અક્ષરોના પ્લેસહોલ્ડર ટેક્સ્ટનો કલર અને સ્ટાઇલ બદલવા.

PART 2: ALL CSS STYLE PROPERTIES (A TO Z)

અહીં CSS ની દરેક ફંક્શનલ અને એડવાન્સ સ્ટાઇલ પ્રોપર્ટીઝને કેટેગરી વાઇઝ વિગતવાર સમજાવવામાં આવી છે.

1. THE BOX MODEL (બોક્સ મોડેલ પ્રોપર્ટીઝ)

* width / height : એલિમેન્ટની મૂળ પહોળાઈ અને ઊંચાઈ સેટ કરે છે (px, %, rem, em માં).

* min-width / max-width : એલિમેન્ટની લઘુત્તમ અને મહત્તમ પહોળાઈ લિમિટ સેટ કરે છે જેથી રિસ્પોન્સિવ લેઆઉટ તૂટે નહિ.

* min-height / max-height : એલિમેન્ટની ઊંચાઈની લઘુત્તમ અને મહત્તમ બોર્ડર નક્કી કરે છે.

* padding (top, right, bottom, left) : બોક્સની બોર્ડર અને અંદરના લખાણ (Content) વચ્ચેની અંદરની જગ્યા સેટ કરે છે.

* margin (top, right, bottom, left) : બોક્સની બોર્ડરની બહાર, બીજા એલિમેન્ટ્સ વચ્ચેની જગ્યા સેટ કરે છે.

`margin: auto` થી બોક્સ સેન્ટરમાં આવે છે.

* box-sizing

- શેના માટે છે?: બોક્સની સાઇઝની ગણતરી બદલવા.

- શું કરે છે?: `content-box` (ડિફોલ્ટ - પેડિંગ ઉમેરતા બોક્સ મોડેલ થઈ જાય) અને `border-box` (પેડિંગ અને બોર્ડર આપો તો પણ બોક્સની મૂળ સાઇઝ એટલી જ રહે છે, જે ડેવલપર્સની પહેલી પસંદ છે).

2. DISPLAY & LAYOUT PROPERTIES (લેઆઉટ અને ગોઠવણ)

* display

- શું કરે છે?: એલિમેન્ટ પેજ પર કેવી રીતે બિહેવ કરશે તે નક્કી કરે છે.

- વેલ્યુઝ: `block` (આખી લાઇન રોકે), `inline` (જેટલી જગ્યા જોઈએ એટલી જ લે, વિડ્થ સેટ ન થાય), `inline-block` (જગ્યા ઇનલાઇન લે પણ વિડ્થ-પેડિંગ સેટ કરવા દે), `none` (પેજ પરથી ગાયબ કરી દે).
- * position
 - શું કરે છે?: એલિમેન્ટને પેજ પર કઈ જગ્યાએ કંટ્રોલ કરવો તેનો મોડ નક્કી કરે છે.
 - વેલ્યુઝ:
 - * `static` (ડિફોલ્ટ, પેજના ફ્લો મુજબ ચાલે).
 - * `relative` (પોતાની મૂળ જગ્યાના સંદર્ભમાં top/left/bottom/right થી ખસેડી શકાય).
 - * `absolute` (પોતાના નજીકના પોઝિશનવાળા પેરન્ટના સંદર્ભમાં ગમે ત્યાં સેટ થાય, નોર્મલ ફ્લોમાંથી બહાર નીકળી જાય).
 - * `fixed` (બ્રાઉઝરની સ્ક્રીન સાથે ચોટી જાય, પેજ ગમે તેટલું સ્ક્રોલ કરો તો પણ ત્યાં જ રહે).
 - * `sticky` (પહેલા નોર્મલ ચાલે, પણ સ્ક્રોલ કરતા જ સ્ક્રીનની ટોપ પર ચોટી જાય).
- * top / right / bottom / left : પોઝિશન સેટ કર્યા પછી એલિમેન્ટને ચોક્કસ દિશામાં કેટલા પિક્સલ ખસેડવો તે નક્કી કરે છે.
- * z-index : એલિમેન્ટ્સ જ્યારે એકબીજાની ઉપર ઓવરલેપ (ચડી જાય) થાય, ત્યારે કયું બોક્સ ઉપર દેખાશે અને કયું નીચે (Layering) તે નક્કી કરે છે (નંબર જેટલો મોટો, એટલું ઉપર).
- * float / clear : જૂની રીત જે એલિમેન્ટને ડાબે કે જમણે ધકેલે છે અને clear તેનો પ્રવાહ રોકે છે.
- * overflow (overflow-x, overflow-y) : જો બોક્સ નાનું હોય અને કન્ટેન્ટ મોટું થઈને બહાર નીકળી જાય, તો તેને હેન્ડલ કરવા માટે. (`visible`, `hidden` - છુપાવી દે, `scroll` - સ્ક્રોલબાર લાવે, `auto`).
- * clip-path : એલિમેન્ટને કસ્ટમ આકારો (Polygon, Circle, Ellipse) માં કાપવા (Crop) માટે.

3. FLEXBOX LAYOUT (આધુનિક વન-ડાયમેન્શનલ લેઆઉટ)

- * display: flex - જે પેરન્ટ બોક્સમાં આ લખો, તેના બધા જ ડાયરેક્ટ ચાઇલ્ડ આપોઆપ ફ્લેક્સ આઇટમ્સ બની આડી લાઇનમાં આવી જાય છે.
- * flex-direction : અંદરની આઇટમ્સ આડી ગોઠવવી છે કે ઉભી તે સેટ કરે છે (`row`, `column`, `row-reverse`, `column-reverse`).
- * flex-wrap : જો આઇટમ્સ વધી જાય, તો તેને નવી લાઇનમાં મોકલવી (`wrap`) કે એક જ લાઇનમાં સંકોચી દેવી (`nowrap`).
- * justify-content : આડી ધરી (Main Axis) પર આઇટમ્સની સ્પેસિંગ સેટ કરવા. (`flex-start`, `flex-end`, `center`, `space-between` - બંને છેડે ગોઠવે, `space-around`, `space-evenly`).
- * align-items : ઉભી ધરી (Cross Axis) પર આઇટમ્સનું એલાઇનમેન્ટ સેટ કરવા. (`flex-start`, `flex-end`, `center`, `baseline`, `stretch`).
- * align-content : મલ્ટિપલ લાઇન્સ જ્યારે ફ્લેક્સમાં બને, ત્યારે તેમની વચ્ચે ઉભી જગ્યાનું મેનેજમેન્ટ કરવા.
- * flex-grow : ખાલી જગ્યા વધે ત્યારે કઈ આઇટમ કેટલી ઝડપથી મોટી થશે તેનો રેશિયો.
- * flex-shrink : સ્ક્રીન નાની થાય ત્યારે કઈ આઇટમ કેટલી દબાશે તેનો રેશિયો.
- * flex-basis : ફ્લેક્સ આઇટમની શરૂઆતની ડિફોલ્ટ લઘુત્તમ પહોળાઈ નક્કી કરે છે.
- * order : કોઈપણ આઇટમનો ક્રમ HTML કોડ બદલ્યા વગર આગળ-પાછળ કરવા માટે.
- * gap (row-gap, column-gap) : ફ્લેક્સની આઇટમ્સ વચ્ચે સ્પેસિંગ (ગેપ) છોડવા માટેની શોર્ટહેન્ડ પ્રોપર્ટી.

4. GRID LAYOUT (આધુનિક ટુ-ડાયમેન્શનલ લેઆઉટ - રો અને કોલમ)

- * display: grid - બોક્સને ગ્રીડ આર્કિટેક્ચરમાં એક્ટિવેટ કરે છે.
- * grid-template-columns : ઉભી કોલમનું માળખું અને સાઇઝ નક્કી કરવા. (દા.ત. `grid-template-columns: repeat(3, 1fr)` એટલે સરખા ભાગની 3 કોલમ).
- * grid-template-rows : આડી રો (લાઇન્સ) નું માળખું સેટ કરવા.
- * grid-template-areas : પેજનો આખો નકશો (Header, Sidebar, Footer) નામ આપીને વિઝ્યુઅલી સેટ કરવા માટે.
- * grid-column-start / grid-column-end (અથવા grid-column) : કોઈ ચોક્કસ બોક્સ ગ્રીડમાં કઈ લાઇનથી શરૂ થઈને કઈ લાઇન પર પૂરું થશે (Span) તે સેટ કરવા.
- * grid-row-start / grid-row-end (અથવા grid-row) : ઉભી રો ની લાઇન સ્પેસિંગ નક્કી કરવા.
- * justify-items / align-items : ગ્રીડ સેલની અંદર એલિમેન્ટ કઈ જગ્યાએ બેસશે તે સેટ કરવા.
- * place-items : `justify-items` અને `align-items` બંનેને એક જ લાઇનમાં સેન્ટર કરવા માટેની શોર્ટકટ પ્રોપર્ટી. (`place-items: center`;

5. TYPOGRAPHY & TEXT PROPERTIES (અક્ષરો અને લખાણની સ્ટાઇલ)

- * color : લખાણના અક્ષરોનો કલર સેટ કરે છે (Hex, RGB, HSL, Color Name).
- * font-family : લખાણ માટે કયો ફોન્ટ વપરાશે તે નક્કી કરે છે (દા.ત. Arial, Sans-serif).
- * font-size : અક્ષરોનું કદ નક્કી કરે છે (px, rem, em, % માં).
- * font-weight : અક્ષરોની જાડાઈ સેટ કરે છે (`normal`, `bold`, અથવા 100 થી 900 સુધીના નંબર્સ).
- * font-style : અક્ષરોને ત્રાંસા (Italic) કે નોર્મલ કરવા.
- * font-variant : અક્ષરોને Small-Caps માં કન્વર્ટ કરવા.
- * line-height : બે આડી લીટીઓ વચ્ચે કેટલી ઉભી જગ્યા (Spacing) રાખવી તે નક્કી કરે છે જેથી લખાણ વાંચવામાં

સરળ રહે.

- * text-align : લખાણને ડાબે (`left`), જમણે (`right`), વચ્ચે (`center`), અથવા આખી લાઇન સરખી કરવા (`justify`) સેટ કરે છે.
- * text-decoration : લખાણની નીચે લાઇન લાવવા (`underline`), ઉપર લાઇન લાવવા (`overline`), લખાણ છેકી નાખવા (`line-through`), અથવા લાઇન હટાવવા (`none` - જે લિંકની અંડરલાઇન હટાવવા વપરાય છે).
- * text-transform : અક્ષરોને કેપિટલ કરવા (`uppercase`), સ્મોલ કરવા (`lowercase`), અથવા દરેક શબ્દનો પહેલો અક્ષર કેપિટલ કરવા (`capitalize`).
- * text-shadow : લખાણના અક્ષરોની પાછળ કસ્ટમ પડછાયો (Shadow) આપવા માટે.
- * letter-spacing : એક-એક અક્ષર (Characters) વચ્ચેની આડી જગ્યા વધારવા-ઘટાડવા.
- * word-spacing : બે આખા શબ્દો (Words) વચ્ચેની સ્પેસ મેનેજ કરવા.
- * white-space : લખાણનો ફ્લો નક્કી કરે છે (`nowrap` કરવાથી લખાણ ક્યારેય નવી લાઇનમાં જતું નથી, સિંગલ લાઇનમાં જ લંબાય છે).
- * text-overflow : જો લખાણ બોક્સની બહાર નીકળી જાય, તો છેલ્લે ત્રણ ટપકાં લાવવા માટે `ellipsis` વેલ્યુ વપરાય છે (white-space: nowrap અને overflow: hidden સાથે).
- * word-break : બહુ મોટો શબ્દ બોક્સ તોડીને બહાર જતો હોય તો તેને વચ્ચેથી તોડીને નવી લાઇનમાં લાવવા (`break-all`).
- * vertical-align : ઇનલાઇન એલિમેન્ટ્સ અથવા ટેબલ સેલની અંદર લખાણને ઉપર, વચ્ચે કે નીચે ઓર્ગેનાઇઝ કરવા.

6. BACKGROUND PROPERTIES (બેકગ્રાઉન્ડ પ્રોપર્ટીઝ)

- * background-color : બોક્સની પાછળનો બેકગ્રાઉન્ડ કલર સેટ કરે છે.
- * background-image : બેકગ્રાઉન્ડમાં ફોટો સેટ કરવા માટે (`url('image.jpg')`) અથવા કલર ગ્રેડિયન્ટ્સ આપવા માટે (`linear-gradient`, `radial-gradient`).
- * background-repeat : બેકગ્રાઉન્ડનો ફોટો જો નાનો હોય તો તેને વારંવાર રીપીટ થતો રોકવા માટે (`no-repeat`, `repeat-x`, `repeat-y`).
- * background-size : ફોટો બેકગ્રાઉન્ડમાં કેવી રીતે ફિટ થશે તે નક્કી કરે છે (`cover` - આખું બોક્સ ભરી દે, `contain` - ફોટો કપાશે નહિ પણ આખો દેખાશે, અથવા ચોક્કસ px / %).
- * background-position : બેકગ્રાઉન્ડનો ફોટો કયા ખૂણેથી દેખાડવો તે સેટ કરે છે (`center`, `top left`, `bottom right`).
- * background-attachment : પેજ સ્ક્રોલ કરતી વખતે બેકગ્રાઉન્ડ ફોટો ત્યાં જ જકડાઈ રહેશે કે પેજ સાથે ઉપર જશે તે નક્કી કરવા (`scroll`, `fixed` - જે પેરાલેક્સ ઇફેક્ટ બનાવે છે).
- * background-clip : બેકગ્રાઉન્ડ કલર બોર્ડર સુધી જશે, પેડિંગ સુધી જશે કે અક્ષરોની અંદર ભરાશે (`text`) તે નક્કી કરવા.
- * background (Shorthand) : આ બધી જ પ્રોપર્ટીઝને એક જ લાઇનમાં લખવા માટેની શોર્ટકટ પ્રોપર્ટી.

7. BORDERS & SHADOWS (બોર્ડર અને શેડો પ્રોપર્ટીઝ)

- * border-width : બોર્ડરની જાડાઈ સેટ કરે છે.
- * border-style : બોર્ડરની ડિઝાઇન કેવી હશે તે નક્કી કરે છે (`none`, `solid` - સળંગ લાઇન, `dashed` - તૂટક લાઇન, `dotted` - ટપકાં વાળી લાઇન, `double`).
- * border-color : બોર્ડરનો કલર સેટ કરે છે.
- * border (Shorthand) : એક જ લાઇનમાં ત્રણેય સેટ કરવા. (`border: 2px solid red;`)
- * border-radius : બોક્સના તીક્ષ્ણ ખૂણાઓને ગોળાકાર (Curve) કરવા માટે. `border-radius: 50%` કરવાથી ચોરસ બોક્સ સંપૂર્ણ ગોળ સર્કલ બની જાય છે.
- * box-shadow
 - શેના માટે છે?: બોક્સની પાછળ કે અંદર સુંદર પડછાયો (3D ડેપ્થ લુક) આપવા.
 - શું કરે છે?: આમાં 5 પેરામીટર હોય છે: `horizontal-offset vertical-offset blur-radius spread-radius color`. જો શરૂઆતમાં `inset` લખો તો પડછાયો બોક્સની અંદરની તરફ પડે છે.

8. TRANSITIONS & ANIMATIONS (એનિમેશન અને સ્મૂથ ઇફેક્ટ્સ)

- * transition-property : કઈ પ્રોપર્ટી બદલાય ત્યારે એનિમેશન થવું જોઈએ તે નક્કી કરે છે (દા.ત. `all`, `background-color`, `width`).
- * transition-duration : એનિમેશન પૂરું થવામાં કેટલો સમય (સેકન્ડ/મિલીસેકન્ડ) લાગશે તે (દા.ત. `0.3s`, `500ms`).
- * transition-timing-function : એનિમેશનની સ્પીડનો ગ્રાફ નક્કી કરે છે (`ease`, `linear` - એકસરખી સ્પીડ, `ease-in` - શરૂઆતમાં ધીમું, `ease-out` - અંતમાં ધીમું, `cubic-bezier`).
- * transition-delay : માઉસ ઓવર કર્યા પછી એનિમેશન કેટલો ટાઇમ વેઇટ કરીને શરૂ થશે તે સમય.
- * transition (Shorthand) : આ ચારેય ટાઇમિંગ પ્રોપર્ટીઝને સિંગલ લાઇનમાં લખવાનો શોર્ટકટ. (`transition: all 0.3s ease-in-out;`)
- * @keyframes
 - શેના માટે છે?: જટિલ અને સ્ટેપ-બાય-સ્ટેપ ચાલતા કસ્ટમ એનિમેશનની આખી વાર્તા (Timeline) લખવા માટે.
 - શું કરે છે?: આની અંદર `from` અને `to` અથવા `0%`, `50%`, `100%` ના સ્ટેજ બનાવીને અલગ-અલગ

સીએસએસ પ્રોપર્ટી ચેન્જ કરાય છે.

- * `animation-name` : `@keyframes` માં જે એનિમેશનનું નામ રાખ્યું હોય તેને એલિમેન્ટ સાથે લિંક કરવા.
- * `animation-duration` : આખું કીફ્રેમ એનિમેશન એક રાઉન્ડ પૂરું કરવા કેટલી સમય લેશે તે.
- * `animation-iteration-count` : એનિમેશન કેટલી વાર ફરશે તે સંખ્યા (દા.ત. `3`, `5`, અથવા આખો દિવસ ચાલુ રાખવા માટે `infinite`).
- * `animation-direction` : એનિમેશન સીધું ચાલશે કે રિવર્સ દિશામાં ફરે તે નક્કી કરવા (`normal`, `reverse`, `alternate`).
- * `animation-fill-mode` : એનિમેશન પૂરું થઈ ગયા પછી એલિમેન્ટ કઈ સ્થિતિમાં અટકી જશે તે નક્કી કરવા (`forwards` કરવાથી એનિમેશન છેલ્લી ફ્રેમ પર જ અટકી જાય છે, જૂની સ્થિતિમાં પાછું નથી જતું).

9. TRANSFORMS (એલિમેન્ટને ફેરવવા કે મોટા-નાના કરવા)

- * `transform` : આ પ્રોપર્ટી એલિમેન્ટના ભૌતિક આકાર કે પોઝિશનને ગ્રાફિક્સ લેવલ પર ચેન્જ કરે છે.
- * વેલ્યુઝ:
 - `translate(x, y)` : એલિમેન્ટને તેની જગ્યા પરથી ડાબે-જમણે કે ઉપર-નીચી ખસેડવા (નોર્મલ લેઆઉટને નુકસાન કર્યા વગર). `translate(-50%, -50%)` સેન્ટરિંગ માટે ફેમસ છે.
 - `scale(x, y)` : એલિમેન્ટને મોટો કે નાનો (Zoom In / Zoom Out) કરવા માટે. `scale(1.2)` એટલે ૨૦% મોટો થશે.
 - `rotate(angle)` : એલિમેન્ટને ઘડિયાળના કાંટાની દિશામાં ગોળ ફેરવવા માટે (દા.ત. `rotate(45deg)`).
 - `skew(x-angle, y-angle)` : એલિમેન્ટના ખૂણા ખેંચીને તેને ત્રાંસો કે લંબચોરસ સમાંતર કરવા માટે.
- * `transform-origin` : ટ્રાન્સફોર્મ (જેમ કે રોટેશન) એલિમેન્ટના કયા બિંદુથી (સેન્ટરથી, ટોપ-લેફ્ટથી કે બોટમથી) ફરશે તે સેન્ટર પોઇન્ટ નક્કી કરવા.

10. TABLES, FILTERS & BLEND MODES (ટેબલ અને ફોટો ઇફેક્ટ્સ)

- * `border-collapse` : ટેબલની અંદર ડબલ બોર્ડર દેખાતી બંધ કરીને સિંગલ ક્લીન લાઇન કરવા માટે આ પ્રોપર્ટીને `collapse` સેટ કરાય છે.
- * `border-spacing` : જો બોર્ડર કોલેપ્સ ન હોય, તો સેલ વચ્ચેની જગ્યા નક્કી કરવા.
- * `table-layout` : ટેબલની પહોળાઈ અંદરના લખાણ પ્રમાણે ઓટોમેટિક વધશે (`auto`) કે પહેલેથી ફિક્સ રાખવી (`fixed`) તે મોડ.
- * `filter` : ફોટોશોપ જેવી ઇફેક્ટ્સ બ્રાઉઝરમાં લાવવા માટે.
 - વેલ્યુઝ: `blur(px)` (ફોટો ઝાંખો કરવા), `grayscale(%)` (ફોટો બ્લેક એન્ડ વ્હાઇટ કરવા), `brightness(%)`, `contrast(%)`, `drop-shadow()`, `opacity()`.
- * `backdrop-filter`
 - શેના માટે છે?: આધુનિક ગ્લાસ જેવો લુક (Glassmorphism) બનાવવા માટે.
 - શું કરે છે?: આ પ્રોપર્ટી એલિમેન્ટની પાછળ બેકગ્રાઉન્ડમાં રહેલી વસ્તુને બ્લર કે ક્લર ફિલ્ટર કરે છે. (`backdrop-filter: blur(10px);`)
- * `mix-blend-mode` : એલિમેન્ટનો ક્લર તેની પાછળ રહેલા પેરન્ટ એલિમેન્ટના ક્લર કે ફોટા સાથે કેવી રીતે બ્લેન્ડ (મિક્સ) થશે તે નક્કી કરવા (`multiply`, `screen`, `overlay`).

11. MODERN & PHD LEVEL PROPERTIES (અત્યંત એડવાન્સ સીએસએસ પ્રોપર્ટીઝ)

- * `pointer-events` : જો આને `none` સેટ કરો, તો એલિમેન્ટ પર માઉસ ક્લિક કે હોવર કોઈ જ અસર નહિ કરે. તે સાવ પારદર્શક બની જશે અને ક્લિક તેની પાછળના એલિમેન્ટ પર જતી રહેશે.
- * `user-select` : યુઝર વેબસાઇટ પરથી લખાણ કોપી કે માઉસથી સિલેક્ટ કરી શકશે કે નહિ તે લોક કરવા માટે (`none` કરવાથી લખાણ સિલેક્ટ જ નહિ થાય).
- * `scroll-behavior` : આખા પેજનું સ્ક્રોલિંગ આંચકા વગર એકદમ સ્મૂથ કરવા માટે આને `smooth` સેટ કરાય છે. (`html { scroll-behavior: smooth; }`)
- * `object-fit` : `img` અથવા `video` ટેગનો ફોટો તેના કન્ટેનર બોક્સમાં કેવી રીતે ફિટ થશે (ખેંચાઈ જશે કે પ્રોપોર્શનમાં રહેશે) તે મેનેજ કરવા. (`cover`, `contain`, `fill`).
- * `object-position` : ઓબ્જેક્ટ ફીટ સેટ કર્યા પછી ફોટાનો કયો મુખ્ય ભાગ કેન્દ્રમાં રાખવો તે.
- * `aspect-ratio` : બોક્સની પહોળાઈ અને ઊંચાઈનો રેશિયો ફિક્સ કરવા (દા.ત. `aspect-ratio: 16 / 9` એટલે વિડિયો પ્લેયર જેવો આકાર ઓટોમેટિક જળવાઈ રહેશે, ઊંચાઈ જાતે ગણાશે).
- * `will-change` : બ્રાઉઝરના ગ્રાફિક્સ કાર્ડ (GPU) ને પહેલેથી એડવાન્સમાં જાણ કરવા માટે કે આ એલિમેન્ટ પર એનિમેશન આવવાનું છે, જેથી બ્રાઉઝર પર્ફોર્મન્સ લેગ ન થાય.
- * `accent-color` : ચેકબોક્સ, રેડિયો બટન, અને પ્રોગ્રેસ બારના ડિફોલ્ટ ક્લરને બદલીને પોતાની બ્રાન્ડનો ક્લર સેટ કરવા માટે.

PART 3: MODERN UTILITY CSS FRAMEWORKS

આજના મોડર્ન વેબ ડેવલપમેન્ટમાં શૂન્યથી CSS લખવાના બદલે પ્રી-બિલ્ટ ફ્રેમવર્ક્સ વપરાય છે.

1. BOOTSTRAP 5 vs TAILWIND CSS (મુખ્ય તફાવત અને ફિલોસોફી)

- * Direct Comparison:

BOOTSTRAP 5 (Component-Driven)	TAILWIND CSS (Utility-First)
આમાં પહેલેથી બનેલા રેડીમેડ ડિઝાઇન	આમાં કોઈ કમ્પોનન્ટ રેડીમેડ નથી હોતા.
વિજેટ્સ (જેમ કે .btn, .card, .navbar) મળે છે. કોડિંગ સ્પીડ વધે છે.	આમાં સિંગલ-ટાસ્ક યુટિલિટી ક્લાસીસ (જેમ કે bg-blue-500, p-4, text-center) મળે છે.
પણ બધી જ વેબસાઇટ્સ ઢેખાવમાં એક્સરખી લાગે છે (UI માળખું ફિક્સ છે).	મળે છે, જેનાથી તમે ધ્યોર કસ્ટમ ડિઝાઇન શૂન્યથી પણ એકદમ યુનિક બનાવી શકો છો.

2. TAILWIND CSS ENGINE & ARCHITECTURE (ટેલવિન્ડની આંતરિક સિસ્ટમ)

- * Just-In-Time (JIT) Engine:

- શેના માટે છે?: પ્રોડક્શન ફાઇલની સાઇઝ નાની કરવા માટે.
- શું કરે છે?: ટેલવિન્ડ તમારા આખા HTML કોડને સ્કેન કરે છે. તમે જે ક્વાસ HTML માં લખ્યો હશે, ફક્ત અને ફક્ત તે જ ક્વાસનો CSS કોડ ફાઇનલ બિલ્ડ ફાઇલમાં ઉમેરાશે. આના કારણે હજારો ક્વાસ હોવા છતાં ફાઇનલ CSS ફાઇલ માત્ર થોડાક જ KB ની બને છે.

* Screen Breakpoints (રિસ્પોન્સિવ ડિઝાઇન):

- ટેલવિઝનમાં મોબાઇલ-ફર્સ્ટ ડિઝાઇન હોય છે. ક્લાસની આગળ પ્રીફિક્સ લગાવાય છે:
 - * `sm:` (Small - ટેબ્લેટ >= 640px)
 - * `md:` (Medium - લેપટોપ >= 768px)
 - * `lg:` (Large - ડેસ્કટોપ >= 1024px)
 - * `xl:` (Extra Large >= 1280px)
- `el.t. `class="w-full md:w-1/2"` એટલે મોબાઇલમાં ફૂલ વિડ્થ અને લેપટોપ સ્ક્રીન આવતા જ અડધી પહોળાઈ થઈ જશે.

- * States & Modifiers:

- ચોક્કસ કન્ડિશન પર ક્લાસ એક્ટિવેટ કરવા માટે આગળ મોડિફાયર લાગે છે:
 - * ``hover:`` (માઉસ ઓવર પર - ``hover:bg-red-500``)
 - * ``focus:`` (ઇનપુટ ફિલ્ડ પર - ``focus:border-black``)
 - * ``dark:`` (ડાર્ક મોડ ચાલુ હોય ત્યારે - ``dark:bg-slate-900``)

3. HOW TO CREATE CUSTOM PROPERTY & CLASSES IN TAILWIND

* Inline Arbitrary Values ([...]):

- જો ટેલેવિઝનના ડિફોલ્ટ સેટિંગ્સ સિવાયની કોઈ ચોક્કસ કસ્ટમ વેલ્યુ સિંગલ ટાઇમ વાપરવી હોય, તો સીએસએસ ફાઇલમાં ગયા વગર ડાયરેક્ટ HTML માં જ સ્ક્રેર બ્રેકેટ `[...]` ની અંદર વેલ્યુ લખી શકાય છે.
- દા.ત. `w-[425px]` (ચોક્કસ 425 પિક્સલ પહોળાઈ), `bg-[#fff573]` (કસ્ટમ હેક્સ કલર કોડ), `text-[19px]`.

* Theme Customization (@theme Directive - Modern Tailwind v4):

- જો તમારી બ્રાન્ડના કસ્ટમર કલર્સ, સ્પેસિંગ કે ફોન્ટ સાઇઝને આપ્યા પ્રોજેક્ટમાં ગ્લોબલ કીવર્ડ તરીકે સેવ કરવા હોય, તો મુખ્ય CSS ફાઇલમાં '@theme' બ્લોકની અંદર વેરીએબલ ડિક્લેર કરાય છે.

- ```
- সেটআপ উদাহরণ:
```css
@theme {
  --color-brand-primary: #8b5cf6;
  --font-display: 'Oswald', sans-serif;
  --breakpoint-xxl: 1600px;
}
```

- એકવાર આ સેવ થઈ ગયા પછી, તમે આખા પ્રોજેક્ટના HTML માં ડાયરેક્ટ `bg-brand-primary` અથવા `font-display` ક્લાસ વાપરી શકો છો, જે ટેલવિઝનના પોતાના ઓરિજિનલ ક્લાસની જેમ જ બિહેવ કરશે.

* Custom Utility Classes & @apply Directive:

- શેના માટે છે?: જ્યારે HTML ની અંદર બહુ બધા ટેલેવિન્ડ ક્લાસીસ ભેગા થઈ જાય અને કોડ ખૂબ ગંદો દેખાવા લાગે (Class Soup), ત્યારે તેને સાફ કરવા માટે.
- શું કરે છે?: આ ડિરેક્ટિવની મદદથી તમે તમારો પોતાનો નવો કસ્ટમ સીએસએસ ક્લાસ બનાવી શકો છો અને તેની અંદર ટેલેવિન્ડના જ રેડીમેડ ક્લાસીસને ઇન્જેક્ટ કરી શકો છો.

- ```
- ସେଟଅପ ଓଢାଢରଣା:
```css
.btn-custom {
  @apply bg-blue-600 text-white font-bold py-2 px-4 rounded-lg shadow-md hover:bg-blue-700
  transition duration-300;
}
```

- હવે તમારે HTML માં આ લાંબી લાઇન લખવાના બદલે ફક્ત `class="btn-custom"` લખવાનું રહેશે. કોડ એકદમ ક્લીન અને કસ્ટમ ક્લાસ મોડલ જેવો વ્યવસ્થિત બની જશે.

=====

CSS COMPLETE MEGA ENCYCLOPEDIA (A-Z)

સંપૂર્ણ CSS માસ્ટર ગાઇડ: દરેક પ્રોપર્ટી, વેલ્યુ અને સિલેક્ટર

૧. તફાવત (Difference): Selectors vs Attributes vs Properties

વેબ ડેવલપમેન્ટમાં HTML અને CSS વચ્ચે કામ કરતી વખતે આ ત્રણ શબ્દો સૌથી મહત્વના છે:

- **Selector (સિલેક્ટર):**
 - તે નક્કી કરે છે કે કયા HTML એલિમેન્ટ પર સ્ટાઇલ લગાવવી છે.
 - ઉદાહરણ: ``p``, ``.my-class``, ``#my-id``. આ બ્રાઉઝરને કહે છે: "આ એલિમેન્ટને શોધો."
- **Attribute (એટ્રિબ્યુટ):**
 - આ HTML નો ભાગ છે. તે HTML ટેગની અંદર વધારાની માહિતી આપે છે.
 - ઉદાહરણ: `<input type="text" class="input-box">` માં ``type`` અને ``class`` એ HTML એટ્રિબ્યુટ્સ છે.
- **Property (પ્રોપર્ટી):**
 - આ CSS નો ભાગ છે. તે એલિમેન્ટનો કયો દેખાવ (સ્ટાઇલ) બદલવો છે તે નક્કી કરે છે.
 - ઉદાહરણ: ``color``, ``background-color``, ``font-size``.
 - દરેક પ્રોપર્ટીની સાથે તેની ચોક્કસ Value (વેલ્યુ) હોય છે, જેમ કે ``color: red;`` (પ્રોપર્ટી: કલર, વેલ્યુ: રેડ).

૨. તમામ CSS Selectors અને Combinators (દરેક પ્રકાર)

બ્રાઉઝરમાં એલિમેન્ટ્સને ટાર્ગેટ કરવા માટેના તમામ રસ્તાઓ:

૧. **Universal Selector (*)**: આખા પેજના બધા જ એલિમેન્ટ્સને સિલેક્ટ કરે છે.
 - સિન્ટેક્સ: ``* { margin: 0; }``
૨. **Element Selector**: ટેગના નામથી સિલેક્ટ કરે છે.
 - સિન્ટેક્સ: ``p { color: gray; }``
૩. **Class Selector (.)**: ચોક્કસ ક્લાસ વાળા એલિમેન્ટ્સ માટે.
 - સિન્ટેક્સ: ``.btn { padding: 10px; }``
૪. **ID Selector (#)**: યુનિક આઈડી વાળા સિંગલ એલિમેન્ટ માટે.
 - સિન્ટેક્સ: ``#header { background: black; }``
૫. **Attribute Selector ([attr])**: HTML એટ્રિબ્યુટના આધારે સિલેક્ટ કરે છે.
 - ``[type="text"]`` -> ફક્ત ટેક્સ્ટ ઇનપુટ માટે.
 - ``[href^="https"]`` -> જેની લિંક https થી શરૂ થતી હોય.
 - ``[src$=".png"]`` -> જે ફોટો .png ફોર્મેટમાં હોય.
૬. **Combinators (જોડાણ સિલેક્ટર્સ)**:
 - **Descendant Selector (space)**: ``div p`` -> div ની અંદર ગમે ત્યાં આવેલા બધા જ p ટેગ.
 - **Child Selector (>)**: ``div > p`` -> div ની અંદર ડાયરેક્ટ આવેલા જ p ટેગ (પૌત્ર નહિ, માત્ર પુત્ર).
 - **Adjacent Sibling (+)**: ``h1 + p`` -> h1 ટેગ પૂરો થયા પછી તરત જ આવતો પહેલો p ટેગ.
 - **General Sibling (~)**: ``h1 ~ p`` -> h1 ટેગ પછી આવતા તેના સ્તરના બધા જ p ટેગ.
૭. **Pseudo-classes (સ્થિતિ દર્શાવતા સિલેક્ટર્સ)**:
 - ``:hover`` -> માઉસ ઉપર જાય ત્યારે.
 - ``:focus`` -> ઇનપુટ બોક્સ પર ક્લિક થાય ત્યારે.
 - ``:nth-child(n)`` -> લિસ્ટમાં ચોક્કસ ક્રમનું બોક્સ સિલેક્ટ કરવા (દા.ત. ``:nth-child(2)`` એટલે બીજું બોક્સ).
 - ``:not(.active)`` -> જે બોક્સમાં .active ક્લાસ ન હોય તેના માટે.
૮. **Pseudo-elements (એલિમેન્ટના ચોક્કસ ભાગ માટે)**:
 - ``::before`` -> એલિમેન્ટની અંદર કન્ટેન્ટ શરૂ થાય તે પહેલા કંઈક ઉમેરવા.
 - ``::after`` -> એલિમેન્ટ પૂરું થાય તે પછી કન્ટેન્ટ ઉમેરવા.
 - ``::placeholder`` -> ઇનપુટ બોક્સના ઝાંખા લખાણની સ્ટાઇલ બદલવા.

૩. CSS PROPERTIES ENCYCLOPEDIA (A-Z વિગતવાર સમજૂતી અને વેલ્યુઝ)

દરેક પ્રોપર્ટી, તેની બધી સંભવિત વેલ્યુઝ અને ડિટેઇલ ગુજરાતી વર્ણન:

--- A ---

- **align-content**: ફ્લેક્સ કે ગ્રીડ લેઆઉટમાં મલ્ટીપલ લાઇનો વચ્ચેની જગ્યા સેટ કરે છે.
 - વેલ્યુઝ: `stretch` (આપોઆપ ખેંચાવું), `center` (વચ્ચે લાવવું), `flex-start`, `flex-end`, `space-between`, `space-around`, `space-evenly`.

around.

- align-items: ફ્લેક્સ કે ગ્રીડ કન્ટેનરની અંદર સિંગલ લાઈનમાં રહેલી આઈટમ્સને ઊભી (vertically) ગોઠવે છે.
 - વેલ્યુઝ: stretch, center, flex-start, flex-end, baseline.
- align-self: કોઈ એક ચોક્કસ ફ્લેક્સ આઈટમનો ઊભો ગોઠવવાનો નિયમ બદલવા માટે.
 - વેલ્યુઝ: auto, stretch, center, flex-start, flex-end.
- animation: એનિમેશન સેટ કરવાનો શોર્ટહેન્ડ નિયમ.
 - વેલ્યુઝ: name | duration | timing-function | delay | iteration-count | direction | fill-mode.
- animation-delay: એનિમેશન પેજ લોડ થયા પછી કેટલી સેકન્ડ મોડું શરૂ થશે તે નક્કી કરે છે.
 - વેલ્યુઝ: 0s, 2s, 500ms.
- animation-direction: એનિમેશન સીધું ચાલશે કે ઊંધું.
 - વેલ્યુઝ: normal, reverse, alternate (એકવાર સીધું, એકવાર ઊંધું), alternate-reverse.
- animation-duration: એનિમેશનને એક ચક્ર (cycle) પૂરું કરવામાં કેટલો સમય લાગશે.
 - વેલ્યુઝ: 1s, 3s, 0.5s.
- animation-fill-mode: એનિમેશન પૂરું થયા પછી એલિમેન્ટ કેવો દેખાશે.
 - વેલ્યુઝ: none, forwards (છેલ્લી સ્થિતિ પર લોક થઈ જશે), backwards, both.
- animation-iteration-count: એનિમેશન કેટલી વાર ફરશે.
 - વેલ્યુઝ: 1, 5, infinite (અનંત સમય સુધી ફર્યા રાખશે).
- animation-name: `@keyframes` માં બનાવેલા એનિમેશનનું નામ.
- animation-play-state: એનિમેશન ચાલુ રાખવું કે અટકાવવું.
 - વેલ્યુઝ: running (ચાલુ), paused (અટકાવેલું - માઉસ હોવર પર કામ આવે).
- animation-timing-function: એનિમેશનની ઝડપની ગતિ કેવી રહેશે.
 - વેલ્યુઝ: ease, linear (એકસરખી ઝડપ), ease-in, ease-out, ease-in-out, cubic-bezier(x1, y1, x2, y2).
- aspect-ratio: એલિમેન્ટની પહોળાઈ અને લંબાઈનો રેશિયો ફિક્સ રાખે છે (રિસ્પોન્સિવ બોક્સ માટે).
 - વેલ્યુઝ: auto, 1/1 (ચોરસ), 16/9 (વીડિયો સાઈઝ), 4/3.

--- B ---

- backdrop-filter: એલિમેન્ટની પાછળ રહેલા બેકગ્રાઉન્ડ કન્ટેન્ટ પર ફિલ્ટર લગાવે છે (ગ્લાસમોર્ફીઝમ કાય લુક માટે).
 - વેલ્યુઝ: none, blur(10px), grayscale(50%), contrast(200%).
- background: બેકગ્રાઉન્ડનો શોર્ટહેન્ડ નિયમ.
- background-attachment: પેજ સ્ક્રોલ કરતી વખતે પાછળનો ફોટો પણ સ્ક્રોલ થશે કે એક જગ્યાએ ફિક્સ રહેશે.
 - વેલ્યુઝ: scroll, fixed (ફોટો ત્યાં જ ચોટી રહેશે, ઉપરનું કન્ટેન્ટ સ્ક્રોલ થશે), local.
- background-blend-mode: બેકગ્રાઉન્ડ ક્લર અને પાછળના ફોટાને મિક્સ કરવા માટે.
 - વેલ્યુઝ: normal, multiply, screen, overlay, darken, lighten.
- background-clip: બેકગ્રાઉન્ડ ક્યાં સુધી ફેલાશે તે નક્કી કરે છે.
 - વેલ્યુઝ: border-box, padding-box, content-box, text (અક્ષરોની અંદર ક્લર કે ગ્રેડિયન્ટ પૂરવા માટે - background-clip: text).
- background-color: પાછળનો ક્લર સેટ કરવા.
 - વેલ્યુઝ: red, #ff0000, rgb(255,0,0), rgba(255,0,0,0.5) (પારદર્શક ક્લર), transparent.
- background-image: બેકગ્રાઉન્ડમાં ફોટો કે ગ્રેડિયન્ટ મુકવા.
 - વેલ્યુઝ: url('img.jpg'), linear-gradient(red, yellow), radial-gradient(circle, red, blue), conic-gradient(red, green, blue).
- background-origin: પાછળનો ફોટો ક્યાંથી શરૂ થશે.
 - વેલ્યુઝ: padding-box, border-box, content-box.
- background-position: ફોટાને બેકગ્રાઉન્ડમાં કઈ જગ્યાએ ગોઠવવો.
 - વેલ્યુઝ: center, top, bottom, left, right, 50% 50%, 10px 20px.
- background-repeat: બેકગ્રાઉન્ડ ફોટો નાનો હોય તો રીપીટ થવા દેવો કે નહિ.
 - વેલ્યુઝ: repeat, repeat-x, repeat-y, no-repeat (ફોટો એક જ વાર દેખાશે).
- background-size: ફોટાનું કદ સેટ કરવું.
 - વેલ્યુઝ: auto, 100px 100px, cover (આખા બોક્સને ક્લર કરી લેશે ખેંચાયા વગર), contain (આખો ફોટો બોક્સની અંદર દેખાશે).
- border: બોર્ડરનો શોર્ટહેન્ડ નિયમ (દા.ત. `1px solid red`).
- border-radius: બોક્સના ખૂણા ગોળ કરવા માટે.
 - વેલ્યુઝ: 0, 5px, 50% (બોક્સને એકદમ ગોળ કરવા માટે).
- box-shadow: બોક્સની પાછળ કે અંદર પડછાયો (Shadow) આપવા માટે.
 - વેલ્યુઝ: none, 10px 10px 5px gray (horizontal-offset vertical-offset blur-radius color), inset (અંદરની તરફ પડછાયો).
- box-sizing: બોક્સની પહોળાઈ અને લંબાઈ ગણવાની પદ્ધતિ બદલે છે.
 - વેલ્યુઝ: content-box (જૂની રીત), border-box (આધુનિક રીત - પેડિંગ અને બોર્ડર આપવાથી બોક્સ પોતાની સાઈઝ બહાર નથી વધારતું, લેઆઉટ તૂટતો નથી).

--- C ---

- **caret-color:** ઇનપુટ બોક્સમાં લખાણ લખતી વખતે જે કર્સર (લાઇન) ઝબકે છે તેનો કલર બદલે છે.
 - વેલ્યુઝ: auto, red, #00ff00.
- **clear:** float થયેલા એલિમેન્ટ્સની બાજુમાંથી કન્ટેન્ટને નીચે ધકેલવા માટે.
 - વેલ્યુઝ: none, left, right, both.
- **clip-path:** એલિમેન્ટને કાપીને મનપસંદ આકાર (Shape) આપવા માટે.
 - વેલ્યુઝ: none, circle(50%), ellipse(), polygon(50% 0%, 0% 100%, 100% 100%) (ત્રિકોણ બનાવવા માટે).
- **color:** અક્ષરોનો (Text) કલર બદલવા માટે.
- **columns:** લખાણને ન્યૂઝપેપરની જેમ ઊભી કોલમમાં વહેંચવા.
 - વેલ્યુઝ: auto, 2, 3, 200px.
- **contain / container-property:** મોડર્ન કન્ટેનર ક્વેરીઝ સેટ કરવા માટે.

--- D ---

- **display:** એલિમેન્ટ બ્રાઉઝરમાં કેવી રીતે બિહેવ કરશે તે નક્કી કરતી સૌથી મુખ્ય પ્રોપર્ટી.
 - વેલ્યુઝ:
 - * **block:** નવી લાઇનથી શરૂ થશે અને આખી પહોળાઈ રોકશે (દા.ત. div, p).
 - * **inline:** નવી લાઇનથી શરૂ નહિ થાય, જેટલી જગ્યા લખાણની હશે એટલી જ રોકશે, આમાં width/height કામ નહિ કરે (દા.ત. span, a).
 - * **inline-block:** લાઇનમાં જ રહેશે પણ વિડ્થ અને હાઇટ સેટ કરી શકાશે.
 - * **flex:** 1D (એક પરિમાણીય) લેઆઉટ એક્રિટવેટ કરશે.
 - * **grid:** 2D (બે પરિમાણીય - રો અને કોલમ) લેઆઉટ એક્રિટવેટ કરશે.
 - * **none:** એલિમેન્ટ પેજ પરથી ગાયબ થઈ જશે અને પોતાની જગ્યા પણ છોડી દેશે.

--- F ---

- **filter:** ઇમેજ કે એલિમેન્ટ પર વિઝ્યુઅલ ઇફેક્ટ્સ આપવા માટે.
 - વેલ્યુઝ: none, blur(5px), grayscale(100%), sepia(100%), brightness(150%), contrast(200%), drop-shadow(2px 2px red).
- **flex:** flex-grow, flex-shrink, અને flex-basis નો શોર્ટહેન્ડ છે.
- **flex-direction:** ફ્લેક્સ બોક્સની આઈટમ્સ આડી ગોઠવાશે કે ઊભી.
 - વેલ્યુઝ: row (આડી લાઇન - ડીફોલ્ટ), row-reverse, column (ઊભી લાઇન), column-reverse.
- **flex-wrap:** જો ફ્લેક્સ આઈટમ્સ સ્ક્રીનની બહાર જતી હોય તો નવી લાઇનમાં મોકલવી કે નહિ.
 - વેલ્યુઝ: nowrap (બધી એક જ લાઇનમાં કચડાઈ જશે), wrap (જગ્યા ન હોય તો નીચેની લાઇનમાં આપોઆપ આવી જશે), wrap-reverse.
- **float:** એલિમેન્ટને ડાબી કે જમણી બાજુ ધકેલવા જેથી લખાણ તેની આજુબાજુ ગોઠવાઈ જાય.
 - વેલ્યુઝ: none, left, right.
- **font-family:** અક્ષરોના ફોન્ટ બદલવા (દા.ત. Arial, 'Times New Roman', sans-serif).
- **font-size:** અક્ષરોનું કદ. વેલ્યુઝ: 16px, 1.2rem, 2em, 120%.
- **font-weight:** અક્ષરોને જાડા (Bold) કરવા. વેલ્યુઝ: normal, bold, 100, 400, 700, 900.

--- G ---

- **gap / row-gap / column-gap:** ફ્લેક્સ કે ગ્રીડ બોક્સની આઈટમ્સ વચ્ચે ખાલી જગ્યા (સ્પેસ) રાખવા માટે.
 - વેલ્યુઝ: 0, 10px, 1rem, 2vw.
- **grid-template-columns:** ગ્રીડ લેઆઉટમાં કેટલી કોલમ રાખવી અને તેની સાઈઝ કેટલી રાખવી તે નક્કી કરે છે.
 - વેલ્યુઝ: 100px 100px 100px (ત્રણ કોલમ ફિક્સ સાઈઝ), repeat(3, 1fr) (એક્સરખી 3 રિસ્પોન્સિવ કોલમ), repeat(auto-fit, minmax(200px, 1fr)).
- **grid-template-rows:** ગ્રીડ લેઆઉટમાં આડી લાઇન (Rows) ની સાઈઝ સેટ કરવા.

--- H ---

- **height:** એલિમેન્ટની લંબાઈ (ઊંચાઈ). વેલ્યુઝ: auto, 500px, 50vh (આખી સ્ક્રીનની અડધી લંબાઈ), 100%.

--- J ---

- **justify-content:** ફ્લેક્સ કે ગ્રીડ લેઆઉટમાં આઈટમ્સને આડી (horizontally) કેવી રીતે ગોઠવવી.
 - વેલ્યુઝ: flex-start, flex-end, center (એકદમ વચ્ચે લાવવા), space-between (વચ્ચે સરખી જગ્યા રાખશે), space-around, space-evenly.

--- L ---

- **letter-spacing:** બે અક્ષરો વચ્ચેની જગ્યા વધારવા-ઘટાડવા.
- **line-height:** બે લાઇનો વચ્ચેનું અંતર સેટ કરવા (Paragraph ને વાંચવામાં સરળ બનાવવા).
- **list-style:** લિસ્ટ ટેગ (ul/ol) ના બુલેટ પોઇન્ટ્સ બદલવા કે કાઢવા.
 - વેલ્યુઝ: none (બુલેટ્સ કાઢી નાખવા), square, circle, decimal.

--- M ---

- **margin:** બોક્સની બોર્ડરની બહારની તરફ ખાલી જગ્યા છોડવા માટે.
 - વેલ્યુઝ: 10px (ચારેય બાજુ), 10px 20px (Top-Bottom Left-Right), auto (બ્લોક બોક્સને સેન્ટરમાં લાવવા)

માટે).

- **margin-block / margin-inline:** આધુનિક લોજિકલ પ્રોપર્ટીઝ. margin-block એટલે top-bottom અને margin-inline એટલે left-right.
- **max-width / max-height:** એલિમેન્ટની વધુમાં વધુ પહોળાઈ કે લંબાઈ મર્યાદિત કરવા (રિસ્પોન્સિવ ડિઝાઇન માટે ખૂબ જરૂરી).
 - વેલ્યુઝ: none, 100% (મોબાઇલમાં બોક્સ બહાર ભાગશે નહિ), 1200px.
- **min-width / min-height:** ઓછામાં ઓછી સાઈઝ ફિક્સ રાખવા.

--- O ---

- **object-fit:** રિસ્પોન્સિવ બોક્સમાં ઇમેજ કે વીડિયોને કેવી રીતે ફિટ કરવો (તે કોપ થશે કે ખેંચાશે).
 - વેલ્યુઝ: fill (ખેંચાઈ જશે), contain (આખો ફોટો અંદર દેખાશે પણ જગ્યા ખાલી રહેશે), cover (ફોટો ખેંચાયા વગર આખા બોક્સમાં ફિટ થશે, જે વધારાનો ભાગ હશે તે કોપ થઈ જશે), scale-down.
- **object-position:** object-fit: cover વાપર્યું હોય ત્યારે ફોટાનો કયો ભાગ કેન્દ્રમાં રાખવો તે સેટ કરે છે (દા.ત. center, top left).
- **opacity:** એલિમેન્ટને પારદર્શક (Transparent) બનાવવા માટે.
 - વેલ્યુઝ: 1 (૧૦૦% દેખાશે), 0.5 (આછું દેખાશે), 0 (એકદમ ગાયબ, પણ પેજ પર જગ્યા રોકી રાખશે અને ક્લિક પણ થશે).
- **order:** ફ્લેક્સ કે ગ્રીડ આઈટમ્સનો ક્રમ (Position) HTML બદલ્યા વગર સીએસએસથી ચેન્જ કરવા.
 - વેલ્યુઝ: 0 (ડીફોલ્ટ), 1, 2, -1.
- **overflow / overflow-x / overflow-y:** જ્યારે બોક્સની અંદરનું લખાણ બોક્સ કરતાં મોટું થઈ જાય ત્યારે શું કરવું.
 - વેલ્યુઝ: visible (બહાર દેખાશે), hidden (વધારાનું લખાણ કપાઈ જશે), scroll (હંમેશા સ્ક્રોલ બાર દેખાશે), auto (લખાણ વધે તો જ આપોઆપ સ્ક્રોલ બાર આવશે).

--- P ---

- **padding:** બોક્સની બોર્ડરની અંદરની તરફ (કન્ટેન્ટ અને બોર્ડર વચ્ચે) જગ્યા રાખવા માટે.
 - વેલ્યુઝ: 10px, 5px 15px.
- **padding-block / padding-inline:** લોજિકલ પેડિંગ (ઊભી અને આડી જગ્યાઓ).
- **pointer-events:** એલિમેન્ટ પર માઉસ ક્લિક કે હોવર અસર કરશે કે નહિ તે કંટ્રોલ કરે છે.
 - વેલ્યુઝ: auto, none (એલિમેન્ટ પર ક્લિક નહિ થઈ શકે, માઉસ તેની આરપાર કન્ટેન્ટને સિલેક્ટ કરી શકશે).
- **position:** એલિમેન્ટને પેજ પર કઈ જગ્યાએ ગોઠવવો તેની મેઈન સિસ્ટમ.
 - વેલ્યુઝ:
 - * **static:** નોર્મલ ફ્લો (ડીફોલ્ટ), આમાં top, left, z-index કામ નહિ કરે.
 - * **relative:** પોતાની અસલી જગ્યાના આધારે ખસેડી શકાય છે (`top: 10px` આપતા પોતાની જગ્યાથી નીચે જશે).
 - * **absolute:** પોતાના પેરેન્ટ બોક્સ (જે relative હોય) ના આધારે ગમે ત્યાં ઉડાડી શકાય છે. આ પેજના નોર્મલ ફ્લોમાંથી બહાર નીકળી જાય છે.
 - * **fixed:** સ્ક્રીનની વિન્ડો સાથે ચોટી જાય છે. પેજ ગમે એટલું સ્ક્રોલ કરો, આ એલિમેન્ટ ત્યાં જ રહેશે (દા.ત. ચેટ બોટ આઇકોન).
 - * **sticky:** નોર્મલ સ્ક્રોલ થાય છે પણ એક ચોક્કસ હદ (દા.ત. `top: 0`) પર પહોંચતા જ ત્યાં ફિક્સ (ચોટી) જાય છે (દા.ત. Sticky Navbar).

--- R ---

- **resize:** યુઝર બોક્સને ખેંચીને મોટું કરી શકે કે નહિ (textarea ટેગમાં ખાસ વપરાય).
 - વેલ્યુઝ: none (ખેંચવાનું બંધ કરવા), both, horizontal, vertical.

--- S ---

- **scroll-behavior:** પેજમાં લિંક પર ક્લિક કરતા સ્ક્રોલિંગ સ્મૂથ થશે કે ઝટકા સાથે.
 - વેલ્યુઝ: auto, smooth (ધીમેથી સરકીને પેજ નીચે જશે).
- **scroll-snap-type / scroll-snap-align:** ઇન્સ્ટાગ્રામ રીલ્સ કે સ્લાઇડર જેવું ટય-લોકિંગ સ્ક્રોલિંગ બનાવવા માટે.

--- T ---

- **text-align:** અક્ષરોને આડા કઈ તરફ ગોઠવવા. વેલ્યુઝ: left, right, center, justify.
- **text-decoration:** અક્ષરોની નીચે લાઈન આપવા કે કાઢવા (દા.ત. લિંકની નીચેથી અંડરલાઈન કાઢવા `text-decoration: none;`).
- **text-overflow:** જો લખાણ એક જ લાઈનમાં બોક્સની બહાર જતું હોય તો પાછળ ત્રણ ટપકાં કરવા માટે.
 - વેલ્યુઝ: clip, ellipsis (પાછળ આપોઆપ ... કરી દેશે, શરત: white-space: nowrap અને overflow: hidden હોવું જોઈએ).
- **text-shadow:** અક્ષરોની પાછળ પડછાયો આપવા.
- **transform:** એલિમેન્ટને ફેરવવા, હલાવવા કે સાઈઝ બદલવા માટેનું જાદુઈ ટૂલ.
 - વેલ્યુઝ:
 - * **translate(x, y) / translateX() / translateY():** એલિમેન્ટને તેની જગ્યાએથી ખસેડવા.
 - * **scale(x, y):** એલિમેન્ટને નાનો કે મોટો કરવા (દા.ત. હોવર પર મોટું કરવા `scale(1.1)`).
 - * **rotate(deg):** એલિમેન્ટને ગોળ ફેરવવા (દા.ત. `rotate(45deg)`).
 - * **skew():** એલિમેન્ટને ત્રાંસો કરવા.

- * 3D વેલ્યુઝ: `rotateX()`, `rotateY()` (કાર્ડ ઊંધું કરવા), `perspective` (3D ઊંડાણ આપવા).
- transform-style: 3D ટ્રાન્સફોર્મમાં યાદવ એલિમેન્ટ્સ 3D સ્પેસમાં રહેશે કે સપાટ (flat) થઈ જશે.
 - વેલ્યુઝ: flat, preserve-3d.
- transition: દેખાવ બદલાવાની પ્રક્રિયાને સ્મૂથ (એનિમેટેડ) બનાવવા માટે.
 - વેલ્યુઝ: property | duration | timing-function | delay (દા.ત. `all 0.3s ease`).

--- U ---

- user-select: યુઝર તમારી વેબસાઈટનું લખાણ માઉસથી ખેંચીને કોપી કે સિલેક્ટ કરી શકે કે નહિ તે નક્કી કરે છે.
 - વેલ્યુઝ: auto, text, all, none (લખાણ કોપી થતું અટકાવવા માટે).

--- V ---

- visibility: એલિમેન્ટને બ્રાઉઝરમાંથી છુપાવવા માટે.
 - વેલ્યુઝ: visible, hidden (એલિમેન્ટ છુપાઈ જશે, પણ પેજ પર પોતાની જગ્યા રોકી રાખશે. આ display: none થી અલગ છે કારણ કે display: none જગ્યા પણ ખાલી કરી દે છે).

--- W ---

- white-space: લખાણની સ્પેસ અને નવી લાઈન કઈ રીતે હેન્ડલ કરવી.
 - વેલ્યુઝ: normal, nowrap (બધું લખાણ એક જ લાઈનમાં સીધું ચાલ્યા રાખશે, નવી લાઈનમાં નહિ જાય).
- width: પહોળાઈ. વેલ્યુઝ: auto, 100px, 100%, 50vw.
- word-break / overflow-wrap: લાંબા સ્પેલિંગો (જેમ કે વિક્સ) બોક્સની બહાર ભાગે ત્યારે તેને વચ્ચેથી તોડીને નવી લાઈનમાં લાવવા માટે.
 - વેલ્યુઝ: normal, break-all, break-word.

--- Z ---

- z-index: કયું બોક્સ ઉપર દેખાશે અને કયું પાછળ (લેયર્સ કંટ્રોલ કરવા). આ પ્રોપર્ટી ફક્ત ત્યારે જ કામ કરે છે જો position પ્રોપર્ટી relative, absolute, fixed કે sticky હોય.
 - વેલ્યુઝ: auto, 1, 999 (જેનો z-index મોટો, તે સ્ક્રીન પર સૌથી ઉપર દેખાશે).

૪. CSS MODERN MATH FUNCTIONS (ગણિતના ફંક્શન્સ)

મીડિયા ક્વેરીઝ વગર ડાયનેમિક કેલ્ક્યુલેશન કરવા માટે:

- calc(): ગણતરી કરવા માટે. દા.ત. `width: calc(100% - 50px);` (આખા પેજમાંથી 50px બાદ કરીને પહોળાઈ સેટ કરશે).
- min(): આપેલી વેલ્યુઝમાંથી જે સૌથી નાની હશે તે બ્રાઉઝર પસંદ કરશે. દા.ત. `width: min(500px, 100%);`
- max(): આપેલી વેલ્યુઝમાંથી જે સૌથી મોટી હશે તે સિલેક્ટ થશે.
- clamp(): રિસ્પોન્સિવ ફોન્ટ માટેનું સૌથી મોટું બ્રહ્માસ્ત્ર.
 - સિન્ટેક્સ: `clamp(MIN, VAL, MAX)`
 - ઉદાહરણ: `font-size: clamp(16px, 5vw, 40px);` -> ફોન્ટ મોબાઈલમાં ઓછામાં ઓછા 16px રહેશે, પીસી પર વધુમાં વધુ 40px થશે, અને વચ્ચેની સ્ક્રીન પર સ્ક્રીનની સાઈઝ (5vw) મુજબ આપોઆપ બદલાશે.

CSS COMPLETE MEGA MASTER ENCYCLOPEDIA

(Level 0 to Hero Blueprint)

1. WHAT IS CSS & THE CORE PRINCIPLES OF CASCADING

- * What is CSS?: Cascading Style Sheets (CSS) is a style sheet language used to describe the presentation of a document written in a markup language like HTML. It separates content from presentation, managing layouts, colors, fonts, and responses.
- * What does 'Cascading' Mean?: The "cascade" is the algorithm that resolves conflicts when multiple CSS rules apply to the same HTML element. It processes rules from top to bottom based on a strict hierarchy:
 1. Importance: Rules marked with `!important` take top priority, followed by inline styles, normal declarations, and browser defaults.
 2. Specificity: A weight system assigned to matching selectors. If importance is equal, the selector with higher specificity wins.
 3. Source Order: If importance and specificity are identical, the last rule declared in the stylesheet wins.

2. THE THREE WAYS TO APPLY CSS

- * Inline CSS: Written directly within the HTML element using the `style` attribute.
Example: `<div style="color: red; font-size: 16px;">Text</div>`
 - Pros: Quick for isolated testing, highest specificity over stylesheets.
 - Cons: Defeats the purpose of separating content from presentation; hard to maintain.
- * Internal CSS: Written inside a `<style>` tag within the `<head>` section of the HTML document.
 - Pros: Great for single-page websites; keeps styling localized to one file.
 - Cons: Cannot be reused across other web pages; increases HTML file size.
- * External CSS: Written in a separate `.css` file and linked using the `<link>` tag.
Example: `<link rel="stylesheet" href="styles.css">`
 - Pros: Best practice; styles apply globally across unlimited pages; enables browser caching.
 - Cons: Requires an extra HTTP request (can be mitigated by CDNs or modern protocols).

3. FOUNDER, HISTORICAL TIMELINE & SPECIFICATIONS

- * The Founder: Proposed by Håkon Wium Lie on October 10, 1994, while working at CERN alongside Web founder Tim Berners-Lee. Co-developed extensively with Bert Bos.
- * Why was it created?: Early HTML lacked styling capabilities. Developers began using presentation tags like `<font>` and `<table>` for layout, making code unmaintainable. CSS was created to restore clean structural markup by extracting style rules entirely.
- * Version Timeline:
 - CSS 1 (December 1996): Introduced basic fonts, text properties, margins, and colors.
 - CSS 2 (May 1998): Added absolute/relative positioning, z-index, and media types.
 - CSS 2.1 (2011): Fixed bugs and brought exact browser implementation standards.
 - CSS 3 & Modern CSS (2005 - Present): Split into independent, modular specifications (Flexbox, Grid, Color, Animations) that update continuously, known as the "CSS Living Standard".
- * Official Documentation: The World Wide Web Consortium (W3C) Style Specs:
<https://www.w3.org/Style/CSS/>

4. ALL CSS SELECTORS & COMBINATORS

- * Basic Selectors:
 - Universal Selector (`*`): Targets every single element on the page.
 - Element/Tag Selector (`div`): Targets all instances of that specific HTML tag.
 - Class Selector (`.classname`): Targets elements containing the matching `class` attribute.
 - ID Selector (`#idname`): Targets a single unique element containing the matching `id` attribute.
 - Attribute Selector (`[type="text"]`): Targets elements based on the presence or value of an attribute.
- * Combinators:
 - Descendant Selector (`space`): Targets all matching children, grandchildren, etc., inside a parent. (e.g., `div p`)
 - Child Selector (`>`): Targets only immediate direct children of a parent. (e.g., `ul > li`)

- Adjacent Sibling Selector (``+``): Targets the very next immediate sibling element. (e.g., ``h1 + p``)
- General Sibling Selector (``~``): Targets all following siblings of the same parent. (e.g., ``h1 ~ p``)
- * Pseudo-Classes (State Tracking):
 - Link States: ``:link``, ``:visited``, ``:hover``, ``:active``, ``:focus``
 - Structural: ``:first-child``, ``:last-child``, ``:nth-child(n)``, ``:nth-of-type(n)``, ``:not(selector)``, ``:empty``
- * Pseudo-Elements (Virtual Element Injection):
 - ``::before``: Injects content right before the element's actual inner HTML.
 - ``::after``: Injects content right after the element's actual inner HTML.
 - ``::first-letter`` / ``::first-line``: Styles the first character or sentence.
 - ``::selection``: Styles the highlighted block of text when selected by a user mouse.

5. THE CSS BOX MODEL & MATHEMATICAL FUNCTIONS

- * The Box Model Hierarchy (Inside Out):
 1. Content: The core text, image, or child element area.
 2. Padding: Transparent interior spacing clearing room around the content. Spaced inside the border.
 3. Border: The outer structural line wrapped tightly around padding and content.
 4. Margin: Transparent exterior spacing clearing room outside the border to push neighboring items away.
- * Sizing Mechanics (``box-sizing``):
 - ``content-box`` (Default): Sizing calculations apply only to the content area. Adding padding or borders expands the structural width/height of the element.
 - ``border-box`` (Industry Standard): The width and height properties encapsulate content, padding, and borders altogether. Prevents layouts from unexpectedly breaking.
- * Math Functions:
 - ``calc()``: Evaluates dynamic calculations on the fly. Example: ``width: calc(100% - 40px);``
 - ``min()``: Sets the absolute smallest value from a provided comma-separated list.
 - ``max()``: Sets the absolute largest value from a provided comma-separated list.
 - ``clamp()``: Locks values between an absolute minimum, a preferred scale, and an absolute maximum. Great for fluid text resizing: ``font-size: clamp(1rem, 2.5vw, 3rem);``

6. EVERY MAJOR ESSENTIAL CSS PROPERTY (A TO Z BY CATEGORY)

--- TYPOGRAPHY & TEXT ---

- * ``font-family``: Declares typeface choices (e.g., ``font-family: 'Roboto', sans-serif;``).
- * ``font-size``: Adjusts text scale using absolute (``px``) or relative (``em``, ``rem``, ``vw``) units.
- * ``font-weight``: Modifies stroke thickness via keywords (``bold``, ``normal``) or numeric values (``100`` to ``900``).
- * ``line-height``: Manages baseline spacing between stacked lines of text.
- * ``text-align``: Sets horizontal alignment (``left``, ``right``, ``center``, ``justify``).
- * ``text-transform``: Alters text casing dynamically (``uppercase``, ``lowercase``, ``capitalize``).
- * ``text-overflow``: Handles clipped strings. ``text-overflow: ellipsis;`` renders an automated ``...`` cutoff.
- * ``white-space``: Governs wraps and spaces (e.g., ``nowrap`` forces layout into a single unbroken line).
- * ``word-break`` / ``overflow-wrap``: Directs whether long unbroken words should tear apart or wrap at boundary limits.

--- VISUALS, BACKGROUNDS & EFFECTS ---

- * ``color``: Controls the foreground text color (HEX, RGB, HSL, Named colors).
- * ``background-color`` / ``background-image``: Configures solid fills or structural images behind content.
- * ``background-size``: Directs scaling parameters (``cover`` fills box completely, ``contain`` retains full aspect ratio).
- * ``opacity``: Adjusts full transparency factors on a scale from ``0.0`` (invisible) to ``1.0`` (fully solid).
- * ``box-shadow`` / ``text-shadow``: Casts soft structural depths or text outlines using offset, blur, and color factors.
- * ``border-radius``: Rounds sharp corner borders to form smooth arcs, capsules, or exact circles (``50%``).
- * ``backdrop-filter``: Applies graphical filters like ``blur()`` or ``contrast()`` to the area directly behind an element (essential for Glassmorphism effects).
- * ``clip-path``: Crops elements into specific custom shapes (polygons, circles, ellipses).

--- DISPLAY, BOX CONTROL & POSITIONING ---

* ``display``: Determines an element's structural layout engine behavior:

- ``block``: Takes up the full horizontal row width; respects all box-model parameters.
- ``inline``: Wraps tight around content; ignores top/bottom margins and width/height settings.
- ``inline-block``: Flows like inline text but respects block-level width, height, and padding settings.
- ``none``: Completely removes the element from the document layout structure entirely.

* ``visibility``: ``hidden`` hides the element from view, but preserves its original physical footprint in the layout.

* ``position``: Dictates the coordinates system for layout placement:

- ``static``: Default structural alignment tracking the document flow.
- ``relative``: Offsets coordinates relative to its own default natural structural position.
- ``absolute``: Pins coordinates relative to its closest non-static parent anchor element.
- ``fixed``: Locks coordinates relative to the screen viewport; stays in place during scroll.
- ``sticky``: Alternates between relative and fixed positioning based on the user's scroll location.

* ``z-index``: Controls stack order layering on overlapping elements (Requires position other than static).

* ``overflow``: Manages overflowing content with structural overrides (``visible``, ``hidden``, ``scroll``, ``auto``).

--- INTERACTION CONTROLS ---

* ``pointer-events``: Toggles interactive click tracking. ``pointer-events: none`` makes elements click-through.

* ``user-select``: Prevents or permits manual highlight-copy text actions (``none`` blocks selection tracking).

* ``cursor``: Modifies pointer graphics when hovering elements (``pointer``, ``grab``, ``not-allowed``, ``zoom-in``).

* ``caret-color``: Explicitly customizes the color of the blinking typing cursor within inputs.

--- SCROLLING EFFECTS ---

* ``scroll-behavior``: Smooths out anchor links across pages when set to ``smooth``.

* ``scroll-snap-type`` / ``scroll-snap-align``: Locks scroll targets cleanly into place (perfect for full-screen carousels or image feeds).

--- MEDIA & OBJECT CONTROLS ---

* ``aspect-ratio``: Forces specific structural proportion constants (e.g., ``aspect-ratio: 16 / 9``).

* ``object-fit``: Governs scaling physics for embedded images or video elements (``cover``, ``contain``).

7. MODERN LAYOUT ARCHITECTURES (FLEXBOX & GRID)

--- CSS FLEXBOX (1-Dimensional Layout Engine) ---

* Container Properties:

- ``display: flex``: Initialized flexbox context on the container.
- ``flex-direction``: Sets primary axis alignment (``row``, ``row-reverse``, ``column``, ``column-reverse``).
- ``flex-wrap``: Toggles wrap permissions (``wrap``, ``nowrap``).
- ``justify-content``: Aligns items horizontally/along the main axis (``flex-start``, ``center``, ``space-between``, ``space-around``).
- ``align-items``: Aligns items vertically/along the cross axis (``flex-start``, ``center``, ``baseline``, ``stretch``).

* Item Properties:

- ``flex-grow`` / ``flex-shrink``: Governs growth expansion factors or reduction behaviors under stress.
- ``flex-basis``: Declares initial default length dimensions prior to distribution math.
- ``align-self``: Overrides default parent ``align-items`` settings on an individual item.

--- CSS GRID LAYOUT (2-Dimensional Layout Engine) ---

* Container Properties:

- ``display: grid``: Initializes 2D grid matrix system.
- ``grid-template-columns`` / ``grid-template-rows``: Defines exact track counts and sizing templates (e.g., ``repeat(3, 1fr)``).
- ``gap`` / ``row-gap`` / ``column-gap``: Sets explicit gutter channels separating items cleanly.
- ``grid-template-areas``: Maps a visual structural layout grid textually using named areas.

* Item Properties:

- ``grid-column`` / ``grid-row``: Combines start and end track line numbers (e.g., ``grid-column: 1 /``

span 3;`).

- `grid-area`: Assigns an individual item into a named grid template cell space.

8. ADVANCED RESPONSIVE MEDIA QUERIES & FEATURES

* Classic Viewport Queries:

- Syntax: `@media screen and (min-width: 768px) { ... }`
- Features: `min-width`, `max-width`, `min-height`, `max-height`, and `orientation` (`portrait` / `landscape`).

* Device Capabilities & Display Integrity:

- `resolution`: Tests output pixel densities (e.g., matching Retina displays via `min-resolution: 2dpp`).
- `display-mode`: Tracks manifest container modes for Progressive Web Apps (`standalone`, `fullscreen`, `browser`).

* Modern User Preference Media Queries:

- `prefers-color-scheme`: Detects user device operating system level theme settings (`light` or `dark`).

Example:

```
@media (prefers-color-scheme: dark) {  
  body { background-color: #121212; color: #ffffff; }  
}
```

- `prefers-reduced-motion`: Identifies if user accessibility profiles request minimized transitions or zero animations to prevent motion sickness.
- `prefers-contrast`: Tailors layouts based on system high/low contrast readability preferences.

* Modern Container Queries:

- Evaluates breakpoints based on the size of a specific parent structural container rather than the entire browser viewport window.
- Requires declaring a container context via `container-type: inline-size;` on the parent, then targeting children via `@container (min-width: 400px) { ... }`.

9. TRANSITIONS, ANIMATIONS & DYNAMIC DESIGN

* CSS Variables (Custom Properties):

- Declared globally within the `:root` pseudo-class using double hyphens: `--main-color: #3498db;`
- Evaluated dynamically within properties using the functional wrapper: `color: var(--main-color);`

* CSS Transitions:

- Creates smooth shifts between style changes. Shorthand properties incorporate: `transition-property`, `transition-duration`, `transition-timing-function` (linear, ease-in-out, cubic-bezier), and `transition-delay`.

* CSS Keyframe Animations:

- Orchestrates intricate timeline animations using targeted keyframe percentages.
- Step 1: Define timeline mapping rules:

```
@keyframes slideIn {  
  0% { transform: translateX(-100%); opacity: 0; }  
  100% { transform: translateX(0); opacity: 1; }  
}
```
- Step 2: Bind rule mappings onto elements: `animation: slideIn 0.5s ease-out forwards;`

10. CODE OPTIMIZATION, STRUCTURAL VALIDATION & TOOLS

* W3C CSS Validation: Always check stylesheet adherence to structural web criteria utilizing the official [W3C CSS Validator](<https://jigsaw.w3.org/css-validator/>).

* Vendor Prefixes: Older experimental implementations utilized browser-specific prefixes to deploy early capabilities safely (`-webkit-` for Chrome/Safari, `-moz-` for Firefox, `-ms-` for IE/Edge). Modern living standards require clean, prefix-free production files.

=====

CSS ULTRA MEGA MASTER DICTIONARY

=====

આ ફાઇલમાં ગૂગલ અને W3C સ્પેસિફિકેશન મુજબની તમામ એડવાન્સ્ડ વિઝ્યુઅલ, મોશન, ટ્રાન્સફોર્મેશન, કન્ટેઈનર ફિટિંગ, ગ્રેડિયન્ટ્સ અને એનિમેશન પ્રોપર્ટીઝને સરળ ગુજરાતી વ્યાખ્યા અને ઉદાહરણો સાથે આવરી લેવામાં આવી છે.

1. ANIMATION & TRANSITION PROPERTIES (એનિમેશન અને મોશન કંટ્રોલ)

- * **transition:** શોર્ટહેન્ડ પ્રોપર્ટી (property duration timing delay).
 - ઉપયોગ: એલિમેન્ટની સ્ટેટ બદલાય (જેમ કે હોવર કરવા પર) ત્યારે સ્પૂષ્ઠ ઇફેક્ટ લાવવા માટે.
 - ઉદાહરણ: `transition: all 0.3s ease-in-out;`
- * **transition-property:** કઈ ચોક્કસ પ્રોપર્ટી પર ટ્રાન્ઝિશન આપવું છે તે નક્કી કરે છે.
 - ઉપયોગ: માત્ર બેકગ્રાઉન્ડ કલર કે સાઈઝ પર ઇફેક્ટ આપવા.
 - ઉદાહરણ: `transition-property: background-color, transform;`
- * **transition-duration:** ટ્રાન્ઝિશન પૂરો થવામાં કેટલો સમય લાગશે (સેકન્ડ કે મિલિસેકન્ડ).
 - ઉદાહરણ: `transition-duration: 0.5s;`
- * **transition-timing-function:** મોશનની સ્પીડ કઈ રીતની રહેશે (શરૂઆતમાં ધીમી, વચ્ચે ફાસ્ટ વગેરે).
 - વેલ્યૂઝ: `ease, linear, ease-in, ease-out, ease-in-out, cubic-bezier(n,n,n,n)`.
 - ઉદાહરણ: `transition-timing-function: cubic-bezier(0.25, 0.1, 0.25, 1);`
- * **transition-delay:** ટ્રાન્ઝિશન એક્શન શરૂ થતા પહેલા કેટલો વેઇટ (સમય) કરવો.
 - ઉદાહરણ: `transition-delay: 0.1s;`
- * **animation:** કસ્ટમ એનિમેશન માટેની શોર્ટહેન્ડ પ્રોપર્ટી (name duration timing delay iteration-count direction fill-mode play-state).
 - ઉદાહરણ: `animation: spin 2s linear infinite;`
- * **@keyframes:** એનિમેશનના અલગ-અલગ ટાઈમ સ્ટેજ (0% થી 100%) પર કઈ ડિઝાઇન રહેશે તે ડિફાઇન કરવાનો નિયમ.
 - ઉદાહરણ:


```
@keyframes spin {
    from { transform: rotate(0deg); }
    to { transform: rotate(360deg); }
}
```
- * **animation-name:** @keyframes માં બનાવેલા એનિમેશનનું નામ જોડવા માટે.
 - ઉદાહરણ: `animation-name: fade-in;`
- * **animation-duration:** એનિમેશન એક ચક્ર (cycle) પૂરું કરવામાં કેટલો સમય લેશે.
 - ઉદાહરણ: `animation-duration: 3s;`
- * **animation-timing-function:** એનિમેશનની સ્પીડ વેગ (acceleration curve).
 - ઉદાહરણ: `animation-timing-function: linear;`
- * **animation-delay:** પેજ લોડ થયા પછી એનિમેશન કેટલી સેકન્ડ પછી સ્ટાર્ટ થશે.
 - ઉદાહરણ: `animation-delay: 1s;`
- * **animation-iteration-count:** એનિમેશન કેટલી વાર ફરશે.
 - વેલ્યૂઝ: સંખ્યા (દા.ત. 1, 3, 5) અથવા `'infinite'` (અનંતકાળ માટે ચાલુ રાખવા).
 - ઉદાહરણ: `animation-iteration-count: infinite;`
- * **animation-direction:** એનિમેશન પૂરું થયા પછી પાછું કઈ દિશામાં ચાલશે.
 - વેલ્યૂઝ: `normal, reverse, alternate` (આગળ-પાછળ ચાલવું), `alternate-reverse`.
 - ઉદાહરણ: `animation-direction: alternate;`
- * **animation-fill-mode:** એનિમેશન ચાલુ ન થયું હોય કે પૂરું થઈ જાય ત્યારે એલિમેન્ટની સ્ટાઇલ કઈ રાખવી.
 - વેલ્યૂઝ: `none, forwards` (એનિમેશન પૂરું થયા પછી છેલ્લી ફ્રેમ પકડી રાખવી), `backwards, both`.
 - ઉદાહરણ: `animation-fill-mode: forwards;`
- * **animation-play-state:** એનિમેશનને લાઈવ ચાલુ રાખવું કે પોઝ (Pause) કરવું.

- વેલ્યુઝ: running, paused.
- ઉપયોગ: હોવર કરવા પર વીડિયો કે લોડરનું એનિમેશન અટકાવવા.
- ઉદાહરણ: animation-play-state: paused;

2. TRANSFORM PROPERTIES (2D અને 3D ટ્રાન્સફોર્મેશન)

- * transform: એલિમેન્ટને આકાર બદલવા, ફેરવવા, ખસેડવા કે ત્રાંસો કરવા માટેનું પાવરફુલ ટૂલ.
 - ફંક્શનલ્સ:
 1. translate(x, y) / translateX(x) / translateY(y): એલિમેન્ટને તેની જગ્યાએથી ખસેડવા.
 2. scale(x, y) / scaleX(x) / scaleY(y): એલિમેન્ટને ઝૂમ (મોટો/નાનો) કરવા.
 3. rotate(deg) / rotateX() / rotateY() / rotateZ(): ડિગ્રી મુજબ ગોળ ફેરવવા.
 4. skew(x-deg, y-deg): એલિમેન્ટને ખેંચીને ત્રાંસો (કાટખૂણો બદલવો) કરવા.
 5. matrix(n,n,n,n,n,n): ઉપરના તમામ ટ્રાન્સફોર્મ્સને એક ગાણિતિક મેટ્રિક્સમાં કમ્બાઈન કરવા.
 - ઉદાહરણ: transform: translate(-50%, -50%) rotate(45deg) scale(1.2);
- * transform-origin: એલિમેન્ટ કયા કેન્દ્રબિંદુથી ફરશે કે ટ્રાન્સફોર્મ થશે તે નક્કી કરે છે.
 - વેલ્યુઝ: top, bottom, center, left, right, અથવા પિક્સલ/ટકાવારી.
 - ઉદાહરણ: transform-origin: top left;
- * transform-style: 3D સ્પેસમાં ચાઇલ્ડ એલિમેન્ટ્સને ફ્લેટ રાખવા કે 3D માં જાળવવા.
 - વેલ્યુઝ: flat, preserve-3d.
 - ઉદાહરણ: transform-style: preserve-3d;
- * perspective: 3D એનિમેશન જોતી વખતે યૂઝરની આંખ અને સ્ક્રીન વચ્ચેનું અંતર (Depth) નક્કી કરે છે જેથી રિયલ 3D લુક આવે.
 - ઉદાહરણ: perspective: 500px;
- * perspective-origin: 3D વ્યુ જોવાનો એન્ગલ (ડાબેથી, જમણેથી કે સેન્ટરમાંથી) સેટ કરવા.
 - ઉદાહરણ: perspective-origin: 50% 50%;
- * backface-visibility: જ્યારે 3D માં કાર્ડ આખું ઊંધું ફરે ત્યારે તેનો પાછળનો ભાગ બતાવવો કે અદ્રશ્ય રાખવો.
 - વેલ્યુઝ: visible, hidden.
 - ઉપયોગ: Flip Card ઇફેક્ટ બનાવવા માટે.
 - ઉદાહરણ: backface-visibility: hidden;

3. MEDIA & IMAGE FITTING PROPERTIES (ઇમેજ, વિડિયો અને કન્ટેન્ટ ફિટિંગ)

- * object-fit: ઇમેજ કે વીડિયો તેના નક્કી કરેલા બોક્સ (પહોળાઈ અને લંબાઈ) માં કઈ રીતે ગોઠવાશે.
 - વેલ્યુઝ:
 1. fill: ડિફોલ્ટ વેલ્યુ. ફોટો ખેંચાઈ (Stretch) જશે પણ આખા બોક્સમાં ફિટ થશે.
 2. contain: ફોટો ખેંચાયા વગર આખો દેખાશે, ભલે બોક્સમાં આજુબાજુ જગ્યા ખાલી રહે.
 3. cover: ફોટો જરાય ખેંચાશે નહીં, ગુણવત્તા જળવાશે અને વધારાનો ભાગ ક્રોપ થઈને આખું બોક્સ ભરાઈ જશે (સૌથી વધુ વપરાય છે).
 4. none: ફોટો પોતાની અસલી ઓરિજિનલ સાઈઝ જ રાખશે.
 5. scale-down: none અને contain માંથી જે સાઈઝ નાની હશે તે પકડી લેશે.
 - ઉદાહરણ: object-fit: cover;
- * object-position: object-fit: cover આપ્યા પછી ફોટાનો કયો ભાગ સેન્ટરમાં રાખવો તે નક્કી કરે છે.
 - ઉદાહરણ: object-position: center top; (ફોટાનો ચહેરો ઉપર હોય તો ક્રોપ થતી વખતે ઉપરનો ભાગ બચાવશે).
- * background-size: બેકગ્રાઉન્ડ ઇમેજની સાઈઝ કંટ્રોલ કરવા માટે.
 - વેલ્યુઝ: auto, length (px), percentage (%), cover (આખું બેકગ્રાઉન્ડ ઢાંકવું), contain (ઇમેજ આખી બતાવવી).
 - ઉદાહરણ: background-size: cover;
- * background-position: બેકગ્રાઉન્ડ ઇમેજ કયા ખૂણેથી દેખાડવી.
 - ઉદાહરણ: background-position: center center;
- * aspect-ratio: એલિમેન્ટની પહોળાઈ અને લંબાઈનો રેશિયો ફિક્સ કરવો જેથી રિસ્પોન્સિવ લેઆઉટમાં બોક્સ તૂટે નહીં.
 - ઉદાહરણ: aspect-ratio: 16 / 9; (યૂટ્યૂબ વીડિયો જેવો પરફેક્ટ રેશિયો).

- * image-rendering: બ્રાઉઝર જ્યારે ઇમેજને મોટી કરે ત્યારે પિક્સલ્સ બ્લર કરવા કે શાર્પ રાખવા.
 - વેલ્યુઝ: auto, crisp-edges, pixelated (જૂની રેટ્રો ગેમ્સ જેવો લુક લાવવા).
 - ઉદાહરણ: image-rendering: pixelated;

4. ADVANCED VISUAL SHAPES & EFFECTS (ગ્રેડિયન્ટ્સ, ક્લિપિંગ અને ફિલ્ટર્સ)

- * clip-path: એલિમેન્ટને માસ્ક કરીને ચોક્કસ ભૌમિતિક આકાર (ગોળ, ત્રિકોણ, પોલીગોન) માં કાપવો.
 - ઉદાહરણ: clip-path: polygon(50% 0%, 0% 100%, 100% 100%); (પરફેક્ટ ત્રિકોણ).
- * mask-image: કોઈપણ ફોટા પર બીજા ફોટા કે ગ્રેડિયન્ટનો માસ્ક ચડાવીને ડાર્ક/લાઇટ ભાગને ગાયબ કરવો.
 - ઉદાહરણ: mask-image: linear-gradient(to bottom, black, transparent);
- * backdrop-filter: એલિમેન્ટની પાછળ રહેલા બેકગ્રાઉન્ડ કન્ટેન્ટ પર ફિલ્ટર લગાવવું (Glassmorphism ઇફેક્ટ).
 - ઉદાહરણ: backdrop-filter: blur(12px) saturate(180%);
- * filter: એલિમેન્ટ કે ફોટા પર લાઇવ ઇફેક્ટ્સ આપવી.
 - વેલ્યુઝ: blur(px), grayscale(%), sepia(%), brightness(%), contrast(%), drop-shadow().
 - ઉદાહરણ: filter: grayscale(100%) blur(2px);
- * mix-blend-mode: એલિમેન્ટના કલર્સ તેના પાછળના લેયર/બેકગ્રાઉન્ડ સાથે કઈ રીતે મિક્સ થશે તે નક્કી કરવું.
 - વેલ્યુઝ: multiply, screen, overlay, darken, color-dodge.
 - ઉદાહરણ: mix-blend-mode: overlay;
- * background-clip: બેકગ્રાઉન્ડ કલર કે ઇમેજ ક્યાં સુધી ફેલાશે (border-box, padding-box, content-box, text).
 - ખાસ ઉપયોગ: `background-clip: text;` કરવાથી અક્ષરોની અંદર લાઇવ બેકગ્રાઉન્ડ ગ્રેડિયન્ટ પૂરી શકાય છે.
 - ઉદાહરણ:


```
background: linear-gradient(to right, red, blue);
-webkit-background-clip: text;
color: transparent;
```

5. GRADIENT FUNCTIONS (તમામ પ્રકારના કલર મિક્સિંગ ફંક્શન્સ)

- * linear-gradient(): બે કે તેથી વધુ કલરને સીધી રેખામાં (કોઈપણ એન્ગલ પર) મિક્સ કરવા.
 - ઉદાહરણ: background: linear-gradient(45deg, #ff007f, #7f00ff);
- * radial-gradient(): કલરને મધ્યબિંદુમાંથી શરૂ કરીને ગોળાકાર કે અંડાકાર બહાર ફેલાવવા.
 - ઉદાહરણ: background: radial-gradient(circle, red, yellow, green);
- * conic-gradient(): ઘડિયાળના કાંટાની જેમ ૩૬૦ ડિગ્રી પર કલર વ્હીલ કે પાઇ-ચાર્ટ જેવી ડિઝાઇન બનાવવા.
 - ઉદાહરણ: background: conic-gradient(red, yellow, green, blue, red);
- * repeating-linear-gradient() / repeating-radial-gradient(): કલર ગ્રેડિયન્ટની પેટર્નને વારંવાર રીપીટ કરીને જેબ્રા કોસિંગ કે પટ્ટાવાળી ડિઝાઇન ઓટોમેટિક તૈયાર કરવા.
 - ઉદાહરણ: background: repeating-linear-gradient(45deg, black, black 10px, white 10px, white 20px);

આ ડિક્શનરીનો ઉપયોગ કરીને તમે કોઈપણ મોડર્ન એનિમેટેડ અને હાઇ-વિઝ્યુઅલ વેબસાઇટ યુઝર ઇન્ટરફેસ (UI) ખૂબ જ સરળતાથી અને ગૂઝલ સ્ટાન્ડર્ડ મુજબ ડિઝાઇન કરી શકશો.

=====

CSS MEGA MASTER ENCYCLOPEDIA (A TO Z)

=====

ભાષા (Language): ગુજરાતી (Gujarati) અને English મિક્સ (સરળ સમજૂતી માટે)
તમામ વિષયો: પાયાથી લઈને એડવાન્સ, હિસ્ટ્રી, બોક્સ મોડેલ, સિલેક્ટર્સ અને તમામ પ્રોપર્ટીઝ.

PART 1: CSS નો પરિચય, ઇતિહાસ અને મૂળભૂત ખ્યાલો (Foundations & History)

1. CSS શું છે? (What is CSS?)

- CSS નું પૂરું નામ Cascading Style Sheets છે.
- તે કોઈ પ્રોગ્રામિંગ ભાષા નથી, પરંતુ એક "Style Sheet Language" છે.
- HTML માત્ર વેબપેજનું માળખું (Skeleton) તૈયાર કરે છે, જ્યારે CSS તે માળખાને સુંદરતા, રંગ, ડિઝાઇન અને લેઆઉટ (Skin & Clothes) પ્રદાન કરે છે.

2. "Cascading" એટલે શું? (What does Cascading Mean?)

- Cascading નો અર્થ થાય છે "ધ ધોધની જેમ ઉપરથી નીચે તરફ વહેવું" (Falling like a waterfall).
- વેબ ડેવલપમેન્ટમાં, જ્યારે એક જ એલિમેન્ટ પર એક કરતાં વધુ CSS નિયમો (Rules) લાગુ પડે છે, ત્યારે કયો નિયમ પ્રાઈઓરિટી (Priority) મેળવશે તે નક્કી કરવા માટે આખી સિસ્ટમ ઉપરથી નીચે તરફ ઓર્ડર ચેક કરે છે. તેને "Cascade" કહેવાય છે.
- આ વ્યવસ્થા 3 મુખ્ય બાબતો પર કામ કરે છે:
 - A) Importance: !important ટેગ સૌથી ઉપર રહે છે.
 - B) Specificity: કયો સિલેક્ટર કેટલો ચોક્કસ (Specific) છે (દા.ત. ID સિલેક્ટર એ ક્લાસ કરતાં વધુ પાવરફુલ છે).
 - C) Source Order: જો બધી બાબતો સરખી હોય, તો કોડમાં જે નિયમ સૌથી છેલ્લે (નીચે) લખાયો હશે તે લાગુ થશે.

3. CSS નો ઇતિહાસ, શોધ અને ફાઉન્ડર (Founder & History)

- શોધક (Founder): CSS ની શોધ Håkon Wium Lie (હાકોન વિયમ લી) એ કરી હતી. તેઓએ 1994 માં આનો પ્રસ્તાવ મૂક્યો હતો, જ્યારે તેઓ CERN ખાતે Tim Berners-Lee (HTML ના શોધક) સાથે કામ કરતા હતા.
- સહ-શોધક: Bert Bos (બર્ટ બોસ), જેમણે બ્રાઉઝરમાં CSS ને સપોર્ટ કરાવવા માટે મોટું યોગદાન આપ્યું.
- શા માટે શોધ થઈ?: શરૂઆતમાં HTML માં જ રંગો અને ફોન્ટ બદલવા માટે કે color એટ્રિબ્યુટ્સ વાપરવા પડતા હતા. આનાથી કોડ બહુ મોટો અને મેનેજ કરવો અશક્ય બની જતો હતો. કન્ટેન્ટ (HTML) અને ડિઝાઇન (CSS) ને અલગ કરવા માટે CSS ની શોધ થઈ.
- ટાઇમલાઇન અને વર્ઝન:
 - * CSS 1.0 (December 1996): સૌપ્રથમ ઓફિશિયલ સ્પેસિફિકેશન (બેઝિક કલર, ફોન્ટ).
 - * CSS 2.0 (May 1998): પોઝિશનિંગ (Absolute, Relative) અને મીડિયા ટાઇપ્સ ઉમેરાયા.
 - * CSS 2.1 (2011): ભૂલો સુધારીને સ્ટેબલ વર્ઝન બનાવાયું.
 - * CSS 3 (1999 - ચાલુ): CSS3 ને કોઈ એક સિંગલ વર્ઝન તરીકે લોન્ચ કરવાને બદલે અલગ-અલગ "Modules" (જેમ કે Flexbox, Grid, Animations) માં વહેંચી દેવામાં આવ્યું.
 - * CSS4 / Next-Gen: હવે CSSLiving Standard ની જેમ ચાલે છે, જેમાં નવા ફીચર્સ (જેમ કે CSS Variables, Nesting, :has() selector) સતત ઉમેરવામાં આવે છે.
- ઓફિશિયલ સાઇટ: W3C (World Wide Web Consortium) CSS સ્ટાન્ડર્ડ મેનેજ કરે છે (w3.org/Style/CSS).

4. CSS કઈ કઈ રીતે વેબપેજમાં ઉમેરી શકાય? (3 Ways to Apply CSS)

- A) Inline CSS: HTML ટેગની અંદર જ 'style' એટ્રિબ્યુટ વાપરીને.
ઉદાહરણ: <h1 style="color: blue; font-size: 20px;">નમસ્તે</h1>
ગેરફાયદો: કોડ ગંદો થાય છે અને દરેક ટેગમાં અલગથી લખવું પડે છે.
- B) Internal / Embedded CSS: HTML પેજના <head> સેક્શનમાં <style> ટેગની અંદર.
ઉદાહરણ:
<style>
 p { color: red; }
</style>
ગેરફાયદો: આ સ્ટાઇલ માત્ર તે એક જ સિંગલ વેબપેજ પૂરતી સીમિત રહે છે.
- C) External CSS: એક અલગ .css ફાઇલ બનાવીને તેને HTML ના <head> માં <link> ટેગથી જોડવી.
ઉદાહરણ: <link rel="stylesheet" href="style.css">
ફાયદો: આ સૌથી બેસ્ટ રીત છે. એક જ CSS ફાઇલથી આખી વેબસાઇટના હજારો પેજ કંટ્રોલ થઈ શકે છે.

PART 2: CSS સિલેક્ટર્સ અને કોમ્બિનેટર્સ (Selectors & Combinators)

સિલેક્ટર એટલે HTML ના કયા ભાગને ડિઝાઇન આપવી છે તે નક્કી કરવું.

1. Basic Selectors (મૂળભૂત સિલેક્ટર્સ):

- Universal Selector (*): બધા જ એલિમેન્ટ્સને એકસાથે સિલેક્ટ કરે છે. (દા.ત. * { margin: 0; })
- Element/Type Selector: ટેગના નામથી સિલેક્ટ કરે. (દા.ત. p { color: green; } - બધા પેરેગ્રાફ ટેગની થશે)
- Class Selector (.): ક્લાસના નામથી સિલેક્ટ કરે. કોડમાં આગળ ડોટ (.) લગાવાય છે. એક ક્લાસ ગમે તેટલા ટેગમાં આપી શકાય. (દા.ત. .my-text { font-weight: bold; })
- ID Selector (#): આઈડીના નામથી સિલેક્ટ કરે. આગળ હેશ (#) લગાવાય છે. આખા પેજમાં ID યુનિક (એક જ) હોવું જોઈએ. (દા.ત. #header-main { background: black; })
- Attribute Selector: ટેગના ચોક્કસ એટ્રિબ્યુટના આધારે સિલેક્ટ કરે. (દા.ત. input[type="text"] { border: 1px solid red; })

2. CSS Combinators (બે કે તેથી વધુ સિલેક્ટર્સને જોડવાની રીત):

- Descendant Selector (space): પેરેન્ટની અંદર રહેલા તમામ ચાઇલ્ડ અને ગ્રાન્ડ-ચાઇલ્ડને સિલેક્ટ કરે. (દા.ત. div p { ... } - div ની અંદર ગમે ત્યાં રહેલા બધા જ p ટેગ)
- Child Selector (>): માત્ર ડાયરેક્ટ (તરત જ નીચેના) ચાઇલ્ડને જ સિલેક્ટ કરે. (દા.ત. div > p { ... })
- Adjacent Sibling Selector (+): એક એલિમેન્ટની તરત જ બાજુમાં/પછી આવતા સગા ભાઈ (Sibling) ને સિલેક્ટ કરે. (દા.ત. h1 + p { ... } - h1 પૂરો થયા પછી તરત આવતો પહેલો પેરેગ્રાફ)
- General Sibling Selector (~): એક એલિમેન્ટની પછી આવતા તમામ ભાઈઓ (Siblings) ને સિલેક્ટ કરે. (દા.ત. h1 ~ p { ... })

3. Pseudo-classes & Pseudo-elements:

- Pseudo-classes (:): એલિમેન્ટની ચોક્કસ સ્ટેટ (સ્થિતિ) બદલવા.
 - * :hover (માઉસ ઉપર લઈ જતાં), :active (ક્લિક કરતી વખતે), :focus (ઇનપુટ બોક્સ ઓપન હોય ત્યારે),
 - * :nth-child(n) (ચોક્કસ નંબરનો ચાઇલ્ડ જેમ કે એકી-બેકી સંખ્યા :nth-child(odd)).
- Pseudo-elements (::): એલિમેન્ટના કોઈ ચોક્કસ ભાગને સ્ટાઇલ કરવા.
 - * ::before (એલિમેન્ટ શરૂ થાય તે પહેલાં કંઈક ઉમેરવા), ::after (પૂરો થયા પછી ઉમેરવા),
 - * ::first-letter (પહેલો અક્ષર મોટો કરવા), ::selection (યુઝર ટેક્સ્ટ સિલેક્ટ કરે ત્યારે બેકગ્રાઉન્ડ બદલવા).

PART 3: CSS બોક્સ મોડેલ (The CSS Box Model) - મોસ્ટ ઇમ્પોર્ટન્ટ

બ્રાઉઝર દરેક HTML એલિમેન્ટને એક લંબચોરસ બોક્સ (Box) તરીકે જુએ છે. આ બોક્સ 4 ભાગોનું બનેલું છે:

1. Content: મુખ્ય કન્ટેન્ટ (લખાણ કે ઇમેજ) જે બોક્સની અંદર દેખાય છે. આની સાઈઝ width અને height થી નક્કી થાય છે.
2. Padding: કન્ટેન્ટ અને બોર્ડર વચ્ચેની અંદરની જગ્યા (Inward Space). આનો કલર બેકગ્રાઉન્ડ જેવો જ રહે છે.
3. Border: પેડિંગની બહાર આવેલી ક્વરિંગ લાઇન. આ જાડી, પાતળી કે કલર હોઈ શકે છે.
4. Margin: બોર્ડરની બહારની ખાલી જગ્યા (Outward Space), જે બીજા બોક્સને આ બોક્સથી દૂર રાખે છે. માર્જિન હંમેશા ટ્રાન્સપરન્ટ હોય છે.

* Box-Sizing નું રહસ્ય:

- ડિફોલ્ટ રૂપે (content-box), જો તમે એલિમેન્ટની width 200px આપો અને padding 20px આપો, તો ટોટલ વિડ્થ 240px થઈ જાય છે.
- આ પ્રોબ્લેમ સોલ્વ કરવા માટે `box-sizing: border-box;` નો ઉપયોગ થાય છે. આનાથી પેડિંગ અને બોર્ડર મૂળ વિડ્થની અંદર જ એડજસ્ટ થઈ જાય છે.

PART 4: તમામ CSS પ્રોપર્ટીઝ, વ્યાખ્યા, ઉપયોગ અને ઉદાહરણો (A to Z Property Guide)

અહીં બધી જ મહત્વની CSS પ્રોપર્ટીઝને કેટેગરી વાઇઝ વર્ણવી છે જેથી એકપણ ટોપિક બાકી ન રહે:

1. TEXT & TYPOGRAPHY PROPERTIES (લખાણ માટે)

- font-family: ફોન્ટની સ્ટાઇલ બદલવા માટે. (દા.ત. font-family: 'Arial', sans-serif;)
- font-size: અક્ષરો નાના-મોટા કરવા. (દા.ત. font-size: 16px; અથવા 1.2rem;)
- font-weight: અક્ષરો જાડા (Bold) કરવા. (દા.ત. font-weight: bold; અથવા 700;)
- font-style: અક્ષરો ત્રાંસા (Italic) કરવા. (દા.ત. font-style: italic;)

- text-align: ટેક્સ્ટને ડાબે, જમણે કે સેન્ટરમાં ગોઠવવા. (દા.ત. text-align: center;)
- text-decoration: લખાણની નીચે, ઉપર કે વચ્ચે લાઇન કરવા. લિંકની અંડરલાઇન હટાવવા `text-decoration: none;` વપરાય છે.
- text-transform: અક્ષરોને Capital કે Small કરવા. (દા.ત. text-transform: uppercase;)
- line-height: બે લાઇન વચ્ચેની જગ્યા સેટ કરવા. (દા.ત. line-height: 1.5;)
- letter-spacing: અક્ષરો વચ્ચેની સ્પેસ વધારવા. (દા.ત. letter-spacing: 2px;)
- word-spacing: બે શબ્દો વચ્ચેની જગ્યા સેટ કરવા.

2. COLOR & BACKGROUND PROPERTIES (રંગ અને બેકગ્રાઉન્ડ)

- color: અક્ષરોનો રંગ બદલવા. (દા.ત. color: #ff5733; અથવા rgb(255,0,0);)
- background-color: બોક્સની પાછળનો બેકગ્રાઉન્ડ કલર સેટ કરવા.
- background-image: બેકગ્રાઉન્ડમાં ફોટો સેટ કરવા. (દા.ત. background-image: url('bg.jpg'));
- background-repeat: જો ફોટો નાનો હોય તો તેને રીપીટ થતો અટકાવવા. (દા.ત. background-repeat: no-repeat;)
- background-size: ફોટો આખા બોક્સમાં ફીટ કરવા માટે `background-size: cover;` વપરાય છે.
- background-position: ફોટો કઈ જગ્યાએ રાખવો (દા.ત. background-position: center;)
- background-attachment: સ્ક્રોલ કરતી વખતે બેકગ્રાઉન્ડ ઈમેજ ફિક્સ રાખવા (fixed) કે સ્ક્રોલ કરવા (scroll).
- opacity: એલિમેન્ટને કાચ જેવો પારદર્શક (Transparent) બનાવવા. વેલ્યુ 0.0 થી 1.0 સુધી હોય.

3. BOX MODEL PROPERTIES (માર્જિન, પેડિંગ, બોર્ડર)

- margin / margin-top / margin-right / margin-bottom / margin-left: બહારની જગ્યા માટે.
- padding / padding-top / padding-right / padding-bottom / padding-left: અંદરની સ્પેસ માટે.
- border: બોર્ડરની શીર્ટહેન્ડ પ્રોપર્ટી. (દા.ત. border: 2px solid red;)
- border-radius: બોક્સના ખૂણાઓને ગોળ (Round) કરવા માટે. (દા.ત. border-radius: 8px; અથવા પૂરું ગોળ કરવા 50%)
- box-shadow: બોક્સની પાછળ પડછાયો (Shadow) લાવવા. (દા.ત. box-shadow: 5px 5px 10px gray;)
- text-shadow: અક્ષરો પાછળ પડછાયો લાવવા.

4. LAYOUT & DISPLAY PROPERTIES (પેજનું માળખું અને ગોઠવણી)

- display: એલિમેન્ટ બ્રાઉઝર પર કેવી રીતે વ્યવહાર કરશે તે નક્કી કરે છે.
 - * display: block; -> આખી લાઇન રોકી લે છે (દા.ત. <div>, <p>).
 - * display: inline; -> જેટલી જગ્યા જોઈએ એટલી જ લે છે, વિડ્થ-હાઇટ સેટ નથી થતી (દા.ત. , <a>).
 - * display: inline-block; -> લાઇન inline જેવી પણ વિડ્થ અને હાઇટ સેટ કરી શકાય.
 - * display: none; -> એલિમેન્ટને વેબપેજ પરથી સંપૂર્ણ ગાયબ (Hide) કરી દે છે અને જગ્યા પણ ખાલી થઈ જાય છે.
- visibility: hidden; -> એલિમેન્ટ ગાયબ થાય છે પણ તેની જગ્યા રોકી રાખે છે.
- width / height: બોક્સની પહોળાઈ અને ઊંચાઈ.
- max-width / min-width: રિસ્પોન્સિવ ડિઝાઇન માટે મેક્સિમમ અને મિનિમમ પહોળાઈ લિમિટ સેટ કરવા.
- overflow: જો કન્ટેન્ટ બોક્સની બહાર નીકળી જાય, તો તેને છુપાવવા (hidden), સ્ક્રોલબાર લાવવા (scroll) કે ઓટો રાખવા (auto).

5. POSITIONING PROPERTIES (ચોક્કસ જગ્યાએ ગોઠવણ)

- position: એલિમેન્ટને સ્ક્રીન પર કઈ જગ્યાએ લોક કરવો તે નક્કી કરે છે.
 - * static: ડિફોલ્ટ પોઝિશન, નોર્મલ ફ્લો મુજબ ચાલે.
 - * relative: પોતાની નોર્મલ જગ્યાથી top, left, right, bottom વડે ખસેડી શકાય.
 - * absolute: પોતાના નજીકના પેરન્ટ (જે રીલેટીવ હોય) ના આધારે ગમે ત્યાં ગોઠવી શકાય. પેજના ફ્લોમાંથી બહાર નીકળી જાય છે.
 - * fixed: સ્ક્રીન (Viewport) સાથે ચોટી જાય છે. પેજ ગમે તેટલું સ્ક્રોલ કરો, તે ત્યાં જ રહે છે (દા.ત. ચેટ બોટન કે ફિક્સ હેડર).
 - * sticky: નોર્મલ સ્ક્રોલ થાય છે, પણ એક ચોક્કસ જગ્યાએ પહોંચ્યા પછી ત્યાં ફિક્સ (ચોટી) જાય છે.
- z-index: એલિમેન્ટ્સ એકબીજાની ઉપર-નીચે ગોઠવાય ત્યારે કયો એલિમેન્ટ આગળ દેખાશે તે નક્કી કરવા લેયર નંબર (દા.ત. z-index: 999;).

PART 5: મોર્ડન લેઆઉટ સિસ્ટમ્સ (Modern CSS Layouts - Flexbox & Grid)

જૂના જમાનામાં લેઆઉટ માટે `float: left;` કે `table` વપરાતા હતા જે બહુ અઘરા હતા. હવે મોર્ડન CSS માં બે ધાંસુ લેઆઉટ છે:

1. CSS Flexbox (Flexible Box Layout - One Dimensional)

- આ લાઇન (Row) અથવા કોલમ (Column) માં આઈટમ્સને સેટ કરવા માટે બેસ્ટ છે.

- પેરન્ટ બોક્સમાં લખો: ``display: flex;``
- મુખ્ય પ્રોપર્ટીઝ:
 - * `flex-direction`: આઈટમ્સ આડી ગોઠવવી (row) કે ઊભી (column).
 - * `justify-content`: આડી લાઈનમાં આઈટમ્સ ક્યાં રાખવી (center, flex-start, flex-end, space-between, space-around).
 - * `align-items`: ઊભી લાઈનમાં (Vertically) આઈટમ્સ સેન્ટર કરવા માટે ``align-items: center;`` ખૂબ જ ફેમસ છે.
 - * `flex-wrap`: જો જગ્યા ઓછી હોય તો આઈટમ્સ આપોઆપ બીજી લાઈનમાં મોકલવા માટે (wrap).

2. CSS Grid Layout (Two Dimensional)

- આ આખા પેજનું ગ્રીડ (રો અને કોલમ બંને એકસાથે) ડિઝાઇન કરવા માટે વપરાય છે.
- પેરન્ટ બોક્સમાં લખો: ``display: grid;``
- મુખ્ય પ્રોપર્ટીઝ:
 - * `grid-template-columns`: કોલમની સંખ્યા અને સાઈઝ નક્કી કરવા. (દા.ત. `grid-template-columns: repeat(3, 1fr);` -> સરખી સાઈઝની 3 કોલમ)
 - * `grid-template-rows`: રો ની સાઈઝ સેટ કરવા.
 - * `gap / grid-gap`: ગ્રીડના બોક્સ વચ્ચે સ્પેસ (ખાલી જગ્યા) રાખવા.

PART 6: એડવાન્સ્ડ CSS અને રિસ્પોન્સિવ વેબ ડિઝાઇન (Advanced CSS & Responsive)

1. Media Queries (@media):

- વેબસાઇટને કમ્પ્યુટર, ટેબ્લેટ અને મોબાઇલ ટ્રાયેલમાં બરાબર બતાવવા (Responsive Design) માટે આનો ઉપયોગ થાય છે.
- ઉદાહરણ:


```
@media (max-width: 768px) {
  body { background-color: lightblue; } /* મોબાઇલમાં બેકગ્રાઉન્ડ બદલાઈ જશે */
  .sidebar { display: none; } /* મોબાઇલમાં સાઈડબાર છુપાવી દો */
}
```

2. CSS Variables (Custom Properties):

- આખા કોડમાં વારંવાર વપરાતા કલર કે વેલ્યુને એક જગ્યાએ સ્ટોર કરવા.
- ડિક્લેરેશન: `:root { --main-color: #007bff; }`
- ઉપયોગ: Use કરવા માટે ``color: var(--main-color);`` લખાય છે. ભવિષ્યમાં ચેન્જ કરવો હોય તો માત્ર `:root` માં બદલવાથી બધે બદલાઈ જશે.

3. Transitions & Animations (એનિમેશન):

- `transition`: કોઈ પણ ચેન્જ (જેમ કે હોવર કરવાથી કલર બદલાવવો) ને ઝટકા સાથે બદલવાને બદલે સ્મૂથ સ્પીડ આપવા. (Ref: `transition: all 0.3s ease;`)
- `@keyframes & animation`: એડવાન્સ્ડ એનિમેશન બનાવવા માટે.
 - * ઉદાહરણ:


```
@keyframes slideIn {
    from { transform: translateX(-100%); }
    to { transform: translateX(0); }
}
.box { animation: slideIn 1s forwards; }
```

4. Transforms:

- `transform`: બોક્સને ત્રાંસું કરવા (skew), ફેરવવા (rotate), મોટું-નાનું કરવા (scale) કે ખસેડવા (translate).
- ઉદાહરણ: ``transform: rotate(45deg);`` કે સેન્ટર કરવા ``transform: translate(-50%, -50%);``

PART 7: CSS વેલિડેશન, આર્કિટેક્ચર અને બેસ્ટ પ્રેક્ટિસિસ

- CSS વેલિડેટ કરવા માટે ઓફિશિયલ [W3C CSS Validator](https://jigsaw.w3.org/css-validator/ "W3C CSS Validation Service") નો ઉપયોગ થાય છે.
- મલ્ટીપલ બ્રાઉઝર સપોર્ટ માટે વેન્ડર પ્રિફિક્સ (Vendor Prefixes) જેવા કે ``-webkit-``, ``-moz-``, ``-ms-`` વપરાય છે.
- મોટા પ્રોજેક્ટ્સમાં CSS કોડ ઓર્ગેનાઇઝ કરવા માટે BEM (Block Element Modifier) જેવી મેથડોલોજી અને Sass/Less જેવા CSS Preprocessors નો ઉપયોગ થાય છે.

=====

CSS MASTER PhD-LEVEL DOCUMENTATION
COMPLETE A TO Z EXPLANATION IN GUJARATI

=====

● Module 1 & 2: CSS Core Architecture & Engine

* CSSOM (CSS Object Model) Parsing Engine

- Definition: બ્રાઉઝર જ્યારે HTML ની જેમ જ CSS રૂલ્સને સમજીને તેનો એક ઓબ્જેક્ટ ટ્રી મેપ તૈયાર કરે છે, તેને CSSOM કહેવાય છે.
- Syntax: બ્રાઉઝર બેકએન્ડ પ્રોસેસ.
- Properties: CSS નિયમોનું આંતરિક પર્સિંગ (Parsing).
- Example: સીએસએસ ફાઇલ લોડ થતાં જ આ મોડેલ મેમરીમાં બને છે.
- Output: બ્રાઉઝર જાવાસ્ક્રિપ્ટ દ્વારા સ્ટાઇલ ચેન્જ એક્સેસ કરવા માટે તૈયાર થાય છે.

* DOM and CSSOM Tree Integration

- Definition: HTML માંથી બનેલું DOM અને CSS માંથી બનેલું CSSOM જ્યારે એકબીજા સાથે જોડાય છે ત્યારે જ પેજ ડિઝાઇન પ્રોસેસ શરૂ થાય છે.
- Syntax: બ્રાઉઝરનું આંતરિક એન્જિન કમ્બિનેશન.
- Example: DOM + CSSOM = Render Tree.
- Output: વેબસાઇટ પર લખાણ અને ડિઝાઇનનું જોડાણ ફાઇનલ થાય છે.

* Render Tree Generation

- Definition: પેજ પર કઈ વસ્તુ દેખાડવાની છે (જેમ કે display: none વગરના એલિમેન્ટ્સ) તેનું ફાઇનલ ટ્રી સ્ટ્રક્ચર એટલે રેન્ડર ટ્રી.
- Syntax: બ્રાઉઝર લેઆઉટ એન્જિન ગણતરી.
- Example: છુપાવેલા ટેગ્સ આ ટ્રીમાંથી આપોઆપ દૂર થઈ જાય છે.
- Output: પેજ પર કન્ટેન્ટ કઈ જગ્યાએ દોરવું (Painting) તેની માહિતી તૈયાર થાય છે.

* The CSS Cascade Algorithm

- Definition: જ્યારે એક જ એલિમેન્ટ પર એક કરતાં વધુ સીએસએસ નિયમો લાગુ પડે, ત્યારે કયો નિયમ ફાઇનલ ગણાશે તે નક્કી કરતી સિસ્ટમ.
- Selectors/Rules: Importance -> Origin -> Specificity -> Order of Appearance.
- Example: જો સરખા ક્લાસ હોય તો છેલ્લે લખેલો કોડ લાગુ પડશે.
- Output: એક જ ક્લર પ્રોપર્ટી માટે ફાઇનલ વિજેતા વેલ્યુ નક્કી થાય છે.

* Cascade Layers (@layer) Declaration

- Definition: સીએસએસ કોડિંગને અલગ અલગ સ્તરોમાં વહેંચવાની નવી રીત જેથી કોડની પોઝિશન ગમે ત્યાં હોય તો પણ લેયરના આધારે પ્રાયોરિટી નક્કી થાય.
- Syntax: @layer layer_name { selectors { property: value; } }
- Example: @layer base { body { color: black; } }
- Output: થર્ડ-પાર્ટી લાઇબ્રેરીના કોડને કસ્ટમ કોડ સાથે મિક્સ થતો અટકાવે છે.

* Nested Cascade Layers

- Definition: એક સીએસએસ લેયરની અંદર બીજું સબ-લેયર બનાવવાની પદ્ધતિ.
- Syntax: @layer parent { @layer child { p { color: blue; } } }
- Example: એડવાન્સ આર્કિટેક્ચર કોડિંગ મેનેજમેન્ટ માટે.
- Output: વધુ જટિલ સીએસએસ પ્રોજેક્ટ્સમાં સ્ટાઇલનું સચોટ આઇસોલેશન.

* Specificity Score Calculations (Inline vs ID vs Class vs Element)

- Definition: બ્રાઉઝર દ્વારા દરેક સિલેક્ટરને મળતા પોઇન્ટ્સ. જેનો સ્કોર વધારે હોય તેની ડિઝાઇન વેલ્યુ પેજ પર પાસ થાય છે.
- Scores: Inline = 1-0-0-0, ID = 0-1-0-0, Class/Pseudo-class = 0-0-1-0, Element = 0-0-0-1.
- Example: #myId (100 પોઇન્ટ) એ .myClass (10 પોઇન્ટ) કરતાં હંમેશા જીતશે.
- Output: ડિઝાઇન કન્ફ્લિક્ટ ઓટોમેટિક સોલ્વ થાય છે.

* Specificity Tie-Breaking Rules

- Definition: જ્યારે બે સીએસએસ નિયમોનો સ્કોર બિલકુલ સરખો હોય, ત્યારે સીએસએસ આલ્ગોરિથમ છેલ્લે લખેલા કોડને પ્રાયોરિટી આપે છે.
- Example: બે અલગ ક્લાસ સરખા સ્કોરના હોય તો ફાઇલમાં જે નીચે હશે તે લાગુ થશે.
- Output: છેલ્લો નિયમ જીતી જાય છે.

* The !important Exception Rule

- Definition: આ કીવર્ડ જે પણ પ્રોપર્ટીની પાછળ લખાય છે, તે તમામ નોર્મલ સ્પેસિફિકિટી સ્કોર તોડીને સૌથી ઉપર (ટોચ પર) આવી જાય છે.

- Syntax: selector { property: value !important; }

- Example: p { color: gold !important; }

- Output: સ્ક્રીન ગમે તેટલી ઓછી હોય તો પણ ગોલ્ડ કલર જ દેખાશે.

* CSS Rule Inheritance Mechanics

- Definition: પેરન્ટ (મુખ્ય) એલિમેન્ટની અમુક પ્રોપર્ટીઝ (જેમ કે font-family, color) તેના ચાઇલ્ડ એલિમેન્ટ્સમાં આપોઆપ આવી જવાની પ્રક્રિયા.

- Properties: color, font, line-height આપોઆપ ટ્રાન્સફર થાય છે (બોર્ડર કે માર્જિન થતા નથી).

- Output: ચાઇલ્ડ એલિમેન્ટ્સ પેરન્ટની થીમ આપોઆપ અપનાવે છે.

* inherit Control Value

- Definition: કોઈ પ્રોપર્ટી જે સામાન્ય રીતે ઇનહેરિટ નથી થતી, તેને જબરદસ્તી પેરન્ટ જેવી જ વેલ્યુ આપવા માટે આ વેલ્યુ વપરાય છે.

- Syntax: selector { property: inherit; }

- Example: .child-box { border: inherit; }

- Output: ચાઇલ્ડ બોક્સને પણ પેરન્ટ જેવી જ બોર્ડર મળી જશે.

* initial Control Value

- Definition: કોઈપણ સીએસએસ પ્રોપર્ટીને બ્રાઉઝરની મૂળ સત્તાવાર ડિફોલ્ટ વેલ્યુ (Default CSS Value) પર રીસેટ કરવા માટે.

- Syntax: selector { property: initial; }

- Example: p { color: initial; }

- Output: લખાણનો કલર બ્રાઉઝરના મૂળ કાળા કલરમાં બદલાઈ જશે.

* unset Control Value

- Definition: આ એક મિક્સ કંટ્રોલ છે. જો પ્રોપર્ટી કુદરતી રીતે ઇનહેરિટ થતી હોય તો તે પેરન્ટની કિંમત લેશે, નહીં તો તે ડિફોલ્ટ વેલ્યુ પર સેટ થઈ જશે.

- Syntax: selector { property: unset; }

- Example: div { color: unset; }

- Output: પ્રોપર્ટીના પ્રકાર પ્રમાણે આપોઆપ રીસેટ પ્રોસેસ.

* revert Control Value

- Definition: સ્ટાઇલને યુઝર કે બ્રાઉઝરની પોતાની ડિફોલ્ટ સ્ટાઇલશીટ (User Agent Stylesheet) ના લેવલ પર પાછી લઈ જવા માટે.

- Syntax: selector { property: revert; }

- Example: h1 { font-size: revert; }

- Output: h1 ની સાઇઝ બ્રાઉઝરની ઓરિજિનલ મોટી સાઇઝ જેવી થઈ જશે.

* revert-layer Control Value

- Definition: ચાલુ કેસેડ લેયર (@layer) ની અંદર કરેલી સ્ટાઇલને કેન્સલ કરી આગળના લેયરમાં જે વેલ્યુ હતી તેને પાછી લાવવા માટે.

- Syntax: selector { property: revert-layer; }

- Example: p { color: revert-layer; }

- Output: અગાઉના લેયરની સ્ટાઇલ આપોઆપ પાછી લાગુ થઈ જશે.

* The all Shorthand Property

- Definition: એલિમેન્ટ પર લાગેલી તમામ સીએસએસ પ્રોપર્ટીઝને એકસાથે રીસેટ કે બદલવા માટેનો મેજિકલ શોર્ટકટ.

- Syntax: selector { all: initial | inherit | unset | revert; }

- Example: .custom-widget { all: unset; }

- Output: બોક્સ પોતાની બધી ડિઝાઇન ગુમાવીને એકદમ સાદા લખાણ જેવું બની જશે.

Module 3: Advanced Selectors Masterclass

* Universal Selector (*)

- Definition: આખા વેબપેજના તમામ એલિમેન્ટ્સને એકસાથે સિલેક્ટ કરવા માટેનો ગ્લોબલ વાઇલ્ડકાર્ડ સિલેક્ટર.

- Syntax: * { property: value; }

- Example: * { margin: 0; padding: 0; }

- Output: પેજના બધા ટેગ્સની ડિફોલ્ટ સ્પેસ ઝીરો થઈ જશે.

* Element / Type Selector

- Definition: સીધા જ HTML ટેગના નામનો ઉપયોગ કરીને તે પ્રકારના બધા જ એલિમેન્ટ્સને ટાર્ગેટ કરવાની પદ્ધતિ.

- Syntax: tagname { property: value; }

- Example: h2 { color: purple; }

- Output: પેજના બધા જ h2 હેડિંગ્સ પર્પલ થઈ જશે.

* Class Selector (.class)

- Definition: HTML માં આપેલા કસ્ટમ ક્લાસ એટ્રિબ્યુટ પરથી એલિમેન્ટ્સ શોધવા માટે. તેની આગળ ડોટ (.) લગાવાય છે.
- Syntax: .classname { property: value; }
- Example: .blue-text { color: blue; }
- Output: જે ટેગમાં class="blue-text" હશે તે બ્લુ થઈ જશે.

* ID Selector (#id)

- Definition: પેજના કોઈ એક જ વિશિષ્ટ યુનિક એલિમેન્ટને તેની ID પરથી ડિઝાઇન આપવા માટે. તેની આગળ હેશ (#) લાગે છે.
- Syntax: #idname { property: value; }
- Example: #main-header { background: black; }
- Output: માત્ર id="main-header" વાળા બોક્સનું બેકગ્રાઉન્ડ બ્લેક થશે.

* Grouping Selector (Comma ,)

- Definition: કોડ ટૂંકો કરવા માટે અલગ-અલગ સિલેક્ટર્સને અલ્પવિરામથી જોડીને એકસરખી સ્ટાઇલ આપવાની રીત.
- Syntax: selector1, selector2 { property: value; }
- Example: h1, h2, p { text-align: center; }
- Output: ત્રણેય એલિમેન્ટનું લખાણ સ્ક્રીનની વચ્ચે આવી જશે.

* Descendant Combinator (Space)

- Definition: મુખ્ય પેરન્ટ એલિમેન્ટની અંદર ગમે તેટલા ઊંડાણમાં રહેલા (ચાઇલ્ડ કે ગ્રાન્ડચાઇલ્ડ) એલિમેન્ટને સિલેક્ટ કરવા માટે વચ્ચે સ્પેસ મુકાય છે.
- Syntax: parent child { property: value; }
- Example: div p { color: green; }
- Output: div ની અંદર ગમે ત્યાં આવેલા બધા પેરેગ્રાફ ગ્રીન થઈ જશે.

* Child Combinator (>)

- Definition: માત્ર અને માત્ર પેરન્ટ એલિમેન્ટના ડાયરેક્ટ (પહેલા લેવલના) ચાઇલ્ડ એલિમેન્ટને જ ટાર્ગેટ કરવા માટે આ ચિહ્ન વપરાય છે.
- Syntax: parent > child { property: value; }
- Example: ul > li { list-style: none; }
- Output: ul ના તાત્કાલિક પુત્ર સમાન li પર જ ડિઝાઇન લાગશે, અંદરના સબ-લિસ્ટ પર નહીં.

* Adjacent Sibling Combinator (+)

- Definition: કોઈ એલિમેન્ટની બરાબર પછી (તરત જ નીચે) આવતા પહેલા સગા ભાઈ જેવા એલિમેન્ટને સિલેક્ટ કરવા માટે પ્લસ ચિહ્ન વપરાય છે.
- Syntax: e1 + e2 { property: value; }
- Example: h1 + p { font-weight: bold; }
- Output: h1 પૂરો થતાં જ જે પહેલો પેરેગ્રાફ શરૂ થશે તે ઘાટો (Bold) થઈ જશે.

* General Sibling Combinator (~)

- Definition: કોઈ એલિમેન્ટની નીચે આવતા તે જ લેવલના તમામ સગા ભાઈઓ (Siblings) ને એકસાથે સિલેક્ટ કરવા માટે ટીલ્ડ (~) ચિહ્ન વપરાય છે.
- Syntax: e1 ~ e2 { property: value; }
- Example: h1 ~ p { color: gray; }
- Output: h1 ની નીચે આવેલા તમામ પેરેગ્રાફ્સ ગ્રે કલરના થઈ જશે.

* Basic Attribute Selector ([attr])

- Definition: HTML ટેગની અંદર કોઈ ચોક્કસ એટ્રિબ્યુટ હાજર છે કે નહીં તેના આધારે ડિઝાઇન આપવાની ટેકનિક.
- Syntax: [attribute] { property: value; }
- Example: [required] { border: 2px solid red; }
- Output: જે ઇનપુટ બોક્સમાં required લખેલું હશે તેની બોર્ડર લાલ થઈ જશે.

* Exact Value Attribute Selector ([attr="value"])

- Definition: એટ્રિબ્યુટની કિંમત બિલકુલ સચોટ મેચ થાય તો જ સ્ટાઇલ આપવા માટે.
- Syntax: [attribute="exact_value"] { property: value; }
- Example: [type="password"] { background: lightyellow; }
- Output: માત્ર પાસવર્ડ વાળા ઇનપુટ બોક્સ જ પીળા બેકગ્રાઉન્ડના બનશે.

* Substring Match Attribute Begins With ([attr^="value"])

- Definition: એટ્રિબ્યુટની વેલ્યુની શરૂઆત કયા ચોક્કસ શબ્દથી થાય છે તેના આધારે એલિમેન્ટ શોધવા માટે (^=).
- Syntax: [attribute^="start_string"] { property: value; }
- Example: a[href^="https"] { color: darkgreen; }
- Output: સિક્યોર વેબસાઇટની લિંક્સ (https વાળી) ગ્રીન કલરની દેખાશે.

- * Substring Match Attribute Ends With ([attr\$="value"])
 - Definition: એટ્રિબ્યુટની વેલ્યુનો અંત કયા ચોક્કસ અક્ષરો કે એક્સટેન્શનથી થાય છે તે ચેક કરવા માટે (`\$=`).
 - Syntax: [attribute\$="end_string"] { property: value; }
 - Example: a[href\$=".pdf"] { color: red; }
 - Output: જે લિંક પર ક્લિક કરવાથી PDF ખુલવાની હશે તે લાલ થઈ જશે.
- * Substring Match Attribute Contains ([attr*="value"])
 - Definition: એટ્રિબ્યુટની વેલ્યુની અંદર વચ્ચે ગમે ત્યાં તે શબ્દ આવતો હોય તો પણ સિલેક્ટ કરવા માટે (`\*=`).
 - Syntax: [attribute*="sub_string"] { property: value; }
 - Example: [class*="btn-"] { border-radius: 4px; }
 - Output: જે ક્લાસના નામમાં વચ્ચે ક્યાંય પણ btn- આવતું હશે તે બધા ગોળ ખૂણાવાળા બનશે.
- * Space-Separated List Attribute Selector ([attr~="value"])
 - Definition: જો એટ્રિબ્યુટમાં સ્પેસ છોડીને ઘણા શબ્દો લખ્યા હોય, તેમાંથી કોઈ એક શબ્દ મેચ કરવો હોય ત્યારે વપરાય છે.
 - Syntax: [attribute~="word"] { property: value; }
 - Example: [title~="important"] { color: darkred; }
 - Output: લિસ્ટમાંથી ઇમ્પોર્ટન્ટ શબ્દ ધરાવતા ટાઇટલ મેચ થશે.
- * Hyphen-Separated Prefix Attribute Selector ([attr|= "value"])
 - Definition: હાઇફન (-) થી જોડાયેલા નામો શોધવા, સામાન્ય રીતે લેંગ્વેજ કોડ (જેમ કે lang="en-US") માટે વપરાય છે.
 - Syntax: [attribute|= "value"] { property: value; }
 - Example: [lang|= "en"] { font-family: sans-serif; }
 - Output: en થી શરૂ થતી તમામ લેંગ્વેજ પ્રોપર્ટી સેટ થશે.
- * Attribute Case-Insensitivity Modifiers (i, s)
 - Definition: એટ્રિબ્યુટની વેલ્યુ કેપિટલ છે કે સ્મોલ તે જોયા વગર મેચ કરવા માટે અંતમાં `i` મુકાય છે, અને કેસ ચેક કરવા `s`.
 - Syntax: [attr="value" i]
 - Example: [href\$=".png" i] { border: 1px solid blue; }
 - Output: .PNG કે .png બંને પ્રકારના ફોટા સિલેક્ટ થઈ જશે.
- * Functional Pseudo-Class :has() (The Parent Selector)
 - Definition: આ નવો સુપર પાવરફુલ સિલેક્ટર છે. જે ચાઇલ્ડની કિંમતના આધારે તેના પેરન્ટ (મુખ્ય બોક્સ) ને સિલેક્ટ કરે છે.
 - Syntax: selector:has(child_selector) { property: value; }
 - Example: .card:has(.btn-danger) { border: 2px solid red; }
 - Output: જે કાર્ડની અંદર ડેન્જર બટન હશે, તે આખા કાર્ડની બોર્ડર લાલ થઈ જશે.
- * Functional Pseudo-Class :is()
 - Definition: કોડ લંબાય નહીં તે માટે ઘણા લાંબા સિલેક્ટર્સ ગ્રુપને એક જ ફંક્શનની અંદર ટૂંકમાં લખવા માટે.
 - Syntax: :is(h1, h2, h3) hover { color: orange; }
 - Example: ત્રણેય હેડિંગ્સ પર હોવર ઇફેક્ટ સેટ કરવા માટે.
 - Output: કોડ એકદમ ક્લીન અને નાનો બને છે.
- * Functional Pseudo-Class :where()
 - Definition: આ બિલકુલ `:is()` જેવું જ કામ કરે છે, પણ આની પોતાની સ્પેસિફિસિટી સ્કોર હંમેશા ઝીરો (0) રહે છે, જેથી ભવિષ્યમાં તેને ઓવરરાઇડ કરવું સાવ સરળ બને છે.
 - Syntax: :where(header, footer) a { text-decoration: none; }
 - Output: હેડર અને ફૂટરની લિંકની અંડરલાઇન હટી જશે, ઝીરો સ્કોર સાથે.
- * Functional Pseudo-Class :not()
 - Definition: નેગેટિવ સિલેક્ટર. આ લિસ્ટમાંથી જે એલિમેન્ટ મેચ ન થતો હોય તેને છોડીને બાકીના બધાને સ્ટાઇલ આપે છે.
 - Syntax: selector:not(.classname) { property: value; }
 - Example: p:not(.special) { color: gray; }
 - Output: special ક્લાસ વગરના તમામ સામાન્ય પેરેગ્રાફ ગ્રે થઈ જશે.
- * User Action Pseudo-Classes
 - Definition: યુઝર જ્યારે પેજ પર કોઈ એક્શન કરે ત્યારે ડિઝાઇનમાં થતો ફેરફાર.
 - Selectors: `:hover` (માઉસ લાવતા), `:focus` (ક્લિક કરતા ઇનપુટમાં), `:active` (બટન દબાવી રાખતા ક્ષણે), `:focus-within` (અંદરના કોઈ ભાગમાં ફોકસ થતા).
 - Example: button:hover { background: blue; }
 - Output: માઉસ બટન પર જતાં જ તે વાદળી રંગનું થઈ જશે.

* Structural Pseudo-Classes

- Definition: HTML સ્ટ્રક્ચરના આધારે કોઈ ચોક્કસ નંબર કે પોઝિશનના યાઇલ્ડને શોધવા માટે.
- Selectors: `:nth-child(n)` (n મો નંબર જેમ કે 2n એટલે બેકી સંખ્યા), `:first-child` (પહેલો પુત્ર), `:last-child` (છેલ્લો પુત્ર).
- Example: `li:nth-child(odd) { background: #eee; }`
- Output: વિસ્ટની એકી સંખ્યા વાળી આઇટમ્સ ઓ પટ્ટી વાળી દેખાશે.

* Typed Structural Pseudo-Classes

- Definition: આ ટેગના પ્રકાર (Type) ને ધ્યાનમાં રાખીને ચોક્કસ સ્થાનના એલિમેન્ટને ટાર્ગેટ કરે છે.
- Selectors: `:nth-of-type()`, `:first-of-type`, `:last-of-type`.
- Example: `p:first-of-type { font-size: 20px; }`
- Output: પેજ પર જે પહેલો પેરેગ્રાફ હશે તેની સાઇઝ મોટી થશે, ભલે તેની ઉપર હેડિંગ કેમ ન હોય.

* Target and Anchor Pseudo-Classes

- Definition: URL ની પાછળ આવતા હેશ આઈડી આઈડેન્ટીફાયર (`#section1`) ના આધારે એક્ટિવ થયેલા ભાગને સજાવવા.
- Selectors: `:target`, `:root` (આખું પેજ), `:scope`.
- Example: `:target { background: lightgreen; }`
- Output: જે સેક્શન પર જમ્પ કરીને યુઝર આવશે તે હાઇલાઇટ થઈ જશે.

* Form State Pseudo-Classes

- Definition: ફોર્મ અને ઇનપુટ બોક્સની વાઇવ કન્ડિશન (સ્ટેટ) ચેક કરવા માટે.
- Selectors: `:enabled`, `:disabled` (બ્લોક કરેલું), `:checked` (ટીક કરેલું), `:invalid` (ખોટો ડેટા).
- Example: `input:disabled { opacity: 0.5; }`
- Output: જે બોક્સ બંધ હશે તે આછું દેખાશે.

* Pseudo-Elements (::before, ::after)

- Definition: HTML કોડમાં લખ્યા વગર જ સીએસએસ દ્વારા એલિમેન્ટની બિલકુલ આગળ કે પાછળ નવું નકલી કન્ટેન્ટ કે ડિઝાઇન ઉમેરવા માટે.
- Syntax: `selector::before { content: "text"; }`
- Example: `.price::before { content: "$"; }`
- Output: કિંમતની આગળ આપોઆપ ડોલરનું ચિહ્ન આવી જશે.

* Typography Pseudo-Elements

- Definition: લખાણની પહેલી વાઇન અથવા પહેલા અક્ષરને ન્યૂઝપેપર સ્ટાઇલમાં સજાવવા.
- Selectors: `::first-line`, `::first-letter`.
- Example: `p::first-letter { font-size: 30px; font-weight: bold; }`
- Output: ફક્તનો પહેલો અક્ષર મોટો અને ઘાટો દેખાશે.

* UI Pseudo-Elements

- Definition: યુઝર ઇન્ટરફેસના ચોક્કસ ભાગો જેમ કે માઉસથી સિલેક્ટ કરેલું લખાણ સજાવવા.
- Selectors: `::selection`, `::placeholder` (ઇનપુટની અંદરનું આછું લખાણ).
- Example: `::selection { background: pink; }`
- Output: માઉસથી લખાણ સિલેક્ટ કરતાં જ તે પિંક બેકગ્રાઉન્ડ સાથે હાઇલાઇટ થશે.

Module 4 & 5: Box Model, Sizing & Typography

* Standard Box Model Mechanics

- Definition: સીએસએસની જૂની ડિફોલ્ટ પદ્ધતિ જેમાં એલિમેન્ટની વિડ્થ આપ્યા પછી પેડિંગ અને બોર્ડર ઉમેરવાથી બોક્સની કુલ સાઇઝ મોટી થઈ જાય છે.
- Syntax: `box-sizing: content-box;`
- Output: લેઆઉટ ઘણીવાર સ્ક્રીનની બહાર નીકળી જાય છે કે તૂટી જાય છે.

* Alternative Box Model (box-sizing: border-box)

- Definition: આધુનિક બેસ્ટ પદ્ધતિ જેમાં પેડિંગ અને બોર્ડર આપમેળે નક્કી કરેલી પહોળાઈ (width) ની અંદર જ સમાઈ જાય છે.
- Syntax: `* { box-sizing: border-box; }`
- Output: બોક્સની કુલ સાઇઝ આપેલા માપથી ક્યારેય વધતી નથી.

* Content-Box vs Border-Box Calculations

- Definition: ગણતરીનો તફાવત. content-box માં કુલ વિડ્થ = width + padding + border થાય છે, જ્યારે border-box માં કુલ વિડ્થ = width જ રહે છે.
- Output: border-box સયોટ ગણતરી અને રિસ્પોન્સિવ ડિઝાઇન માટે બેસ્ટ છે.

* Margin Collapse Mechanics

- Definition: જ્યારે બે બ્લોક એલિમેન્ટ્સ ઉપર-નીચે ગોઠવાય છે, ત્યારે તેમની માર્જિન પ્લસ થવાને બદલે જેની માર્જિન મોટી હોય એટલી જ જગ્યા વચ્ચે છૂટે છે.
- Output: બે બ્લોક વચ્ચે અપેક્ષા કરતાં ઓછી જગ્યા દેખાય છે.

* Padding and Border Interaction

- Definition: પેડિંગ હંમેશા બોર્ડરની અંદરની તરફ કન્ટેન્ટને દબાવે છે અને બોર્ડર તેની કિનારી નક્કી કરે છે.
- Output: બોક્સનો અંદરનો લુક વ્યવસ્થિત થાય છે.

* Extrinsically Sizing

- Definition: યુઝર દ્વારા કમ્પ્યુટરને પિક્સલ કે ફિક્સ વેલ્યુ આપીને એલિમેન્ટની સાઇઝ બહારથી નક્કી કરવાની પ્રક્રિયા.
- Properties: `width`, `height`, `max-width`, `min-width`.
- Example: width: 400px;
- Output: એલિમેન્ટ કન્ટેન્ટ ગમે તેટલો હોય પણ 400px નો જ બનશે.

* Intrinsic Sizing max-content

- Definition: કન્ટેન્ટ આધારિત સાઇઝિંગ. આ બોક્સને લાઇન તોડ્યા વગર આખું લખાણ સમાવવા માટે જેટલી આડી પહોળાઈ જોઈએ એટલું લાંબુ કરી દે છે.
- Syntax: width: max-content;
- Output: વાંચ્ય ગમે તેટલું લાંબુ હશે, એક જ લાઇનમાં સીધું સેટ થઈ જશે.

* Intrinsic Sizing min-content

- Definition: આ બોક્સને સૌથી લાંબા સિંગલ શબ્દ જેટલું સાંકડું કરી દે છે અને બાકીના બધા શબ્દો નવી લાઇનમાં મોકલે છે.
- Syntax: width: min-content;
- Output: બોક્સ એકદમ ઊભી પટ્ટી જેવું સાંકડું બની જશે.

* Intrinsic Sizing fit-content / fit-content()

- Definition: સ્માર્ટ સાઇઝિંગ. સામાન્ય રીતે આ max-content જેવું ખુલે છે પણ સ્ક્રીનની બોર્ડર આવતા જ તે આપોઆપ નોર્મલ ટેક્સ્ટની જેમ નીચે વળી જાય છે.
- Syntax: width: fit-content;
- Output: લખાણ નાનું હોય તો બોક્સ નાનું, મોટું હોય તો સ્ક્રીન પ્રમાણે સેટ.

* Absolute Length Units (px, cm, mm, in, pt, pc)

- Definition: સ્ક્રીનની સાઇઝ ગમે તેવી હોય, આ એકમોનું માપ ક્યારેય બદલાતું નથી, તે હંમેશા સ્થિર (Fixed) રહે છે.
- Example: width: 200px;
- Output: દરેક ડિવાઇસ પર બિલકુલ ૨૦૦ પિક્સલ જ રોકશે.

* Font-Relative Length Units (em, rem, ex, ch)

- Definition: ફોન્ટની સાઇઝના આધારે કદ બદલતા ડાયનેમિક એકમો. `rem` એ આખા પેજના રૂટ (html) ફોન્ટ સાઇઝ (16px) ના ગુણાંકમાં કામ કરે છે.
- Example: font-size: 2rem; (એટલે કે 16 x 2 = 32px).
- Output: યુઝર બ્રાઉઝરના ફોન્ટ મોટા કરશે તો આખી ડિઝાઇન આપોઆપ મોટી થશે.

* Viewport Linear Units (vw, vh, vmin, vmax)

- Definition: ડિવાઇસની સ્ક્રીનની ઊંચાઈ અને પહોળાઈ ટકાવારીના આધારે કામ કરતા એકમો.
- Details: `100vw` = આખી સ્ક્રીનની પહોળાઈ, `100vh` = આખી સ્ક્રીનની ઊંચાઈ.
- Output: પેજ કમ્પ્યુટર કે મોબાઇલ સ્ક્રીન બરાબર ફિટ થઈ જાય છે.

* Modern Large, Small, and Dynamic Viewport Units (lvh, svh, dvh)

- Definition: મોબાઇલ બ્રાઉઝરમાં જ્યારે નીચેથી એડ્રેસ બાર કે કીબોર્ડ ખુલે કે બંધ થાય ત્યારે સ્ક્રીનની સાઇઝ બદલાય છે તેને કન્ટ્રોલ કરવા માટેના આધુનિક એકમો.
- Details: `svh` (નાની સ્ક્રીન), `lvh` (મોટી સ્ક્રીન), `dvh` (ડાયનેમિકલી આપોઆપ એડજસ્ટ થતી ઊંચાઈ).
- Output: મોબાઇલ લેઆઉટ અકસ્માતે કટ કે બગડતો નથી.

* Container Query Length Units (cqww, cqhh, cqiw, cqhb)

- Definition: આખી સ્ક્રીનને બદલે તેના પેરન્ટ કન્ટેનર (બોક્સ) ની વિડ્થના આધારે અંદરની વસ્તુનું માપ નક્કી કરવા માટેના એકમો.
- Example: font-size: 5cqww;
- Output: બોક્સ મોટું થશે તેમ અંદરના અક્ષરો પણ આપોઆપ મોટા થશે.

* Custom Web Fonts Integration (@font-face)

- Definition: કમ્પ્યુટરમાં ન હોય તેવા બહારના સુંદર કસ્ટમ ફોન્ટ્સને ડાઉનલોડ કરી વેબસાઇટમાં લિંક કરવાની ઓફિશિયલ રીત.

- Syntax: `@font-face { font-family: 'MyFont'; src: url('font.woff2'); }`
- Output: વેબસાઇટ પર યુનિક બ્રાન્ડેડ ફોન્ટ દેખાય છે.

* Font Display Timeline Control (font-display: swap)

- Definition: ઇન્ટરનેટ ધીમું હોય ત્યારે કસ્ટમ ફોન્ટ લોડ ન થાય ત્યાં સુધી બ્રાઉઝરને નોર્મલ ફોન્ટ બતાવવાની સૂચના આપવી જેથી પેજ ખાલી ન દેખાય.
- Syntax: `font-display: swap;`
- Output: વેબસાઇટ ફાસ્ટ લોડ થાય છે અને ટેક્સ્ટ તરત દેખાય છે.

* Text Overflow Controls

- Definition: જ્યારે લાંબુ લખાણ બોક્સ તોડીને બહાર ભાગે ત્યારે તેને કન્ટ્રોલ કરવાની પદ્ધતિ.
- Properties: ``white-space: nowrap;` (લાઇન ન તોડવી), ``overflow: hidden;`, ``text-overflow: ellipsis;` (પાછળ ત્રણ ટપકા કરવા).
- Output: લખાણ કટ થઈને પાછળ સુંદર ત્રણ ટપકા (...) દેખાશે.

● Module 6: Positioning & Stacking Context

* position: static Mechanics

- Definition: સીએસએસની નોર્મલ ડિફોલ્ટ પોઝિશન. આમાં કોઈ એલિમેન્ટ પોતાની નોર્મલ જગ્યા છોડતો નથી અને તેની પર top, left પ્રોપર્ટી કામ કરતી નથી.

* position: relative Mechanics

- Definition: આ પોઝિશન આપવાથી એલિમેન્ટ પોતાની મૂળ જગ્યા પર જ રહે છે, પણ તેને `top` કે `left` આપીને મૂળ જગ્યાથી સહેજ આગળ-પાછળ સરકાવી શકાય છે.
- Example: `position: relative; left: 10px;`
- Output: એલિમેન્ટ પોતાની ઓરિજિનલ જગ્યાથી 10 પિક્સલ જમણી તરફ ખસશે.

* position: absolute Layout Behavior

- Definition: આ એલિમેન્ટ નોર્મલ પેજ ફ્લોમાંથી સાવ મુક્ત થઈ હવામાં આવી જાય છે અને પોતાના પેરન્ટ (જે relative હોય) તેના ખૂણાના આધારે સેટ થાય છે.
- Example: `position: absolute; top: 0; right: 0;`
- Output: બોક્સ પેરન્ટના જમણા ખૂણામાં બિલકુલ ઉપર ફિટ થઈ જશે.

* position: fixed Viewport Binding

- Definition: આ એલિમેન્ટ આખા બ્રાઉઝરની સ્ક્રીન (Viewport) સાથે ચોટી જાય છે. પેજ ગમે તેટલું સ્ક્રોલ કરો, આ પોતાની જગ્યા ક્યારેય છોડતું નથી.
- Example: `position: fixed; bottom: 0;`
- Output: બોટમ બાર કે ચેટ આઇકોન સ્ક્રીન પર નીચે હંમેશા ફિક્સ દેખાશે.

* position: sticky Scroll Threshold Mechanics

- Definition: આ એક સુપર હાઇબ્રિડ પોઝિશન છે. સામાન્ય રીતે આ નોર્મલ સ્ટીકરની જેમ વર્તે છે, પણ સ્ક્રોલ કરતાં કરતાં જ્યારે તેની શરત (જેમ કે top: 0) આવે ત્યારે તે fixed ની જેમ ચોટી જાય છે.
- Example: `position: sticky; top: 0;`
- Output: પેજ હેડર સ્ક્રોલ કરતાં કરતાં ઉપર આવીને આપોઆપ લોક થઈ જશે.

* Modern CSS Logical Placement Properties (inset)

- Definition: ``top`, `right`, `bottom`, `left`` ચારેય અલગ લખવાને બદલે એક જ લાઇનમાં શોર્ટકટથી પોઝિશનના માપ આપવાની આધુનિક રીત.
- Syntax: `inset: top right bottom left;` (અથવા `inset: 0` એટલે ચારેય બાજુ 0).
- Output: કોડ ઓછો લખવો પડે છે.

* The z-index Property Structure

- Definition: પોઝિશન આપેલા એલિમેન્ટ્સ જ્યારે એકબીજાની ઉપર-નીચે ગોઠવાય છે, ત્યારે કયો લેયર સૌથી ઉપર દેખાશે તેનો નંબર નક્કી કરવા.
- Syntax: `z-index: number;` (મોટો નંબર ઉપર દેખાશે).
- Example: `z-index: 999;`
- Output: પોપ-અપ બોક્સ બાકીના બધા કન્ટેન્ટની ઉપર સક્સેસફુલી આવી જશે.

* Stacking Context Creation Triggers

- Definition: બ્રાઉઝર બેકગ્રાઉન્ડમાં ક્યારે નવો 3D લેયર મેપ બનાવે છે તેના નિયમો. position + z-index અથવા `opacity`, `transform` આપવાથી પણ નવો સ્ટેકિંગ કોન્ટેક્સ્ટ બને છે.
- Output: લેયર્સનું મેનેજમેન્ટ આઇસોલેટ થાય છે.

Module 7: Master Flexbox Layout Engine

* **display: flex Framework**

- Definition: વન-ડાયમેન્શનલ (આડી અથવા ઊભી લાઇન) લેઆઉટ બનાવવા માટેનું સૌથી શક્તિશાળી એન્જિન. આ લખતા જ અંદરની બધી વસ્તુઓ લાઇનમાં આવી જાય છે.
- Syntax: `.container { display: flex; }`
- Output: બધા યાઇલ્ડ બોક્સ (Flex Items) આડા પટ્ટામાં એકસાથે ગોઠવાઈ જશે.

* **Main Axis and Cross Axis Mechanics**

- Definition: ફ્લેક્સબોક્સની બે મુખ્ય દિશાઓ. `flex-direction: row` હોય ત્યારે આડી લાઇન મેઇન એક્સિસ બને અને ઊભી લાઇન ક્રોસ એક્સિસ બને છે.
- Output: બધી અલાઇનમેન્ટ પ્રોપર્ટીઝ આ બે ધરી પર જ કામ કરે છે.

* **Flex Container: flex-direction**

- Definition: અંદરની આઇટમ્સને આડી લાઇનમાં રાખવી કે ઊભી લાઇનમાં તે નક્કી કરવાનો પેરન્ટ ક્વાસ.
- Values: `row` (ડિફોલ્ટ આડું), `column` (ઊભું), `row-reverse`, `column-reverse`.
- Example: `flex-direction: column;`
- Output: બધી આઇટમ્સ એક નીચે એક ઊભી ગોઠવાઈ જશે.

* **Flex Container: flex-wrap**

- Definition: જો સ્ક્રીન નાની થાય અને આઇટમ્સ સમાય તેમ ન હોય, તો તેને તોડીને આપોઆપ નવી લાઇનમાં મોકલવી કે નહીં તે નક્કી કરવા.
- Values: `nowrap` (એક જ લાઇનમાં સંકોચાઈ જશે), `wrap` (નવી લાઇન લેશે).
- Example: `flex-wrap: wrap;`
- Output: રિસ્પોન્સિવ કાર્ડ લેઆઉટ આપોઆપ નીચે શિફ્ટ થાય છે.

* **justify-content**

- Definition: મેઇન એક્સિસ (આડી લાઇન) પર આઇટમ્સ વચ્ચે ખાલી જગ્યાનું ડિસ્ટ્રીબ્યુશન કઈ રીતે કરવું.
- Values: `flex-start`, `flex-end`, `center`, `space-between` (વચ્ચે જગ્યા), `space-around` (આજુબાજુ જગ્યા).
- Example: `justify-content: space-between;`
- Output: પહેલી આઇટમ ડાબે ચોટશે, છેલ્લી જમણે અને બાકીની વચ્ચે સરખી સ્પેસ મેળવશે.

* **align-items**

- Definition: ક્રોસ એક્સિસ (ઊભી લાઇન) પર સિંગલ લાઇનની આઇટમ્સને ઉપર, નીચે કે સેન્ટરમાં કઈ રીતે લાઇનઅપ કરવી.
- Values: `flex-start` (ટોપ), `flex-end` (બોટમ), `center` (વચ્ચે), `stretch` (આખી ઊંચાઈ રોકવી).
- Example: `align-items: center;`
- Output: નાની-મોટી ઊંચાઈ વાળા બધા બોક્સ ઊભી દિશામાં બરાબર સેન્ટર થઈ જશે.

* **gap**

- Definition: માર્જિન આપ્યા વગર ફ્લેક્સ આઇટમ્સ વચ્ચે આપમેળે ફિક્સ જગ્યા (Spacing) છોડવા માટેનો આધુનિક ક્વાસ.
- Syntax: `gap: 20px;`
- Output: દરેક બોક્સ વચ્ચે બરાબર ૨૦ પિક્સલની ગેપ ઓટોમેટિક સેટ થઈ જશે.

* **flex-grow, flex-shrink, and flex-basis**

- Definition: યાઇલ્ડ આઇટમ્સ પોતાની સાઇઝ કઈ રીતે વધારશે કે ઘટાડશે તેનું ગણિત.
- Details: `flex-grow: 1` (વધારેલી બધી ખાલી જગ્યા પોતે રોકી લેવી), `flex-shrink` (સંકોચાવવાની સ્પીડ), `flex-basis` (શરૂઆતની વિડ્થ).
- Example: `flex: 1 1 auto;` (Shorthand).
- Output: બોક્સ સ્ક્રીનની સાઇઝ પ્રમાણે ઓટોમેટિક નાનું-મોટું થયા કરે છે.

Module 8: Master CSS Grid Layout Engine

* **display: grid Architecture**

- Definition: ટુ-ડાયમેન્શનલ (એકસાથે રો અને કોલમ બંને) લેઆઉટ બનાવવા માટેનું માસ્ટર એન્જિન. આ આખી સ્ક્રીનને ગ્રાફ પેપરની જેમ ખાનાઓમાં વહેંચી દે છે.
- Syntax: `.grid-box { display: grid; }`

* **Explicit Grid Definition (grid-template-columns)**

- Definition: આપણે જાતે જ નક્કી કરવું કે ગ્રીડમાં કેટલી ઊભી કોલમ રાખવી છે અને દરેક કોલમની પહોળાઈ કેટલી હશે.
- Syntax: `grid-template-columns: 1fr 1fr 1fr;` (સરખા ત્રણ ભાગ).
- Output: આખા પેજ પર આપોઆપ ત્રણ સરખી કોલમનું લેઆઉટ બની જશે.

* **Grid Track Repeat Layouts: repeat()**

- Definition: જો સરખા માપની ઘણી બધી કોલમ બનાવવી હોય, તો કોડ વારંવાર ન લખવો પડે તે માટેનું શોર્ટ ફંક્શન.
- Syntax: `grid-template-columns: repeat(4, 1fr);`
- Output: એકસરખા માપની ૪ કોલમ ઓટોમેટિક ક્રિએટ થઈ જશે.

* Dynamic Sizing Boundaries: minmax()

- Definition: ગ્રીડના ખાનાની સાઇઝ ઓછામાં ઓછી અને વધુમાં વધુ કેટલી રાખી શકાય તેની ફ્લેક્સિબલ લિમિટ સેટ કરવા.
- Syntax: `grid-template-columns: repeat(3, minmax(200px, 1fr));`
- Output: ખાનું ક્યારેય 200px થી નાનું નહીં થાય, પણ જગ્યા હશે તો 1fr જેટલું મોટું થઈ જશે.

* Auto-Placement Engine: auto-fit and auto-fill

- Definition: જાવાસ્ક્રિપ્ટ કે મીડિયા ક્વેરી વગર આખા ગ્રીડ લેઆઉટને સંપૂર્ણ રિસ્પોન્સિવ બનાવવાની અદ્યતન ટેકનિક.
- Syntax: `grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));`
- Output: સ્ક્રીન નાની-મોટી થતા જ બોક્સ આપોઆપ નવી લાઇનમાં શિફ્ટ થઈને સ્ક્રીન સાઇઝ પ્રમાણે એડજસ્ટ થઈ જશે.

* Advanced Nested Component Layouts: Subgrid

- Definition: ગ્રીડની અંદર રહેલા ચાઇલ્ડ બોક્સની અંદર પણ જો ગ્રીડ બનાવવી હોય, અને તે તેના મેઇન પેરન્ટ ગ્રીડની લાઇન સાથે જ બિલકુલ કનેક્ટેડ રહે તે માટેની અલ્ટ્રા-એડવાન્સ ટેકનિક.
- Syntax: `grid-template-columns: subgrid;`
- Output: અંદરના કમ્પોનન્ટ્સ પણ મેઇન ગ્રીડના ખાના સાથે ૧૦૦% પર્ફેક્ટ લાઇનઅપમાં સેટ થાય છે.

● Module 9 & 10: Visuals, Graphics, Color Spaces & Masking

* Modern Color Spaces (oklch)

- Definition: માનવ આંખ કલર્સને કઈ રીતે જુએ છે તેના પર બનેલો આધુનિક કલર સ્પેસ, જે સામાન્ય હેક્સ કોડ કરતાં કરોડો નવા અને બ્રાઇટ કલર્સ સપોર્ટ કરે છે.
- Syntax: `color: oklch(Lightness Chroma Hue);`
- Example: `color: oklch(0.6 0.25 140);`
- Output: આધુનિક ડિસ્પ્લે સ્ક્રીન પર અત્યંત આકર્ષક અને સચોટ કલર ટોન દેખાય છે.

* The color-mix() Function Mechanics

- Definition: કોઈપણ ત્રીજી ફાઇલ કે પ્રોગ્રામ વગર સીએસએસ કોડની અંદર જ બે અલગ અલગ કલર્સને ચોક્કસ ટકાવારીમાં મિક્સ કરીને નવો કલર બનાવવો.
- Syntax: `background: color-mix(in oklab, blue 60%, yellow);`
- Output: વાદળી અને પીળા કલરનું મિશ્રણ થઈને નવો શેડ આપોઆપ જનરેટ થશે.

* The filter Property Functions

- Definition: ફોટોશોપ સોફ્ટવેરની જેમ ઇમેજ કે એલિમેન્ટ પર ડાયરેક્ટ ફિલ્ટર ઇફેક્ટ્સ એપ્લાય કરવાની રીત.
- Values: ``blur()`` (ઝાંખું કરવું), ``grayscale()`` (બ્લેક એન્ડ વ્હાઇટ), ``brightness()``, ``drop-shadow()``.
- Example: `filter: blur(5px) grayscale(100%);`
- Output: ફોટો આખો બ્લેક એન્ડ વ્હાઇટ અને બ્લર થઈ જશે.

* Backdrop Filtering Layout (backdrop-filter)

- Definition: એલિમેન્ટની પોતાની પાછળ જે બેકગ્રાઉન્ડ દેખાય છે, તેને બ્લર કે ફિલ્ટર કરવાની ટેકનિક. આનાથી ગ્લાસમોર્ફિઝમ ડિઝાઇન બને છે.
- Syntax: `backdrop-filter: blur(10px); background: rgba(255,255,255,0.3);`
- Output: એલિમેન્ટની પાછળનો ભાગ કાય (Frosted Glass) જેવો ધૂંધળો આકર્ષક દેખાશે.

* The clip-path Geometric Shapes

- Definition: કોઈપણ લંબચોરસ બોક્સ કે ફોટાને ચોક્કસ ભૂમિતિના આકારો (વર્તુળ, ત્રિકોણ, સ્ટાર કે પોલીગોન) માં કાપી નાખવા (Clip કરવા) માટે.
- Syntax: `clip-path: polygon(50% 0%, 0% 100%, 100% 100%);` (ત્રિકોણ આકાર).
- Output: તમારો સામાન્ય ચોરસ ફોટો આપોઆપ ત્રિકોણ આકારમાં બદલાઈ જશે.

* The shape-outside Layout Routing

- Definition: લખાણને સામાન્ય સીધી લાઇનમાં રાખવાને બદલે કોઈ ગોળાકાર કે ત્રિકોણ ફોટાની વળાંક વાળી બોર્ડરની ફરતે આપોઆપ વહેંચવાની (wrap કરવાની) ટેકનિક.
- Syntax: `shape-outside: circle(50%);`
- Output: કન્ટેન્ટ ટેક્સ્ટ ફોટાના વળાંક પ્રમાણે આજુબાજુ ગોઠવાઈ જશે.

● Module 11 & 12: Animations, Logic & Math Functions

* 3D Transform Matrix Mechanics

- Definition: એલિમેન્ટને માત્ર આડું-ઊભું જ નહીં પણ 3D સ્પેસમાં (Z-axis ઊંડાણ સાથે) ફેરવવા, સરકાવવા કે

ત્રિપરિમાણીય લુક આપવા માટે.

- Properties: `transform: translate3d(x, y, z) rotateX(45deg) perspective(500px);``
- Output: બોક્સ સ્ક્રીનની બહાર નીકળતું હોય કે અંદર જતું હોય તેવો રિયલ 3D ભાસ થાય છે.

* Custom Timing Bezier Curves (cubic-bezier)

- Definition: એનિમેશન કે ટ્રાન્ઝિશનની સ્પીડ ક્યારે ફાસ્ટ થશે અને ક્યારે સ્લો થશે તેનો ગ્રાફ જાતે કસ્ટમાઇઝ કરવાની એડવાન્સ રીત.

- Syntax: `transition-timing-function: cubic-bezier(0.25, 1, 0.5, 1);`
- Output: એનિમેશન એકદમ સ્મૂથ, બાઉન્સિંગ અથવા પ્રોફેશનલ બને છે.

* Multi-Step Keyframe Architecture (@keyframes)

- Definition: એનિમેશનની શરૂઆતથી અંત વચ્ચે અલગ-અલગ ટકાવારી (0% થી 100%) પર અલગ-અલગ ડિઝાઇન ચેન્જ સેટ કરીને મોટો ગ્રાફ બનાવવો.

- Syntax: `@keyframes myAnim { 0% { top:0; } 50% { top:50px; } 100% { top:0; } }`
- Output: એલિમેન્ટ સતત કસ્ટમ ઓટોમેટેડ હલનચલન કરતો રહેશે.

* Fluid Layout Range Interpolation (clamp)

- Definition: રિસ્પોન્સિવ ટેક્સ્ટ કે સાઇઝ માટેનું જાદુઈ ફંક્શન. આમાં એક મિનિમમ વેલ્યુ, એક સેન્ટર ડાયનેમિક વેલ્યુ અને એક મેક્સિમમ લિમિટ વેલ્યુ સેટ થાય છે.

- Syntax: `font-size: clamp(1rem, 5vw, 3rem);`
- Output: મોબાઇલમાં ફોન્ટ ઓટોમેટિક 1rem થઈ જશે, કમ્પ્યુટરમાં મોટો થઈને 3rem પર લોક થઈ જશે, વચ્ચે સ્ક્રીન પ્રમાણે આપોઆપ નાનો-મોટો થશે.

* Trigonometric Math Operations (sin, cos, tan)

- Definition: જાવાસ્ક્રિપ્ટ વગર ધ્યોર સીએસએસ કોડિંગની અંદર જ ત્રિકોણમિતિના ફંક્શન્સ વાપરીને વર્તુળાકાર પાથ કે એડવાન્સ વેવ (તરંગ) એનિમેશન્સ બનાવવા.

- Syntax: `transform: translateY(calc(sin(45deg) * 100px));`
- Output: અત્યંત જટિલ મેથમેટિકલ ગ્રાફિક્સ લેઆઉટ ધ્યોર CSS થી બને છે.

● Module 13, 14 & 15: Modern Queries, Snapping & Performance

* Container Query Target Definition (@container)

- Definition: મીડિયા ક્વેરીઝ આખી સ્ક્રીન જોઈને ડિઝાઇન બદલે છે, જ્યારે કન્ટેનર ક્વેરીઝ કોઈ ચોક્કસ પેરન્ટ બોક્સની પહોળાઈ ચેક કરીને તેની અંદરની વસ્તુની ડિઝાઇન બદલી શકે છે.

- Syntax: `.parent { container-type: inline-size; } @container (min-width: 400px) { .child { color: red; } }`
- Output: વિજેટ આર્કિટેક્ચર માટે આ બેસ્ટ છે, બોક્સ નાનું થતાં જ અંદરની ડિઝાઇન આપોઆપ બદલાઈ જાય છે.

* Scroll Snapping Framework (scroll-snap-type)

- Definition: યુઝર જ્યારે આંગળીથી પેજ સ્ક્રોલ કરે, ત્યારે પેજ ગમે ત્યાં અટકવાને બદલે આપોઆપ સરકીને બિલકુલ ફોટા કે સેક્શનના પરફેક્ટ ખૂણા પર આવીને જ ચુબકની જેમ ચોટી જાય તે સિસ્ટમ.

- Syntax: `.parent-scroll { scroll-snap-type: y mandatory; } .child-slide { scroll-snap-align: start; }`
- Output: મોબાઇલ એપ્લિકેશન જેવું સ્મૂથ ફ્લૂ-સ્ક્રીન સ્વાઇપિંગ પેજ વેબસાઇટ પર બને છે.

* Hardware GPU Rendering Acceleration (will-change)

- Definition: બ્રાઉઝરને અગાઉથી જ જણાવી દેવું કે આ એલિમેન્ટ ભવિષ્યમાં એનિમેટ થવાનો છે, જેથી બ્રાઉઝર તેને કમ્પ્યુટરના ગ્રાફિક્સ કાર્ડ (GPU) પર શિફ્ટ કરી દે છે.

- Syntax: `will-change: transform, opacity;`
- Output: વેબસાઇટ અટક્યા વગર કે લેગ (Lag) થયા વગર સુપર ફાસ્ટ 60fps પર ચાલે છે.

* Content Render Layout Postponement (content-visibility: auto)

- Definition: જે સ્ક્રીનની નીચે આવેલો ભાગ છે (જે અત્યારે યુઝરને દેખાતો નથી), તેનો કોડ બ્રાઉઝર અત્યારે રેન્ડર ન કરે જેથી પેજ લોડિંગ સ્પીડ રોકેટ જેવી ફાસ્ટ થઈ જાય.

- Syntax: `content-visibility: auto; contain-intrinsic-size: 1000px;`
- Output: વિશાળ વેબસાઇટ્સનું પર્ફોર્મન્સ અદ્ભુત રીતે વધી જાય છે.

* BEM (Block Element Modifier) Naming Methodology

- Definition: પ્રોફેશનલ આર્કિટેક્ચરમાં સીએસએસના ક્લાસના નામો લખવાની વિશ્વપ્રસિદ્ધ વ્યવસ્થિત પદ્ધતિ.
- Syntax: `.block__element--modifier { }`
- Example: `.card__title--large { font-size: 24px; }`
- Output: કોડ વાંચવો અને ભવિષ્યમાં મેઇન્ટેન કરવો એકદમ આસાન બને છે.

=====

```
=====
CSS MASTER PhD-LEVEL SYLLABUS
COMPLETE A TO Z TOPICS ROADMAP LIST
=====
```

● Module 1 & 2: CSS Core Architecture & Engine

- ```

```
- \* CSSOM (CSS Object Model) Parsing Engine
  - \* DOM and CSSOM Tree Integration
  - \* Render Tree Generation
  - \* The CSS Cascade Algorithm
  - \* Cascade Layers (`@layer`) Declaration
  - \* Nested Cascade Layers
  - \* Specificity Score Calculations (Inline vs ID vs Class vs Element)
  - \* Specificity Tie-Breaking Rules
  - \* The `!important` Exception Rule
  - \* CSS Rule Inheritance Mechanics
  - \* `inherit` Control Value
  - \* `initial` Control Value
  - \* `unset` Control Value
  - \* `revert` Control Value
  - \* `revert-layer` Control Value
  - \* The `all` Shorthand Property

## ● Module 3: Advanced Selectors Masterclass

- ```
-----
```
- * Universal Selector (`\*`)
 - * Element / Type Selector
 - * Class Selector (`.class`)
 - * ID Selector (`#id`)
 - * Grouping Selector (Comma `,`)
 - * Descendant Combinator (Space)
 - * Child Combinator (`>`)
 - * Adjacent Sibling Combinator (`+`)
 - * General Sibling Combinator (`~`)
 - * Basic Attribute Selector (`[attr]`)
 - * Exact Value Attribute Selector (`[attr="value"]`)
 - * Substring Match Attribute Begins With (`[attr^="value"]`)
 - * Substring Match Attribute Ends With (`[attr\$="value"]`)
 - * Substring Match Attribute Contains (`[attr\*="value"]`)
 - * Space-Separated List Attribute Selector (`[attr~="value"]`)
 - * Hyphen-Separated Prefix Attribute Selector (`[attr|="value"]`)
 - * Attribute Case-Insensitivity Modifiers (`i`, `s`)
 - * Functional Pseudo-Class `:has()` (The Parent Selector)
 - * Functional Pseudo-Class `:is()`
 - * Functional Pseudo-Class `:where()`
 - * Functional Pseudo-Class `:not()`
 - * User Action Pseudo-Classes (`:hover`, `:focus`, `:active`, `:focus-within`, `:focus-visible`)
 - * Structural Pseudo-Classes (`:nth-child()`, `:nth-last-child()`, `:first-child`, `:last-child`, `:only-child`)
 - * Typed Structural Pseudo-Classes (`:nth-of-type()`, `:nth-last-of-type()`, `:first-of-type`, `:last-of-type`, `:only-of-type`)
 - * Target and Anchor Pseudo-Classes (`:target`, `:root`, `:scope`)
 - * Form State Pseudo-Classes (`:enabled`, `:disabled`, `:checked`, `:required`, `:optional`, `:valid`, `:invalid`, `:placeholder-shown`)
 - * Pseudo-Elements (`::before`, `::after`)
 - * Typography Pseudo-Elements (`::first-line`, `::first-letter`)
 - * UI Pseudo-Elements (`::selection`, `::placeholder`, `::marker`)

● Module 4 & 5: Box Model, Sizing & Typography

- ```

```
- \* Standard Box Model Mechanics
  - \* Alternative Box Model (`box-sizing: border-box`)
  - \* Content-Box vs Border-Box Calculations
  - \* Margin Collapse Mechanics
  - \* Padding and Border Interaction
  - \* Extrinsically Sizing (`width`, `height`, `min-width`, `max-width`, `min-height`, `max-height`)
  - \* Intrinsic Sizing `max-content`

- \* Intrinsic Sizing ``min-content``
- \* Intrinsic Sizing ``fit-content`` / ``fit-content()``
- \* Absolute Absolute Length Units (``px``, ``cm``, ``mm``, ``in``, ``pt``, ``pc``)
- \* Font-Relative Length Units (``em``, ``rem``, ``ex``, ``ch``)
- \* Viewport Linear Units (``vw``, ``vh``, ``vmin``, ``vmax``)
- \* Modern Large Viewport Units (``lvw``, ``lvh``)
- \* Modern Small Viewport Units (``svw``, ``svh``)
- \* Modern Dynamic Viewport Units (``dvw``, ``dvh``)
- \* Container Query Length Units (``cqw``, ``cqh``, ``cqi``, ``cqb``, ``cqmin``, ``cqmax``)
- \* Percentage Unit (``%``) Calculations
- \* Custom Web Fonts Integration (``@font-face``)
- \* Font Formats (``woff2``, ``woff``, ``ttf``)
- \* Font Display Timeline Control (``font-display: swap``)
- \* Advanced Font Properties (``font-family``, ``font-size``, ``font-weight``, ``font-style``, ``font-stretch``)
- \* Variable Fonts Layout Control
- \* Text Layout Properties (``line-height``, ``letter-spacing``, ``word-spacing``, ``text-align``, ``text-align-last``)
- \* Text Decoration and Styling (``text-decoration``, ``text-transform``, ``text-shadow``, ``text-indent``)
- \* Text Overflow Controls (``white-space``, ``overflow-wrap``, ``word-break``)
- \* Truncating Text with Multi-Line Ellipsis (``text-overflow: ellipsis``, ``-webkit-line-clamp``)

## ● Module 6: Positioning & Stacking Context

- \* ``position: static`` Mechanics
- \* ``position: relative`` Mechanics
- \* ``position: absolute`` Layout Behavior
- \* ``position: fixed`` Viewport Binding
- \* ``position: sticky`` Scroll Threshold Mechanics
- \* Containing Block Invalidation Rules
- \* Offset Control Properties (``top``, ``right``, ``bottom``, ``left``)
- \* Modern CSS Logical Placement Properties (``inset``, ``inset-block``, ``inset-inline``)
- \* The ``z-index`` Property Structure
- \* Stacking Context Creation Triggers
- \* Atomic Stacking Context Constraints
- \* Hardware GPU Acceleration Overrides (``will-change``, ``transform``)

## ● Module 7: Master Flexbox Layout Engine

- \* ``display: flex`` and ``display: inline-flex`` Framework
- \* Flex Container vs Flex Item Separation
- \* Main Axis and Cross Axis Mechanics
- \* Flex Container: ``flex-direction`` (``row``, ``row-reverse``, ``column``, ``column-reverse``)
- \* Flex Container: ``flex-wrap`` (``nowrap``, ``wrap``, ``wrap-reverse``)
- \* Flex Container: ``flex-flow`` Shorthand
- \* Flex Container Axis Alignment: ``justify-content`` (``flex-start``, ``flex-end``, ``center``, ``space-between``, ``space-around``, ``space-evenly``)
- \* Flex Container Cross Axis Alignment: ``align-items`` (``stretch``, ``flex-start``, ``flex-end``, ``center``, ``baseline``)
- \* Flex Container Multi-Line Alignment: ``align-content`` (``stretch``, ``flex-start``, ``flex-end``, ``center``, ``space-between``, ``space-around``, ``space-evenly``)
- \* Flex Container Spacing: ``gap``, ``row-gap``, ``column-gap``
- \* Flex Item Sizing Control: ``flex-basis`` (Content-based vs Fixed sizing)
- \* Flex Item Scaling Mechanics: ``flex-grow`` Proportion Calculations
- \* Flex Item Scaling Mechanics: ``flex-shrink`` Dynamic Calculations
- \* Flex Item: ``flex`` Shorthand Property Optimization
- \* Flex Item Custom Alignment: ``align-self`` Override Control
- \* Flex Item Layout Reordering: ``order`` Attribute Mechanics

## ● Module 8: Master CSS Grid Layout Engine

- \* ``display: grid`` and ``display: inline-grid`` Architecture
- \* Grid Tracks, Grid Cells, Grid Lines, and Grid Areas
- \* Explicit Grid Definition: ``grid-template-columns`` and ``grid-template-rows``
- \* Grid Absolute & Flexible Units (``px``, ``%``, ``em``, ``rem``, ``fr``)
- \* Grid Track Repeat Layouts: ``repeat()`` Function
- \* Dynamic Sizing Boundaries: ``minmax()`` Function
- \* Auto-Placement Engine: ``auto-fill`` Keyword Mechanics
- \* Auto-Placement Engine: ``auto-fit`` Keyword Mechanics

- \* Implicit Grid Generation: ``grid-auto-columns`` and ``grid-auto-rows``
- \* Implicit Flow Management: ``grid-auto-flow`` (``row``, ``column``, ``dense``)
- \* Grid Gutters Control: ``gap``, ``row-gap``, ``column-gap``
- \* Line-Based Placement: ``grid-column-start``, ``grid-column-end``, ``grid-row-start``, ``grid-row-end``
- \* Line-Based Shorthands: ``grid-column`` and ``grid-row``
- \* Area-Based Placement Design: ``grid-template-areas`` Syntax
- \* Area-Based Item Binding: ``grid-area`` Reference Control
- \* Grid Box Realignment: ``justify-items`` and ``align-items``
- \* Grid Content Realignment: ``justify-content`` and ``align-content``
- \* Grid Item Custom Realignment: ``justify-self`` and ``align-self``
- \* Advanced Nested Component Layouts: Subgrid (``grid-template-columns: subgrid``)
- \* Advanced Nested Component Layouts: Subgrid (``grid-template-rows: subgrid``)

## ● Module 9: Visuals, Graphics & Color Spaces

---

- \* Traditional Color Spaces (``rgb()``, ``rgba()``, ``hsl()``, ``hsla()``, Hex codes)
- \* Modern Color Spaces (``lch()``, ``lab()``, ``oklch()``, ``oklab()``)
- \* The ``color-mix()`` Function Mechanics
- \* Dynamic Color Switching: ``currentcolor`` Keyword
- \* Linear Gradients Design (``linear-gradient()``)
- \* Radial Gradients Design (``radial-gradient()``)
- \* Conic Gradients Design (``conic-gradient()``)
- \* Repeating Gradients Framework (``repeating-linear-gradient()``, ``repeating-radial-gradient()``)
- \* Advanced Box Shadows (``box-shadow`` inset vs outset)
- \* Advanced Text Shadows (``text-shadow``)
- \* The ``filter`` Property Functions (``blur()``, ``brightness()``, ``contrast()``, ``grayscale()``, ``hue-rotate()``, ``invert()``, ``opacity()``, ``saturate()``, ``sepia()``, ``drop-shadow()``)
- \* The SVG Filters Integration (``url()``)
- \* Backdrop Filtering Layout (``backdrop-filter``)
- \* Glassmorphism Design Implementation
- \* Neumorphism Design Implementation

## ● Module 10: Advanced Composition, Shapes & Masking

---

- \* The ``mix-blend-mode`` Compositing
- \* The ``background-blend-mode`` Layering
- \* Vector Vector Graphics Integration (``svg``)
- \* The ``clip-path`` Geometric Shapes (``inset()``, ``circle()``, ``ellipse()``, ``polygon()``, ``path()``)
- \* The ``mask-image`` Architecture
- \* Masking Rules Management (``mask-size``, ``mask-repeat``, ``mask-position``, ``mask-mode``)
- \* The ``shape-outside`` Layout Routing (``circle()``, ``ellipse()``, ``polygon()``)
- \* Text Flow Controls (``shape-margin``, ``shape-image-threshold``)
- \* Image Rendering Optimization (``image-rendering``, ``object-fit``, ``object-position``)
- \* The ``isolation: isolate`` Stacking Safeguards

## ● Module 11: Animations & 3D Transformations

---

- \* 2D Transform Properties (``translate()``, ``rotate()``, ``scale()``, ``skew()``)
- \* 3D Transform Matrix Mechanics (``translate3d()``, ``rotate3d()``, ``scale3d()``)
- \* The ``transform-origin`` Control Placement
- \* The ``transform-style: preserve-3d`` Layering
- \* The ``perspective`` and ``perspective-origin`` Depth Engines
- \* The ``backface-visibility`` Render Hiding
- \* Transition Timing Engine (``transition-property``, ``transition-duration``, ``transition-delay``)
- \* Acceleration Curve Logic (``transition-timing-function``: ``linear``, ``ease``, ``ease-in``, ``ease-out``, ``ease-in-out``)
- \* Custom Timing Bezier Curves (``cubic-bezier()``)
- \* Multi-Step Keyframe Architecture (``@keyframes``)
- \* Animation Control System (``animation-name``, ``animation-duration``, ``animation-delay``)
- \* Animation Loop Control (``animation-iteration-count``, ``animation-direction``)
- \* State Persistence Management (``animation-fill-mode``: ``none``, ``forwards``, ``backwards``, ``both``)
- \* Live Animation Interruption (``animation-play-state``: ``running``, ``paused``)
- \* Animation Speed Adjustments (``animation-timing-function``)

## Module 12: CSS Logic, Variables & Math Functions

---

- \* Custom Properties Declaration (``--variable-name``)
- \* Global Scope Routing (``:root``)
- \* Local Scope Isolation Overrides

- \* The `var()` Extraction Mechanics
- \* Variable Fallback Parameters (`var(--name, fallback)`)
- \* Dynamic Arithmetic Calculations (`calc()`)
- \* Dynamic Boundary Limits (`min()`)
- \* Dynamic Boundary Limits (`max()`)
- \* Fluid Layout Range Interpolation (`clamp()`)
- \* Trigonometric Math Operations (`sin()`, `cos()`, `tan()`, `asin()`, `acos()`, `atan()`, `atan2()`)
- \* Mathematical Power Engine (`pow()`, `sqrt()`, `hypot()`, `log()`, `exp()`)
- \* Sign Controls Logic (`abs()`, `sign()`, `rem()`, `mod()`)

#### Module 13: Responsive Frameworks & Ultra-Modern Queries

- \* Media Queries Layout Framework (`@media`)
- \* Media Query Viewport Feature Modifiers (`width`, `height`, `orientation`)
- \* Media Query Range Syntax Optimization (`width >= 768px`)
- \* Container Query Target Definition (`container-type: size / inline-size`)
- \* Container Name Isolation (`container-name`)
- \* The `@container` Size Queries Syntax
- \* Modern Container Scroll-State Queries (`scroll-state`)
- \* Feature Capability Testing (`@supports`)
- \* Logical Operator Integration (`and`, `or`, `not` inside `@supports`)
- \* User Accessibility Preferences (`prefers-color-scheme`)
- \* User Motion Accessibility Controls (`prefers-reduced-motion`)
- \* Device Interaction Queries (`hover`, `pointer`)

#### ● Module 14: Custom Shapes, Layout Routing & Scroll Snapping

- \* The Scroll Containers Framework (`scroll-snap-type`)
- \* Scroll Alignment Mechanics (`scroll-snap-align`: `start`, `center`, `end`)
- \* Scroll Stop Behaviors Control (`scroll-snap-stop`: `normal`, `always`)
- \* Visual Area Offsets (`scroll-padding`, `scroll-margin`)
- \* Geometric Path Clipping (`clip-path`)
- \* Text Text Wrapping Layouts (`shape-outside`)
- \* Shape Layout Boundary Offsets (`shape-margin`)
- \* Grid Column Newspaper Multi-Layouts (`column-count`, `column-gap`, `column-rule`)
- \* Pointer Pointer Interaction Blocking (`pointer-events: none`)
- \* Dynamic Element Sizing Interactivity (`resize: both`)

#### ● Module 15: CSS Architecture, Optimization & Rendering Performance

- \* Hardware GPU Rendering Acceleration (`will-change`)
- \* Content Render Layout Postponement (`content-visibility: auto`)
- \* Component Component Containment Isolation (`contain`)
- \* DOM and CSSOM Structural Optimization
- \* Critical CSS Rendering Optimization
- \* BEM (Block Element Modifier) Naming Methodology
- \* OOCSS (Object-Oriented CSS) Structural Separation
- \* SMACSS (Scalable and Modular Architecture for CSS) Categories
- \* SASS/SCSS Variables and Component Nesting Fundamentals
- \* SASS/SCSS Reusable Logic Mixins and `@extend` Functions

=====

CSS COMPLETE HERO ENCYCLOPEDIA (A-Z)

EVERY PROPERTY, SELECTOR, ATTR VS PROPERTY & DETAILED VALUES

=====

ભાષા: સરળ અને સ્પષ્ટ ગુજરાતી (Pure, Simple Gujarati)

હેતુ: ગૂગલ, MDN, W3Schools, HTMLReference.io ની તમામ સીએસએસ ટોપિક્સ અને પ્રોપર્ટીઝનું ગાઈડ.

-----

=====

PART 1: HTML ATTRIBUTE VS CSS PROPERTY (એટ્રિબ્યુટ અને પ્રોપર્ટી વચ્ચેનો તફાવત)

=====

1. HTML Attribute (એટ્રિબ્યુટ):

- વ્યાખ્યા: HTML ટેગની અંદર જે વધારાની માહિતી આપવા માટે લખાય તેને એટ્રિબ્યુટ કહેવાય. તે HTML ના સ્ટ્રક્ચરનો ભાગ છે.

- ઉદાહરણ: `<input type="text" id="username" disabled>`

અહીં 'type', 'id', અને 'disabled' એ HTML Attributes છે. તે બ્રાઉઝરને જણાવે છે કે આ કેવા પ્રકારનું ઇનપુટ બોક્સ છે.

2. CSS Property (પ્રોપર્ટી):

- વ્યાખ્યા: HTML એલિમેન્ટને કેવો લુક આપવો (સ્ટાઈલિંગ), તેની ડિઝાઇન કે શણગાર કેવો કરવો તે માટે જે નિયમ વપરાય તેને પ્રોપર્ટી કહેવાય. તે CSS નો ભાગ છે.

- ઉદાહરણ: `color: red; font-size: 16px;`

અહીં 'color' અને 'font-size' એ CSS Properties છે અને 'red' તથા '16px' તેની Values છે.

=====

PART 2: ALL CSS SELECTORS & COMBINATORS (તમામ સિલેક્ટર્સ અને કોમ્બિનેટર્સનું આઉટપુટ)

=====

1. Basic Selectors (મૂળભૂત સિલેક્ટર્સ):

- Universal Selector (`*`): પેજના તમામ HTML એલિમેન્ટ્સને એકસાથે સિલેક્ટ કરે છે.

\* આઉટપુટ: `* { margin: 0; }` લખવાથી પેજના દરેક બોક્સની બહારની જગ્યા ઝીરો થઈ જશે.

- Element Selector (ટેગ નામ): ચોક્કસ HTML ટેગને ટાર્ગેટ કરે છે.

\* આઉટપુટ: `p { color: blue; }` પેજના બધા જ પેરેગ્રાફ બ્લુ કલરના થઈ જશે.

- Class Selector (`.className`): જે ટેગમાં તે ક્લાસ લખ્યો હોય તેને જ સિલેક્ટ કરે છે.

\* આઉટપુટ: `.btn { background: green; }` જે ટેગમાં `class="btn"` હશે તેની બેકગ્રાઉન્ડ ગ્રીન થશે.

- ID Selector (`#idName`): કોઈ એક જ વિશિષ્ટ એલિમેન્ટને ટાર્ગેટ કરવા માટે.

\* આઉટપુટ: `#header { height: 80px; }` જે ટેગમાં `id="header"` હશે તેની જ હાઇટ સેટ થશે.

- Attribute Selector (`[attr="value"]`): HTML એટ્રિબ્યુટના આધારે સિલેક્ટ કરે છે.

\* આઉટપુટ: `input[type="password"] { border: 2px solid red; }` માત્ર પાસવર્ડ ઇનપુટ બોક્સની બોર્ડર રેડ થશે.

2. Combinators (જોડાણ સિલેક્ટર્સ):

- Descendant Selector (જગ્યા/Space): પેરેન્ટની અંદર આવેલા તમામ ચાઇલ્ડ અને સબ-ચાઇલ્ડને સિલેક્ટ કરે છે.

\* આઉટપુટ: `div p { color: gray; }` div ની અંદર ગમે ત્યાં આવેલા બધા જ પેરેગ્રાફ ગ્રે કલરના થશે.

- Child Selector (`>`): માત્ર ડાયરેક્ટ (પહેલા લેવલના) ચાઇલ્ડને જ સિલેક્ટ કરે છે.

\* આઉટપુટ: `div > p { color: black; }` div ના સગા ચાઇલ્ડ હોય તેવા જ પેરેગ્રાફ બદલાશે, તેની અંદર બીજા બોક્સમાં છુપાયેલા પેરેગ્રાફ નહીં.

- Adjacent Sibling Selector (`+`): એલિમેન્ટની તરત જ પછી આવતા સગા ભાઈ (Sibling) ને સિલેક્ટ કરે છે.

\* આઉટપુટ: `h1 + p { font-weight: bold; }` h1 ટેગ પૂરો થયા પછી જે પહેલો જ પેરેગ્રાફ આવશે તે બોલ્ડ થશે.

- General Sibling Selector (`~`): એલિમેન્ટની પછી આવતા તમામ ભાઈઓ (Siblings) ને સિલેક્ટ કરે છે.

\* આઉટપુટ: `h1 ~ p { color: red; }` h1 પછી નીચે લાઈનમાં ગમે તેટલા પેરેગ્રાફ આવશે તે બધા રેડ થઈ જશે.

3. Pseudo-classes & Pseudo-elements:

- `:hover`: માઉસ એલિમેન્ટ પર લઈ જવાથી આઉટપુટ બદલાય. દા.ત. બટન પર માઉસ જતા કલર બદલવો.

- `:focus`: જ્યારે ઇનપુટ બોક્સ પર ક્લિક કરીને લખાણ ટાઈપ કરવાનું શરૂ કરીએ ત્યારે આઉટપુટ બદલાય.

- `:nth-child(n)`: લિસ્ટ કે ટેબલની લાઈન ટાર્ગેટ કરવા. દા.ત. `:nth-child(even)` થી ટેબલની એક છોડીને એક લાઈન ગ્રે થાય.

- `::before`: એલિમેન્ટ શરૂ થાય તે પહેલાં કોડ દ્વારા નવી લખાણ કે ડિઝાઇન ઉમેરે છે.

- `::after`: એલિમેન્ટ પૂરો થાય તેની તરત પછી બેકગ્રાઉન્ડમાં કંઈક ડિઝાઇન કે આઇકોન ઉમેરે છે.

## PART 3: ALL CSS PROPERTIES, VALUES AND OUTCOME (તમામ પ્રોપર્ટીઝ, વેલ્યુઝ અને આઉટપુટ)

### 1. BOX MODEL & DISPLAY PROPERTIES (બોક્સ મોડેલ અને વેઆઉટ)

- \* **display:** એલિમેન્ટ સ્ક્રીન પર કેવી રીતે જગ્યા રોકશે તે નક્કી કરે છે.
  - **block:** એલિમેન્ટ નવી લાઇનથી શરૂ થશે અને આખી પહોળાઈ (100% width) રોકશે.
  - **inline:** લખાણ જેટલી જ જગ્યા રોકશે, નવી લાઇન નહીં લે. આમાં width/height આપી શકાતી નથી.
  - **inline-block:** લખાણ પ્રમાણે લાઇનમાં ગોઠવાશે પણ તમે મનગમતી width અને height આપી શકશો.
  - **flex:** આખું પેરેન્ટ બોક્સ ફ્લેક્સ મોડમાં આવી જશે. અંદરની વસ્તુઓ લાઇનમાં ગોઠવાઈ જશે (1D Layout).
  - **grid:** આખું બોક્સ ખાના વાળી ગ્રીડ સિસ્ટમ બની જશે (2D Layout - આડી અને ઊભી લાઇનો).
  - **none:** એલિમેન્ટ સ્ક્રીન પરથી ગાયબ થઈ જશે અને પોતાની જગ્યા પણ છોડી દેશે.
- \* **visibility:** એલિમેન્ટને જોવો કે છુપાવવો.
  - **visible:** એલિમેન્ટ નોર્મલ દેખાશે.
  - **hidden:** એલિમેન્ટ ગાયબ થઈ જશે પણ તે પોતાની જગ્યા છોડશે નહીં, ત્યાં ખાલી જગ્યા દેખાશે.
- \* **margin / padding:**
  - **margin:** બોક્સની બહારની તરફ જગ્યા છોડશે.
  - **padding:** બોક્સની અંદર બોર્ડર અને લખાણ વચ્ચે જગ્યા છોડશે.
  - **margin-inline / padding-block:** (Modern Logical Properties) ભાષા ડાબેથી જમણે કે જમણેથી ડાબે હોય તે મુજબ આપોઆપ ડાબી/જમણી કે ઉપર/નીચેની જગ્યા એડજસ્ટ કરશે.
- \* **box-sizing:** બોક્સનું કુલ માપ કેવી રીતે ગણવું.
  - **content-box:** બાય-ડિફોલ્ટ વેલ્યુ. આમાં width ની અંદર padding અને border પ્લસ થાય છે, જેથી વેઆઉટ મોટો થઈને તૂટી જાય છે.
  - **border-box:** બેસ્ટ વેલ્યુ. તમે જે width આપશો તેની અંદર જ padding અને border આપોઆપ સમાઈ જશે, બોક્સ મોટું નહીં થાય.
- \* **overflow:** કન્ટેન્ટ જ્યારે બોક્સની બહાર ભાગે ત્યારે શું કરવું.
  - **visible:** કન્ટેન્ટ બોક્સની બહાર નીકળીને નીચેના લખાણ પર ઓવરલેપ થશે.
  - **hidden:** બહાર જતો વધારાનો ભાગ આપોઆપ કપાઈ જશે, દેખાશે નહીં.
  - **scroll:** બોક્સમાં આડી અને ઊભી સ્ક્રોલ બાર આવી જશે, ભલે લખાણ નાનું હોય.
  - **auto:** જો લખાણ મોટું હશે તો જ આપોઆપ સ્ક્રોલ બાર દેખાશે, નહીંતર નોર્મલ રહેશે.

### 2. POSITIONING PROPERTIES (જગ્યા અને ગોઠવણી)

- \* **position:** એલિમેન્ટને સ્ક્રીન પર કઈ જગ્યાએ સેટ કરવો.
  - **static:** બાય-ડિફોલ્ટ નોર્મલ પોઝિશન. આમાં top, bottom, left, right પ્રોપર્ટી કામ નહીં કરે.
  - **relative:** એલિમેન્ટ પોતાની અસલી જગ્યા પર જ રહેશે, પણ ત્યાંથી top કે left આપવાથી થોડો આગળ-પાછળ ખસી શકશે.
  - **absolute:** એલિમેન્ટ તેના નજીકના પેરેન્ટ બોક્સના આધારે આખા પેજમાં ગમે ત્યાં ઉડીને સેટ થઈ શકે છે. તે નોર્મલ ડોક્યુમેન્ટ ફ્લોમાંથી બહાર નીકળી જાય છે.
  - **fixed:** એલિમેન્ટ કોમ્પ્યુટર/મોબાઇલની સ્ક્રીન સાથે ચોટી જશે. પેજ ગમે તેટલું સ્ક્રોલ કરો, તે ત્યાં જ રહેશે (જેમ કે હેડર કે હેલ્પ બટન).
  - **sticky:** જ્યાં સુધી સ્ક્રોલ ન કરીએ ત્યાં સુધી નોર્મલ રહે છે, પણ જેવું તે સ્ક્રીનની ઉપર પહોંચશે એટલે ત્યાં ચોટી જશે (Sticky Header માટે વપરાય છે).
- \* **z-index:** લેયર્સ નક્કી કરવા (કઈ વસ્તુ ઉપર દેખાશે અને કઈ પાછળ).
  - **auto:** બ્રાઉઝર પોતે ક્રમ નક્કી કરશે.
  - **positive number (દા.ત. 1, 10, 999):** જેનો નંબર મોટો હશે તે એલિમેન્ટ નાના નંબર વાળા એલિમેન્ટની ઉપર (આગળ) દેખાશે.
  - **negative number (દા.ત. -1):** એલિમેન્ટ બેકગ્રાઉન્ડમાં પાછળ જતો રહેશે.

### 3. FLEXBOX & GRID PROPERTIES (મોર્ડન વેઆઉટ એન્જિન)

- \* **flex-direction (Flex Parent):**
  - **row:** અંદરના આઈટમ્સ ડાબેથી જમણે આડી લાઇનમાં ગોઠવાશે.
  - **column:** અંદરના આઈટમ્સ ઉપરથી નીચે ઊભી લાઇનમાં ગોઠવાશે.

- \* justify-content (Flex/Grid Parent - Main Axis Alignment):
  - flex-start: બધી વસ્તુઓ શરૂઆતમાં (ડાબી બાજુ) ગોઠવાશે.
  - flex-end: બધી વસ્તુઓ લાઈનના અંતમાં (જમણી બાજુ) જતી રહેશે.
  - center: બધી વસ્તુઓ બોક્સની બિલકુલ વચ્ચે (Center) આવી જશે.
  - space-between: પહેલી અને છેલ્લી વસ્તુ ખૂણામાં ચોટી જશે અને વચ્ચે સરખી જગ્યા છૂટશે.
  - space-around: દરેક આઈટમની આજુબાજુ સરખી જગ્યા છૂટશે.
- \* align-items (Flex/Grid Parent - Cross Axis Alignment):
  - stretch: બધા આઈટમ્સ પેરેન્ટ બોક્સની આખી ઊંચાઈ જેટલા લાંબા થઈ જશે.
  - center: આડી લાઈનમાં રહેલી બધી વસ્તુઓ ઊભી રીતે સેન્ટરમાં આવી જશે.
- \* grid-template-columns (Grid Parent):
  - `repeat(3, 1fr)`: બોક્સની અંદર સરખા ભાગના 3 ઊભા ખાના (Columns) બની જશે.
- \* gap: ફ્લેક્સ કે ગ્રીડના બે ખાના કે આઈટમ્સ વચ્ચે સ્પેસ (જગ્યા) સેટ કરવા માટે.

#### 4. MODERN VISUALS & GRAPHICS ENGINE (ગ્રાફિક્સ અને આધુનિક પ્રોપર્ટીઝ)

- \* object-fit: ઇમેજ કે વિડિયો પોતાના બોક્સમાં કેવી રીતે એડજસ્ટ થશે.
  - fill: ઇમેજ બોક્સમાં ફિટ થવા માટે આડી-ઊભી ખેંચાઈ (Stretch) જશે અને ચહેરો/આકાર બગડી જશે.
  - contain: ઇમેજ પોતાનો અસલી રેશિયો સાચવશે અને બોક્સની અંદર આખી દેખાશે, ભલે આજુબાજુ ખાલી જગ્યા રહે.
  - cover: ફોટો ખેંચાયા વગર બોક્સને આખું ભરી દેશે. વધારાનો ભાગ આપોઆપ ક્રોપ થઈ જશે (વેબસાઈટ માટે બેસ્ટ).
  - scale-down: ફોટો જો નાનો હશે તો નાનો જ રહેશે, મોટો થઈને બ્લર નહીં થાય.
- \* aspect-ratio: લંબાઈ-પહોળાઈનો રેશિયો ફિક્સ કરવો.
  - `16/9`: યુટ્યુબ વીડિયો જેવું બોક્સ બનશે. width બદલાશે તો height આપોઆપ રેશિયો મુજબ સેટ થઈ જશે.
  - `1/1`: બોક્સ હંમેશા ચોરસ (Square) જ રહેશે.
- \* clip-path: એલિમેન્ટને કસ્ટમ શેપમાં કાપવો.
  - `circle(50%)`: ચોરસ ડિવ કે ઇમેજ એકદમ ગોળ સર્કલ બની જશે.
  - `polygon(...)`: તમે આપેલા કો-ઓર્ડિનેટ્સ મુજબ એલિમેન્ટ ત્રિકોણ, સ્ટાર કે ડાયમંડ આકારમાં કપાઈ જશે.
- \* backdrop-filter: એલિમેન્ટની પાછળ રહેલી વસ્તુઓ પર ફિલ્ટર લગાવવું (Glassmorphism).
  - `blur(10px)`: એલિમેન્ટ કાય જેવો દેખાશે અને તેની પાછળ જે પણ બેકગ્રાઉન્ડ હશે તે આઈફોનની જેમ બ્લર દેખાશે.
- \* background-clip:
  - text: આ પ્રોપર્ટી `color: transparent;` સાથે વાપરવાથી બેકગ્રાઉન્ડ કલર કે ગ્રેડિયન્ટ અક્ષરોની (Text) ની અંદર પુરાઈ જશે, જેથી ફેન્સી અક્ષરો બનશે.
- \* pointer-events: માઉસ ક્લિક કંટ્રોલ કરવી.
  - auto: નોર્મલ ક્લિક કામ કરશે.
  - none: એલિમેન્ટ આરપાર (Transparent) થઈ જશે. તેના પર માઉસ ક્લિક કામ નહીં કરે, ક્લિક તેની પાછળ રહેલી વસ્તુ પર લાગશે.
- \* user-select: લખાણ કોપી કંટ્રોલ.
  - auto: યુઝર લખાણ સિલેક્ટ અને કોપી કરી શકશે.
  - none: યુઝર આ એલિમેન્ટનું લખાણ માઉસથી સિલેક્ટ કે કોપી કરી શકશે નહીં (સિક્યોરિટી માટે).

#### 5. ANIMATIONS, TRANSITIONS & GRADIENTS (એનિમેશન અને કલર મિક્સિંગ)

- \* linear-gradient(direction, color1, color2): બે કે તેથી વધુ કલર સીધી રેખામાં મિક્સ થઈને સુંદર બેકગ્રાઉન્ડ બનાવશે.
- \* radial-gradient(): સેન્ટરમાંથી કલર બહારની તરફ રાઉન્ડમાં ફેલાશે.
- \* conic-gradient(): ઘડિયાળના કાંટાની જેમ ૩૬૦ ડિગ્રીમાં કલર વ્હીલ કે પાઇ-ચાર્ટ જેવો લુક આપશે.
- \* transition: ડિઝાઇનમાં થતો ફેરફાર સ્મૂથ કરવો.
  - `all 0.3s ease`: બટન પર માઉસ લઈ જતા કલર કે સાઈઝ અચાનક બદલાવાને બદલે 0.3 સેકન્ડમાં ધીમેથી એકદમ સ્મૂથલી બદલાશે.
- \* transform: એલિમેન્ટને ફેરવવો કે મોટો કરવો.
  - `rotate(45deg)`: એલિમેન્ટ ૪૫ ડિગ્રી નમી જશે.
  - `scale(1.2)`: એલિમેન્ટ પોતાની અસલી સાઈઝથી ૨૦% મોટો થઈ જશે.
  - `translate(20px, 50px)`: એલિમેન્ટ પોતાની જગ્યાએથી ૨૦ પિક્સલ જમણે અને ૫૦ પિક્સલ નીચે ખસી જશે.

- 3D Transforms (`rotateY(180deg)`): એલિમેન્ટ આડો પલટી મારી જશે (કાર્ડ ફ્લિપિંગ એનિમેશન માટે).

#### PART 4: MODERN CSS RESPONSIVE MATH FUNCTIONS (રિસ્પોન્સિવ ગણિત ફંક્શન્સ)

##### 1. `calc()` [Calculation]:

- ઉપયોગ: બે અલગ અલગ યુનિટ્સનું ગણિત કરવા.
- આઉટપુટ: `width: calc(100% - 50px);` બોક્સ સ્ક્રીનની આખી પહોળાઈમાંથી બરાબર 50 પિક્સલ ઓછી જગ્યા રોકશે. રિસ્પોન્સિવ સાઈડબાર માટે બેસ્ટ છે.

##### 2. `min()` [Minimum Value]:

- ઉપયોગ: લિસ્ટમાંથી જે વેલ્યુ સૌથી નાની હશે તે બ્રાઉઝર સિલેક્ટ કરશે.
- આઉટપુટ: `width: min(80%, 1200px);` મોટી સ્ક્રીન પર બોક્સ 1200px નું રહેશે, પણ જો મોબાઈલની સ્ક્રીન નાની થશે તો તે આપોઆપ સ્ક્રીનના 80% થઈ જશે જેથી પેજ બહાર નહીં ભાગે.

##### 3. `max()` [Maximum Value]:

- ઉપયોગ: આપેલા ઓપ્શન્સમાંથી જે વેલ્યુ મોટી હશે તે સિલેક્ટ થશે.
- આઉટપુટ: `width: max(50%, 300px);` મોબાઈલમાં પણ બોક્સ ઓછામાં ઓછું 300px નું તો રહેશે જ, તેનાથી નાનું ક્યારેય નહીં થાય.

##### 4. `clamp(minimum, preferred, maximum)` [The Magic Function]:

- ઉપયોગ: મીડિયા ક્વેરીઝ લખ્યા વગર અક્ષરો કે બોક્સને મોબાઈલથી પીસી સુધી ઓટો-રિસ્પોન્સિવ કરવા.
- આઉટપુટ: `font-size: clamp(16px, 5vw, 40px);`  
મોબાઈલ ગમે તેટલો નાનો થાય, ફોન્ટ 16 પિક્સલથી નાના નહીં થાય. લેપટોપ ગમે તેટલું મોટું થાય, ફોન્ટ 40 પિક્સલથી મોટા નહીં થાય. અને વચ્ચેની સ્ક્રીન પર તે વ્યુપોર્ટ (`5vw`) મુજબ આપોઆપ નાના-મોટા થતા રહેશે. આને Fluid Typography કહેવાય છે.

# CSS સંપૂર્ણ માસ્ટર જ્ઞાનકોશ (Complete CSS A-Z Encyclopedia)  
 # સ્રોત સોસિફિકેશન: W3C / MDN Web Docs / HTMLReference.io / W3Schools

## 1. ATTRIBUTE અને PROPERTY વચ્ચેનો અસલી તફાવત (Difference between Attribute and Property)

ડેવલપમેન્ટમાં ઘણા લોકો આ બે શબ્દોમાં ભૂલ કરે છે. બંનેનો અર્થ તદ્દન અલગ છે:

\* HTML Attribute (એટ્રિબ્યુટ):

- આ HTML ટેગની અંદર લખવામાં આવે છે.
- તે બ્રાઉઝરને એલિમેન્ટની ઓળખ અથવા કન્ટેન્ટની મૂળ માહિતી આપે છે.
- ઉદાહરણ: `<input type="text" id="username" disabled>`  
 અહીં 'type', 'id', અને 'disabled' એ HTML Attributes છે.

\* CSS Property (પ્રોપર્ટી):

- આ CSS નિયમોની અંદર લખવામાં આવે છે.
- તે એલિમેન્ટના દેખાવ, કલર, લેઆઉટ અને ડિઝાઇનને કંટ્રોલ કરે છે.
- ઉદાહરણ: `color: red; font-size: 16px;`  
 અહીં 'color' અને 'font-size' એ CSS Properties છે અને 'red' તથા '16px' તેની Values છે.

## 2. CSS SELECTORS અને COMBINATORS (તમામ પ્રકારના સિલેક્ટર્સ અને તેનું આઉટપુટ)

બ્રાઉઝરને કયા એલિમેન્ટ પર ડિઝાઇન અપ્લાય કરવી તે આનાથી ખબર પડે છે.

\* \* (Universal Selector):

- ઉપયોગ: `* { margin: 0; }`
- આઉટપુટ: પેજના દરેક સિંગલ એલિમેન્ટ પર આ નિયમ લાગુ થઈ જશે.

\* Element Selector:

- ઉપયોગ: `p { color: blue; }`
- આઉટપુટ: પેજના તમામ `<p>` (પેરેગ્રાફ) એલિમેન્ટ બ્લુ કલરના થઈ જશે.

\* .class Selector:

- ઉપયોગ: `.card { border: 1px solid black; }`
- આઉટપુટ: જે જે ટેગમાં `class="card"` લખ્યું હશે તે બધાની ફરતે બોર્ડર આવી જશે.

\* #id Selector:

- ઉપયોગ: `#main-header { background: yellow; }`
- આઉટપુટ: આખા પેજમાં જે એકમાત્ર એલિમેન્ટનું `id="main-header"` હશે તેની બેકગ્રાઉન્ડ યેલો થશે.

\* Attribute Selector:

- ઉપયોગ: `input[type="password"] { background: red; }`
- આઉટપુટ: માત્ર એવા જ ઇનપુટ બોક્સ લાલ થશે જેમાં પાસવર્ડ ટાઇપ કરવાનો હોય.

\* Descendant Combinator (Space - ખાલી જગ્યા):

- ઉપયોગ: `div p { color: green; }`
- આઉટપુટ: `<div>` ની અંદર ગમે તેટલા ઊંડાણમાં રહેલા તમામ `<p>` ટેગ લીલા રંગના થશે.

\* Child Combinator (`>`):

- ઉપયોગ: `div > p { color: green; }`
- આઉટપુટ: માત્ર એવા જ `<p>` લીલા થશે જે સીધા જ `<div>` ના સંતાન (Direct Child) હોય.

\* Adjacent Sibling Combinator (`+`):

- ઉપયોગ: `h1 + p { font-weight: bold; }`
- આઉટપુટ: `<h1>` ની તરત જ નીચે આવતો પહેલો `<p>` ટેગ ઘાટો (Bold) થશે.

\* General Sibling Combinator (`~`):

- ઉપયોગ: `h1 ~ p { color: grey; }`
- આઉટપુટ: `<h1>` ની નીચે આવતા તે જ લેવલના તમામ `<p>` ટેગ ગ્રે કલરના થઈ જશે.

## 3. CSS PROPERTIES, VALUES અને તેનું ચોક્કસ OUTPUT (A to Z Master Guide)

## [A]

- \* align-content (ફ્લેક્સ કે ગ્રીડ લાઇનો ગોઠવવા):
  - flex-start: બધી લાઇનો બોક્સની શરૂઆતમાં ભેગી થઈ જશે.
  - flex-end: બધી લાઇનો બોક્સના છેડે ભેગી થઈ જશે.
  - center: બધી લાઇનો બોક્સની બરાબર વચ્ચે ગોઠવાઈ જશે.
  - space-between: પહેલી લાઇન ઉપર, છેલ્લી નીચે અને બાકીની વચ્ચે સરખી જગ્યા છોડશે.
  - space-around: દરેક લાઇનની આજુબાજુ સરખી જગ્યા છૂટશે.
  - stretch (Default): બધી લાઇનો આખી જગ્યા રોકવા માટે ખેંચાઈ જશે.
- \* align-items (એક જ લાઇનની અંદર આઈટમ્સને ઊભા લેવલમાં સેટ કરવા):
  - flex-start: બધા આઈટમ્સ લાઇનની ઉપર અડીને ગોઠવાશે.
  - flex-end: બધા આઈટમ્સ લાઇનના નીચેના ભાગે અડીને ગોઠવાશે.
  - center: બધા આઈટમ્સ ઊભી લાઇનમાં બરાબર વચ્ચે (Vertically Centered) થઈ જશે.
  - baseline: બધા અક્ષરોના નીચેના બેઝ લાઇન લેવલ એક સરખા લાઇનમાં આવી જશે.
  - stretch (Default): જો હાઇટ ન આપી હોય, તો બધા આઈટમ્સ પેરેન્ટ બોક્સ જેટલા ઊંચા ખેંચાઈ જશે.
- \* align-self (કોઈ એક ચોક્કસ આઈટમનો ઊભો લેઆઉટ બદલવા):
  - values: auto, flex-start, flex-end, center, baseline, stretch
  - આઉટપુટ: બાકીના આઈટમ્સ પેરેન્ટના નિયમ પ્રમાણે ચાલશે, પણ આ એક આઈટમ પોતાની વેલ્યુ મુજબ અલગ આઉટપુટ આપશે.
- \* animation-name & animation-duration:
  - values: [કોઈપણ નામ] અને ટાઈમ જેમ કે 2s, 500ms
  - આઉટપુટ: @keyframes માં બનાવેલું એનિમેશન એલિમેન્ટ પર રન થવાનું શરૂ થશે.
- \* animation-fill-mode:
  - none: એનિમેશન પૂરું થતાં જ એલિમેન્ટ પોતાની જુની અસલી સ્ટેટ પર આવી જશે.
  - forwards: એનિમેશન પૂરું થયા પછી એલિમેન્ટ છેલ્લી ફ્રેમના દેખાવ પર જ લોક થઈ જશે (પાછો જૂનો નહિ થાય).
  - backwards: એનિમેશન શરૂ થતા પહેલાં (ડીલે ટાઈમ દરમિયાન) એલિમેન્ટ પહેલી ફ્રેમનો લુક પકડી લેશે.
  - both: forwards અને backwards બંનેના નિયમો એકસાથે લાગુ પડશે.
- \* animation-iteration-count:
  - [નંબર]: એનિમેશન એટલી જ વાર ફરી ફરીને શાંત થઈ જશે (દા.ત. 3).
  - infinite: લોડર આઇકોનની જેમ એનિમેશન ક્યારેય બંધ નહીં થાય, સતત ચાલતું જ રહેશે.
- \* animation-play-state:
  - running: એનિમેશન ચાલુ રહેશે.
  - paused: એનિમેશન જ્યાં હશે ત્યાં જ થીજી (Stuck) જશે (માઉસ હોવર પર એનિમેશન રોકવા બેસ્ટ).
- \* aspect-ratio (બોક્સનો રેશિયો ફિક્સ કરવા):
  - 16/9: વિડિયો પ્લેયર જેવો લેપટોપ સ્ક્રીન રેશિયો આપોઆપ જળવાઈ રહેશે, પહોળાઈ વધતા લંબાઈ આપોઆપ વધશે.
  - 1/1: બોક્સ હંમેશા પરફેક્ટ ચોરસ (Square) જ રહેશે, ભલે તે ગમે તેટલું રિસ્પોન્સિવ થાય.
  - 4/3: જૂના ટીવી કે મોનિટર જેવો લુક મળશે.

## [B]

- \* backdrop-filter (એલિમેન્ટની પાછળ રહેલી વસ્તુઓ પર ફિલ્ટર લગાવવું - Glassmorphism):
  - blur(10px): એલિમેન્ટની પાછળ જે કંઈ હશે તે 10 પિક્સલ બ્લર દેખાશે, જેનાથી આઈફોન જેવો મોર્ડન કાયનો લુક (Glass Look) બનશે.
  - grayscale(100%): પાછળનો આખો ભાગ બ્લેક એન્ડ વ્હાઇટ દેખાશે.
- \* background-clip:
  - border-box: બેકગ્રાઉન્ડ કલર બોર્ડરની નીચે સુધી ફેલાશે.
  - padding-box: બેકગ્રાઉન્ડ માત્ર પેડિંગ સુધી રહેશે, બોર્ડરની નીચે કલર નહિ જાય.
  - content-box: બેકગ્રાઉન્ડ કલર માત્ર અસલી લખાણ કે કન્ટેન્ટ હોય એટલા જ ભાગમાં દેખાશે.
  - text: બેકગ્રાઉન્ડ કલર કે ફોન્ટ અક્ષરોની અંદર પુરાઈ જશે (ટેક્સ્ટ ગ્રેડિયન્ટ માટે ઉપયોગી).
- \* background-size:
  - auto: ફોટો પોતાની અસલી સાઈઝમાં જ રહેશે.
  - cover: ફોટો ખેંચાયા વગર આખા બેકગ્રાઉન્ડ બોક્સને ભરી દેશે (જો ફોટો મોટો હશે તો સાઈડમાંથી કોપ થઈ જશે પણ

બોક્સ ખાલી નહિ રહે).

- contain: આખો ફોટો બોક્સમાં દેખાશે, ફોટો ક્યાંયથી કોપ નહિ થાય (પણ જો બોક્સ મોટું હશે તો આજુબાજુ ખાલી જગ્યા બચશે).

- 100% 100%: ફોટો બોક્સમાં ફિટ થવા માટે ઊંચો-પહોળો ખેંચાઈ (Stretch) જશે.

\* border-radius (ખૂણા ગોળ કરવા):

- 10px: બોક્સના ચારેય ખૂણા સહેજ વળાંકવાળા ગોળ થશે.

- 50%: જો બોક્સ ચોરસ હશે તો તે પરફેક્ટ ગોળ (Circle) બની જશે (પ્રોફાઇલ પિક્સલ માટે બેસ્ટ).

\* box-sizing (બોક્સની સાઈઝ ગણવાની રીત):

- content-box (Default): જો તમે width: 100px આપો અને પછી padding: 20px આપો, તો બોક્સની અસલી પહોળાઈ 140px થઈ જશે અને લેઆઉટ તૂટી જશે.

- border-box: બોક્સની પહોળાઈ 100px જ રહેશે. પેડિંગ કે બોર્ડર આપોઆપ બોક્સની અંદર જ એડજસ્ટ થઈ જશે (દરેક વેબ ડિઝાઇનમાં આ વાપરવું જ જોઈએ).

[C]

\* caret-color (ઇનપુટ કર્સરનો કલર):

- red: ઇનપુટ બોક્સમાં લખાણ લખતી વખતે જે ઊભી લીટી ઝબકતી (Blink) થાય છે તેનો કલર લાલ થઈ જશે.

\* clear:

- left / right / both: float થયેલા એલિમેન્ટ્સની આજુબાજુની ખાલી જગ્યામાં નીચેનું લખાણ ઉપર આવતું અટકશે અને નવી લાઈનથી જ શરૂ થશે.

\* clip-path (એલિમેન્ટને આડા-અવળા આકારોમાં કાપવો):

- circle(50%): આખું બોક્સ ગોળ આકારમાં કપાઈ જશે.

- polygon(50% 0%, 0% 100%, 100% 100%): બોક્સ પરફેક્ટ ત્રિકોણ (Triangle) આકારમાં કપાઈ જશે.

- polygon(...) થી સ્ટાર કે કોઈ પણ આડો-અવળો શેપ બનાવી શકાય છે.

[D]

\* display (એલિમેન્ટનો મૂળ સ્વભાવ નક્કી કરવો):

- block: એલિમેન્ટ નવી લાઈનથી શરૂ થશે અને આખી પહોળાઈ (100% width) રોકી લેશે (દા.ત. div, p).

- inline: એલિમેન્ટ જેટલું લખાણ હશે એટલી જ જગ્યા રોકશે, નવી લાઈન નહિ લે. આમાં width/height સેટ નહિ કરી શકાય (દા.ત. span, a).

- inline-block: લખાણ જેટલી જગ્યા રોકશે પણ તમે તેની ઉપર width, height, margin, padding બધું જ પરફેક્ટ સેટ કરી શકશો.

- flex: આખું બોક્સ ૧-પરિમાણીય ફ્લેક્સ કન્ટેનર બની જશે. અંદરના બધા ચાઇલ્ડ આપોઆપ એક જ લાઈનમાં બાજુ-બાજુમાં ગોઠવાઈ જશે.

- grid: આખું બોક્સ ૨-પરિમાણીય ગ્રીડ બની જશે. તમે ઊભી કોલમ અને આડી રો નો આખો ગ્રાફ નક્કી બનાવી શકશો.

- none: એલિમેન્ટ સ્ક્રીન પરથી ગાયબ થઈ જશે અને પોતાની રોકેલી જગ્યા પણ ખાલી કરી દેશે (જાણે કોડમાં છે જ નહીં).

[F]

\* filter (ફોટો ઇફેક્ટ્સ):

- blur(5px): આખો ફોટો ધૂંધળો (Blur) થઈ જશે.

- grayscale(100%): ફોટો સંપૂર્ણ બ્લેક એન્ડ વ્હાઇટ થઈ જશે.

- brightness(150%): ફોટામાં લાઈટ/પ્રકાશ વધી જશે.

- drop-shadow(2px 2px 5px black): પીએનજી ફોટોગ્રાફના અસલી શેપ પ્રમાણે પાછળ પડછાયો પડશે.

\* flex-direction:

- row (Default): અંદરના આઈટમ્સ ડાબેથી જમણે એક જ લાઈનમાં ગોઠવાશે.

- row-reverse: આઈટમ્સનો ક્રમ ઉલટો થઈ જશે અને જમણેથી ડાબે ગોઠવાશે.

- column: આઈટમ્સ એકબીજાની નીચે ઊભી લાઈનમાં ગોઠવાઈ જશે (મોબાઈલ વ્યુ માટે બેસ્ટ).

- column-reverse: ઊભી લાઈનમાં નીચેથી ઉપરના ક્રમમાં ગોઠવાશે.

\* flex-wrap:

- nowrap (Default): બધા આઈટમ્સ ગમે તેટલા મોટા હોય, એક જ લાઈનમાં કચડાઈને પણ એકસાથે જ રહેશે, લાઈન નહિ બદલે.

- wrap: જો સ્ક્રીન નાની થશે અને જગ્યા નહિ હોય, તો વધારાના આઈટમ્સ આપોઆપ તૂટીને નીચેની નવી લાઈનમાં આવી

જશે (રિસ્પોન્સિવ માટે જરૂરી).

\* float:

- left / right: એલિમેન્ટ આખા પેજમાં ડાબી કે જમણી બાજુ ચોટી જશે અને બાકીનું લખાણ તેની આજુબાજુ વીંટળાઈ (Wrap) જશે.

-----  
[G]  
-----

\* gap (Flex કે Grid ની વચ્ચે જગ્યા છોડવી):

- 20px: બધા આઈટમ્સ વચ્ચે અંદરોઅંદર ૨૦ પિક્સલની પરફેક્ટ જગ્યા છૂટશે (બહારની સાઈડ જગ્યા નહિ છૂટે).

\* grid-template-columns (ગ્રીડની ઊભી કોલમ બનાવવી):

- repeat(3, 1fr): એક સરખા માપના ૩ ઊભા ખાના (3 Columns) બની જશે.  
- 200px 1fr: પહેલી કોલમ ફિક્સ 200px ની રહેશે અને બીજી બાકી બચેલી બધી જગ્યા રોકી લેશે (સાઇડબાર લેઆઉટ).

-----  
[J]  
-----

\* justify-content (આડા લેવલમાં આઈટમ્સ વચ્ચે જગ્યા સેટ કરવી - હોરિઝોન્ટલ અલાઈનમેન્ટ):

- flex-start: બધા આઈટમ્સ ડાબી બાજુ ભેગા થઈ જશે.  
- flex-end: બધા આઈટમ્સ જમણી બાજુ ખૂણામાં ભેગા થઈ જશે.  
- center: બધા આઈટમ્સ આડી લાઈનમાં બરાબર વચ્ચે આવી જશે.  
- space-between: પહેલી અને છેલ્લી આઈટમ ખૂણામાં ચોટી જશે અને વચ્ચે બચેલી જગ્યા સરખા ભાગે વહેંચાઈ જશે.  
- space-around: દરેક આઈટમની ડાબી અને જમણી બાજુ એક સરખી સ્પેસ છૂટશે.  
- space-evenly: દરેક આઈટમની વચ્ચે અને છેડા પર બધી જગ્યા એક સરખી જ ગણતરીથી છૂટશે.

-----  
[M]  
-----

\* margin vs padding (સૌથી મોટો કન્સેપ્ટ):

- margin: 20px -> આ બોર્ડરની બહારના ભાગમાં જગ્યા છોડશે. બે અલગ-અલગ બોક્સને એકબીજાથી દૂર રાખવા આ વપરાય.  
- padding: 20px -> આ બોર્ડરની અંદરના ભાગમાં જગ્યા છોડશે. બોક્સની બોર્ડર અને અંદર લખેલા લખાણ વચ્ચે ગેપ રાખવા આ વપરાય.

\* mix-blend-mode:

- multiply / screen / overlay: અક્ષરોનો કલર પાછળ રહેલા ફોટા કે કલર સાથે ફોટોશોપના લેયરની જેમ બ્લેન્ડ (મિક્સ) થઈને નવો જ કલર આઉટપુટ આપશે.

-----  
[O]  
-----

\* object-fit (ઇમેજ બોક્સમાં કેવી રીતે ફિટ થશે):

- fill (Default): ફોટો ગમે તેવો હોય, બોક્સમાં ફિટ થવા માટે કદરૂપો ખેંચાઈ કે દબાઈ (Stretch) જશે.  
- contain: ફોટો પોતાનો અસલી રેશિયો સાચવશે અને આખો દેખાશે (જો બોક્સનો શેપ અલગ હશે તો સાઇડમાં ખાલી જગ્યા બચશે).  
- cover: ફોટો જરાય ખેંચાશે નહીં, બોક્સને આખું ભરી દેશે. વધારાનો ભાગ આપોઆપ કોપ (કપાઈ) થઈ જશે (પ્રોડક્ટ ઇમેજ અને બેનર માટે બેસ્ટ).  
- scale-down: ફોટો જો નાનો હશે તો નાનો જ રહેશે, બોક્સ કરતા મોટો હશે તો ખેંચાયા વગર contain ની જેમ નાનો થઈ જશે.

\* opacity (પારદર્શકતા):

- 1: સંપૂર્ણ દેખાશે.  
- 0.5: અડધું પારદર્શક (આરપાર જોઈ શકાય તેવું સેમી-ટ્રાન્સપરન્ટ) થઈ જશે.  
- 0: એલિમેન્ટ સંપૂર્ણ કાચ જેવો અદ્રશ્ય થઈ જશે (પણ તે જગ્યા રોકી રાખશે અને ત્યાં માઉસ ક્લિક પણ કામ કરશે).

\* overflow (બોક્સ નાનું હોય અને લખાણ બહાર ભાગે ત્યારે):

- visible (Default): બહાર નીકળેલું લખાણ બોક્સ તોડીને બહાર લટકતું દેખાશે (ખરાબ લુક).  
- hidden: બોક્સની બહાર જતું બધું જ લખાણ કપાઈ જશે અને અદ્રશ્ય થઈ જશે.  
- scroll: બોક્સની અંદર ઊભી અને આડી સ્ક્રોલ બાર આવી જશે, જેથી યુઝર તેને સ્ક્રોલ કરીને વાંચી શકશે ભલે બોક્સ નાનું હોય.  
- auto: જો લખાણ વધશે તો જ આપોઆપ સ્ક્રોલ બાર દેખાશે, નહીંતર નોર્મલ રહેશે (બેસ્ટ પ્રેક્ટિસ).

\* overflow-wrap / word-break:

- break-word / break-all: જો કોઈ બહુ મોટી લિંક કે મોટો શબ્દ હોય જે બોક્સની બહાર ભાગતો હોય, તો તેને અધવચ્ચેથી તોડીને નીચેની નવી લાઈનમાં મોકલી દેશે.

[P]

\* pointer-events (માઉસ એક્શન કંટ્રોલ):

- auto: નોર્મલ ક્લિક, હોવર બધું જ કામ કરશે.
- none: એલિમેન્ટ પર માઉસની કોઈ ક્લિક કે હોવર ઇફેક્ટ કામ જ નહીં કરે. માઉસ ક્લિક આ એલિમેન્ટની આરપાર પાછળના લેયર પર જતી રહેશે.

\* position (પેજ પર ગોઠવણની આખી ગેમ):

- static (Default): નોર્મલ લાઈન મુજબ જ્યાં આવતું હોય ત્યાં જ ગોઠવાશે. આમાં top, left, z-index પ્રોપર્ટી કામ નહીં કરે.
- relative: એલિમેન્ટ પોતાની અસલી જગ્યાએ જ રહેશે, પણ તમે top: 10px આપશો તો તે પોતાની અસલી જગ્યા પરથી સહેજ ૧૦ પિક્સલ નીચે ખસશે (પોતાની જગ્યા ખાલી નહિ કરે).
- absolute: આ પોતાની અસલી જગ્યા કાયમ માટે છોડી દેશે (પાછળના એલિમેન્ટ તેની જગ્યા રોકી લેશે). આ એલિમેન્ટ પોતાના નજીકના 'relative' પેરેન્ટ બોક્સના આધારે આખા પેજમાં ગમે ત્યાં ઉડીને સેટ થઈ શકે છે.
- fixed: આ એલિમેન્ટ બ્રાઉઝરની સ્ક્રીન સાથે ફેવિકોલથી ચોટી જશે. પેજ ગમે તેટલું ઉપર-ચે નીચે સ્ક્રોલ કરો, તે સ્ક્રીન પર એક જ જગ્યાએ જકડાયેલું રહેશે (દા.ત. હેલ્પ ચેટ બટન).
- sticky: આ બહુ મસ્ત છે. પેજ સ્ક્રોલ કરતી વખતે તે નોર્મલ ઉપર જશે, પણ જેવા તમે સેટ કરેલા પોઇન્ટ (દા.ત. top: ૦) પર પહોંચશે એટલે ત્યાં લોક (ચોટી) થઈ જશે. પેજ નીચે સ્ક્રોલ થતું રહેશે પણ આ મેનુ ત્યાં જ ચોટેલું રહેશે.

[S]

\* scroll-behavior:

- smooth: જ્યારે તમે પેજમાં કોઈ લિંક પર ક્લિક કરશો, ત્યારે પેજ આંચકો મારીને જવાને બદલે ધીમેથી સરકતું (Smooth Slide) નીચે જશે.

\* scroll-snap-type & scroll-snap-align:

- values: y mandatory / start
- આઉટપુટ: ઇન્સ્ટાગ્રામ રીલ્સ કે સ્વાઇડર જેવો લુક બનશે. યુઝર અડધું સ્ક્રોલ કરીને છોડી દેશે તો પણ બોક્સ આપોઆપ સરકીને પરફેક્ટ આગલા બોક્સ પર લોક (Snap) થઈ જશે.

[T]

\* text-overflow:

- ellipsis: જો લખાણ બહુ મોટું હોય અને સિંગલ લાઈનમાં સમાતું ન હોય, તો કદરૂપું કપાવાને બદલે છેલ્લે ત્રણ ટપકાં (... આઉટપુટ) આવી જશે.

\* transform (2D & 3D જાદુ):

- translate(20px, 50px): એલિમેન્ટ પોતાની જગ્યા પરથી ૨૦ પિક્સલ આડો અને ૫૦ પિક્સલ ઊભો ખસી જશે (લેઆઉટ બગાડ્યા વગર).
- scale(1.2): એલિમેન્ટ પોતાની અસલી સાઈઝથી ૨૦% મોટો થઈ જશે (હોવર ઇફેક્ટ માટે બેસ્ટ).
- rotate(45deg): એલિમેન્ટ ૪૫ ડિગ્રી ઘડિયાળના કાંટાની દિશામાં કોસ નમી જશે.
- rotateY(180deg): એલિમેન્ટ ઊંધો ફરી જશે (3D કાર્ડ ફ્લિપ કરવા માટેની અસલી વેલ્યુ).

\* transform-style & backface-visibility:

- preserve-3d & hidden: જ્યારે 3D કાર્ડ પાછળની તરફ ફરે (Flip થાય), ત્યારે તેનો પાછળનો ભાગ આપોઆપ છુપાઈ જશે અને કાર્ડની પાછળ છુપાવેલી અસલી ડીઝાઇન આઉટપુટમાં લાઈવ દેખાશે.

\* transition (એનિમેશન સ્મૂથનેસ):

- all 0.3s ease-in-out: એલિમેન્ટ પર માઉસ લઈ જતા ક્લિક કે સાઈઝનો ફેરફાર કોઈ ઝટકા વગર એકદમ સ્મૂથ અને ધીમેથી ૦.૩ સેકન્ડમાં પૂરો થશે.

[U]

\* user-select:

- none: વેબસાઈટ પર આવેલો કોઈ પણ યુઝર તમારા લખાણને માઉસથી સિલેક્ટ કે ડ્રેગ કરીને કોપી (Copy) નહીં કરી શકે. ટેક્સ્ટ સંપૂર્ણ લોક થઈ જશે.

-----  
[V]  
-----

\* **visibility** (દ્રશ્યતા):

- **visible**: નોર્મલ દેખાશે.
- **hidden**: એલિમેન્ટ સ્ક્રીન પરથી ગાયબ થઈ જશે (પણ તે પેજ પર પોતાની જગ્યા રોકી રાખશે, ત્યાં ખાલી મોટો ગેપ દેખાશે. `display: none` ની જેમ જગ્યા ખાલી નહિ કરે).

-----  
[Z]  
-----

\* **z-index** (વેયરની ઉપર-નીચે ગોઠવણી):

- [નંબર જેમ કે 1, 10, 999]: જેનો z-index નંબર સૌથી મોટો હશે, તે વેયર બાકીના બધા જ એલિમેન્ટ્સની ઉપર (Top Layer) તરતું દેખાશે (પોપ-અપ મોડલ અને હેડર માટે જરૂરી). આ માત્ર position relative/absolute/fixed સાથે જ કામ કરે છે.

#### =====

#### ૪. MODERN CSS MATH FUNCTIONS (રિસ્પોન્સિવ ગણિતના ફંક્શન્સ)

#### =====

આ આધુનિક ફંક્શન્સ મીડિયા ક્વેરીઝ વગર વેબસાઇટને આપોઆપ રિસ્પોન્સિવ બનાવે છે:

\* **calc()** - સાયનેમિક ગણતરી:

- ઉપયોગ: width: calc(100% - 50px);
- આઉટપુટ: બોક્સ આખી સ્ક્રીનની પહોળાઈમાંથી બરાબર ૫૦ પિક્સલ બાદ કરીને બાકીની જગ્યામાં પરફેક્ટ સેટ થઈ જશે.

\* **min()** - નાની વેલ્યુ પસંદ કરવી:

- ઉપયોગ: width: min(50%, 400px);
- આઉટપુટ: બ્રાઉઝર આ બંનેમાંથી જે નાનું હશે તે રાખશે. જો 50% ની સાઈઝ 400px થી ઓછી હશે તો ટકાવારી રાખશે, નહીંતર 400px પર લોક કરી દેશે.

\* **max()** - મોટી વેલ્યુ પસંદ કરવી:

- ઉપયોગ: width: max(10%, 200px);
- આઉટપુટ: ગમે તે થાય બોક્સ rbase પિક્સલથી નાનું નહીં થાય. સ્ક્રીન ગમે તેટલી નાની થાય, તે 200px ની સાઈઝ પકડી રાખશે.

\* **clamp()** - ફ્લુઇડ ટાઇપોગ્રાફી (ટેક્સ્ટ કંટ્રોલ):

- ઉપયોગ: font-size: clamp(16px, 5vw, 32px);
- આઉટપુટ: આમાં ૩ વેલ્યુ છે: [ન્યૂનતમ, રિસ્પોન્સિવ, મહત્તમ].  
મોબાઇલમાં અક્ષરો ક્યારેય 16px થી નાના નહીં થાય, પીસીમાં ક્યારેય 32px થી મોટા નહીં થાય, અને વચ્ચેની સ્ક્રીનમાં વ્યુપોર્ટ (`5vw`) ના આધારે આપોઆપ એકદમ સ્મૂથ નાના-મોટા થતા રહેશે. કોઈ મીડિયા ક્વેરી લખવાની જરૂર જ નહીં પડે!

## CSS COMPLETE EXPLANATION IN SIMPLE GUJARATI (સરળ ગુજરાતીમાં)

આ માર્ગદર્શિકામાં CSS ના તમામ પાયાના ખ્યાલો, લેઆઉટ પ્રોપર્ટીઝ, મીડિયા ક્વેરીઝ અને વેબસાઇટને રિસ્પોન્સિવ (Responsive) બનાવવાની તમામ રીતો ખૂબ જ સરળ ગુજરાતીમાં સમજાવવામાં આવી છે.

### ૧. પાયાની મહત્વની પ્રોપર્ટીઝ (Core CSS Properties)

#### ■ Margin (માર્જિન):

- આ પ્રોપર્ટી એલિમેન્ટની બોર્ડરની બહારની બાજુએ ખાલી જગ્યા (Space) છોડવા માટે વપરાય છે.
- આનાથી બે અલગ-અલગ બોક્સ કે એલિમેન્ટ વચ્ચે અંતર રાખી શકાય છે.
- ઉદાહરણ: `margin: 20px;` (ચારેય બાજુ ૨૦ પિક્સલ જગ્યા છૂટશે).
- અલગ-અલગ બાજુ માટે: `margin-top`, `margin-right`, `margin-bottom`, `margin-left`.

#### ■ Padding (પેડિંગ):

- આ પ્રોપર્ટી એલિમેન્ટની બોર્ડરની અંદરની બાજુએ (કન્ટેન્ટ અને બોર્ડર વચ્ચે) જગ્યા છોડવા માટે વપરાય છે.
- આનાથી બોક્સની અંદર રહેલું લખાણ બોર્ડર સાથે ચોટી જતું નથી.
- ઉદાહરણ: `padding: 15px;` (બોક્સની અંદર ચારેય બાજુ જગ્યા છૂટશે).
- અલગ-અલગ બાજુ માટે: `padding-top`, `padding-right`, `padding-bottom`, `padding-left`.

#### ■ Opacity (ઓપેસિટી):

- આ પ્રોપર્ટી કોઈપણ એલિમેન્ટની પારદર્શકતા (Transparency) નક્કી કરે છે.
- તેની વેલ્યુ 0.0 થી 1.0 ની વચ્ચે હોય છે.
- 0.0 એટલે એલિમેન્ટ સંપૂર્ણ અદ્રશ્ય (ગાયબ) થઈ જશે, અને 1.0 એટલે તે સંપૂર્ણ દેખાશે (Default).
- ઉદાહરણ: `opacity: 0.5;` (એલિમેન્ટ અડધો કાચ જેવો પારદર્શક દેખાશે).

#### ■ Visibility (વિઝિબિલિટી):

- આ પ્રોપર્ટી એલિમેન્ટ દેખાશે કે નહીં તે નક્કી કરે છે.
- મુખ્ય વેલ્યુ: `visible` (દેખાશે) અને `hidden` (છુપાઈ જશે).
- મહત્વની વાત: જ્યારે તમે `visibility: hidden;` કરો છો, ત્યારે એલિમેન્ટ સ્ક્રીન પરથી છુપાઈ જાય છે પણ તે વેબપેજ પર પોતાની જગ્યા રોકી રાખે છે (જગ્યા ખાલી રહે છે).

#### ■ Overflow (ઓવરફ્લો):

- જ્યારે બોક્સ નાનું હોય અને તેની અંદરનું લખાણ કે કન્ટેન્ટ બોક્સની બહાર નીકળી જાય (ઓવરફ્લો થાય), ત્યારે તેને કંટ્રોલ કરવા આ પ્રોપર્ટી વપરાય છે.
- મુખ્ય વેલ્યુઝ:
  1. `visible`: બહાર નીકળેલું લખાણ બોક્સની બહાર પણ દેખાશે (Default).
  2. `hidden`: બહાર નીકળેલું લખાણ કપાઈ જશે અને દેખાશે નહીં.
  3. `scroll`: બોક્સમાં સ્ક્રોલ બાર આવી જશે, જેથી યુઝર સ્વાઈડ કરીને લખાણ વાંચી શકે.
  4. `auto`: જો લખાણ મોટું હશે તો જ બ્રાઉઝર આપોઆપ સ્ક્રોલ બાર બતાવશે.

### ૨. લેઆઉટ અને પોઝિશન પ્રોપર્ટીઝ (Layout & Positioning)

#### ■ Float (ફ્લોટ):

- આ જૂની પણ મહત્વની પ્રોપર્ટી છે. તેનો ઉપયોગ ઈમેજ કે ટેક્સ્ટને ડાબી (left) કે જમણી (right) બાજુ ખસેડવા માટે થાય છે, જેથી બાકીનું લખાણ તેની આસપાસ ગોઠવાઈ જાય.
- ઉદાહરણ: `float: left;` (એલિમેન્ટ ડાબી બાજુ આવી જશે અને તેની બાજુમાં લખાણ શરૂ થઈ જશે).

#### ■ Position (પોઝિશન):

- વેબપેજ પર કોઈ એલિમેન્ટ કઈ જગ્યાએ દેખાશે તેનું ચોક્કસ લોકેશન સેટ કરવા માટે આ પ્રોપર્ટી વપરાય છે. આની સાથે `top`, `bottom`, `left`, `right` પ્રોપર્ટી વાપરવામાં આવે છે.
- પ્રકારો:
  1. `static`: આ નોર્મલ મોડ છે. એલિમેન્ટ જેમ લાઈનમાં આવતો હોય તેમ જ ગોઠવાય છે.
  2. `relative`: એલિમેન્ટ પોતાની અસલી જગ્યા પર જ રહે છે, પણ ત્યાંથી તેને થોડો ઉપર-નીચે કે ડાબે-જમણે ખસેડી શકાય છે.
  3. `absolute`: આ એલિમેન્ટ તેના સૌથી નજીકના 'relative' પેરેન્ટ બોક્સના આધારે આખા પેજમાં ગમે ત્યાં ગોઠવી શકાય છે. તે નોર્મલ લાઈનમાંથી બહાર નીકળી જાય છે.

4. fixed: આ એલિમેન્ટ સ્ક્રીન સાથે ચોટી જાય છે. પેજ ગમે તેટલું સ્ક્રોલ કરો, તે પોતાની જગ્યાએથી હલતો નથી (જેમ કે ફિક્સ હેડર કે ફેટ બટન).

5. sticky: આ નોર્મલ રીતે સ્ક્રોલ થાય છે, પણ જ્યારે તે સ્ક્રીનની ટોચ પર પહોંચે ત્યારે ત્યાં ફિક્સ (ચોટી) જાય છે.

#### ■ CSS Flexbox (ફ્લેક્સબોક્સ):

- આ ૧-પરિમાણીય (1-Dimensional) લેઆઉટ સિસ્ટમ છે. તેનો ઉપયોગ એલિમેન્ટ્સને એક જ આડી લાઈન (Row) અથવા ઊભી લાઈન (Column) માં ગોઠવવા માટે થાય છે.

- પેરેન્ટ બોક્સમાં `display: flex;` લખવાથી તે ચાલુ થાય છે.

- મહત્વના કમાન્ડ્સ:

\* justify-content: આડી લાઈનમાં એલિમેન્ટ્સ વચ્ચે કેવી જગ્યા રાખવી (center, space-between, space-around).

\* align-items: ઊભી લાઈનમાં એલિમેન્ટ્સને વચ્ચે કે ઉપર-નીચે સેટ કરવા (center, flex-start, flex-end).

\* flex-direction: આડી લાઈન રાખવી કે ઊભી (row અથવા column).

#### ■ CSS Grid (ગ્રીડ):

- આ ૨-પરિમાણીય (2-Dimensional) લેઆઉટ સિસ્ટમ છે. તેનો ઉપયોગ આખી વેબસાઇટનું મોટું માળખું એટલે કે આડી લાઈનો (Rows) અને ઊભા ખાના (Columns) બંને એકસાથે કંટ્રોલ કરવા થાય છે.

- પેરેન્ટ બોક્સમાં `display: grid;` લખવાથી ચાલુ થાય છે.

- મહત્વના કમાન્ડ્સ:

\* grid-template-columns: ઊભા કેટલા ખાના પાડવા અને કેટલી સાઈઝના રાખવા (દા.ત. grid-template-columns: 1fr 1fr 1fr; એટલે એક સરખા ૩ ખાના પડશે).

\* gap: ખાનાઓ વચ્ચે કેટલી સ્પેસ (જગ્યા) રાખવી.

### ----- 3. વેબસાઇટને રિસ્પોન્સિવ બનાવવી અને મીડિયા ક્વેરીઝ (Responsive Web Design) -----

#### ■ Responsive (રિસ્પોન્સિવ) એટલે શું?:

- વેબસાઇટ કમ્પ્યુટર, ટેબ્લેટ કે મોબાઇલ ગમે તે સ્ક્રીન પર ખુલે, તે સ્ક્રીનની સાઈઝ પ્રમાણે આપોઆપ બરાબર ગોઠવાઈ જાય અને યુઝરને સરસ દેખાય તેને રિસ્પોન્સિવ ડિઝાઇન કહેવાય.

#### ■ Media Queries (@media) શું છે?:

- મીડિયા ક્વેરીઝ એ CSS નો એવો જાદુઈ નિયમ છે જે બ્રાઉઝરની સાઈઝ ચેક કરે છે. જો સાઈઝ આપણા નિયમ મુજબની હોય, તો જ તે અંદર લખેલી CSS પ્રોપર્ટીઝને વેબસાઇટ પર લાગુ કરે છે.

#### ■ Breakpoints (બ્રેકપોઇન્ટ્સ) એટલે શું?:

- બ્રેકપોઇન્ટ્સ એટલે એ પિક્સલ (px) ની સાઈઝ જ્યાંથી આપણી વેબસાઇટનું લેઆઉટ બદલાય છે. દા.ત. કમ્પ્યુટરમાં જે ૩ ખાના દેખાતા હતા, તે મોબાઇલની નાની સ્ક્રીનમાં આવતા જ એકની નીચે એક આવી જવા જોઈએ.

#### ■ સ્ટાન્ડર્ડ બ્રેકપોઇન્ટ્સનું લિસ્ટ (Standard Breakpoints Guide):

૧. મોબાઇલ સ્ક્રીન (Mobile - નાના ફોન): up to 480px

૨. ટેબ્લેટ સ્ક્રીન (Tablets): 481px થી 768px

૩. લેપટોપ સ્ક્રીન (Laptops/Small Screens): 769px થી 1024px

૪. કમ્પ્યુટર મોનિટર (Desktops/Large Screens): 1025px થી 1200px

૫. એક્સ્ટ્રા મોટી સ્ક્રીન (Extra Large Screens): 1201px થી વધુ

#### ■ મીડિયા ક્વેરી લખવાનું પ્રેક્ટિકલ ઉદાહરણ:

```
/* નોર્મલ કમ્પ્યુટર માટેની CSS (જેમાં બેકગ્રાઉન્ડ લાલ રહેશે) */
```

```
body {
 background-color: red;
}
```

```
/* જ્યારે સ્ક્રીનની સાઈઝ ૭૬૮ પિક્સલ કે તેનાથી નાની થાય (મોબાઇલ/ટેબ્લેટ) ત્યારે આ નિયમ ચાલુ થશે */
```

```
@media screen and (max-width: 768px) {
 body {
 background-color: blue; /* મોબાઇલમાં બેકગ્રાઉન્ડ બ્લુ થઈ જશે */
 }
}
```

```
=====
```

## CSS & WEB RESPONSIVENESS MEGA ENCYCLOPEDIA (તમામ બાકી રહેલા એડવાન્સ કન્સેપ્ટ્સ અને વિગતો)

અહીં વેબસાઇટને રિસ્પોન્સિવ (Responsive) બનાવવા માટેના તમામ આધુનિક નિયમો, ખૂટતી પ્રોપર્ટીઝ, અને બ્રાઉઝર સેટિંગ્સની સંપૂર્ણ એ ટુ ઝેડ (A to Z) સમજૂતી એકદમ સરળ ગુજરાતીમાં આપેલી છે.

### ૧. META VIEWPORT (મેટા વ્યુપોર્ટ નો અસલી જાદુ)

જ્યારે તમે મોબાઇલમાં કોઈ જૂની વેબસાઇટ ખોલો છો, ત્યારે અક્ષરો એકદમ નાના કીડી જેવા દેખાય છે અને આખું પેજ ઝૂમ આઉટ થઈ જાય છે. આ સમસ્યાને ઉકેલવા માટે HTML ના <head> ટેગમાં નીચેની લાઇન લખવી જ પડે છે:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

આમાં વપરાતા દરેક શબ્દનો સરળ અર્થ:

- meta: બ્રાઉઝરને પેજ વિશે વધારાની માહિતિ (Metadata) આપે છે.
- name="viewport": બ્રાઉઝરને કહે છે કે આ નિયમ સ્ક્રીનના દેખાતા ભાગ (Viewport) માટે છે.
- width=device-width: વેબસાઇટની પહોળાઈ એટલી જ કરો જેટલી યુઝરના મોબાઇલ કે કમ્પ્યુટરની સ્ક્રીનની છે. આનાથી કન્ટેન્ટ મોબાઇલની બહાર સ્ક્રોલ થતો નથી.
- initial-scale=1.0: જ્યારે વેબસાઇટ પહેલીવાર મોબાઇલમાં ખુલે, ત્યારે ઝૂમ લેવલ ૧૦૦% (Normal) હોવું જોઈએ.
- user-scalable=no (વધારાનો ઓપ્શન): આ લખવાથી યુઝર આંગળીથી પેજને ઝૂમ (Zoom In/Out) નથી કરી શકતો. (નોંધ: એક્સેસિબિલિટીના નિયમો મુજબ આ ન વાપરવાની સલાહ આપવામાં આવે છે).

### ૨. RESPONSIVE WEB UNITS (માપવાના આધુનિક અને રિસ્પોન્સિવ એકમો)

વેબસાઇટને મોબાઇલ અને કમ્પ્યુટર બંનેમાં સરખી બતાવવા માટે પિક્સલ (px) ના બદલે રિલેટિવ યુનિટ્સ વપરાય છે.

- px (Pixels): આ ફિક્સ માપ છે. જો તમે કોઈ બોક્સને width: 400px; આપો, તો ૩૨૦ પિક્સલના નાના મોબાઇલમાં તે બોક્સ સ્ક્રીનની બહાર નીકળી જશે અને નીચે આડી સ્ક્રોલબાર આવી જશે.
- % (Percentage): પેરેન્ટ (Parent) બોક્સના ટકાના આધારે સાઈઝ નક્કી થાય છે. width: 50%; એટલે તેના મેઈન બોક્સ કરતા અડધી સાઈઝ.
- vw (Viewport Width): આખી સ્ક્રીનની પહોળાઈના ટકા. 100vw એટલે સ્ક્રીનની પૂરેપૂરી પહોળાઈ (૧૦૦%).
- vh (Viewport Height): આખી સ્ક્રીનની ઊંચાઈ (લંબાઈ) ના ટકા. 100vh એટલે મોબાઇલ કે મોનિટરની ફૂલ હાઈટ.
- em: આ યુનિટ તેના પેરેન્ટ બોક્સના ફોન્ટ સાઈઝ પર આધાર રાખે છે. જો પેરેન્ટની સાઈઝ 16px હોય, તો 2em એટલે 32px થાય.
- rem (Root EM): આ સૌથી બેસ્ટ અને સેફ યુનિટ છે. આ આખા પેજના મેઈન રૂટ (<html> ટેગ) ની સાઈઝ પર આધાર રાખે છે. સામાન્ય રીતે બ્રાઉઝરમાં મેઈન સાઈઝ 16px હોય છે, એટલે 1rem = 16px, 2rem = 32px. યુઝર જો પોતાના ફોનના સેટિંગ્સમાંથી અક્ષરો મોટા કરશે, તો rem વાપરેલો કોડ આપોઆપ મોટો થઈ જશે.

### ૩. FLUID TYPOGRAPHY & MATH FUNCTIONS (ગણિતના જાદુઈ ફંક્શન્સ)

મીડિયા ક્વેરીઝ વગર અક્ષરોને કે બોક્સને આપોઆપ નાના-મોટા કરવા માટે CSS માં ગણિતના ફંક્શન્સ આવ્યા છે:

- calc(): ડાયનેમિક ગણતરી કરવા માટે.  
ઉદાહરણ: width: calc(100% - 40px); (આખા પેજમાંથી ૪૦ પિક્સલ બાદ કરીને બાકીની જગ્યા આપશે).
- min() અને max(): બ્રાઉઝરને નાની અથવા મોટી વેલ્યુ પસંદ કરવા માટે મજબૂર કરે છે.
- clamp(minimum, preferred, maximum): આ સુપર ફંક્શન છે!  
ઉદાહરણ: font-size: clamp(16px, 5vw, 32px);  
આનો અર્થ એ થાય કે મોબાઇલમાં અક્ષરો ક્યારેય 16px થી નાના નહીં થાય, કમ્પ્યુટરમાં 32px થી મોટા નહીં થાય, અને વચ્ચેની સ્ક્રીનમાં સ્ક્રીનની પહોળાઈ (5vw) મુજબ આપોઆપ એડજસ્ટ (Fluid) થતા રહેશે.

### ૪. RESPONSIVE IMAGES & MEDIA (ઇમેજ ફાટી કે બહાર નીકળી ન જાય તે માટે)

જો તમે મોટી ઇમેજ વેબસાઇટમાં મુકો, તો તે મોબાઇલમાં બહાર ભાગી જાય છે. તેને રોકવા માટે ગોલ્ડન રૂલ:

```
img, video {
 max-width: 100%;
```

```
height: auto;
}
```

- max-width: 100%; આનો અર્થ એ કે ઇમેજ પોતાના પેરેન્ટ બોક્સ કરતા ક્યારેય મોટી નહીં થાય. જો સ્ક્રીન નાની થશે, તો ઇમેજ પણ આપોઆપ નાની થઈ જશે.
- height: auto; આનાથી ઇમેજનો રેશિયો (લાંબી-પહોળી) જળવાઈ રહે છે અને ઇમેજ ખેંચાતી (Stretch) નથી.
- aspect-ratio: ઇમેજ કે વીડિયોનો ચોક્કસ રેશિયો ફિક્સ કરવા માટે.  
ઉદાહરણ: aspect-ratio: 16 / 9; (યૂટ્યુબ વીડિયો જેવો પરફેક્ટ શેપ આપશે).
- object-fit: cover; : જો તમે ઇમેજની હાઈટ અને વિડ્થ ફિક્સ આપી હોય, તો આ પ્રોપર્ટી લખવાથી ઇમેજ ખેંચાયા વગર બોક્સની અંદર પરફેક્ટ સેટ (Crop) થઈ જશે.

#### ૫. ADVANCED MEDIA FEATURES (આધુનિક મીડિયા ક્વેરીઝ)

ફક્ત સ્ક્રીનની પહોળાઈ (width) જ નહીં, પણ યુઝરના ફોનના સેટિંગ્સ જાણીને વેબસાઈટ બદલવાની ટેકનિક:

- prefers-color-scheme: dark { ... }  
જો યુઝરે પોતાના આખા ફોનમાં 'ડાર્ક મોડ' ચાલુ રાખ્યો હશે, તો આ ક્વેરી આપોઆપ એક્ટિવ થઈ જશે અને તમારી વેબસાઈટ પણ આપોઆપ ડાર્ક કલરની થઈ જશે!
- prefers-reduced-motion: reduce { ... }  
જો યુઝરને આંખની તકલીફ હોય કે ચક્કર આવતા હોય અને તેમણે ફોનમાં એનિમેશન બંધ રાખ્યા હોય, તેમના માટે આ ક્વેરી દ્વારા તમારી વેબસાઈટના એનિમેશન્સ અને સ્લાઈડ્સ બંધ કે ધીમા કરી શકાય છે.
- orientation: landscape અથવા portrait;  
ફોન આડો (Landscape) છે કે ઊભો (Portrait) તે જાણીને અલગ અલગ ડિઝાઇન બતાવી શકાય છે.

#### ૬. MODERN CONTAINER QUERIES (કન્ટેનર ક્વેરીઝ - મીડિયા ક્વેરીનો બાપ)

મીડિયા ક્વેરી આખી સ્ક્રીનની સાઈઝ જુએ છે. પણ જો તમારે આખી સ્ક્રીન નહીં, પણ એક નાના બોક્સ (જેમ કે સાઈડબાર) ની સાઈઝ નાની થાય ત્યારે તેની અંદરની ડિઝાઇન બદલવી હોય, તો મોડર્ન CSS માં @container નો ઉપયોગ થાય છે. આનાથી કમ્પોનન્ટ લેવલનું રિસ્પોન્સિવ કોડિંગ ખૂબ સરળ બની ગયું છે.

#### ૭. LOGICAL PROPERTIES (ભાષા પ્રમાણે આપોઆપ બદલાતી પ્રોપર્ટીઝ)

જૂના જમાનામાં આપણે margin-left કે padding-right વાપરતા હતા. પણ જો વેબસાઈટ ઉર્દૂ કે અરબી જેવી જમણેથી ડાબે (RTL - Right to Left) લખાતી ભાષાઓમાં ટ્રાન્સલેટ થાય, તો ડિઝાઇન બગડી જતી હતી. તેના ઉકેલ માટે લોજિકલ પ્રોપર્ટીઝ આવી છે:

- margin-inline-start: ડાબેથી લખાતી ભાષામાં આ ડાબી બાજુ માર્જિન આપશે, અને જમણેથી લખાતી ભાષામાં જમણી બાજુ!
- padding-block-end: નીચેની બાજુ પેડિંગ આપવા માટે.

આ તમામ કન્સેપ્ટ્સને મિલાવીને એક પ્રોપર મોડર્ન રિસ્પોન્સિવ વેબસાઈટ તૈયાર થાય છે.

આ ફાઈલને સાચવી રાખો અને કોડિંગ કરતી વખતે આનો રેફરન્સ તરીકે ઉપયોગ કરો!

# CSS Reference

(from w3schools.com <http://www.w3schools.com/css/>)

Prepared by Marc Seguin, December 2013

## CSS Properties

### CSS Property Groups

- Animation
- Background
- Border and outline
- Box
- Color
- Content Paged Media
- Dimension
- Flexible Box
- Font
- Generated content
- Grid
- Hyperlink
- Linebox
- List
- Margin
- Marquee
- Multi-column
- Padding
- Paged Media
- Positioning
- Print
- Ruby
- Speech
- Table
- Text
- 2D/3D Transform
- Transition
- User-interface

The "Ver." column indicates in which CSS version the property is defined (CSS1, CSS2, or CSS3).

### Animation Properties

Property	Description	Ver.
@keyframes	Specifies the animation	3
animation	A shorthand property for all the animation properties below, except the animation-play-state property	3
animation-name	Specifies a name for the @keyframes animation	3
animation-duration	Specifies how many seconds or milliseconds an animation takes to complete one cycle	3
animation-timing-function	Specifies the speed curve of the animation	3
animation-delay	Specifies when the animation will start	3
animation-iteration-count	Specifies the number of times an animation should be played	3
animation-direction	Specifies whether or not the animation should play in reverse on alternate cycles	3
animation-play-state	Specifies whether the animation is running or paused	3

### Background Properties

Property	Description	Ver.
background	Sets all the background properties in one declaration	1
background-attachment	Sets whether a background image is fixed or scrolls with the rest of the page	1
background-color	Sets the background color of an element	1
background-image	Sets the background image for an element	1
background-position	Sets the starting position of a background image	1
background-repeat	Sets how a background image will be repeated	1
background-clip	Specifies the painting area of the background	3
background-origin	Specifies the positioning area of the background images	3
background-size	Specifies the size of the background images	3

## **Border and Outline Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
border	Sets all the border properties in one declaration	1
border-bottom	Sets all the bottom border properties in one declaration	1
border-bottom-color	Sets the color of the bottom border	1
border-bottom-style	Sets the style of the bottom border	1
border-bottom-width	Sets the width of the bottom border	1
border-color	Sets the color of the four borders	1
border-left	Sets all the left border properties in one declaration	1
border-left-color	Sets the color of the left border	1
border-left-style	Sets the style of the left border	1
border-left-width	Sets the width of the left border	1
border-right	Sets all the right border properties in one declaration	1
border-right-color	Sets the color of the right border	1
border-right-style	Sets the style of the right border	1
border-right-width	Sets the width of the right border	1
border-style	Sets the style of the four borders	1
border-top	Sets all the top border properties in one declaration	1
border-top-color	Sets the color of the top border	1
border-top-style	Sets the style of the top border	1
border-top-width	Sets the width of the top border	1
border-width	Sets the width of the four borders	1
outline	Sets all the outline properties in one declaration	2
outline-color	Sets the color of an outline	2
outline-style	Sets the style of an outline	2
outline-width	Sets the width of an outline	2
border-bottom-left-radius	Defines the shape of the border of the bottom-left corner	3
border-bottom-right-radius	Defines the shape of the border of the bottom-right corner	3
border-image	A shorthand property for setting all the border-image-* properties	3
border-image-outset	Specifies the amount by which the border image area extends beyond the border box	3
border-image-repeat	Specifies whether the image-border should be repeated, rounded or stretched	3
border-image-slice	Specifies the inward offsets of the image-border	3
border-image-source	Specifies an image to be used as a border	3
border-image-width	Specifies the widths of the image-border	3
border-radius	A shorthand property for setting all the four border-*-radius properties	3
border-top-left-radius	Defines the shape of the border of the top-left corner	3
border-top-right-radius	Defines the shape of the border of the top-right corner	3
box-decoration-break		3
box-shadow	Attaches one or more drop-shadows to the box	3

## **Box Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
overflow-x	Specifies whether or not to clip the left/right edges of the content, if it overflows the element's content area	3
overflow-y	Specifies whether or not to clip the top/bottom edges of the content, if it overflows the element's content area	3
overflow-style	Specifies the preferred scrolling method for elements that overflow	3
rotation	Rotates an element around a given point defined by the rotation-point property	3
rotation-point	Defines a point as an offset from the top left border edge	3

## **Color Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
color-profile	Permits the specification of a source color profile other than the default	3
opacity	Sets the opacity level for an element	3
rendering-intent	Permits the specification of a color profile rendering intent other than the default	3

## **Content for Paged Media Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
bookmark-label	Specifies the label of the bookmark	3
bookmark-level	Specifies the level of the bookmark	3
bookmark-target	Specifies the target of the bookmark link	3
float-offset	Pushes floated elements in the opposite direction of the where they have been floated with float	3
hyphenate-after	Specifies the minimum number of characters in a hyphenated word after the hyphenation character	3
hyphenate-before	Specifies the minimum number of characters in a hyphenated word before the hyphenation character	3
hyphenate-character	Specifies a string that is shown when a hyphenate-break occurs	3
hyphenate-lines	Indicates the maximum number of successive hyphenated lines in an element	3
hyphenate-resource	Specifies a comma-separated list of external resources that can help the browser determine hyphenation points	3
hyphens	Sets how to split words to improve the layout of paragraphs	3
image-resolution	Specifies the correct resolution of images	3
marks	Adds crop and/or cross marks to the document	3
string-set		3

## **Dimension Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
height	Sets the height of an element	1
max-height	Sets the maximum height of an element	2

max-width	Sets the maximum width of an element	2
min-height	Sets the minimum height of an element	2
min-width	Sets the minimum width of an element	2
width	Sets the width of an element	1

### **Flexible Box Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
box-align	Specifies how to align the child elements of a box	3
box-direction	Specifies in which direction the children of a box are displayed	3
box-flex	Specifies whether the children of a box is flexible or inflexible in size	3
box-flex-group	Assigns flexible elements to flex groups	3
box-lines	Specifies whether columns will go onto a new line whenever it runs out of space in the parent box	3
box-ordinal-group	Specifies the display order of the child elements of a box	3
box-orient	Specifies whether the children of a box should be laid out horizontally or vertically	3
box-pack	Specifies the horizontal position in horizontal boxes and the vertical position in vertical boxes	3

### **Font Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
font	Sets all the font properties in one declaration	1
font-family	Specifies the font family for text	1
font-size	Specifies the font size of text	1
font-style	Specifies the font style for text	1
font-variant	Specifies whether or not a text should be displayed in a small-caps font	1
font-weight	Specifies the weight of a font	1
@font-face	A rule that allows websites to download and use fonts other than the "web-safe" fonts	3
font-size-adjust	Preserves the readability of text when font fallback occurs	3
font-stretch	Selects a normal, condensed, or expanded face from a font family	3

### **Generated Content Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
content	Used with the :before and :after pseudo-elements, to insert generated content	2
counter-increment	Increments one or more counters	2
counter-reset	Creates or resets one or more counters	2
quotes	Sets the type of quotation marks for embedded quotations	2
crop	Allows a replaced element to be just a rectangular area of an object, instead of the whole object	3

move-to	Causes an element to be removed from the flow and reinserted at a later point in the document	3
page-policy	Determines which page-based occurrence of a given element is applied to a counter or string value	3

### **Grid Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
grid-columns	Specifies the width of each column in a grid	3
grid-rows	Specifies the height of each column in a grid	3

### **Hyperlink Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
target	A shorthand property for setting the target-name, target-new, and target-position properties	3
target-name	Specifies where to open links (target destination)	3
target-new	Specifies whether new destination links should open in a new window or in a new tab of an existing window	3
target-position	Specifies where new destination links should be placed	3

### **Linebox Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
alignment-adjust	Allows more precise alignment of elements	3
alignment-baseline	Specifies how an inline-level element is aligned with respect to its parent	3
baseline-shift	Allows repositioning of the dominant-baseline relative to the dominant-baseline	3
dominant-baseline	Specifies a scaled-baseline-table	3
drop-initial-after-adjust	Sets the alignment point of the drop initial for the primary connection point	3
drop-initial-after-align	Sets which alignment line within the initial line box is used at the primary connection point with the initial letter box	3
drop-initial-before-adjust	Sets the alignment point of the drop initial for the secondary connection point	3
drop-initial-before-align	Sets which alignment line within the initial line box is used at the secondary connection point with the initial letter box	3
drop-initial-size	Controls the partial sinking of the initial letter	3
drop-initial-value	Activates a drop-initial effect	3
inline-box-align	Sets which line of a multi-line inline block align with the previous and next inline elements within a line	3
line-stacking	A shorthand property for setting the line-stacking-strategy, line-stacking-ruby, and line-stacking-shift properties	3
line-stacking-ruby	Sets the line stacking method for block elements containing ruby annotation elements	3
line-stacking-shift	Sets the line stacking method for block elements containing elements with base-shift	3

line-stacking-strategy	Sets the line stacking strategy for stacked line boxes within a containing block element	3
text-height	Sets the block-progression dimension of the text content area of an inline box	3

### **List Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
list-style	Sets all the properties for a list in one declaration	1
list-style-image	Specifies an image as the list-item marker	1
list-style-position	Specifies if the list-item markers should appear inside or outside the content flow	1
list-style-type	Specifies the type of list-item marker	1

### **Margin Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
margin	Sets all the margin properties in one declaration	1
margin-bottom	Sets the bottom margin of an element	1
margin-left	Sets the left margin of an element	1
margin-right	Sets the right margin of an element	1
margin-top	Sets the top margin of an element	1

### **Marquee Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
marquee-direction	Sets the direction of the moving content	3
marquee-play-count	Sets how many times the content move	3
marquee-speed	Sets how fast the content scrolls	3
marquee-style	Sets the style of the moving content	3

### **Multi-column Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
column-count	Specifies the number of columns an element should be divided into	3
column-fill	Specifies how to fill columns	3
column-gap	Specifies the gap between the columns	3
column-rule	A shorthand property for setting all the column-rule-* properties	3
column-rule-color	Specifies the color of the rule between columns	3
column-rule-style	Specifies the style of the rule between columns	3
column-rule-width	Specifies the width of the rule between columns	3
column-span	Specifies how many columns an element should span across	3
column-width	Specifies the width of the columns	3
columns	A shorthand property for setting column-width and column-count	3

### **Padding Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
padding	Sets all the padding properties in one declaration	1
padding-bottom	Sets the bottom padding of an element	1
padding-left	Sets the left padding of an element	1
padding-right	Sets the right padding of an element	1
padding-top	Sets the top padding of an element	1

### **Paged Media Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
fit	Gives a hint for how to scale a replaced element if neither its width nor its height property is auto	3
fit-position	Determines the alignment of the object inside the box	3
image-orientation	Specifies a rotation in the right or clockwise direction that a user agent applies to an image	3
page	Specifies a particular type of page where an element SHOULD be displayed	3
size	Specifies the size and orientation of the containing box for page content	3

### **Positioning Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
bottom	Specifies the bottom position of a positioned element	2
clear	Specifies which sides of an element where other floating elements are not allowed	1
clip	Clips an absolutely positioned element	2
cursor	Specifies the type of cursor to be displayed	2
display	Specifies how a certain HTML element should be displayed	1
float	Specifies whether or not a box should float	1
left	Specifies the left position of a positioned element	2
overflow	Specifies what happens if content overflows an element's box	2
position	Specifies the type of positioning method used for an element (static, relative, absolute or fixed)	2
right	Specifies the right position of a positioned element	2
top	Specifies the top position of a positioned element	2
visibility	Specifies whether or not an element is visible	2
z-index	Sets the stack order of a positioned element	2

### **Print Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
orphans	Sets the minimum number of lines that must be left at the bottom of a page when a page break occurs inside an element	2
page-break-after	Sets the page-breaking behavior after an element	2
page-break-before	Sets the page-breaking behavior before an element	2

page-break-inside	Sets the page-breaking behavior inside an element	2
widows	Sets the minimum number of lines that must be left at the top of a page when a page break occurs inside an element	2

### **Ruby Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
ruby-align	Controls the text alignment of the ruby text and ruby base contents relative to each other	3
ruby-overhang	Determines whether, and on which side, ruby text is allowed to partially overhang any adjacent text in addition to its own base, when the ruby text is wider than the ruby base	3
ruby-position	Controls the position of the ruby text with respect to its base	3
ruby-span	Controls the spanning behavior of annotation elements	3

### **Speech Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
mark	A shorthand property for setting the mark-before and mark-after properties	3
mark-after	Allows named markers to be attached to the audio stream	3
mark-before	Allows named markers to be attached to the audio stream	3
phonemes	Specifies a phonetic pronunciation for the text contained by the corresponding element	3
rest	A shorthand property for setting the rest-before and rest-after properties	3
rest-after	Specifies a rest or prosodic boundary to be observed after speaking an element's content	3
rest-before	Specifies a rest or prosodic boundary to be observed before speaking an element's content	3
voice-balance	Specifies the balance between left and right channels	3
voice-duration	Specifies how long it should take to render the selected element's content	3
voice-pitch	Specifies the average pitch (a frequency) of the speaking voice	3
voice-pitch-range	Specifies variation in average pitch	3
voice-rate	Controls the speaking rate	3
voice-stress	Indicates the strength of emphasis to be applied	3
voice-volume	Refers to the amplitude of the waveform output by the speech synthesises	3

### **Table Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
border-collapse	Specifies whether or not table borders should be collapsed	2
border-spacing	Specifies the distance between the borders of adjacent cells	2
caption-side	Specifies the placement of a table caption	2
empty-cells	Specifies whether or not to display borders and background on empty cells in a table	2
table-layout	Sets the layout algorithm to be used for a table	2

## **Text Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
color	Sets the color of text	1
direction	Specifies the text direction/writing direction	2
letter-spacing	Increases or decreases the space between characters in a text	1
line-height	Sets the line height	1
text-align	Specifies the horizontal alignment of text	1
text-decoration	Specifies the decoration added to text	1
text-indent	Specifies the indentation of the first line in a text-block	1
text-transform	Controls the capitalization of text	1
unicode-bidi	Used together with the direction property to set or return whether the text should be overridden to support multiple languages in the same document	2
vertical-align	Sets the vertical alignment of an element	1
white-space	Specifies how white-space inside an element is handled	1
word-spacing	Increases or decreases the space between words in a text	1
hanging-punctuation	Specifies whether a punctuation character may be placed outside the line box	3
punctuation-trim	Specifies whether a punctuation character should be trimmed	3
text-align-last	Describes how the last line of a block or a line right before a forced line break is aligned when text-align is "justify"	3
text-justify	Specifies the justification method used when text-align is "justify"	3
text-outline	Specifies a text outline	3
text-overflow	Specifies what should happen when text overflows the containing element	3
text-shadow	Adds shadow to text	3
text-wrap	Specifies line breaking rules for text	3
word-break	Specifies line breaking rules for non-CJK scripts	3
word-wrap	Allows long, unbreakable words to be broken and wrap to the next line	3

## **2D/3D Transform Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
transform	Applies a 2D or 3D transformation to an element	3
transform-origin	Allows you to change the position on transformed elements	3
transform-style	Specifies how nested elements are rendered in 3D space	3
perspective	Specifies the perspective on how 3D elements are viewed	3
perspective-origin	Specifies the bottom position of 3D elements	3
backface-visibility	Defines whether or not an element should be visible when not facing the screen	3

## **Transition Properties**

<b>Property</b>	<b>Description</b>	<b>Ver.</b>
transition	A shorthand property for setting the four transition properties	3
transition-property	Specifies the name of the CSS property the transition effect is for	3

transition-duration	Specifies how many seconds or milliseconds a transition effect takes to complete	3
transition-timing-function	Specifies the speed curve of the transition effect	3
transition-delay	Specifies when the transition effect will start	3

### **User-interface Properties**

Property	Description	Ver.
appearance	Allows you to make an element look like a standard user interface element	3
box-sizing	Allows you to define certain elements to fit an area in a certain way	3
icon	Provides the author the ability to style an element with an iconic equivalent	3
nav-down	Specifies where to navigate when using the arrow-down navigation key	3
nav-index	Specifies the tabbing order for an element	3
nav-left	Specifies where to navigate when using the arrow-left navigation key	3
nav-right	Specifies where to navigate when using the arrow-right navigation key	3
nav-up	Specifies where to navigate when using the arrow-up navigation key	3
outline-offset	Offsets an outline, and draws it beyond the border edge	3
resize	Specifies whether or not an element is resizable by the user	3

### **CSS Selectors**

In CSS, selectors are patterns used to select the element(s) you want to style.

The "CSS" column indicates in which CSS version the property is defined (CSS1, CSS2, or CSS3).

Selector	Example	Example description	CSS
<i>.class</i>	intro	Selects all elements with class="intro"	1
<i>#id</i>	#firstname	Selects the element with id="firstname"	1
<i>*</i>	*	Selects all elements	2
<i>element</i>	p	Selects all <p> elements	1
<i>element,element</i>	div,p	Selects all <div> elements and all <p> elements	1
<i>element element</i>	div p	Selects all <p> elements inside <div> elements	1
<i>element&gt;element</i>	div>p	Selects all <p> elements where the parent is a <div> element	2
<i>element+element</i>	div+p	Selects all <p> elements that are placed immediately after <div> elements	2
<i>[attribute]</i>	[target]	Selects all elements with a target attribute	2
<i>[attribute=value]</i>	[target=_blank]	Selects all elements with target="_blank"	2
<i>[attribute~=value]</i>	[title~=flower]	Selects all elements with a title attribute containing the word "flower"	2
<i>[attribute =value]</i>	[lang=en]	Selects all elements with a lang attribute value starting with "en"	2
<i>:link</i>	a:link	Selects all unvisited links	1
<i>:visited</i>	a:visited	Selects all visited links	1

:active	a:active	Selects the active link	1
:hover	a:hover	Selects links on mouse over	1
:focus	input:focus	Selects the input element which has focus	2
:first-letter	p:first-letter	Selects the first letter of every <p> element	1
:first-line	p:first-line	Selects the first line of every <p> element	1
:first-child	p:first-child	Selects every <p> element that is the first child of its parent	2
:before	p:before	Insert content before the content of every <p> element	2
:after	p:after	Insert content after every <p> element	2
:lang( <i>language</i> )	p:lang(it)	Selects every <p> element with a lang attribute equal to "it" (Italian)	2
<i>element1~element2</i>	p~ul	Selects every    element that are preceded by a <p> element	3
[ <i>attribute^=value</i> ]	a[src^="https"]	Selects every <a> element whose src attribute value begins with "https"	3
[ <i>attribute\$=value</i> ]	a[src\$=".pdf"]	Selects every <a> element whose src attribute value ends with ".pdf"	3
[ <i>attribute*=value</i> ]	a[src*="w3schools"]	Selects every <a> element whose src attribute value contains the substring "w3schools"	3
:first-of-type	p:first-of-type	Selects every <p> element that is the first <p> element of its parent	3
:last-of-type	p:last-of-type	Selects every <p> element that is the last <p> element of its parent	3
:only-of-type	p:only-of-type	Selects every <p> element that is the only <p> element of its parent	3
:only-child	p:only-child	Selects every <p> element that is the only child of its parent	3
:nth-child( <i>n</i> )	p:nth-child(2)	Selects every <p> element that is the second child of its parent	3
:nth-last-child( <i>n</i> )	p:nth-last-child(2)	Selects every <p> element that is the second child of its parent, counting from the last child	3
:nth-of-type( <i>n</i> )	p:nth-of-type(2)	Selects every <p> element that is the second <p> element of its parent	3
:nth-last-of-type( <i>n</i> )	p:nth-last-of-type(2)	Selects every <p> element that is the second <p> element of its parent, counting from the last child	3
:last-child	p:last-child	Selects every <p> element that is the last child of its parent	3
:root	:root	Selects the document's root element	3
:empty	p:empty	Selects every <p> element that has no children (including text nodes)	3
:target	#news:target	Selects the current active #news element (clicked on a URL containing that anchor name)	3
:enabled	input:enabled	Selects every enabled <input> element	3
:disabled	input:disabled	Selects every disabled <input> element	3
:checked	input:checked	Selects every checked <input> element	3
:not( <i>selector</i> )	:not(p)	Selects every element that is not a <p> element	3
::selection	::selection	Selects the portion of an element that is selected by a user	3

## Font Reference

Example: `p{font-family:"Times New Roman", Times, serif}`

### Serif Fonts

font-family	Example text
Georgia, serif	<b>This is a heading</b> This is a paragraph
"Palatino Linotype", "Book Antiqua", Palatino, serif	<b>This is a heading</b> This is a paragraph
"Times New Roman", Times, serif	<b>This is a heading</b> This is a paragraph

### Sans-Serif Fonts

font-family	Example text
Arial, Helvetica, sans-serif	<b>This is a heading</b> This is a paragraph
"Arial Black", Gadget, sans-serif	<b>This is a heading</b> <b>This is a paragraph</b>
"Comic Sans MS", cursive, sans-serif	<b>This is a heading</b> This is a paragraph
Impact, Charcoal, sans-serif	<b>This is a heading</b> <b>This is a paragraph</b>
"Lucida Sans Unicode", "Lucida Grande", sans-serif	<b>This is a heading</b> This is a paragraph
Tahoma, Geneva, sans-serif	<b>This is a heading</b> This is a paragraph
"Trebuchet MS", Helvetica, sans-serif	<b>This is a heading</b> This is a paragraph
Verdana, Geneva, sans-serif	<b>This is a heading</b> This is a paragraph

### Monospace Fonts

font-family	Example text
"Courier New", Courier, monospace	<b>This is a heading</b> This is a paragraph
"Lucida Console", Monaco, monospace	<b>This is a heading</b> This is a paragraph

## **Colour Reference**

Colors are displayed combining RED, GREEN, and BLUE light.

### **Color Values**

CSS colors are defined using a **hexadecimal (hex) notation** for the combination of Red, Green, and Blue color values (RGB). The lowest value that can be given to one of the light sources is 0 (hex 00). The highest value is 255 (hex FF). Hex values are written as 3 double digit numbers, **starting with a # sign**.

### **Color Examples**

Color	Color HEX	Color RGB	Name
	#000000	rgb(0,0,0)	black
	#FF0000	rgb(255,0,0)	red
	#00FF00	rgb(0,255,0)	green
	#0000FF	rgb(0,0,255)	blue
	#FFFF00	rgb(255,255,0)	yellow
	#00FFFF	rgb(0,255,255)	cyan
	#FF00FF	rgb(255,0,255)	magenta
	#C0C0C0	rgb(192,192,192)	grey
	#FFFFFF	rgb(255,255,255)	white

### **16 Million Different Colors**

The combination of Red, Green and Blue values from 0 to 255 gives a total of more than 16 million different colors to play with (256 x 256 x 256).

Most modern monitors are capable of displaying at least 16384 different colors.

See: [http://www.w3schools.com/cssref/css\\_colors.asp](http://www.w3schools.com/cssref/css_colors.asp)



ଓଡ଼ିଶା ରାଜ୍ୟ ମୁକ୍ତ ବିଶ୍ୱବିଦ୍ୟାଳୟ, ସମ୍ବଲପୁର, ଓଡ଼ିଶା  
**Odisha State Open University, Sambalpur, Odisha**  
Established by an Act of Government of Odisha.

---

**DIPLOMA IN COMPUTER APPLICATION**

**DCA-04**

**WEB -DESIGN**

**Block**

**2**

**CASCADING STYLE SHEETS (CSS)**

---

**Unit -3**

**Getting Started With CSS**

---

**Unit -4**

**CSS Advanced**

---



ଓଡ଼ିଶା ରାଜ୍ୟ ମୁକ୍ତ ବିଶ୍ୱବିଦ୍ୟାଳୟ, ସମ୍ବଲପୁର, ଓଡ଼ିଶା  
Odisha State Open University, Sambalpur, Odisha  
Established by an Act of Government of Odisha.

### EXPERT COMMITTEE

<b>Dr.P.K.Behera</b> Reader in Computer Science Utkal University , Odisha	<b>(Chairman)</b>
<b>Dr.J.R.Mohanty</b> Prof And HOD KIIT University, Odisha	<b>(Member)</b>
<b>ShPabitrandaPattnaik</b> Scientist –E,NIC NIC, ,BhubaneswarOdisha	<b>(Member)</b>
<b>Sh Malaya Kumar Das</b> Scientist –E , NIC Bhubaneswar , Odisha	<b>(Member)</b>
<b>Dr.BhagirathiNayak</b> Professor And Head (IT & System) Sri Sri University, Odisha	<b>(Member)</b>
<b>Dr.ManoranjanPradhan</b> Professor and Head G.I.T.A Bhubaneswar, Odisha	<b>(Member)</b>
<b>Mr.V.S.Sandilya</b> Academic Consultant I.T Odisha State Open University, Sambalpur,Odisha	<b>(Convener)</b>

### DIPLOMA IN COMPUTER APPLICATION

### COURSE WRITER

**Mr. Dhruba Charan Pradhan**  
Department of Computer Science,  
OAVS, Th. Rampur, Bhabanipatna,  
Kalahandi, Odisha

**Mr. Sushanta Kumar Mohanty**  
Department of Information Technology,  
Shailabala Women's College, Cuttack,  
Odisha

---

## Unit -3

# Getting Started With CSS

---

### Learning objectives:

After the Completion of this unit you should be able to know

- ✎ What a style sheet is and how it actually styles a web page
- ✎ The importance of CSS.
- ✎ How to create a style sheet and link an html document to the style sheet.
- ✎ The basic building blocks of any style sheet: rules, selectors, properties and values.
- ✎ How External Style Sheets are stored in CSS files
- ✎ Three ways of inserting a style sheet
- ✎ Different selectors using CSS
- ✎ Define background properties ,Background colour & images handling
- ✎ Font family, Style and Size using CSS
- ✎ Designing Tables using CSS

### Structure

- 3.1 Introduction of CSS
  - 3.1.1 Definition
  - 3.1.2 Advantages
  - 3.1.3 Parts of CSS
- 3.2 CSS Syntax
  - 3.2.1 Rules / Principle of CSS
  - 3.2.2 Parts of style sheet
- 3.3 CSS Selectors
  - 3.3.1 The element selectors
  - 3.3.2 The ID selectors
  - 3.3.3 The Class selectors
  - 3.3.4 Universal Selector
  - 3.3.5 Attribute Selector
- 3.4 Ways to Insert CSS
  - 3.4.1 External style sheet
  - 3.4.2 Internal style sheet
  - 3.4.3 Inline style
- 3.5 Background image handling
  - 3.5.1 Repeat the Background Image
  - 3.5.2 Set the background image position
- 3.6 Background colour management using CSS
  - 3.6.1 CSS color hex code
  - 3.6.2 CSS color -short hex code
  - 3.6.3 CSS color RGB value
- 3.7 Text management using CSS
  - 3.7.1 Set the text color

- 3.7.2 Set the text direction
  - 3.7.3 Set the space between character
  - 3.7.4 Set the space between word
  - 3.7.5 Set the text indent
  - 3.7.6 Set the text alignment
  - 3.7.7 Decorating the text
  - 3.7.8 Set the text cases
- 3.8 Font management using CSS
  - 3.8.1. Set the font family
  - 3.8.2 Set the style
  - 3.8.3 Set the font variant
  - 3.8.4 Set the font weight
  - 3.8.5 Set the Font size
- 3.9 Managing Hyperlinks using CSS
  - 3.9.1 Set the color of Visited link
  - 3.9.2 Change the Color of Links when Mouse is Over
  - 3.9.3 Change the color of active link
- 3.10 Managing Lists using CSS
  - 3.10.1 The list style type Property
  - 3.10.2 The list-style-position Property
- 3.11 Designing Tables using CSS
  - 3.11.1 The empty cells Property
  - 3.11.2 The border-spacing Property
- 3.12 Working with the BOX Model
  - 3.12.1 The Height and Width Properties
  - 3.12.2 The line height property
  - 3.12.3 The max height property
  - 3.12.4 The max width property
- 3.13 Designing Borders using CSS
  - 3.13.1 The border color property
  - 3.13.2 The border-style Property
- 3.14 Designing outline using CSS
  - 3.14.1 The outline width property
  - 3.14.2 The outline style
  - 3.14.3 The outline-color property
- 3.15 Setting Page Margin using CSS
  - 3.15.1 The Margin Property
  - 3.15.2 The Margin Bottom Property
  - 3.15.3 The Margin left Property
- 3.16 Let us sum up
- 3.17 Reference
- 3.18 Check Your Progress-possible answers

---

## 3.1 Introduction of CSS

---

CSS stands for Cascading Style Sheets. It is a simple design language intended to simplify the process of making web pages presentable. CSS handles the look and feel part of a web page. Using CSS, you can control the color of the text, the style of fonts, the spacing between paragraphs, how columns are sized and laid out, what background images or colors are used, as well as a variety of other effects.

CSS works with HTML and other Markup Languages (such as XHTML and XML) to control the way the content is presented. Cascading Style Sheets is a means to separate the appearance of a webpage from the content of a webpage.

### 3.1.1 Definition

**Cascading Style Sheets (CSS)** is a simple mechanism used to format the layout of Web Pages and adding **style** (e.g., fonts, colors, spacing...) to web documents that previously could only be defined in a page's HTML. CSS describes how HTML elements are to be displayed on screen, paper, or in other media. It can control the layout of multiple web pages all at once.

### 3.1.2 Advantages

The advantages of CSS are:

- ❖ **CSS saves time** - You can write CSS once and then reuse the same sheet in multiple HTML pages.
- ❖ **Pages load faster** – Increases Download Speed
- ❖ **Easy maintenance** - To make a global change, all the elements in all the web pages will be updated automatically.
- ❖ **Superior styles to HTML** – It is better look to your HTML page in comparison to HTML attributes.
- ❖ **Multiple Device Compatibility** - Style sheets allow content to be optimized for more than one type of device.
- ❖ **Global web standards** - Now HTML attributes are being deprecated and it is being recommended to use CSS

### 3.1.3 What is the “Cascade” part of CSS?

The cascade part of CSS means that more than one style sheet can be attached to a document, and all of them can influence the presentation. For example, a designer can have a global style sheet for the whole site, but a local one for say, controlling the link color and background of a specific page. Or, a user can use own style sheet if s/he has problems seeing the page, or if s/he just prefers a certain look.

---

## 3.2 CSS Syntax

---

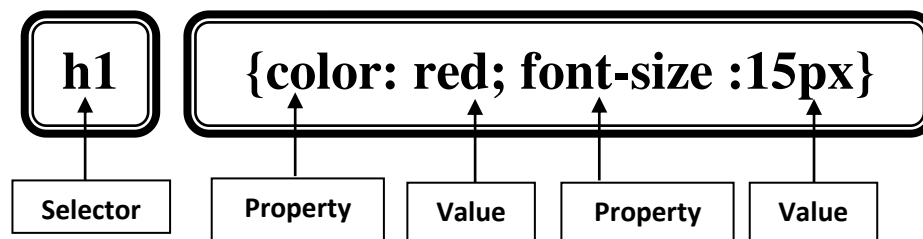
A CSS style rule is made of three parts:

1. **Selector:** A selector is an HTML tag at which a style will be applied. This could be any tag like <h1>, <p> or <table> etc.
2. **Property:** A property is a type of attribute of HTML tag. Put simply, all the HTML attributes are converted into CSS properties. They could be *color*, *border*, *bgcolor* etc.
3. **Value:** Values are assigned to properties. For example, *color* property can have the value either *red* or *#F1F1F1* etc.

The format or syntax of CSS is:

**selector {property:**

**Example:** You can define a heading as follows:



Here **h1** is a selector , **color and font-size** are properties and the given value **red, and 15px** are the value of that property.

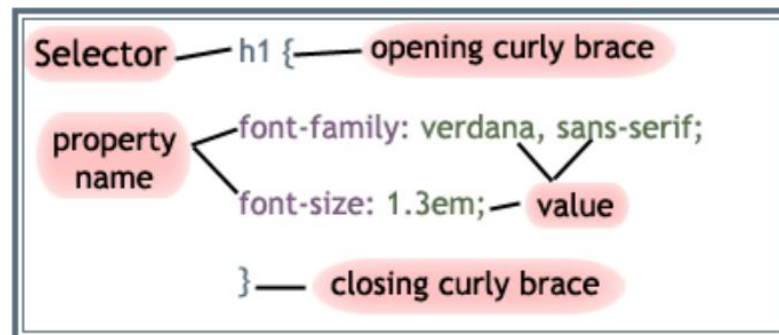
- ✂ The selector is normally the HTML element you want to style.
- ✂ Each declaration consists of a property and a value.
- ✂ The property is the style attribute you want to change. Each property has a value.

### 3.2.1 Rules/ Principle of CSS

1. Every statement must have a selector and a declaration. The declaration comes immediately after the selector and is contained in a pair of curly braces.
2. The declaration is one or more properties separated by semicolons.
3. Each property has a property name followed by a colon and then the value for that property. There are many different types of values, but any given property can only take certain values as set down in the specification.
4. Sometimes a property can take a number of values, as in the font-family. The values in the list should be separated by a comma and a space.
5. Sometimes a value will have a unit as well as the actual value, as in the 1.3em. You must not put a space between the value and its unit.
6. As with HTML, white space can be used to make your style sheet easier to read and write.

### 3.2.2Parts of style sheet

A style sheet consists of one or more rules that describe how document elements should be displayed. A rule in CSS has two parts: the selector and the declaration. The declaration also has two parts, the property and the value. Let's take a look at a rule for a heading 1 style: `h1 { font-family: verdana, "sans serif"; font-size: 1.3em }` This expression is a rule that says every h1 tag will be verdana or other sans-serif font and the font size will be 1.3em. Let's take a look at the different parts of this rule.



Selector

```
{
 property1: some value;
 property2: some value;
}
```

The declaration contains the property and value for the selector. The property is the attribute you wish to change and each property can take a value. The property and value are separated by a colon and surrounded by curly braces:

```
body { background-color: black }
```

If the value of a property is more than one word, put quotes around that value: `body { font-family: "sans serif"; }` If you wish to specify more than one property, you must use a semi-colon to separate each property. This rule defines a paragraph that will have blue text that is centered.

```
p { text-align: center; color: blue }
```

You can group selectors. Separate each selector with a comma. The example below groups headers 1, 2, and 3 and makes them all yellow. `h1, h2, h3 { color: yellow }`

---

#### Check your progress 1

---

**Q1. Write the abbreviation of CSS?**

Answer: \_\_\_\_\_

**Q2. Write the three parts of CSS syntax?**

Answer: \_\_\_\_\_

---

### 3.3 CSS Selectors

---

You can define selectors in various simple ways based on your comfort. Let me put these selectors one by one. Three types of CSS Selectors

1. The Element selectors
2. The ID Selectors
3. The Class Selectors

#### **3.3.1 The Element selectors**

A CSS declaration always ends with a semicolon, and declaration groups are surrounded by curly brackets: example -

```
p {color:red;text-align:center;}
```

To make the CSS more readable, you can put one declaration on each line, like this:

```
p
{
color:red;
text-align:center;
}
```

#### **3.3.2 The ID selectors**

The id selector is used to specify a style for a single, unique element. The id selector uses the id attribute of the HTML element, and is defined with a "#". Example –

Imagine within the body element of our html page, we have the following paragraph element

```
<p id="welcome">welcome to the 1st CSS
Document </p>
```

We can then create a CSS rule with the id selector:

```
#welcome
{
color:red;
text-align:center;
}
```

#### **3.3.3 The Class selectors**

The class selector is used to specify a style for a group of elements. Unlike the id selector, the class selector is most often used on several elements. This allows you to set a particular style for many HTML elements with the same class. The class selector uses the HTML class attribute, and is defined with a ".". In the example below, all HTML elements with class="center" will be center-aligned:

Imagine within the body element of our html page, we have the following header element

```
<h2 class="center">Summary</h2>
```

We can then create a CSS rule with the class selector:

```
.center {text-align:center;}
```

You can also specify that only specific HTML elements should be affected by a class. In the example below, all p elements with class="center" will be center-aligned: example

```
p.center {text-align:center;}
```

Some of the other selectors are used in CSS, they are :

#### 3.3.3.1 **Universal selector**

An asterisk (\*) is the universal selector for CSS. It matches a single element of any type. Omitting the asterisk with simple selectors has the same effect. For instance, \*.warning and . warning are considered equal. Rather than selecting elements of a specific type, the universal selector quite simply matches the name of any element type: Example-

```
*
{
 color:#000000;
}
```

This rule renders the content of every element in our document in black.

#### 3.3.3.2 **Attribute Selector**

You can also apply styles to HTML elements with particular attributes. The style rule below will match all the input elements having a type attribute with a value of text:

```
input[type="text"]
{
 color: #000000;
}
```

The advantage to this method is that the <input type="submit" /> element is unaffected, and the color applied only to the desired text fields. There are following rules applied to attribute selector.

- p[lang] - Selects all paragraph elements with a lang attribute.
- p[lang="fr"] - Selects all paragraph elements whose lang attribute has a value of exactly "fr".
- p[lang~="fr"] - Selects all paragraph elements whose lang attribute contains the word "fr".
- p[lang|="en"] - Selects all paragraph elements whose lang attribute contains values that are exactly "en", or begin with "en-".

---

### **Check your progress 2**

---

**Q1. What are the 3 types of selectors?.**

**Answer:**\_\_\_\_\_

**Q2. What is universal selector ?.**

**Answer:**\_\_\_\_\_

---

### 3.4 Ways to insert CSS

---

There are three ways of inserting a style sheet:

1. External style sheet
2. Internal style sheet
3. Inline style

#### **3.4.1 External style sheet**

An external style sheet is ideal when the style is applied to many pages. With an external style sheet, you can change the look of an entire Web site by changing one file. Each page must link to the style sheet using the <link> tag. The <link> tag goes inside the head section:

```
<head>
```

```
<link rel="stylesheet" type="text/css" href="mystyle.css" />
```

```
</head>
```

An external style sheet can be written in any text editor. The file should not contain any html tags. Your style sheet should be saved with a .css extension. An example of a style sheet file is shown below:

```
hr {color:sienna;}
p {margin-left:20px;}
body {background-image:url("images/back40.gif");}
```

*Notes* : Do not leave spaces between the property value and the units! "margin-left:20 px" (instead of "margin-left:20px") will work in IE, but not in Firefox or Opera.

#### **3.4.2 Internal style sheet**

An internal style sheet should be used when a single document has a unique style. You define internal styles in the head section of an HTML page, by using the <style> tag, like this:

```
<head>
```

```
<style type="text/css">
```

```
hr {color:sienna;}
p {margin-left:20px;}
body {background-image:url("images/back1.gif");}
```

```
</style>
```

```
</head>
```

#### **3.4.3 Inline style sheet**

An inline style loses many of the advantages of style sheets by mixing content with presentation. Use this method sparingly! To use inline styles you use the style attribute in the relevant tag. The style attribute can contain any CSS property. The example shows how to change the color and the left margin of a paragraph:

`<p style="color:sienna;margin-left:20px">This is a  
paragraph.</p>`

---

### 3.5 Background image handling

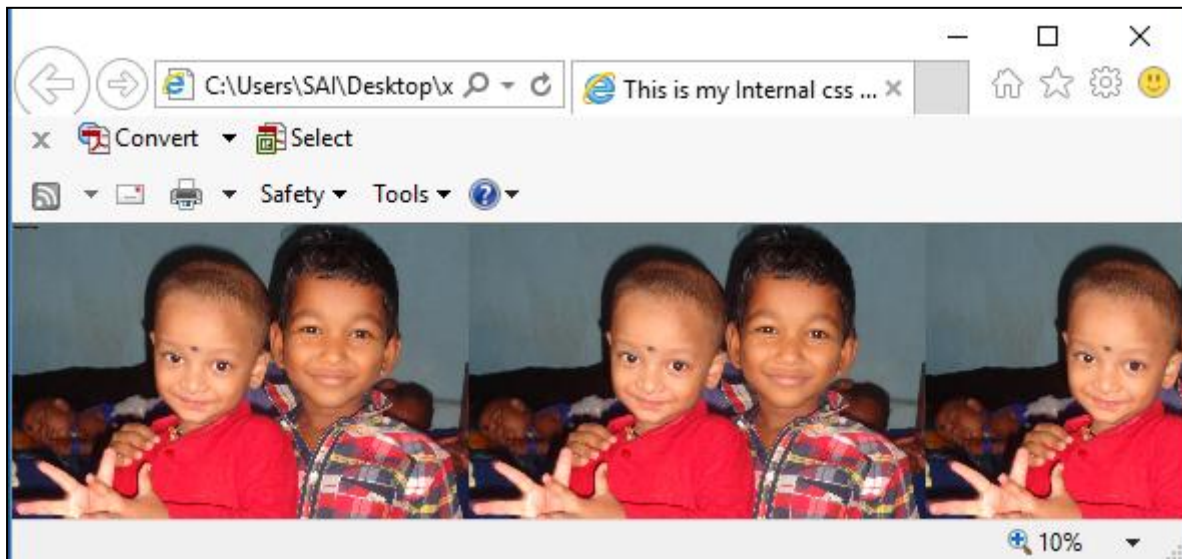
---

The background-image property specifies an image to use as the background of an element. By default, the image is repeated so it covers the entire element. The background image for a page can be set like this: `body {background-image:url('paper.gif');}`

Example

```
<html>
<head>
 <title>This is my Internal css page</title>
 <style type="text/css">
 body
 {
 background-image:url
 ("C:/Users/SAI/Desktop/Desktop/100MSDCF/11.
 jpg");
 }
 </style>
</head>
<body>
 Background Image
</body>
</html>
```

The output of the above example is :



You can set the following background properties of an element:

- The **background-image** property is used to set the background image of an element.
- The **background-repeat** property is used to control the repetition of an image in the background.
- The **background-position** property is used to control the position of an image in the background.
- The **background-attachment** property is used to control the scrolling of an image in the background.
- The background property is used as shorthand to specify a number of other background properties.

### **3.5.1 Repeat the Background Image**

The following example demonstrates how to repeat the background image if an image is small. You can use no-repeat value for the background-repeat property if you don't want to repeat an image. In this case, the image will display only once. By default, the background-repeat property will have a repeat value.

```
<table style="background-image:url(/images/pattern1.gif); background-repeat: repeat;">
<tr><td>
This table has background image which repeats multiple times.
</td></tr>
</table>
```

The following example which demonstrates how to repeat the background image **vertically**.

```
<table style="background-image:url(/images/pattern1.gif); background-repeat: repeat-y;">
<tr><td>
This table has background image set which will repeat vertically. </td></tr>
</table>
```

The following example demonstrates how to repeat the background image **horizontally**.

```
<table style="background-image:url(/images/pattern1.gif); background-repeat: repeat-x;">
<tr><td>
This table has background image set which will repeat horizontally. </td></tr>
</table>
```

### **3.5.2 Set the Background Image Position**

The following example demonstrates how to set the background image position 100 pixels away from the left side.

```
<table style="background-image:url(/images/pattern1.gif);
background-position:100px;">
<tr><td>
Background image positioned 100 pixels away from the left.
</td></tr>
</table>
```

---

### Check your progress 3

---

**Q1. How many ways to build style sheet ? what are they ?**

**Answer:** \_\_\_\_\_

**Q2. Write the background properties of CSS.**

**Answer:** \_\_\_\_\_

---

### 3.6 Background colour Management using CSS

---

The background-color property specifies the background color of an element. The background color of a page is defined in the body selector: Example

```
body {background-color:#b0c4de;}
<p style="background-color:yellow;">
This text has a yellow background color. </p>
```

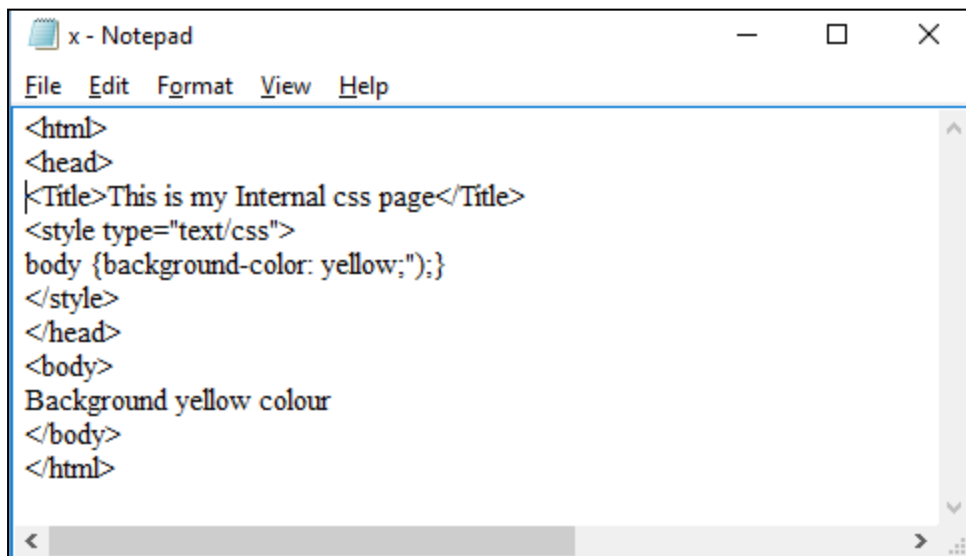
With CSS, a color is most often specified by:

1. a HEX value - like "#ff0000"
2. an RGB value - like "rgb(255,0,0)"
3. a color name - like "red"

In the example below, the h1, p, and div elements have different background colors:

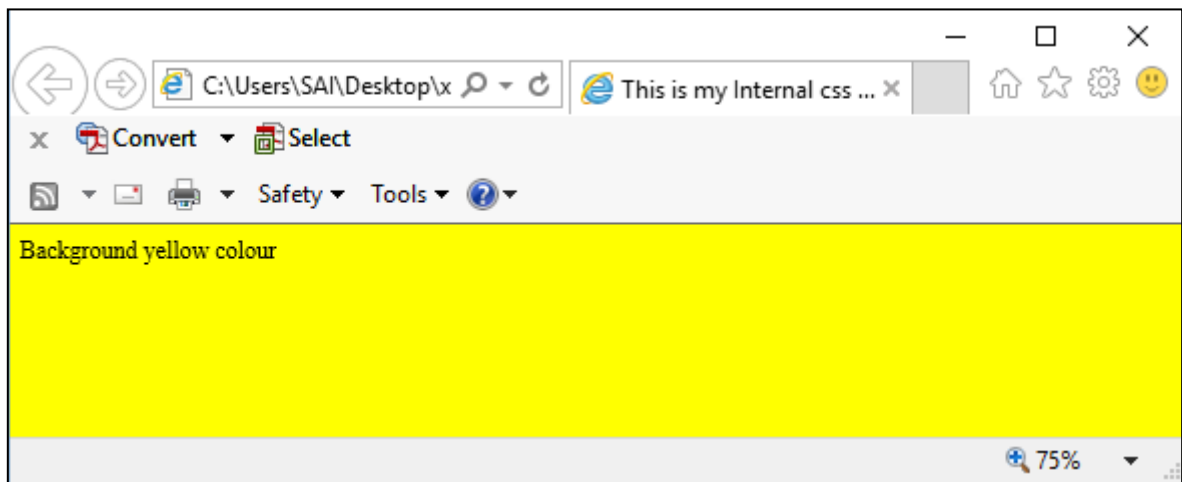
```
h1 {background-color:#6495ed;}
p {background-color:#e0ffff;}
div {background-color:#b0c4de;}
```

Example

A screenshot of a Notepad window titled 'x - Notepad'. The window contains the following HTML code:

```
<html>
<head>
<Title>This is my Internal css page</Title>
<style type="text/css">
body {background-color: yellow;};
</style>
</head>
<body>
Background yellow colour
</body>
</html>
```

The code defines a CSS rule for the body element with a yellow background color. The body content is 'Background yellow colour'.



### **3.6.1 CSS Colors - Hex Codes**

A hexadecimal is a 6 digit representation of a color. The first two digits (RR) represent a red value, the next two are a green value (GG), and the last are the blue value (BB). Each hexadecimal code will be preceded by a pound or hash sign '#'. Following are the examples to use Hexadecimal notation.

### **3.6.2 CSS Colors - Short Hex Codes**

This is a shorter form of the six-digit notation. In this format, each digit is replicated to arrive at an equivalent six-digit value. For example: #6A7 becomes #66AA77.

Color	Color HEX
	#000000
	#FF0000
	#00FF00
	#0000FF
	#FFFF00
	#00FFFF
	#FF00FF

Color	Color HEX
	#000
	#F00
	#0F0
	#0FF
	#FF0
	#0FF
	#F0F
	#FFF

### 3.6.3 CSS Colors - RGB Values

This color value is specified using the `rgb( )` property. This property takes three values, one each for red, green, and blue. The value can be an integer between 0 and 255 or a percentage. NOTE: All the browsers does not support `rgb()` property of color, so it is recommended not to use it.

Color	Color RGB
	<code>rgb(0,0,0)</code>
	<code>rgb(255,0,0)</code>
	<code>rgb(0,255,0)</code>
	<code>rgb(0,0,255)</code>
	<code>rgb(255,255,0)</code>
	<code>rgb(0,255,255)</code>

---

### 3.7 Text Management using CSS

---

**CSS** is a language that describes the style of an HTML document. You can set the following text properties of an element:

- The **color** property is used to set the color of a text.
- The **direction** property is used to set the text direction.
- The **letter-spacing** property is used to add or subtract space between the letters that make up a word.
- The **word-spacing** property is used to add or subtract space between the words of a sentence.
- The **text-indent** property is used to indent the text of a paragraph.
- The **text-align** property is used to align the text of a document.
- The **text-decoration** property is used to underline, overline, and strikethrough text.

- The **text-transform** property is used to capitalize text or convert text to uppercase or lowercase letters.
- The **white-space** property is used to control the flow and formatting of text.
- The **text-shadow** property is used to set the text shadow around a text.

### **3.7.1 Set the Text Color**

The following example demonstrates how to set the text color. Possible value could be any color name in any valid format.

```
<p style="color:red;">This text will be written in red. </p>
```

### **3.7.2 Set the Text Direction**

The following example demonstrates how to set the direction of a text. Possible values are ltr or rtl.

```
<p style="direction:rtl;">This text will be rendered from right to left </p>
```

### **3.7.3. Set the Space between Characters**

The following example demonstrates how to set the space between characters. Possible values are normal or a number specifying space.

```
<p style="letter-spacing:5px;">This text is having space between letters. </p>
```

### **3.7.4 Set the Space between Words**

The following example demonstrates how to set the space between words. Possible values are normal or a number specifying space.

```
<p style="word-spacing:5px;">This text is having space between words. </p>
```

### **3.7.5 Set the Text Indent**

The following example demonstrates how to indent the first line of a paragraph. Possible values are % or a number specifying indent space.

```
<p style="text-indent:1cm;">
 This text will have first line indented by 1cm and this line will
 remain at its actual position this is done by CSS text-indent
 property.
</p>
```

### **3.7.6 Set the Text Alignment**

The following example demonstrates how to align a text. Possible values are left, right, center, justify.

```
<p style="text-align:right;"> This will be right aligned. </p>
<p style="text-align:center;"> This will be center aligned. </p>
<p style="text-align:left;"> This will be left aligned. </p>
```

### 3.7.7 Decorating the Text

The following example demonstrates how to decorate a text. Possible values are none, underline, overline, line-through, blink.

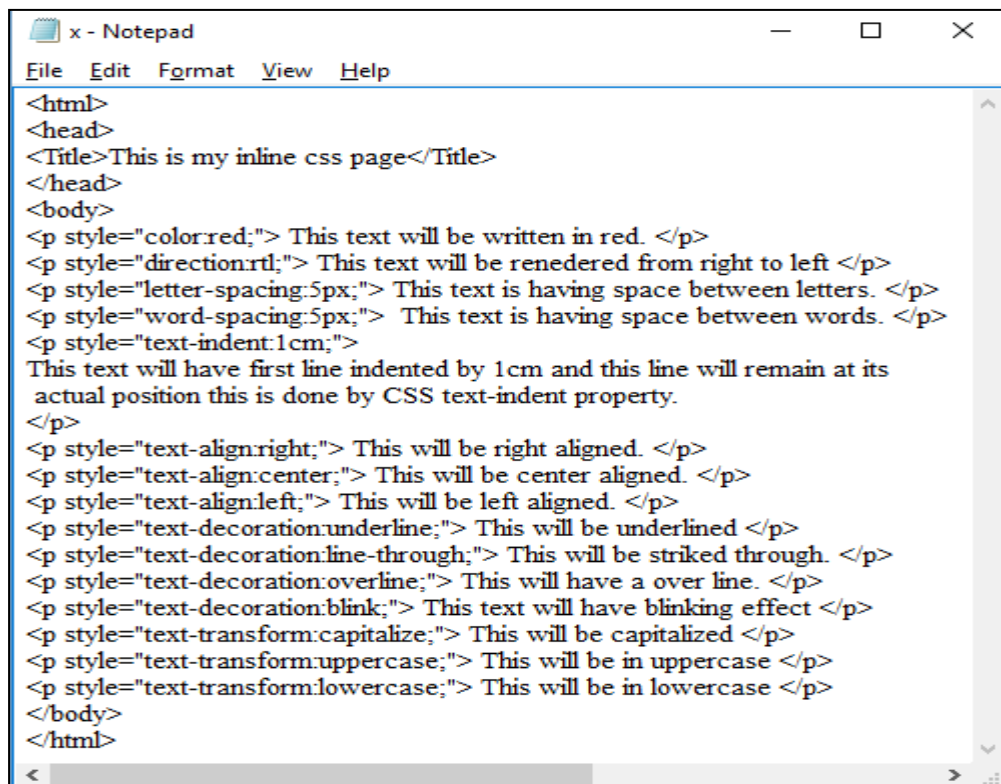
```
<p style="text-decoration:underline;"> This will be underlined </p>
<p style="text-decoration:line-through;"> This will be striked through.
</p>
<p style="text-decoration:overline;"> This will have a over line. </p>
<p style="text-decoration:blink;">This text will have blinking effect </p>
```

### 3.7.8 Set the Text Cases

The following example demonstrates how to set the cases for a text. Possible values are none, capitalize, uppercase, lowercase.

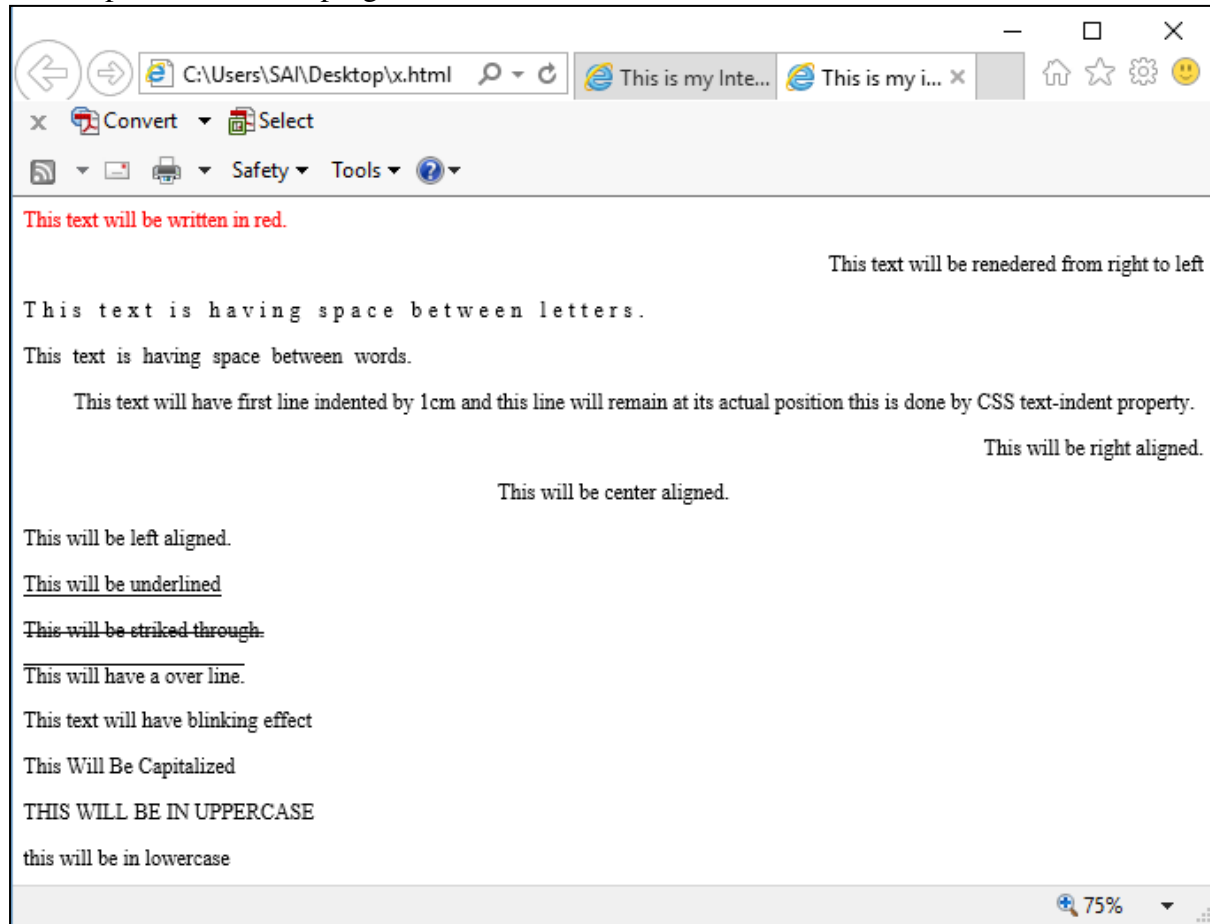
```
<p style="text-transform:capitalize;"> This will be capitalized </p>
<p style="text-transform:uppercase;"> This will be in uppercase </p>
<p style="text-transform:lowercase;"> This will be in lowercase </p>
```

Example  
:

A screenshot of a Notepad window titled 'x - Notepad'. The window contains HTML code for text decoration and case transformation. The code is as follows:

```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="color:red;"> This text will be written in red. </p>
<p style="direction:rtl;"> This text will be rendered from right to left </p>
<p style="letter-spacing:5px;"> This text is having space between letters. </p>
<p style="word-spacing:5px;"> This text is having space between words. </p>
<p style="text-indent:1cm;">
This text will have first line indented by 1cm and this line will remain at its
actual position this is done by CSS text-indent property.
</p>
<p style="text-align:right;"> This will be right aligned. </p>
<p style="text-align:center;"> This will be center aligned. </p>
<p style="text-align:left;"> This will be left aligned. </p>
<p style="text-decoration:underline;"> This will be underlined </p>
<p style="text-decoration:line-through;"> This will be striked through. </p>
<p style="text-decoration:overline;"> This will have a over line. </p>
<p style="text-decoration:blink;"> This text will have blinking effect </p>
<p style="text-transform:capitalize;"> This will be capitalized </p>
<p style="text-transform:uppercase;"> This will be in uppercase </p>
<p style="text-transform:lowercase;"> This will be in lowercase </p>
</body>
</html>
```

The output of the above program is :



---

### 3.8 Font Management using CSS

---

A **font** is the combination of typeface and other qualities, such as size, pitch, and spacing. For example, Times Roman is a typeface that defines the shape of each character. Within Times Roman, however, there are many **fonts** to choose from -- different sizes, italic, bold, and so on. You can set the following font properties of an element:

- The **font-family** property is used to change the face of a font.
- The **font-style** property is used to make a font italic or oblique.
- The **font-variant** property is used to create a small-caps effect.
- The **font-weight** property is used to increase or decrease how bold or light a font appears.
- The **font-size** property is used to increase or decrease the size of a font.
- The **font** property is used as shorthand to specify a number of other font properties.

#### 3.8.1 Set the Font Family

Following is the example, which demonstrates how to set the font family of an element. Possible value could be any font family name.

`<p style="font-family:georgia,garamond,serif;">`  
This text is rendered in either georgia, garamond, or the default serif font  
depending on which font you have at your system. `</p>`

### **3.8.2 Set the Font Style**

The following example demonstrates how to set the font style of an element. Possible values are normal, italic and oblique.

`<p style="font-style:italic;">`This text will be rendered in italic  
style `</p>`

### **3.8.3 Set the Font Variant**

The following example demonstrates how to set the font variant of an element. Possible values are normal and small-caps.

`<p style="font-variant:small-caps;">` This text will be rendered as small  
caps `</p>`

### **3.8.4 Set the Font Weight**

The following example demonstrates how to set the font weight of an element. The font-weight property provides the functionality to specify how bold a font is. Possible values could be normal, bold, bolder, lighter, 100, 200, 300, 400, 500, 600, 700, 800, 900.

`<p style="font-weight:bold;">` This font is bold. `</p>`  
`<p style="font-weight:bolder;">` This font is bolder. `</p>`  
`<p style="font-weight:900;">` This font is 900 weight. `</p>`

### **3.8.5 Set the Font Size**

The following example demonstrates how to set the font size of an element. The font-size property is used to control the size of fonts. Possible values could be xx-small, x-small, small, medium, large, x-large, xx-large, smaller, larger, size in pixels or in %.

`<p style="font-size:20px;">` This font size is 20 pixels `</p>`  
`<p style="font-size:small;">` This font size is small `</p>`  
`<p style="font-size:large;">` This font size is large `</p>`

### **3.8.6 Set the Font Size Adjust**

The following example demonstrates how to set the font size adjust of an element. This property enables you to adjust the x-height to make fonts more legible. Possible value could be any number.

`<p style="font-size-adjust:0.61;">`This text is using a font-size-adjust  
value. `</p>`

### **3.8.7 Set the Font Stretch**

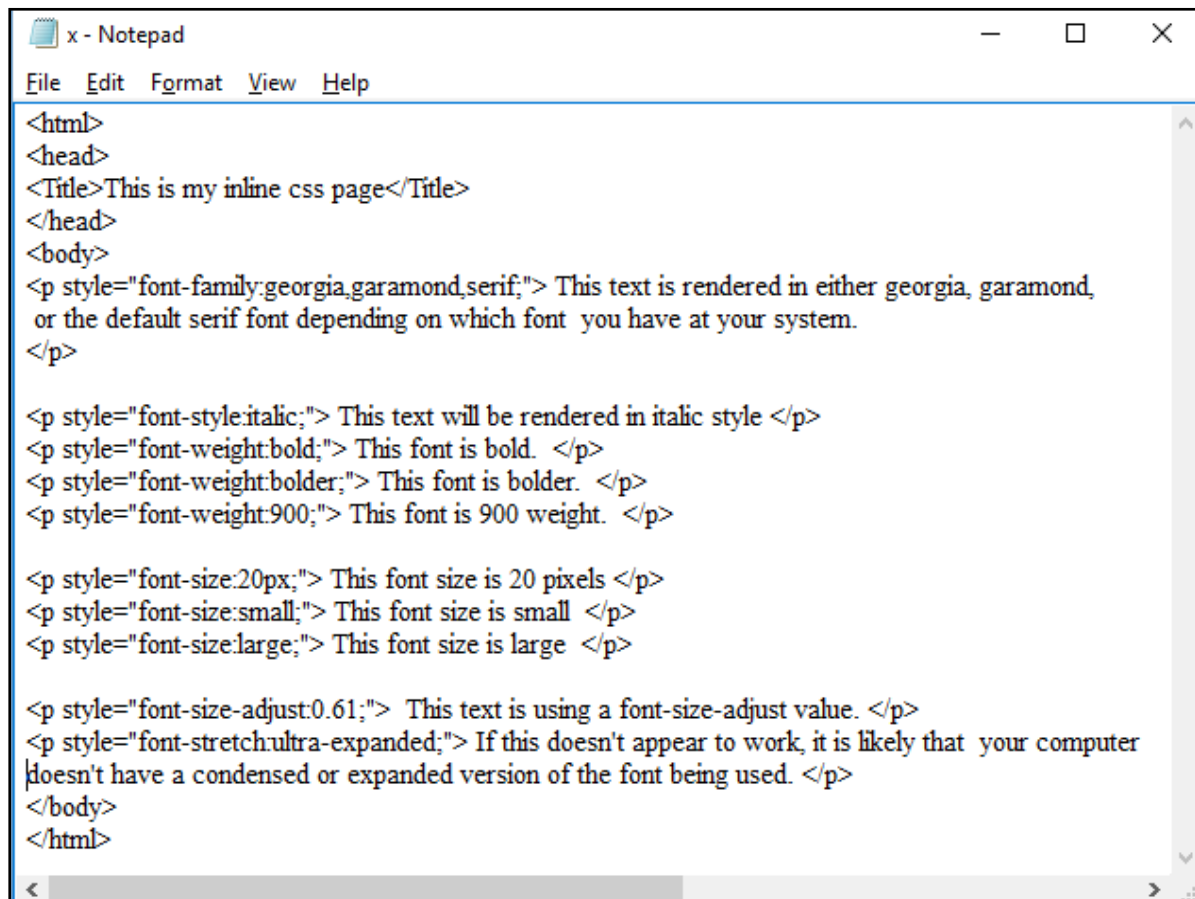
The following example demonstrates how to set the font stretch of an element. This property relies on the user's computer to have an expanded or condensed version of the font being used.

Possible values could be normal, wider, narrower, ultra-condensed, extra-condensed, condensed, semi-condensed, semi-expanded, expanded, extra-expanded, ultra-expanded.

```
<p style="font-stretch:ultra-expanded;">
```

If this doesn't appear to work, it is likely that your computer doesn't have a condensed or expanded version of the font being used. </p>

Example:-



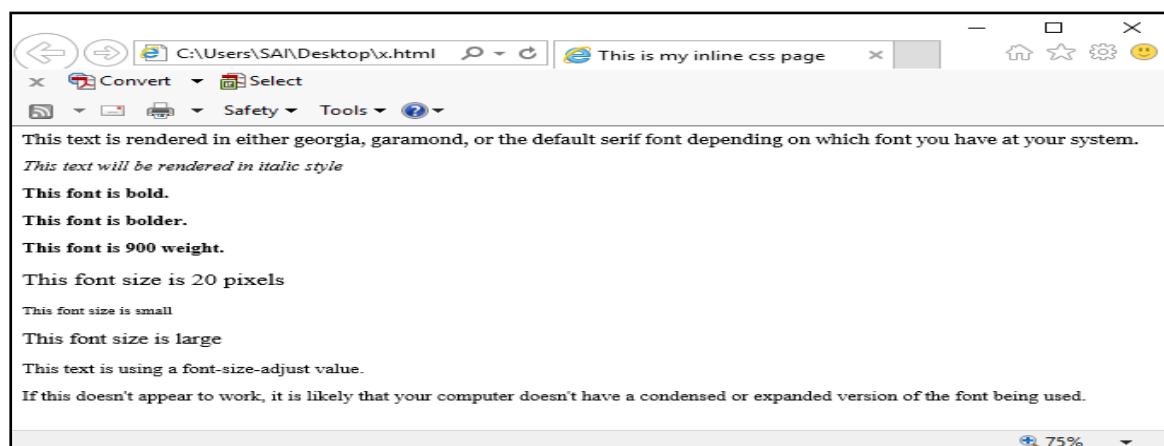
```
x - Notepad
File Edit Format View Help
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="font-family:georgia,garamond,serif;"> This text is rendered in either georgia, garamond,
or the default serif font depending on which font you have at your system.
</p>

<p style="font-style:italic;"> This text will be rendered in italic style </p>
<p style="font-weight:bold;"> This font is bold. </p>
<p style="font-weight:bolder;"> This font is bolder. </p>
<p style="font-weight:900;"> This font is 900 weight. </p>

<p style="font-size:20px;"> This font size is 20 pixels </p>
<p style="font-size:small;"> This font size is small </p>
<p style="font-size:large;"> This font size is large </p>

<p style="font-size-adjust:0.61;"> This text is using a font-size-adjust value. </p>
<p style="font-stretch:ultra-expanded;"> If this doesn't appear to work, it is likely that your computer
doesn't have a condensed or expanded version of the font being used. </p>
</body>
</html>
```

The output of the above program is :



---

### 3.9 Managing hyperlinks using CSS

---

An element in an electronic document that links to another place in the same document or to an entirely different document. A **hyperlink**, or simply a **link**, is a reference to data that the reader can directly follow either by clicking, tapping, or hovering. A **hyperlink** points to a whole document or to a specific element within a document. Hypertext is text with **hyperlinks**. The text that is linked is called anchor text. You can set the following properties of a hyperlink:

- The **:link** signifies unvisited hyperlinks.
- The **:visited** signifies visited hyperlinks.
- The **:hover** signifies an element that currently has the user's mouse pointer hovering over it.
- The **:active** signifies an element on which the user is currently clicking.

Usually, all these properties are kept in the header part of the HTML document. Remember **a:hover** MUST come after **a:link** and **a:visited** in the CSS definition in order to be effective. Also, **a:active** MUST come after **a:hover** in the CSS definition as follows:

```
<style type="text/css">
 a:link {color: #000000}
 a:visited {color: #006600}
 a:hover {color: #FFCC00}
 a:active {color: #FF00CC}
</style>
```

#### **3.9.1 Set the Color of Links**

The following example demonstrates how to set the link color. Possible values could be any color name in any valid format.

```
<style type="text/css">
 a:link {color:#000000}
</style>

Black Link
```

#### **3.9.2 Set the Color of Visited Links**

The following example demonstrates how to set the color of the visited links. Possible values could be any color name in any valid format.

```
<style type="text/css">
 a:visited {color: #006600}
</style>

Click this link
```

### **3.9.3 Change the Color of Links when Mouse is Over**

The following example demonstrates how to change the color of links when we bring a mouse pointer over that link. Possible values could be any color name in any valid format.

```
<style type="text/css">
 a:hover {color: #FFCC00}
</style>
```

```
Bring Mouse Here
```

### **3.9.4 Change the Color of Active Links**

The following example demonstrates how to change the color of active links. Possible values could be any color name in any valid format.

```
<style type="text/css">
 a:active {color: #FF00CC}
</style>
```

```
Click This Link
```

---

### **Check your progress 4**

---

**Q1. What is the representation of 6 digit Hexa code ?**

**Answer:** \_\_\_\_\_  
\_\_\_\_\_

**Q2. What are the different properties of Font ?**

**Answer:** \_\_\_\_\_  
\_\_\_\_\_

**Q3. Write the different properties of Hyperlink ?**

**Answer:** \_\_\_\_\_  
\_\_\_\_\_

---

### **3.10 Managing List using CSS**

---

Lists are very helpful in conveying a set of either numbered or bulleted points. This chapter teaches you how to control list type, position, style, etc., using CSS. We have the following five CSS properties, which can be used to control lists:

- The **list-style-type** allows you to control the shape or appearance of the marker.
- The **list-style-position** specifies whether a long point that wraps to a second line should align with the first line or start underneath the start of the marker.
- The **list-style-image** specifies an image for the marker rather than a bullet point or number.
- The **list-style** serves as shorthand for the preceding properties.

- The **marker-offset** specifies the distance between a marker and the text in the list.

### **3.10.1 The list-style-type Property**

The list-style-type property allows you to control the shape or style of a bullet point (also known as a marker) in case of unordered lists and the style of numbering characters in ordered lists. Here are the values, which can be used for an **unordered list**:

Value	Description
None	NA
disc (default)	A filled-in circle
Circle	An empty circle
Square	A filled-in square

Here are the values, which can be used for an **ordered list**:

Value	Description	Example
Decimal	Number	1,2,3,4.....
decimal-leading-zero	0 before the number	01, 02, ..... .....
lower-alpha	Lowercase alphanumeric characters	a, b, c, ....
upper-alpha	Uppercase alphanumeric characters	A, B, C, D, E..
lower-roman	Lowercase Roman numerals	i, ii, iii, iv, v
upper-roman	Uppercase Roman numerals	I, II, III, IV, V

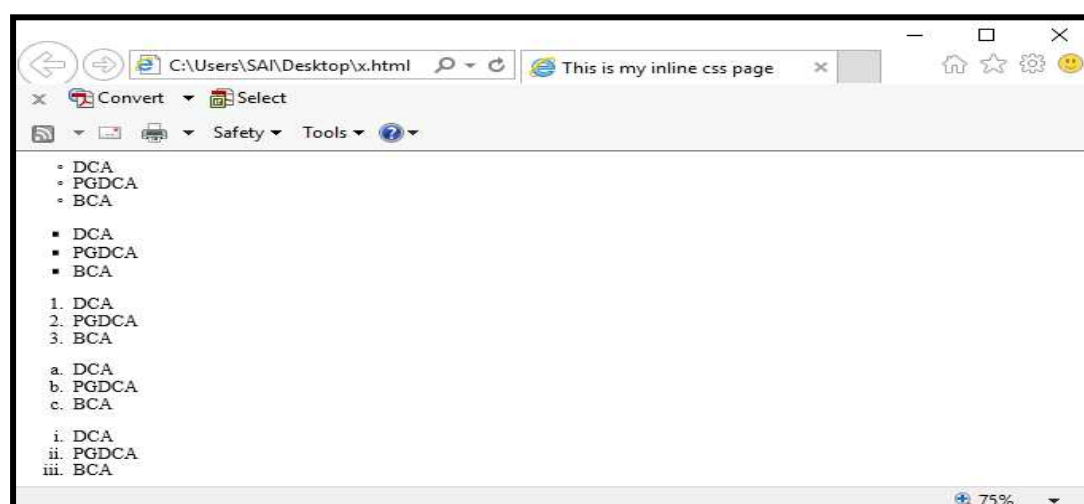
Here is an example:

```

<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<ul style="list-style-type:circle;"> DCAPGDCABCA
<ul style="list-style-type:square;"> DCAPGDCABCA
<ol style="list-style-type:decimal;"> DCAPGDCABCA
<ol style="list-style-type:lower-alpha;"> DCAPGDCABCA
<ol style="list-style-type:lower-roman;"> DCAPGDCABCA
</body>
</html>

```

The output of the above program is :



### **3.10.2 The list-style-position Property**

The list-style-position property indicates whether the marker should appear inside or outside of the box containing the bullet points. It can have one of the two values:

Value	Description
None	NA
Inside	If the text goes onto a second line, the text will wrap underneath the marker. It will also appear indented to where the text would have started if the list had a value of outside.
Outside	If the text goes onto a second line, the text will be aligned with the start of the first line (to the right of the bullet).

Example :

```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<ul style="list-style-type:circle; list-stlye-
position:outside;">Maths
Social Science
Physics

<ul style="list-style-type:square;list-style-
position:inside;">Maths
Social Science
Physics

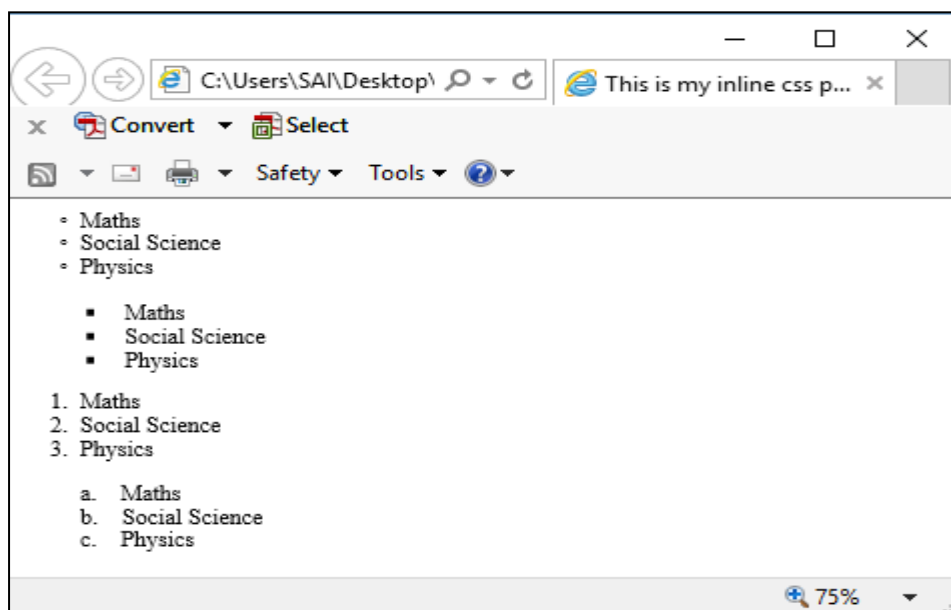
<ol style="list-style-type:decimal;list-stlye-
position:outside;">Maths
Social Science
Physics

<ol style="list-style-type:lower-alpha;list-style-position:inside;">

Maths
Social SciencePhysics

</body>
</html>
```

The output of the above program is :



---

### 3.11 Designing Tables using CSS

---

Tables are an excellent way to organize and display information on a page. You can set the following properties of a table:

- The **border-collapse** specifies whether the browser should control the appearance of the adjacent borders that touch each other or whether each cell should maintain its style.
- The **border-spacing** specifies the width that should appear between table cells.
- The **caption-side** captions are presented in the <caption> element. By default, these are rendered above the table in the document. You use the caption-side property to control the placement of the table caption.
- The **empty-cells** specify whether the border should be shown if a cell is empty.
- The **table-layout** allows browsers to speed up the layout of a table by using the first width properties it comes across for the rest of a column rather than having to load the whole table before rendering it.

#### 3.11.1 The border-collapse Property

This property can have two values collapse and separate. The following example uses both the values:

```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<style type="text/css">
table.one
 {border-collapse:collapse;}
table.two
 {border-collapse:separate;}
td.a
 {
 border-style:dotted;
 border-width:3px;
 border-color:#000000;
 padding: 10px;
 }
td.b
 {
 border-style:solid;
 border-width:3px;
 border-color:#333333;
 padding:10px;
 }
```

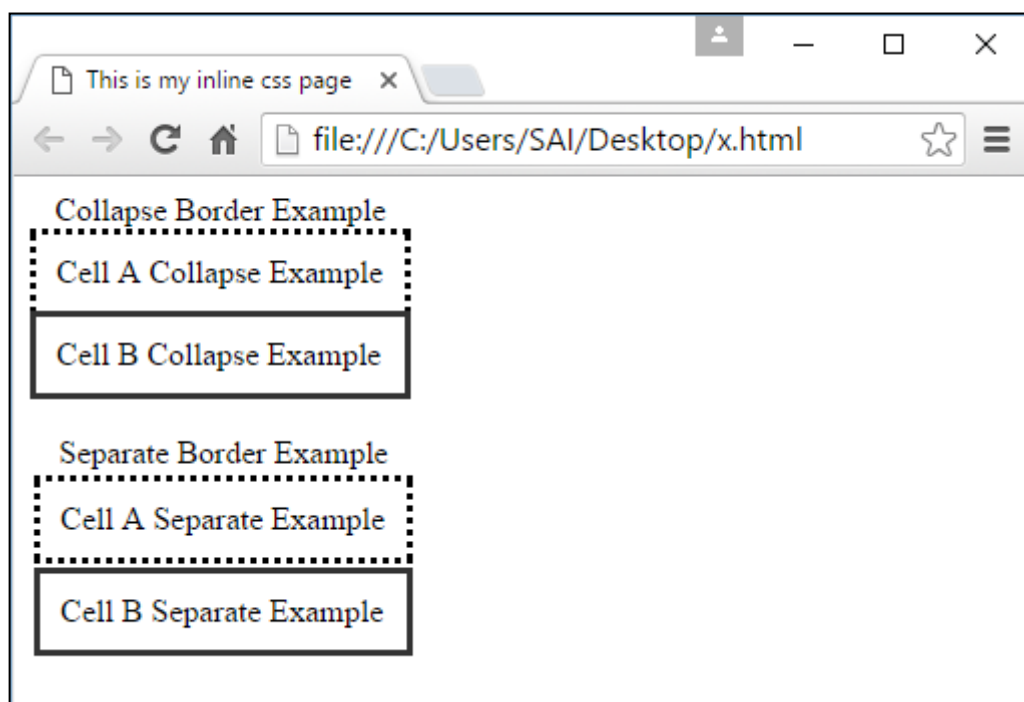
```

</style>
<table class="one">
 <caption>Collapse Border Example</caption>
 <tr><td class="a"> Cell A Collapse Example</td></tr>
 <tr><td class="b"> Cell B Collapse Example</td></tr>
</table>

<table class="two">
 <caption>Separate Border Example</caption>
 <tr><td class="a"> Cell A Separate Example</td></tr>
 <tr><td class="b"> Cell B Separate Example</td></tr>
</table>
</body>
</html>

```

The output of the above program is :



### **3.11.2 The border-spacing Property**

The border-spacing property specifies the distance that separates the adjacent cells' borders. It can take either one or two values; these should be units of length. If you provide one value, it applies to both vertical and horizontal borders. Or you can specify two values, in which case, the first refers to the horizontal spacing and the second to the vertical spacing:

**NOTE:** Unfortunately, this property does not work in Netscape 7 or IE 6.

Now let's modify the previous example and see the effect:

```

<style type="text/css">
table.one
{
border-collapse:separate; width:400px;
border-spacing:10px;
}
table.two
{
border-collapse:separate; width:400px;
border-spacing:10px 50px;
}
</style>
<table class="one" border="1">
<caption>Separate Border Example with border-spacing</caption>
<tr><td> Cell A Collapse Example</td></tr>
<tr><td> Cell B Collapse Example</td></tr>
</table>

<table class="two" border="1">
<caption>Separate Border Example with border-spacing</caption>
<tr><td> Cell A Separate Example</td></tr>
<tr><td> Cell B Separate Example</td></tr>
</table>

```

### **3.11.3 The empty-cells Property**

The empty-cells property indicates whether a cell without any content should have a border displayed. This property can have one of the three values - show, hide, or inherit. Here is the empty-cells property used to hide borders of empty cells in the <table> element.

#### **Example:**

```

<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<style type="text/css">
table.empty
{
width:350px;
border-collapse:separate;
empty-cells:hide;
}
td.empty
{
padding:5px;
border-style:solid;

```

```

border-width:1px;
border-color:#999999;}
</style>
<table class="empty">
<tr>
<th></th>
<th>Title one</th>
<th>Title two</th>
</tr>
<tr>
<th>Row Title</th>
<td class="empty">value</td><td class="empty">value</td></tr>
<tr>
<th>Row Title</th>
<td class="empty">value</td><td class="empty"></td>
</tr>
</table>
</body>
</html>

```

The output of the above program is :

	Title one	Title two
Row Title		
Row Title	value	value

---

### Check your progress 5

---

**Q1. What is the different properties of table in css ?**

**Answer:** \_\_\_\_\_  
 \_\_\_\_\_

**Q2. What are the different values of ordered and unordered list in css ?**

**Answer:** \_\_\_\_\_  
 \_\_\_\_\_

---

### 3.12 Working with Box Model

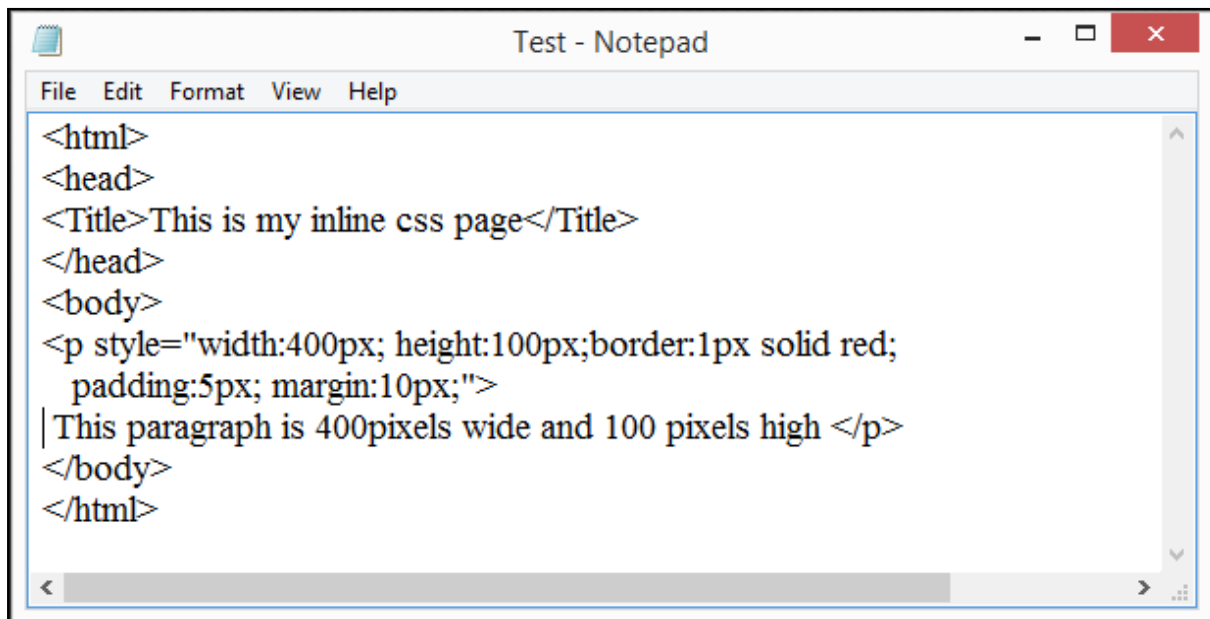
---

You have seen the border that surrounds every box i.e. element, the padding that can appear inside each box, and the margin that can go around them. In this chapter, we will learn how to change the dimensions of boxes. We have the following properties that allow you to control the dimensions of a box.

- The **height** property is used to set the height of a box.
- The **width** property is used to set the width of a box.
- The **line-height** property is used to set the height of a line of text.
- The **max-height** property is used to set a maximum height that a box can be.
- The **min-height** property is used to set the minimum height that a box can be.
- The **max-width** property is used to set the maximum width that a box can be.
- The **min-width** property is used to set the minimum width that a box can be.

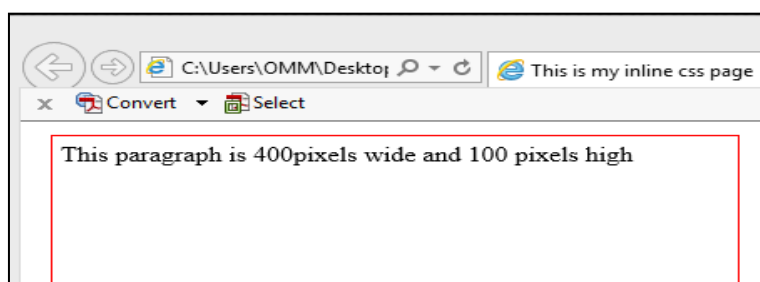
#### 3.12.1 The Height and Width Properties

The height and width properties allow you to set the height and width for boxes. They can take values of a length, a percentage, or the keyword auto. Here is an example:



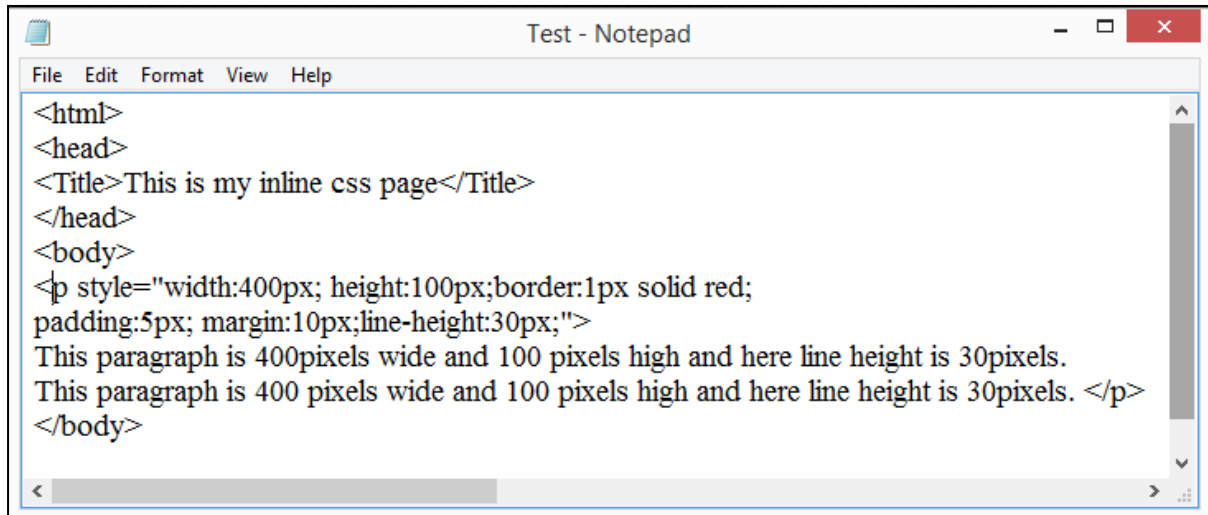
```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="width:400px; height:100px;border:1px solid red;
padding:5px; margin:10px;">
| This paragraph is 400pixels wide and 100 pixels high </p>
</body>
</html>
```

The output of the above program is:



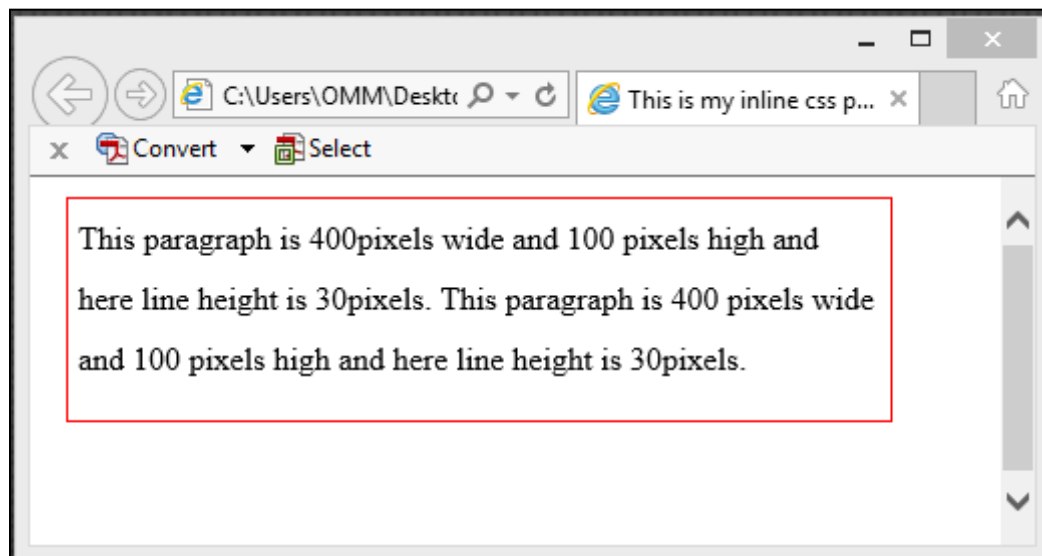
### **3.12.2 The line-height Property**

The line-height property allows you to increase the space between lines of text. The value of the line-height property can be a number, a length, or a percentage. Here is an example:



```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="width:400px; height:100px;border:1px solid red;
padding:5px; margin:10px;line-height:30px;">
This paragraph is 400pixels wide and 100 pixels high and here line height is 30pixels.
This paragraph is 400 pixels wide and 100 pixels high and here line height is 30pixels. </p>
</body>
```

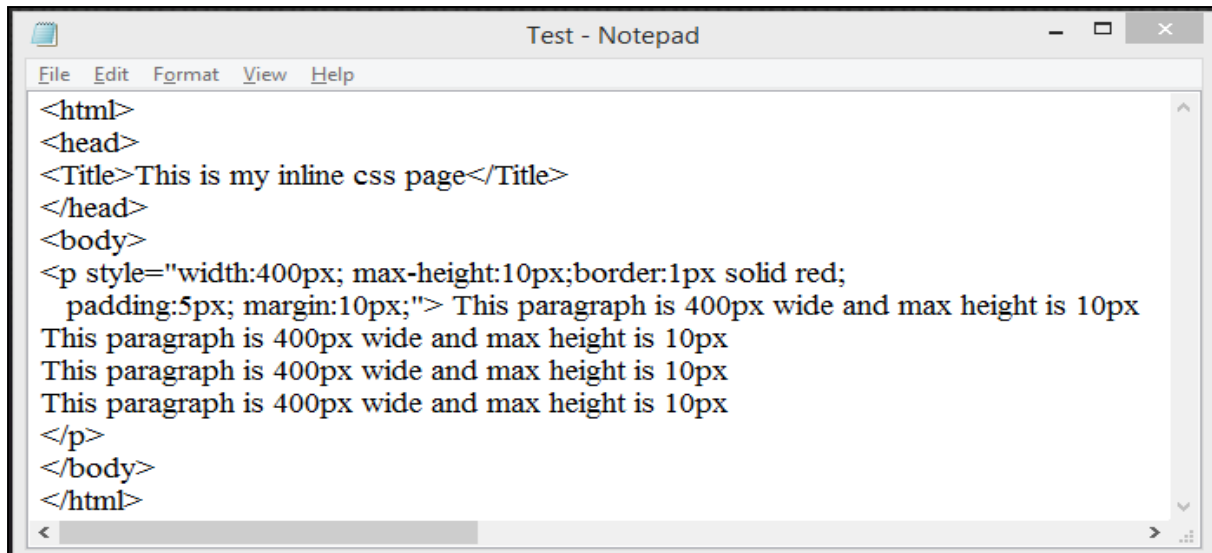
The output of the above program is



### **3.12.3 The max-height Property**

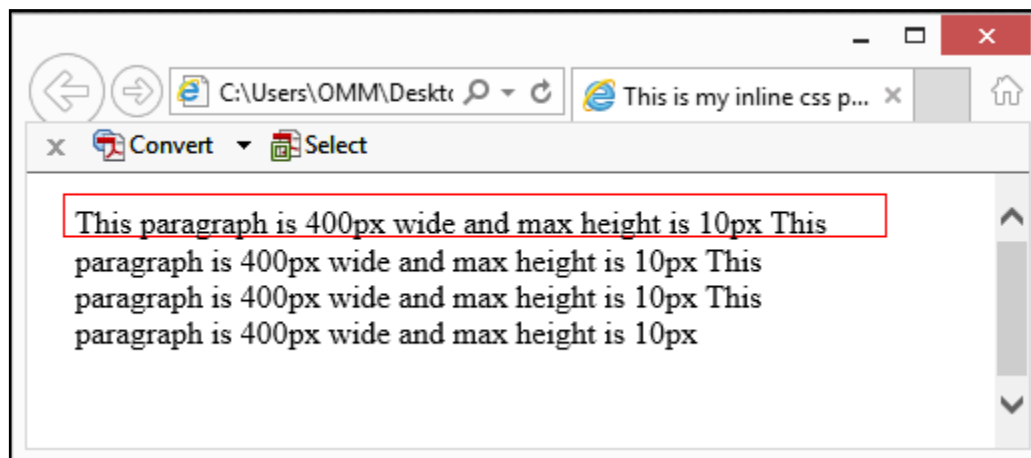
The max-height property allows you to specify the maximum height of a box. The value of the max-height property can be a number, a length, or a percentage.

NOTE: This property does not work in either Netscape 7 or IE 6.  
Here is an example:



```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="width:400px; max-height:10px;border:1px solid red;
padding:5px; margin:10px;"> This paragraph is 400px wide and max height is 10px
This paragraph is 400px wide and max height is 10px
This paragraph is 400px wide and max height is 10px
This paragraph is 400px wide and max height is 10px
</p>
</body>
</html>
```

The output of the above program is

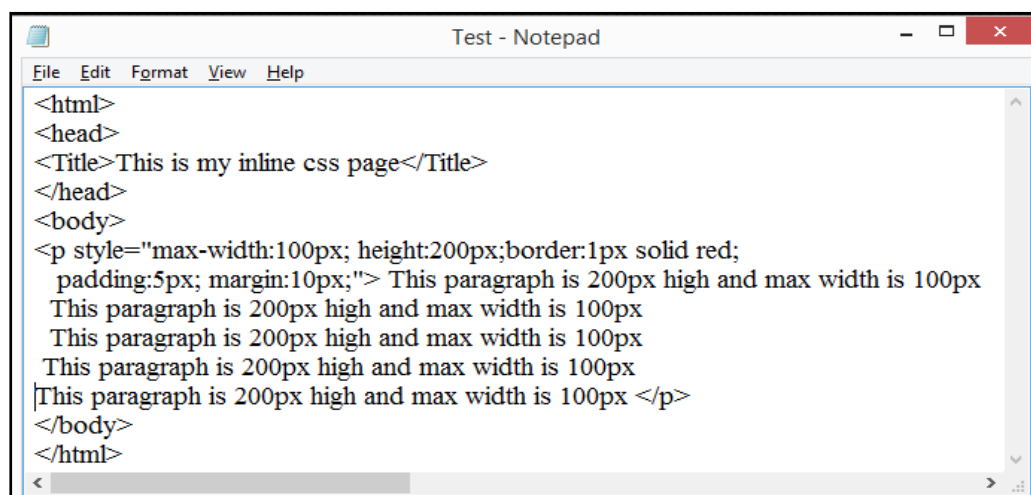


#### **3.12.4 The max-width Property**

The max-width property allows you to specify the maximum width of a box. The value of the max-width property can be a number, a length, or a percentage.

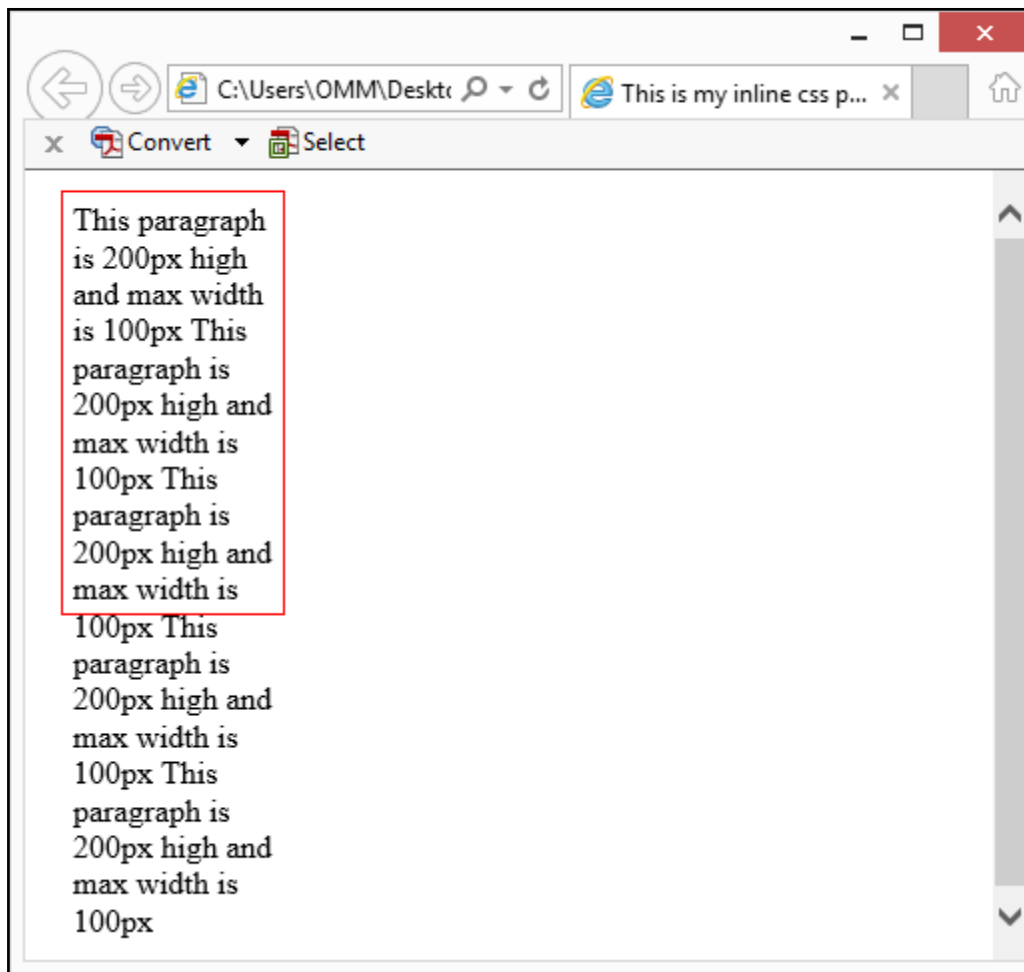
NOTE: This property does not work in either Netscape 7 or IE 6.

Here is an example:



```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="max-width:100px; height:200px;border:1px solid red;
padding:5px; margin:10px;"> This paragraph is 200px high and max width is 100px
This paragraph is 200px high and max width is 100px
This paragraph is 200px high and max width is 100px
This paragraph is 200px high and max width is 100px
This paragraph is 200px high and max width is 100px </p>
</body>
</html>
```

The output of the above program is



---

### 3.13 Designing borders using CSS

---

The *border* properties allow you to specify how the border of the box representing an element should look. There are three properties of a border you can change:

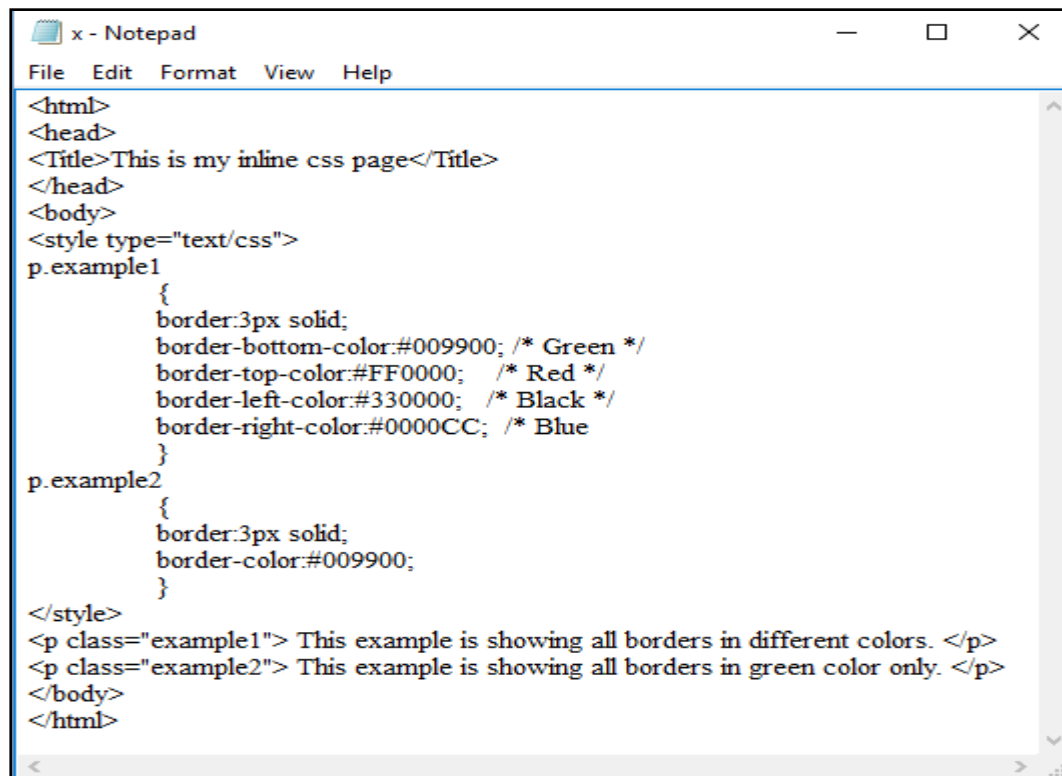
- The **border-color** specifies the color of a border.
- The **border-style** specifies whether a border should be solid, dashed line, double line, or one of the other possible values.
- The **border-width** specifies the width of a border.

#### 3.13.1 The border-color Property

The *border-color* property allows you to change the color of the border surrounding an element. You can individually change the color of the bottom, left, top and right sides of an element's border using the properties:

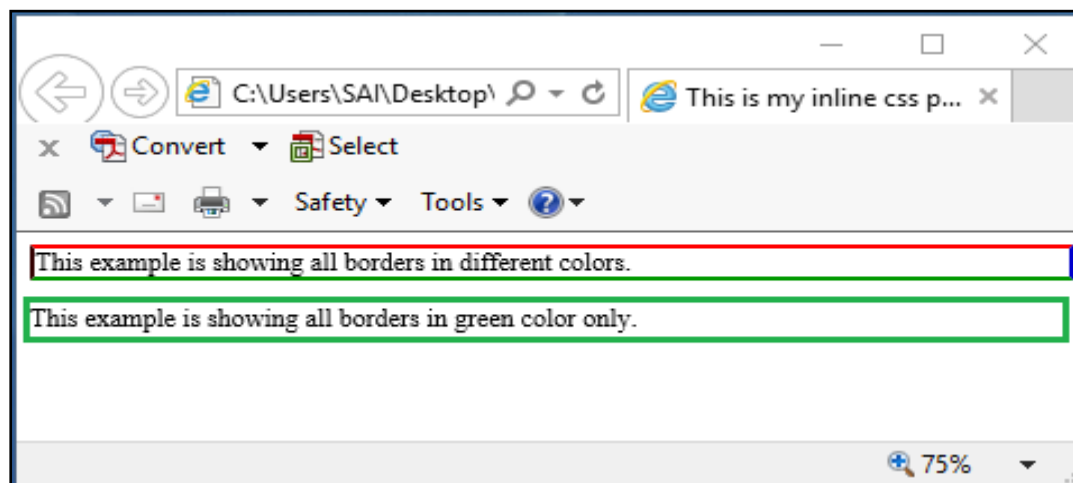
- *border-bottom-color* changes the color of bottom border.
- *border-top-color* changes the color of top border.
- *border-left-color* changes the color of left border.
- *border-right-color* changes the color of right border.

The following example shows the effect of all these properties:



```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<style type="text/css">
p.example1
{
border:3px solid;
border-bottom-color:#009900; /* Green */
border-top-color:#FF0000; /* Red */
border-left-color:#330000; /* Black */
border-right-color:#0000CC; /* Blue
}
p.example2
{
border:3px solid;
border-color:#009900;
}
</style>
<p class="example1"> This example is showing all borders in different colors. </p>
<p class="example2"> This example is showing all borders in green color only. </p>
</body>
</html>
```

The output of the above program is :



### 3.13.2 The border-style Property

The border-style property allows you to select one of the following styles of border:

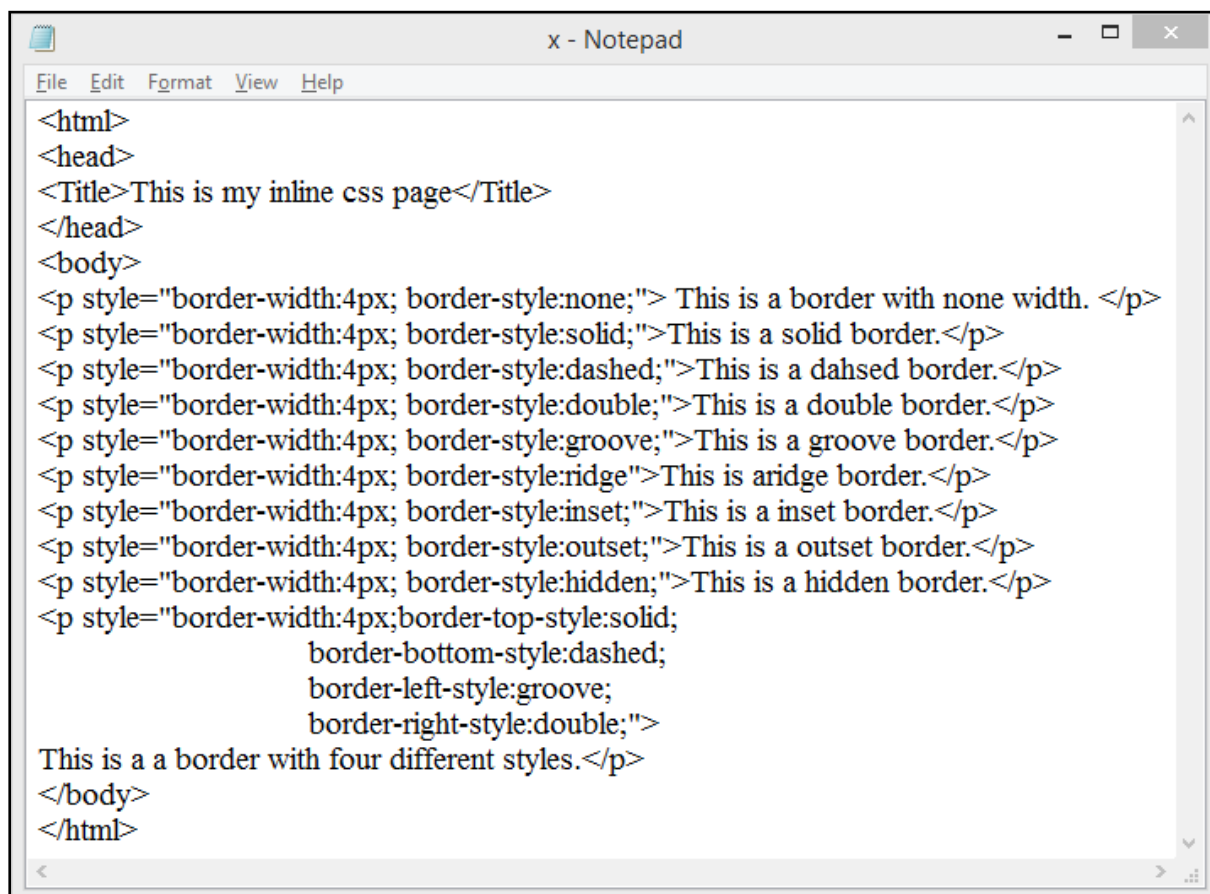
- **none**: No border. (Equivalent of border-width:0;)
- **solid**: Border is a single solid line.
- **dotted**: Border is a series of dots.
- **dashed**: Border is a series of short lines.
- **double**: Border is two solid lines.

- **groove:** Border looks as though it is carved into the page.
- **ridge:** Border looks the opposite of groove.
- **inset:** Border makes the box look like it is embedded in the page.
- **outset:** Border makes the box look like it is coming out of the canvas.
- **hidden:** Same as none, except in terms of border-conflict resolution for table elements.

You can individually change the style of the bottom, left, top, and right borders of an element using the following properties:

- **border-bottom-style** changes the style of bottom border.
- **border-top-style** changes the style of top border.
- **border-left-style** changes the style of left border.
- **border-right-style** changes the style of right border.

The following example shows all these border styles:

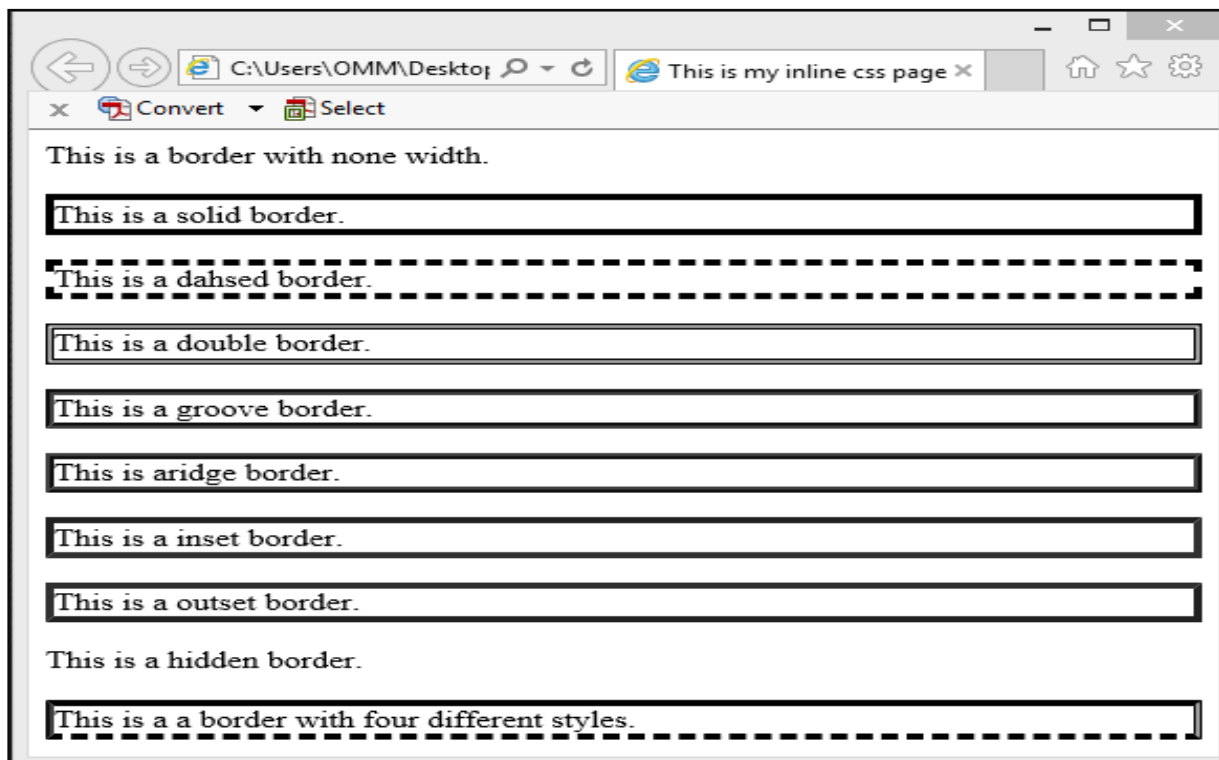


```

x - Notepad
File Edit Format View Help
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="border-width:4px; border-style:none;"> This is a border with none width. </p>
<p style="border-width:4px; border-style:solid;">This is a solid border.</p>
<p style="border-width:4px; border-style:dashed;">This is a dahsed border.</p>
<p style="border-width:4px; border-style:double;">This is a double border.</p>
<p style="border-width:4px; border-style:groove;">This is a groove border.</p>
<p style="border-width:4px; border-style:ridge">This is aridge border.</p>
<p style="border-width:4px; border-style:inset;">This is a inset border.</p>
<p style="border-width:4px; border-style:outset;">This is a outset border.</p>
<p style="border-width:4px; border-style:hidden;">This is a hidden border.</p>
<p style="border-width:4px;border-top-style:solid;
 border-bottom-style:dashed;
 border-left-style:groove;
 border-right-style:double;">
This is a a border with four different styles.</p>
</body>
</html>

```

It will produce the following result:



---

### 3.14 Designing outline using CSS

---

Outlines are very similar to borders, but there are few major differences as well:

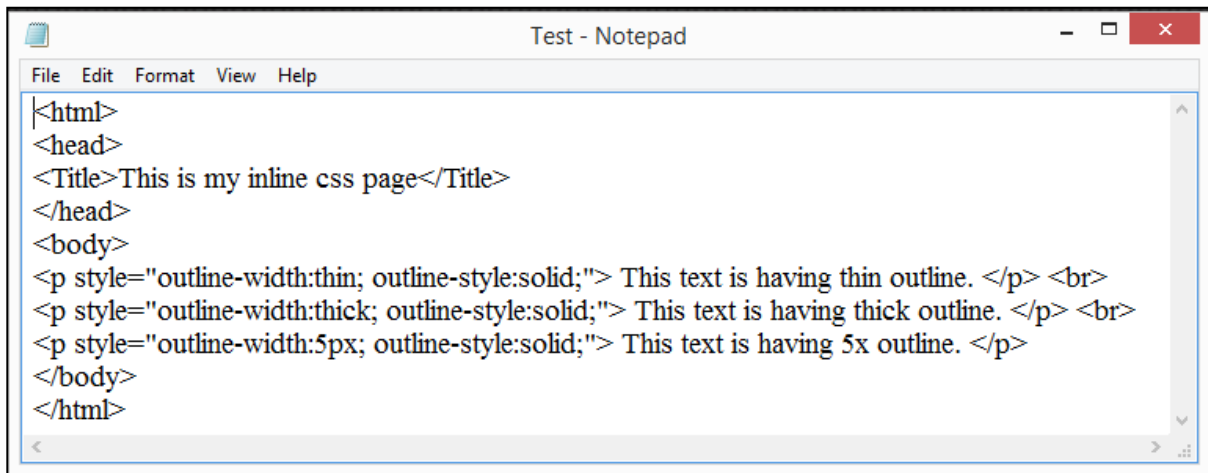
- An outline does not take up space.
- Outlines do not have to be rectangular.
- Outline is always the same on all sides; you cannot specify different values for different sides of an element.

**NOTE:** The outline properties are not supported by IE 6 or Netscape 7. You can set the following outline properties using CSS.

- The outline-width property is used to set the width of the outline.
- The outline-style property is used to set the line style for the outline.
- The outline-color property is used to set the color of the outline.
- The outline property is used to set all the above three properties in a single statement.

#### 3.14.1 The outline-width Property

The outline-width property specifies the width of the outline to be added to the box. Its value should be a length or one of the values thin, medium, or thick, just like the border-width attribute. A width of zero pixels means no outline. Here is an example:

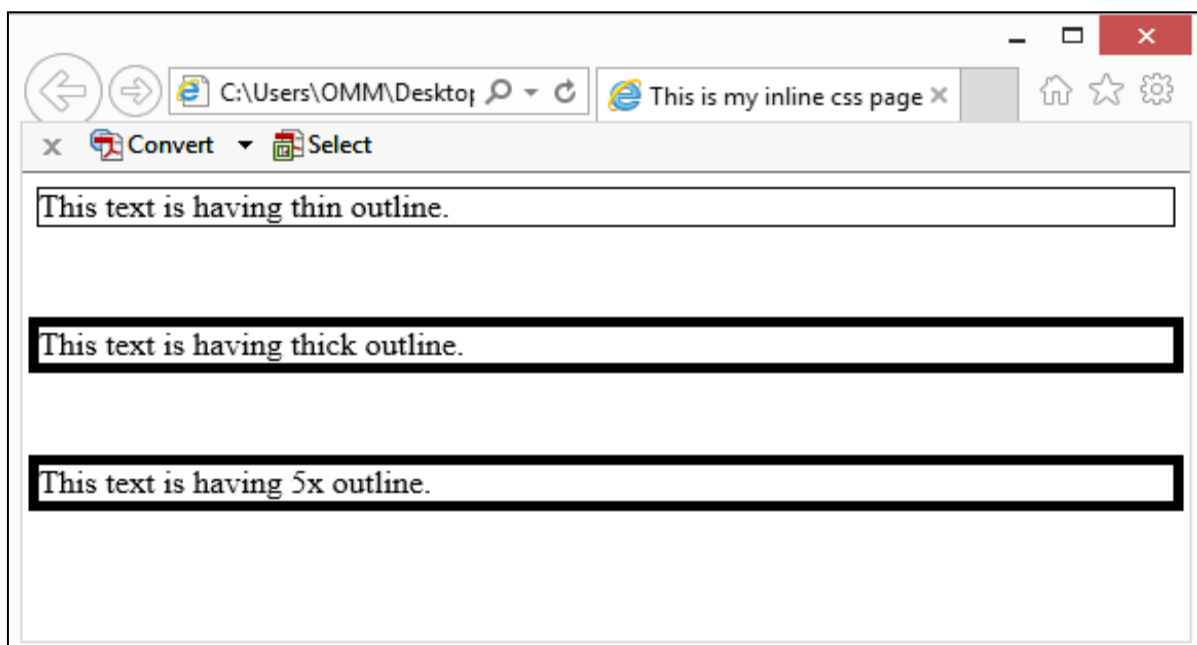


```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="outline-width:thin; outline-style:solid;"> This text is having thin outline. </p>

<p style="outline-width:thick; outline-style:solid;"> This text is having thick outline. </p>

<p style="outline-width:5px; outline-style:solid;"> This text is having 5x outline. </p>
</body>
</html>
```

The above program will produce the following result:

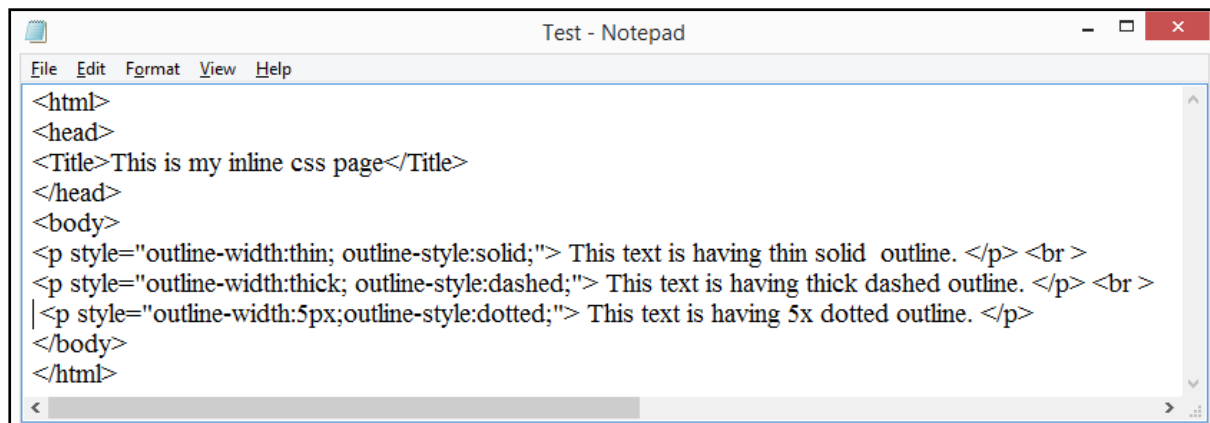


### **3.14.2 The outline-style Property**

The outline-style property specifies the style for the line (solid, dotted, or dashed) that goes around an element. It can take one of the following values:

- none: No border. (Equivalent of outline-width:0;)
- solid: Outline is a single solid line.
- dotted: Outline is a series of dots.
- dashed: Outline is a series of short lines.
- double: Outline is two solid lines.
- groove: Outline looks as though it is carved into the page.
- ridge: Outline looks the opposite of groove.
- inset: Outline makes the box look like it is embedded in the page.
- outset: Outline makes the box look like it is coming out of the canvas.
- hidden: Same as none.

Here is an example:

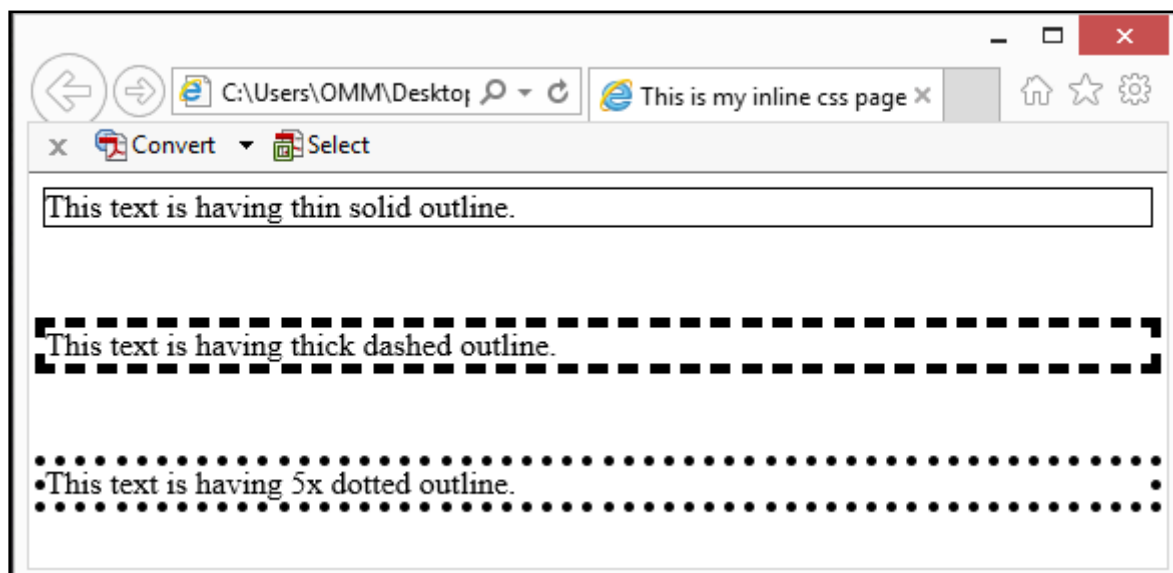


```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="outline-width:thin; outline-style:solid;"> This text is having thin solid outline. </p>

<p style="outline-width:thick; outline-style:dashed;"> This text is having thick dashed outline. </p>

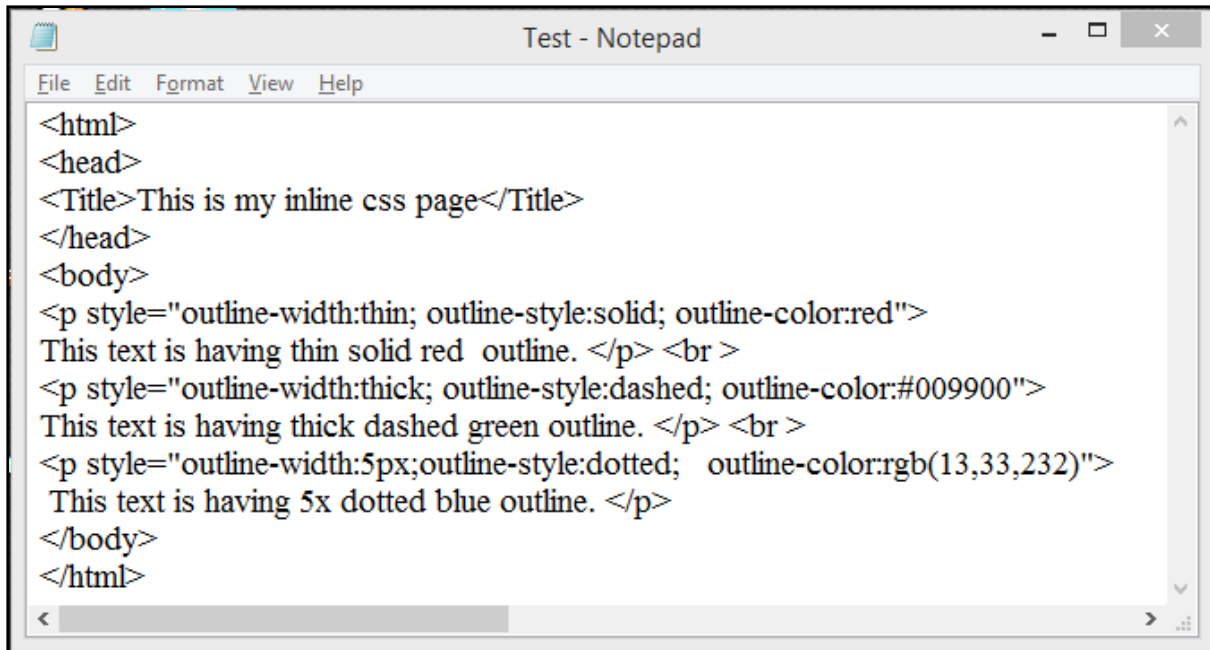
<p style="outline-width:5px;outline-style:dotted;"> This text is having 5x dotted outline. </p>
</body>
</html>
```

The above program will produce the following result:



### **3.14.3 The outline-color Property**

The outline-color property allows you to specify the color of the outline. Its value should either be a color name, a hex color, or an RGB value, as with the color and border-color properties. Here is an example:

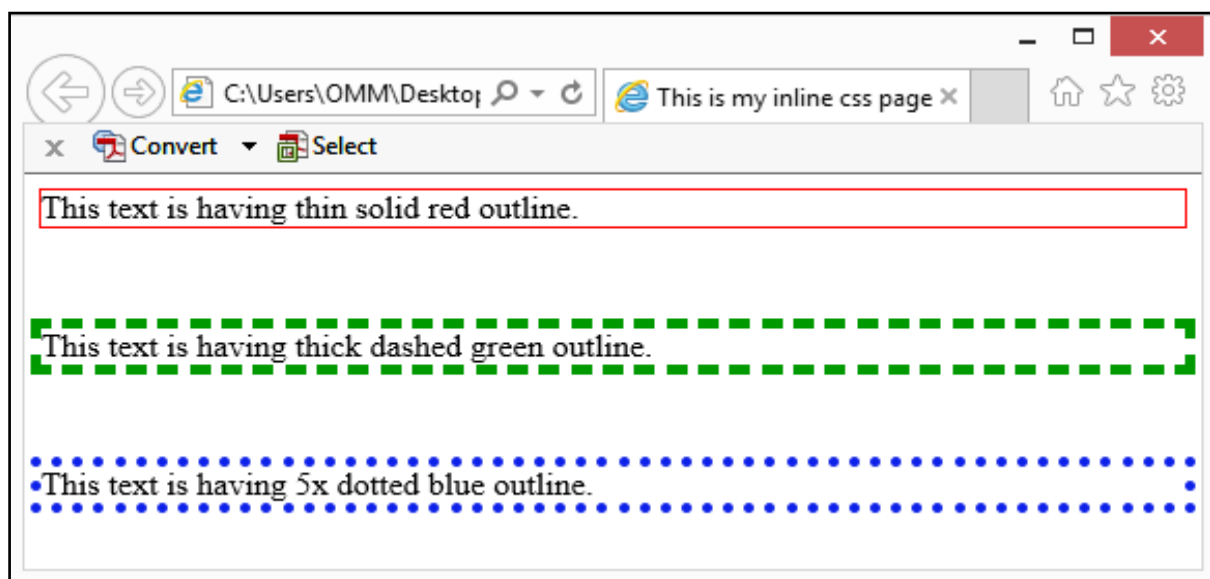


```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="outline-width:thin; outline-style:solid; outline-color:red">
This text is having thin solid red outline. </p>

<p style="outline-width:thick; outline-style:dashed; outline-color:#009900">
This text is having thick dashed green outline. </p>

<p style="outline-width:5px;outline-style:dotted; outline-color:rgb(13,33,232)">
This text is having 5x dotted blue outline. </p>
</body>
</html>
```

The above program will produce the following result:



---

### 3.15 Setting Page Margin using CSS

---

The margin property defines the space around an HTML element. It is possible to use negative values to overlap content. The values of the margin property are not inherited by the child elements. Remember that the adjacent vertical margins (top and bottom margins) will collapse into each other so that the distance between the blocks is not the sum of the margins, but only the greater of the two margins or the same size as one margin if both are equal. We have the following properties to set an element margin.

- The **margin** specifies a shorthand property for setting the margin properties in one declaration.
- The **margin-bottom** specifies the bottom margin of an element.
- The **margin-top** specifies the top margin of an element.
- The **margin-left** specifies the left margin of an element.
- The **margin-right** specifies the right margin of an element.

### 3.15.1 The Margin Property

The margin property allows you to set all of the properties for the four margins in one declaration. Here is the syntax to set margin around a paragraph:

```
<style type="text/css">
```

```
p {margin: 15px}
```

*all four margins will be 15px*

```
p {margin: 10px 2%}
```

*top and bottom margin will be 10px, left and right margin will be 2% of the total width of the document.*

```
p {margin: 10px 2% -10px}
```

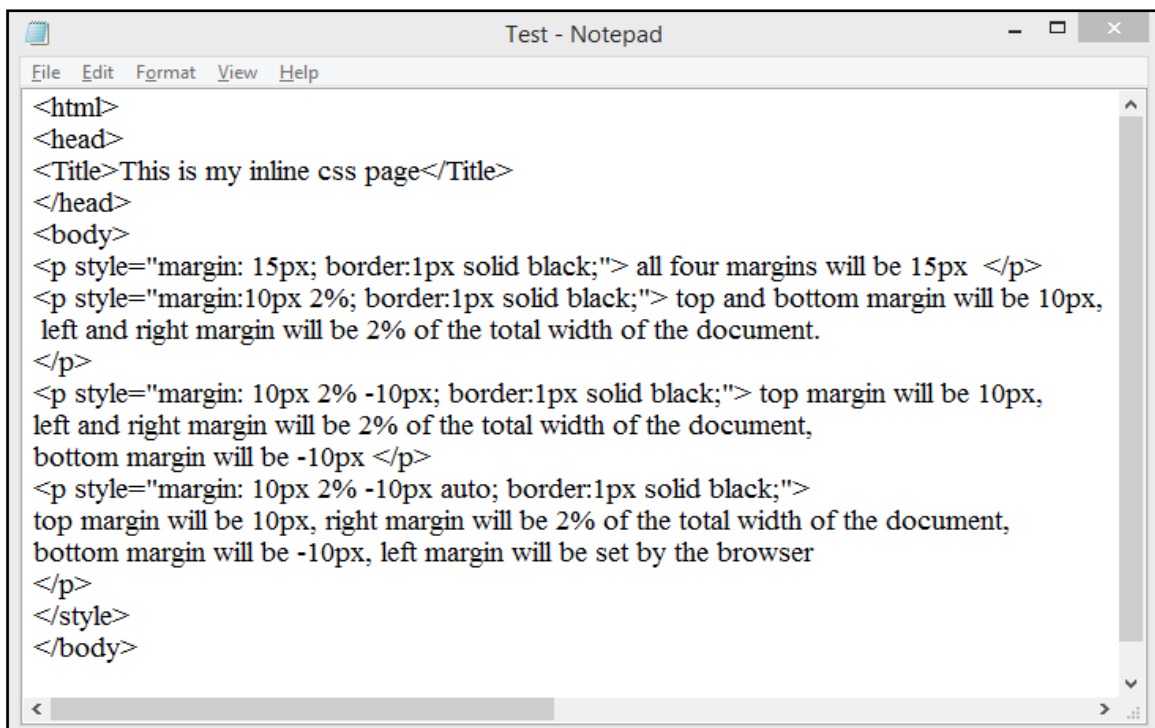
*top margin will be 10px, left and right margin will be 2% of the total width of the document, bottom margin will be -10px*

```
p {margin: 10px 2% -10px auto}
```

*top margin will be 10px, right margin will be 2% of the total width of the document, bottom margin will be -10px, left margin will be set by the browser*

```
</style>
```

Here is an example:

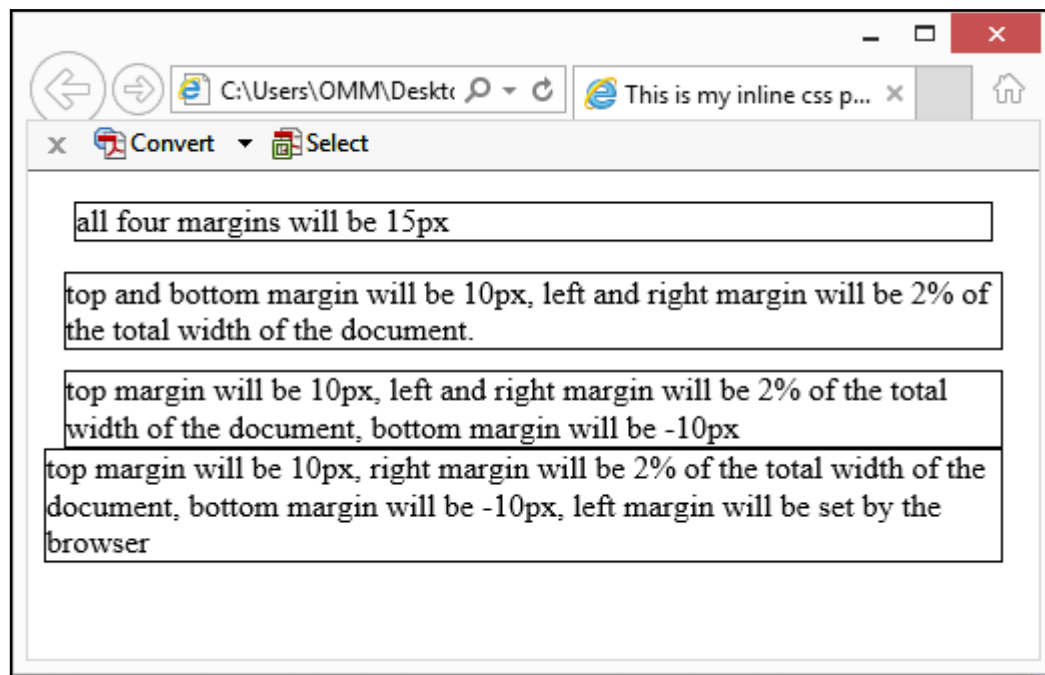


```

Test - Notepad
File Edit Format View Help
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="margin: 15px; border:1px solid black;"> all four margins will be 15px </p>
<p style="margin:10px 2%; border:1px solid black;"> top and bottom margin will be 10px,
left and right margin will be 2% of the total width of the document.
</p>
<p style="margin: 10px 2% -10px; border:1px solid black;"> top margin will be 10px,
left and right margin will be 2% of the total width of the document,
bottom margin will be -10px </p>
<p style="margin: 10px 2% -10px auto; border:1px solid black;">
top margin will be 10px, right margin will be 2% of the total width of the document,
bottom margin will be -10px, left margin will be set by the browser
</p>
</style>
</body>

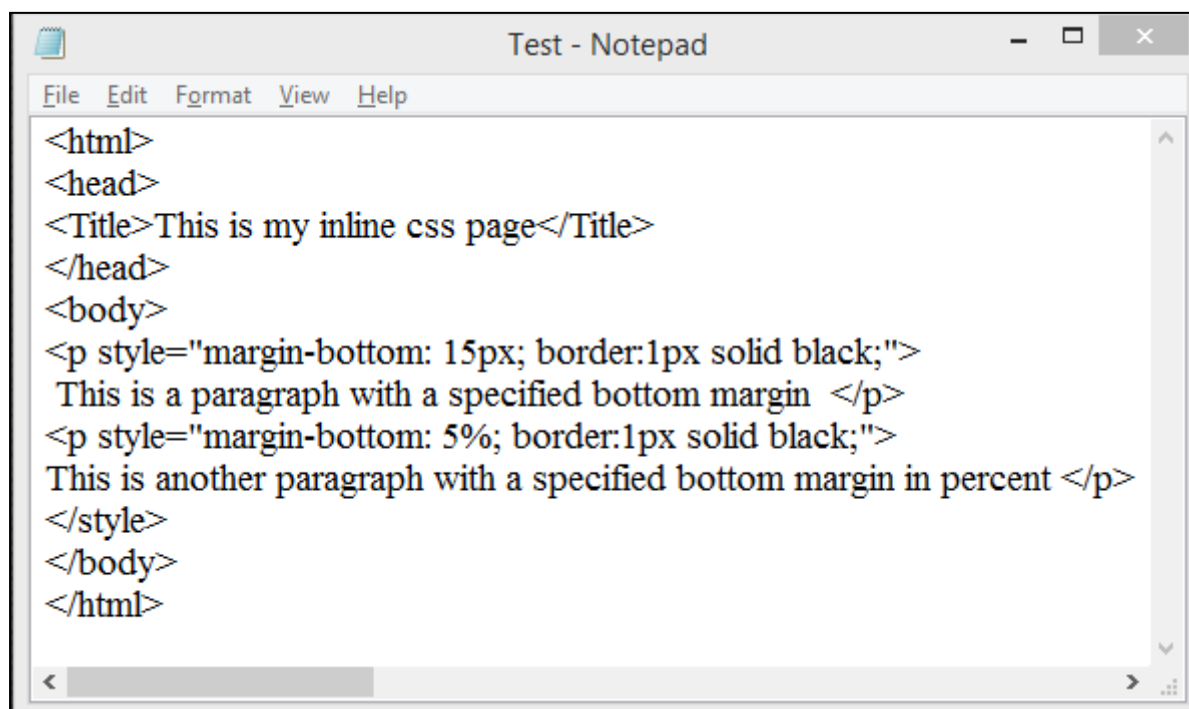
```

The above program will produce the following result:

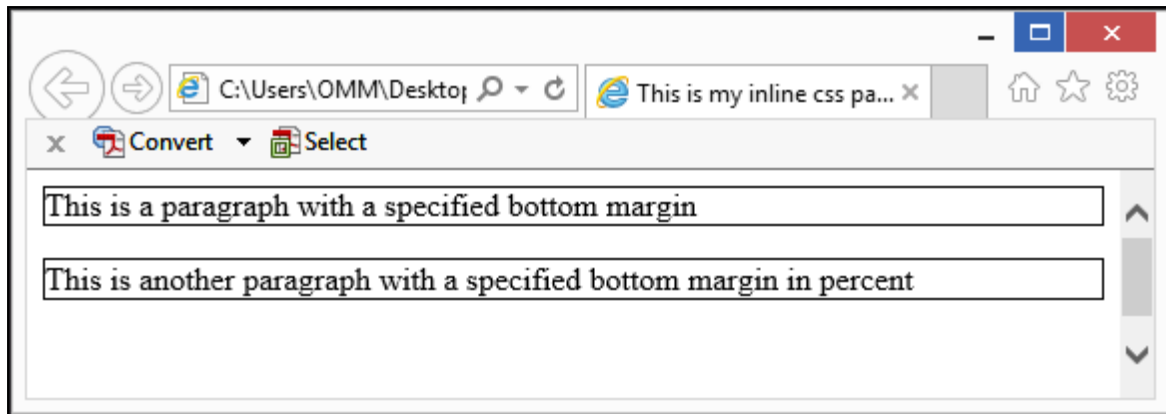


### **3.15.2 The margin-bottom Property**

The margin-bottom property allows you to set the bottom margin of an element. It can have a value in length, %, or auto. Here is an example:



The above program will produce the following result:



### 3.15.3 The margin-left Property

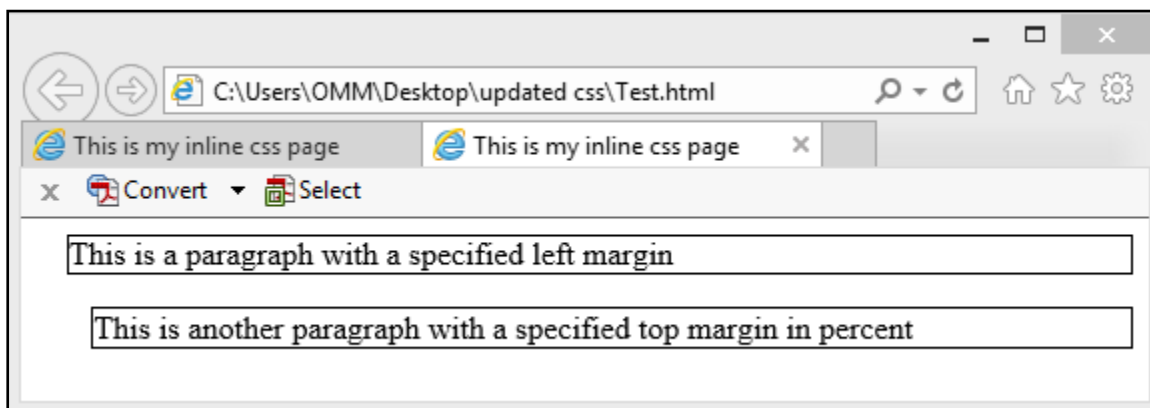
The margin-left property allows you to set the left margin of an element. It can have a value in length, %, or auto. Here is an example:

```

Test - Notepad
File Edit Format View Help
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="margin-left: 15px; border:1px solid black;">
This is a paragraph with a specified left margin </p>
<p style="margin-left: 5%; border:1px solid black;">
This is another paragraph with a specified top margin in percent </p>
</style>
</body>
</html>

```

The above program will produce the following result



---

### Check your progress 6

---

**Q1. Write the different properties of the dimensions of a box.**

**Answer:** \_\_\_\_\_

**Q2. Write the three properties of border.**

**Answer:** \_\_\_\_\_

**Q3. Write the few major difference of outline in CSS.**

**Answer:** \_\_\_\_\_

**Q4. What is a margin property?**

**Answer:** \_\_\_\_\_

**Q5. Write the border-style property in CSS.**

**Answer:** \_\_\_\_\_

---

### 3.16 Let us sum up

---

In this unit we have understood what is CSS, advantages of CSS, Parts of CSS, CSS syntax, CSS selectors, ways to insert CSS, background image handling, background colour management, text management, font management, managing hyperlinks, managing lists, designing tables, working with box model, designing borders, designing outline, setting page margin

---

### 3.17 Reference

---

1. W3Schools.com
2. Google.com
3. Tutorialpoints.com

---

### 3.18 Check your progress possible answers

---

#### Check your progress 1

**Q1. Write the abbreviation of CSS?**

**Answer**

CSS stands for Cascading Style Sheets. It is a simple design language intended to simplify the process of making web pages presentable.

## **Q2. Write the three parts of CSS syntax?**

### **Answer**

CSS style is made of three parts:

1. **Selector:** A selector is an HTML tag at which a style will be applied. This could be any tag like <h1>, <p> or <table> etc.
2. **Property:** A property is a type of attribute of HTML tag. Put simply, all the HTML attributes are converted into CSS properties. They could be *color*, *border*, *bgcolor* etc.
3. **Value:** Values are assigned to properties. For example, *color* property can have the value either *red* or *#F1F1F1* etc.

## **Check your progress 2**

### **Q1. What are the 3 types of selectors.**

#### **Answer**

Three types of CSS Selectors

1. The Element selectors
2. The ID Selectors
3. The Class Selectors

### **Q2. What is universal selector ?.**

#### **Answer**

#### **Universal selector**

An asterisk (\*) is the universal selector for CSS. It matches a single element of any type. Omitting the asterisk with simple selectors has the same effect. For instance, \*.warning and .warning are considered equal. Rather than selecting elements of a specific type

## **Check your progress 3**

### **Q1. How many ways to build style sheet ?what are they ?**

#### **Answer**

There are three ways of inserting a style sheet:

1. External style sheet
2. Internal style sheet
3. Inline style

## Q2. Write the background properties of CSS

### Answer

You can set the following background properties of an element:

- The **background-image** property is used to set the background image of an element.
- The **background-repeat** property is used to control the repetition of an image in the background.
- The **background-position** property is used to control the position of an image in the background.
- The **background-attachment** property is used to control the scrolling of an image in the background.
- The background property is used as shorthand to specify a number of other background properties.

### Check your progress 4

## Q1. What is the representation of 6 digit Hexa code ?

### Answer

A hexadecimal is a 6 digit representation of a color. The first two digits (RR) represent a red value, the next two are a green value (GG), and the last are the blue value (BB). Each hexadecimal code will be preceded by a pound or hash sign '#'. Following are the example to use Hexadecimal notation.

#66AA77

## Q2. What are the different properties of Font ?

### Answer

Font properties of an element:

- The **font-family** property is used to change the face of a font.
- The **font-style** property is used to make a font italic or oblique.
- The **font-variant** property is used to create a small-caps effect.
- The **font-weight** property is used to increase or decrease how bold or light a font appears.
- The **font-size** property is used to increase or decrease the size of a font.
- The **font** property is used as shorthand to specify a number of other font properties.

## Q3. Write the different properties of Hyperlink ?

### Answer

Properties of a hyperlink:

- The **:link** signifies unvisited hyperlinks.
- The **:visited** signifies visited hyperlinks.
- The **:hover** signifies an element that currently has the user's mouse pointer hovering over it.
- The **:active** signifies an element on which the user is currently clicking.

## **Check your progress 5**

### **Q1. What is the different properties of table in css ?**

#### **Answer**

Following properties of a table:

- The **border-collapse** specifies whether the browser should control the appearance of the adjacent borders that touch each other or whether each cell should maintain its style.
- The **border-spacing** specifies the width that should appear between table cells.
- The **caption-side** captions are presented in the <caption> element. By default, these are rendered above the table in the document. You use the caption-side property to control the placement of the table caption.
- The **empty-cells** specify whether the border should be shown if a cell is empty.
- The **table-layout** allows browsers to speed up the layout of a table by using the first width properties it comes across for the rest of a column rather than having to load the whole table before rendering it.

### **Q2. What are the different values of ordered and unordered list in CSS ?**

#### **Answer**

The values, for an **unordered list**:

disc (default), Circle, Square.

The values, for an **ordered list**:

Decimal, decimal-leading-zero, lower-alpha, upper-alpha, lower-roman, upper-roman.

## **Check your progress 6**

### **Q1. Write the different properties of the dimensions of a box.**

#### **Answer**

Properties of dimensions of a box.

- The **height** property is used to set the height of a box.
- The **width** property is used to set the width of a box.
- The **line-height** property is used to set the height of a line of text.
- The **max-height** property is used to set a maximum height that a box can be.
- The **min-height** property is used to set the minimum height that a box can be.
- The **max-width** property is used to set the maximum width that a box can be.
- The **min-width** property is used to set the minimum width that a box can be.

## **Q2. Write the three properties of border.**

### **Answer**

There are three properties of a border:

- The **border-color** specifies the color of a border.
- The **border-style** specifies whether a border should be solid, dashed line, double line, or one of the other possible values.
- The **border-width** specifies the width of a border

## **Q3. Write the few major difference of outline in CSS.**

### **Answer**

There are few major differences are :

- An outline does not take up space.
- Outlines do not have to be rectangular.
- Outline is always the same on all sides; you cannot specify different values for different sides of an element.

## **Q4. What is a margin property?**

### **Answer**

Following properties to set an element margin.

1. The **margin** specifies a shorthand property for setting the margin properties in one declaration.
2. The **margin-bottom** specifies the bottom margin of an element.
3. The **margin-top** specifies the top margin of an element.
4. The **margin-left** specifies the left margin of an element.
5. The **margin-right** specifies the right margin of an element

## **Q5. Write the border-style property in CSS.**

### **Answer**

The border-style property :

- **none**: No border. (Equivalent of border-width:0;)
- **solid**: Border is a single solid line.
- **dotted**: Border is a series of dots.
- **dashed**: Border is a series of short lines.
- **double**: Border is two solid lines.
- **groove**: Border looks as though it is carved into the page.
- **ridge**: Border looks the opposite of groove.
- **inset**: Border makes the box look like it is embedded in the page.
- **outset**: Border makes the box look like it is coming out of the canvas.
- **hidden**: Same as none, except in terms of border-conflict resolution for table elements.

---

## Unit -4

### CSS Advanced

---

#### Learning objectives:

After the Completion of this unit you should be able to know

- ✎ Paragraph Padding
- ✎ Image height and width setting
- ✎ Block level and inline elements
- ✎ Pseudo-class and elements
- ✎ Various attribute of image
- ✎ Different attribute values of selectors

#### Structure

- 4.1 Padding using CSS
  - 4.1.1 The padding-bottom Property
  - 4.1.2 The padding-top Property
- 4.2 Setting Display Using CSS
  - 4.2.1 Block-level Elements
  - 4.2.2 Inline element
- 4.3 Setting Width And Max Width using CSS
- 4.4 Setting Position using CSS
  - 4.4.1 Relative Positioning
  - 4.4.2 Absolute Positioning
  - 4.4.3 Fixed Positioning
- 4.5 Setting the Float Property using CSS
  - 4.5.1 Element float
  - 4.5.2 Turning off Float - Using Clear
  - 4.5.3 All CSS float
- 4.6 Inline Block property
- 4.7 Horizontal Alignment in CSS
  - 4.7.1 Center Align Elements
  - 4.7.2 Center alignment Text
  - 4.7.3 Center an image
  - 4.7.4 Left and Right Align - Using position
- 4.8 Working With Combinatory
  - 4.8.1 Descendant Selector
  - 4.8.2 Child Selector
  - 4.8.3 Adjacent Sibling Selector
  - 4.8.4 General sibling selector

- 4.9 Working with Pseudo-class
  - 4.9.1 Anchor Pseudo-classes
  - 4.9.2 Pseudo classes and CSS classes
  - 4.9.3 Simple Tooltip Hover
  - 4.3.4 The first child pseudo classes
  - 4.3.5 Match the first <p> element
- 4.10 working with Pseudo-elements
  - 4.10.1 The: first-line pseudo-element
  - 4.10.2 The: first-letter pseudo-element
  - 4.10.3 The: before pseudo-element
  - 4.10.4 The: after pseudo-element
  - 4.10.5 The: selection pseudo-element
- 4.11 Creating a navigation bar
  - 4.11.1\_Navigation Bar = List of Links
  - 4.11.2Vertical navigation bar
  - 4.11.3 Center Links & Add Borders
  - 4.11.4 Horizontal Navigation Bar
- 4.12 Working with images
  - 4.12.1 The Image Border Property
  - 4.12.2 The Image width Property
- 4.13 Working with Attribute selectors
  - 4.13.1Multiple style rules
  - 4.13.2 CSS [attribute] Selector
  - 4.13.3 CSS [attribute="value"] Selector
  - 4.13.4 CSS [attribute ~ ="value"] Selector
  - 4.13.5 CSS [attribute |= "value"] Selector
  - 4.13.6 CSS [attribute ^= "value"] Selector
  - 4.13.7 CSS [attribute \$= "value"] Selector
  - 4.13.8 CSS [attribute \*= "value"] Selector
- 4.14 Check Your Progress Possible answer
- 4.15 Reference

---

## 4.1 Padding using CSS

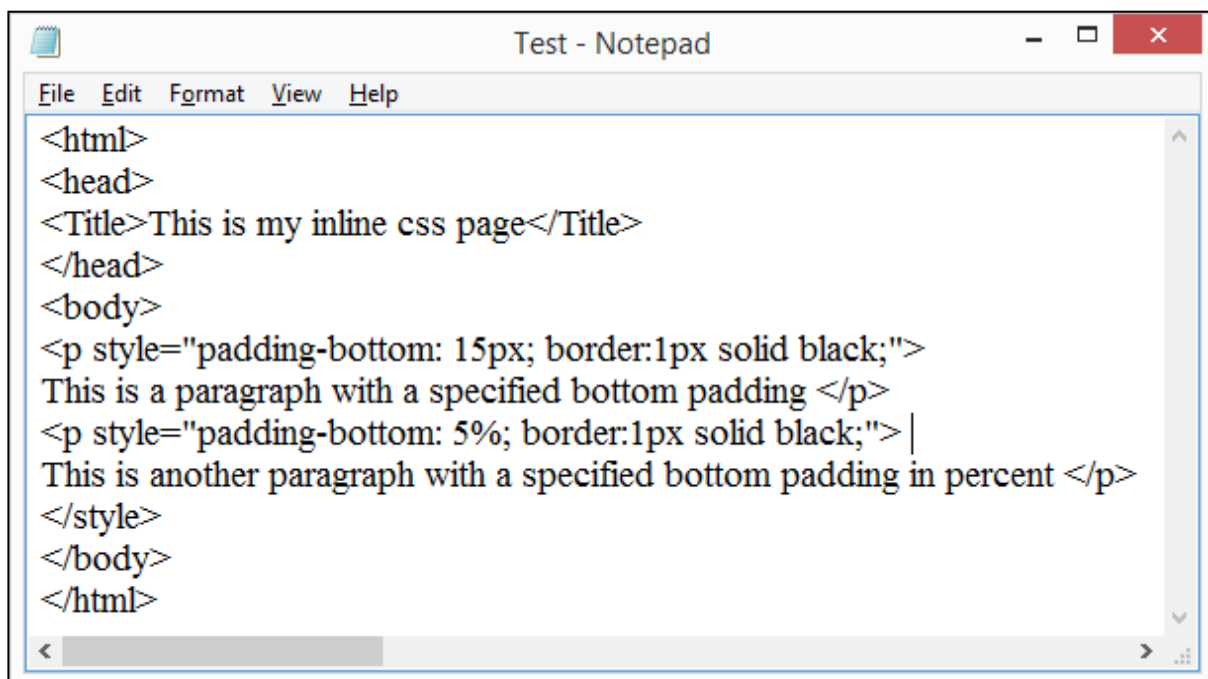
---

The padding property allows you to specify how much space should appear between the content of an element and its border: The value of this attribute should be either a length, a percentage, or the word inherits. If the value is inherit, it will have the same padding as its parent element. If a percentage is used, the percentage is of the containing box. The following CSS properties can be used to control lists. You can also set different values for the padding on each side of the box using the following properties:

- The padding-bottom specifies the bottom padding of an element.
- The padding-top specifies the top padding of an element.
- The padding-left specifies the left padding of an element.
- The padding-right specifies the right padding of an element.
- The padding serves as shorthand for the preceding properties.

### **4.1.1 The padding-bottom Property**

The padding-bottom property sets the bottom padding (space) of an element. This can take a value in terms of length of %. Here is an example:



```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="padding-bottom: 15px; border:1px solid black;">
This is a paragraph with a specified bottom padding </p>
<p style="padding-bottom: 5%; border:1px solid black;"> |
This is another paragraph with a specified bottom padding in percent </p>
</style>
</body>
</html>
```

The above program will produce the following result



#### 4.1.2 The padding-top Property

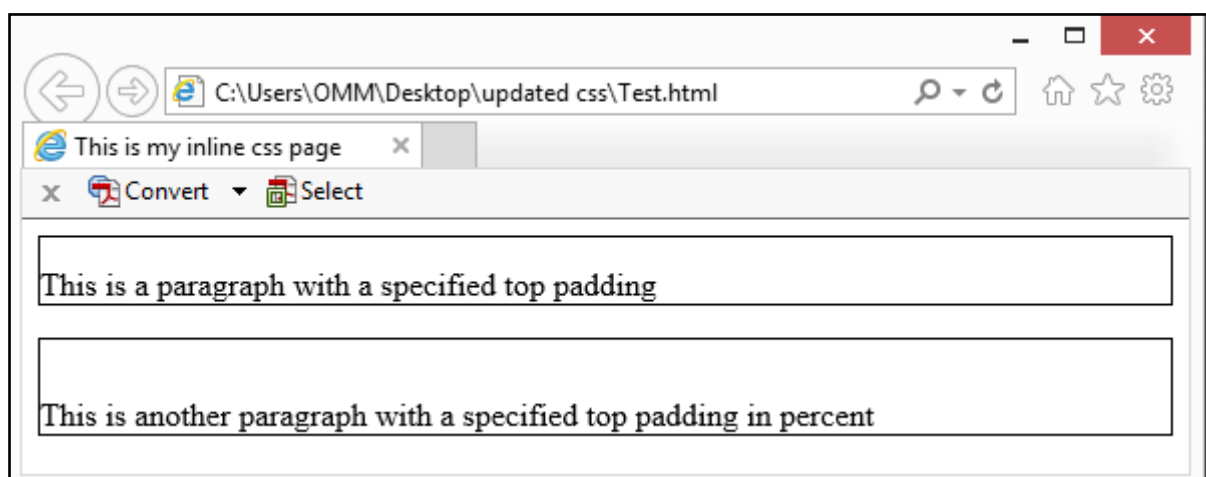
The padding-top property sets the top padding (space) of an element. This can take a value in terms of length of %. Here is an example:

```

Test - Notepad
File Edit Format View Help
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<p style="padding-top: 15px; border:1px solid black;">
This is a paragraph with a specified top padding </p>
<p style="padding-top: 5%; border:1px solid black;">
This is another paragraph with a specified top padding in percent </p>
</body>
</html>

```

The above program will produce the following result:



---

## 4.2 Setting Display Using CSS

---

The display property is the most important CSS property for controlling layout. The display property specifies if/how an element is displayed. Every HTML element has a default display value depending on what type of element it is. The default display value for most elements is block or inline. A block element is often called a block-level element. An inline element is always just called an inline element.

### 4.2.1 Block-level Elements

A block-level element always starts on a new line and takes up the full width available (stretches out to the left and right as far as it can).

The <div> element is a block-level element.

Examples of block-level elements:

- <div>
- <h1> - <h6>
- <p>
- <form>
- <header>
- <footer>
- <section>

<div>

div is the standard block-level element. A block-level element starts on a new line and stretches out to the left and right as far as it can. Other common block-level elements are *p* and *form*, and new in HTML5 are header, footer, section, and more.

</div>

### 4.2.2 Inline Elements

An inline element does not start on a new line and only takes up as much width as necessary. This is an inline <span>

element inside a paragraph.

Span is the standard inline element. An inline element can wrap some text inside a paragraph <span> like this </span> without disrupting the flow of that paragraph. The *a* element is the most common inline element, since you use them for links.

Examples of inline elements:

- <span>
- <a>
- <img>

## CSS Syntax

*Display: value;*

The values of ***display property*** have the following meanings:

- **Block** - This value causes an element to generate a block box.
- **Inline-block** - This value causes an element to generate an inline-level block container. The inside of an inline-block is formatted as a block box, and the element itself is formatted as an atomic inline-level box.
- **Inline** - This value causes an element to generate one or more inline boxes.
- **list-item** - This value causes an element (e.g., LI in HTML) to generate a principal block box and a marker box.
- **None** - This value causes an element to not appear in the formatting structure

Here are some examples of the display property:

```
p { display: block }
em { display: inline }
li { display: list-item }
```

---

### 4.3 Setting width and Maxwidth using CSS

---

Setting the **width** of a block-level element will prevent it from stretching out to the edges of its container. Then, you can set the margins to auto, to horizontally center the element within its container. The element will take up the specified width, and the remaining space will be split equally between the two margins:

This <div> element has a width of 500px, and margin set to auto.

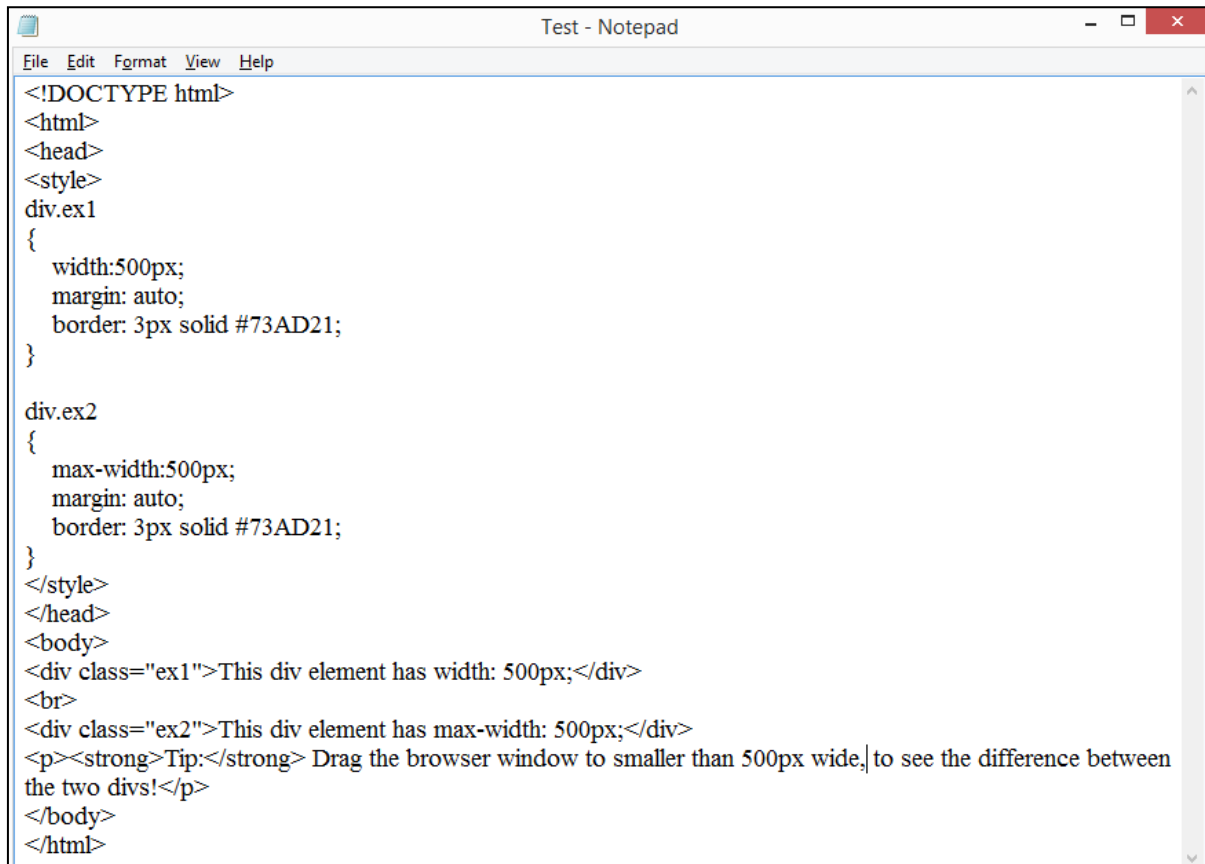
**Note:** The problem with the <div> above occurs when the browser window is smaller than the width of the element. The browser then adds a horizontal scrollbar to the page.

Using **max-width** instead, in this situation, will improve the browser's handling of small windows. This is important when making a site usable on small devices:

This <div> element has a max-width of 500px, and margin set to auto.

**Tip:** Resize the browser window to less than 500px wide, to see the difference between the two divs!

Here is an example of the two divs above:



```
<!DOCTYPE html>
<html>
<head>
<style>
div.ex1
{
 width:500px;
 margin: auto;
 border: 3px solid #73AD21;
}

div.ex2
{
 max-width:500px;
 margin: auto;
 border: 3px solid #73AD21;
}
</style>
</head>
<body>
<div class="ex1">This div element has width: 500px;</div>

<div class="ex2">This div element has max-width: 500px;</div>
<p>Tip: Drag the browser window to smaller than 500px wide, to see the difference between
the two divs!</p>
</body>
</html>
```

The output of the above program is following bellow:

---

## 4.4 Setting Position using CSS

---

CSS helps you to position your HTML element. You can put any HTML element at whatever location you like. You can specify whether you want the element positioned relative to its natural position in the page or absolute based on its parent element. Now, we will see all the CSS positioning related properties with examples.

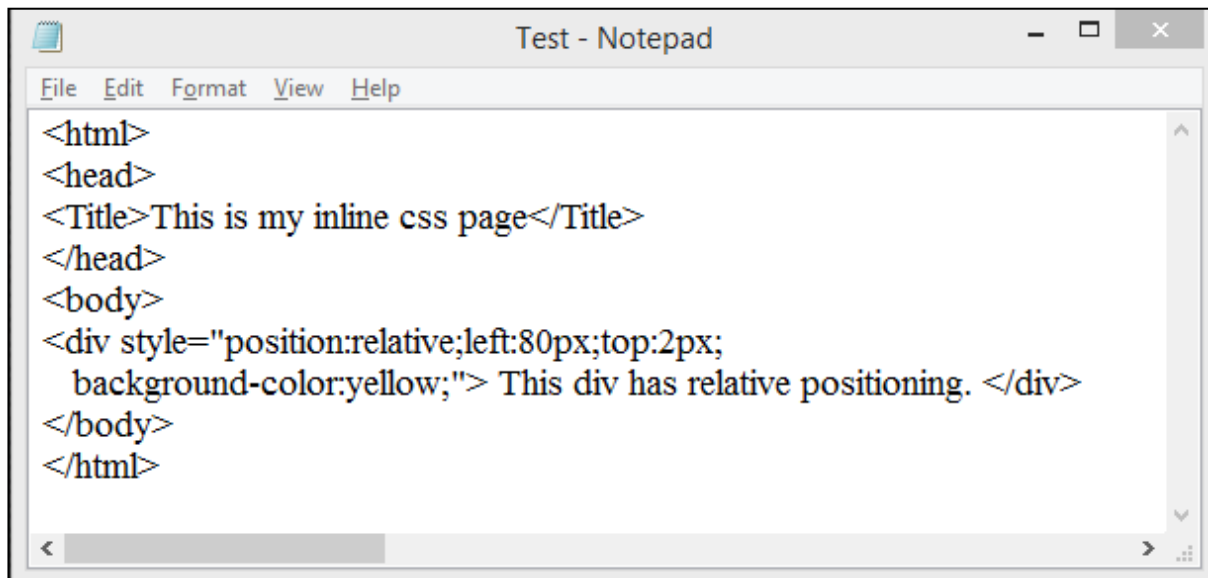
### 4.4.1 Relative Positioning

Relative positioning changes the position of the HTML element relative to where it normally appears. So "left:20" adds 20 pixels to the element's LEFT position. You can use two values top and left along with the position property to move an HTML element anywhere in an HTML document.

- Move Left - Use a negative value for left.
- Move Right - Use a positive value for left.
- Move Up - Use a negative value for top.
- Move Down - Use a positive value for top.

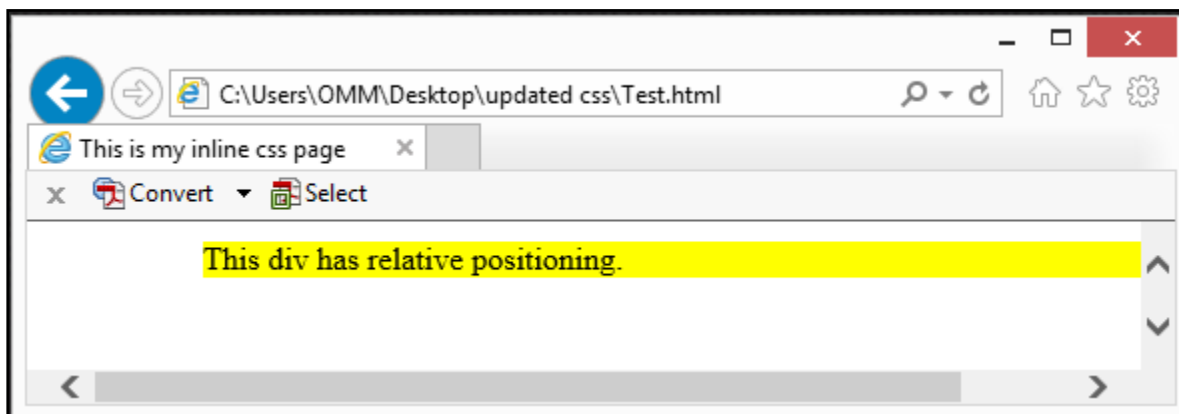
NOTE: You can use the bottom or right values as well in the same way as top and left.

Here is an example:



```
<html>
<head>
<Title>This is my inline css page</Title>
</head>
<body>
<div style="position:relative;left:80px;top:2px;
background-color:yellow;"> This div has relative positioning. </div>
</body>
</html>
```

The above program will produce the following result:



#### **4.4.2 Absolute Positioning**

An element with position: absolute is positioned at the specified coordinates relative to your screen top-left corner. You can use two values top and left along with the position property to move an HTML element anywhere in HTML document.

- Move Left - Use a negative value for left.
- Move Right - Use a positive value for left.
- Move Up - Use a negative value for top.
- Move Down - Use a positive value for top.

**NOTE:** You can use bottom or right values as well in the same way as top and left.

Here is an example:

```
<div style="position: absolute; left: 80px; top: 20px; background-color:
yellow ;">
This div has absolute positioning.
```

</div>

#### **4.4.3 Fixed Positioning**

Fixed positioning allows you to fix the position of an element to a particular spot on the page, regardless of scrolling. Specified coordinates will be relative to the browser window. You can use two values top and left along with the position property to move an HTML element anywhere in the HTML document.

- Move Left - Use a negative value for left.
- Move Right - Use a positive value for left.
- Move Up - Use a negative value for top.
- Move Down - Use a positive value for top.

NOTE: You can use bottom or right values as well in the same way as top and left.

Here is an example:

```
<div style="position:fixed;left:80px;top:20px; background-
color:yellow;">
This div has fixed positioning.
</div>
```

---

### **4.5 Setting the float property using CSS**

---

A float is a box that is shifted to the left or right on the current line. The most interesting characteristic of a float (or "floated" or "floating" box) is that content may flow along its side (or be prohibited from doing so by the property).

With CSS float, an element can be pushed to the left or right, allowing other elements to wrap around it. Float is very often used for images, but it is also useful when working with layouts.

A line box is next to a float when there exists a vertical position that satisfies all of these four conditions:

- (a) At or below the top of the line box,
- (b) At or above the bottom of the line box
- (c) Below the top margin edge of the float, and
- (d) Above the bottom margin edge of the float.

#### **4.5.1 Elements Float**

Elements are floated horizontally; this means that an element can only be floated left or right, not up or down. A floated element will move as far to the left or right as it can. Usually this means all the way to the left or right of the containing element. The elements after the floating element will flow around it. The elements before the floating element will not be affected. If an image is floated to the right, a following text flows around it, to the left:

**Example:**

```
img { float:right; }
```

If you place several floating elements after each other, they will float next to each other if there is room. Here we have made an image gallery using the float property:

**Example**

```
.thumbnail { float:left; width:110px; height:90px; margin:5px; }
```

**4.5.2 Turning off Float - Using Clear**

Elements after the floating element will flow around it. To avoid this, use the clear property. The clear property specifies which sides of an element other floating elements are not allowed. Add a text line into the image gallery, using the clear property:

**Example:**

```
.text_line { clear:both; }
```

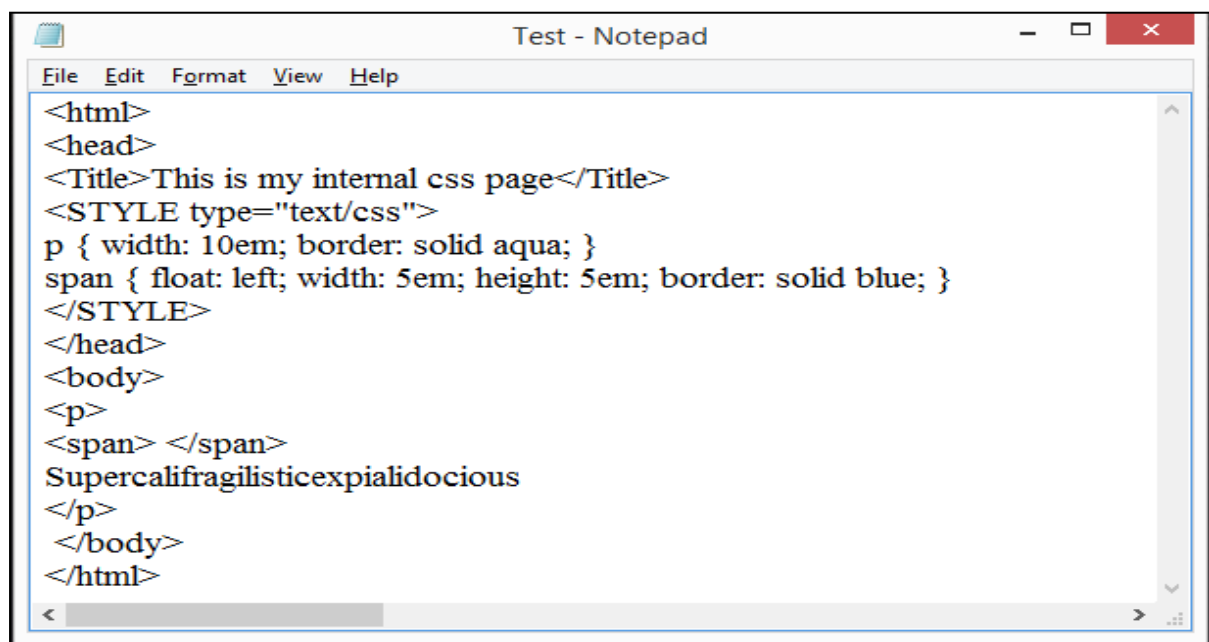
**4.5.3 All CSS Float Properties**

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
clear	Specifies which sides of an element where other floating elements are not allowed	left right both none inherit	1
float	Specifies whether or not a box should float	left right none inherit	1

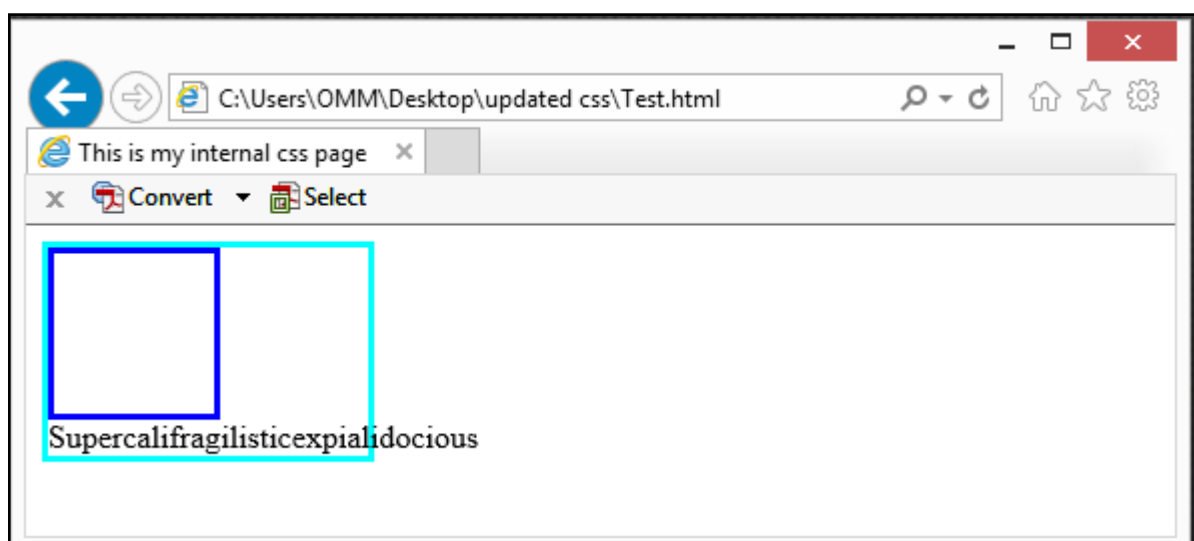
Example:

The above program will produce the following result:



```
<html>
<head>
<Title>This is my internal css page</Title>
<STYLE type="text/css">
p { width: 10em; border: solid aqua; }
span { float: left; width: 5em; height: 5em; border: solid blue; }
</STYLE>
</head>
<body>
<p>

Supercalifragilisticexpialidocious
</p>
</body>
</html>
```



---

## Check your progress 1

---

**Q1. What is Float ?**

**Answer:** \_\_\_\_\_  
\_\_\_\_\_

**Q2. Write the different properties of position?**

**Answer:** \_\_\_\_\_  
\_\_\_\_\_

**Q3. What is padding?**

**Answer:** \_\_\_\_\_  
\_\_\_\_\_

**Q4. Write the use of Display properties?**

**Answer:** \_\_\_\_\_  
\_\_\_\_\_

---

## 4.6 Inline Block Properties

---

It has been possible for a long time to create a grid of boxes that fills the browser width and wraps nicely (when the browser is resized), by using the float property. However, the inline-block value of the display property makes this even easier. Inline-block elements are like inline elements but they can have a width and a height.

### Examples

The old way - using float (notice that we also need to specify a clear property for the element after the floating boxes):

```
<!DOCTYPE html>
<html>
<head>
<style>
.floating-box
{
 float: left;
 width: 150px;
 height: 75px;
 margin: 10px;
 border: 3px solid #73AD21;
}

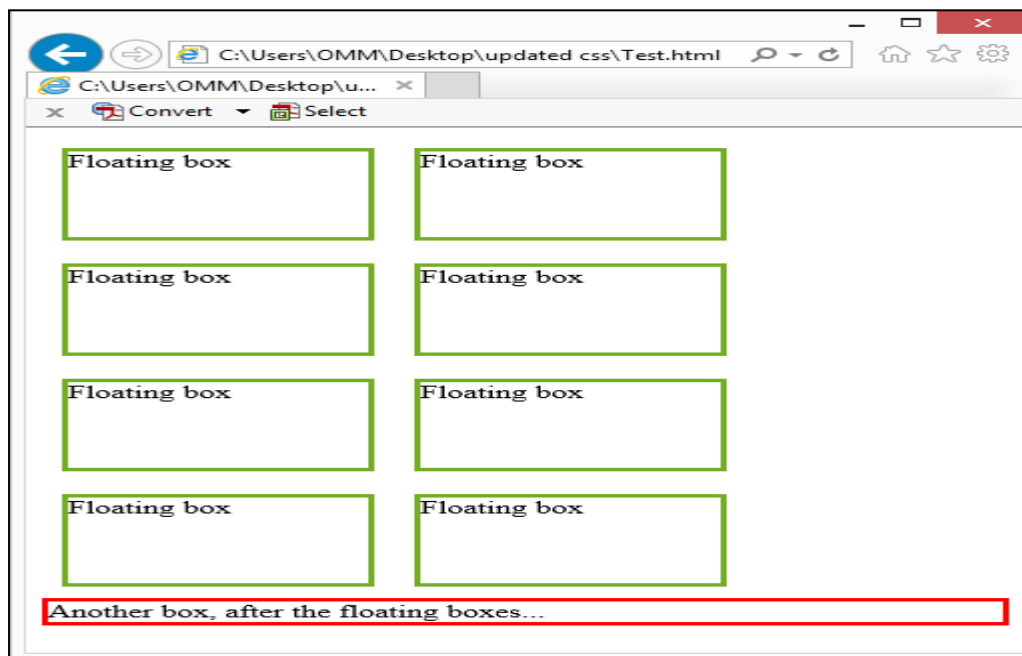
.after-box
{
 clear: left;
 border: 3px solid red;
}
```

```

</style>
</head>
<body>
<div class="floating-box">Floating box</div>
<div class="floating-box">Floating box</div>
<div class="floating-box">Floating box</div>
<div class="floating-box">Floating box</div>
<div class="floating-box">Floating box</div>
<div class="floating-box">Floating box</div>
<div class="floating-box">Floating box</div>
<div class="after-box">Another box, after the floating boxes...</div>
</body>
</html>

```

The above program will produce the following result




---

## 4.7 Horizontal Alignment in CSS

---

In CSS, several properties can be used to align elements horizontally and vertically.

### 4.7.1 Center Align Elements

To horizontally center a block element (like <div>), use `margin: auto;` Setting the width of the element will prevent it from stretching out to the edges of its container. The element will then take up the specified width, and the remaining space will be split equally between the two margins:

Example:

```
.center
{
 margin: auto;
 width: 50%;
 border: 3px solid green;
 padding: 10px;
}
```

**Note:** Center aligning has no effect if the width property is not set (or set to 100%).

#### **4.7.2 Center Align Text**

To just center the text inside an element, use `text-align: center;`

Example:

```
.center
{
 text-align: center;
 border: 3px solid green;
}
```

#### **4.7.3 Center an Image**

To center an image, use `margin: auto;` and make it into a block element:

```
img
{
 display: block;
 margin: auto;
 width: 40%;
}
```

#### **4.7.4 Left and Right Align - Using position**

One method for aligning elements is to use `position: absolute;;`

Example:

```
.right {
 position: absolute;
 right: 0px;
 width: 300px;
 border: 3px solid #73AD21;
 padding: 10px;
}
```

**Note:** Absolute positioned elements are removed from the normal flow, and can overlap elements.

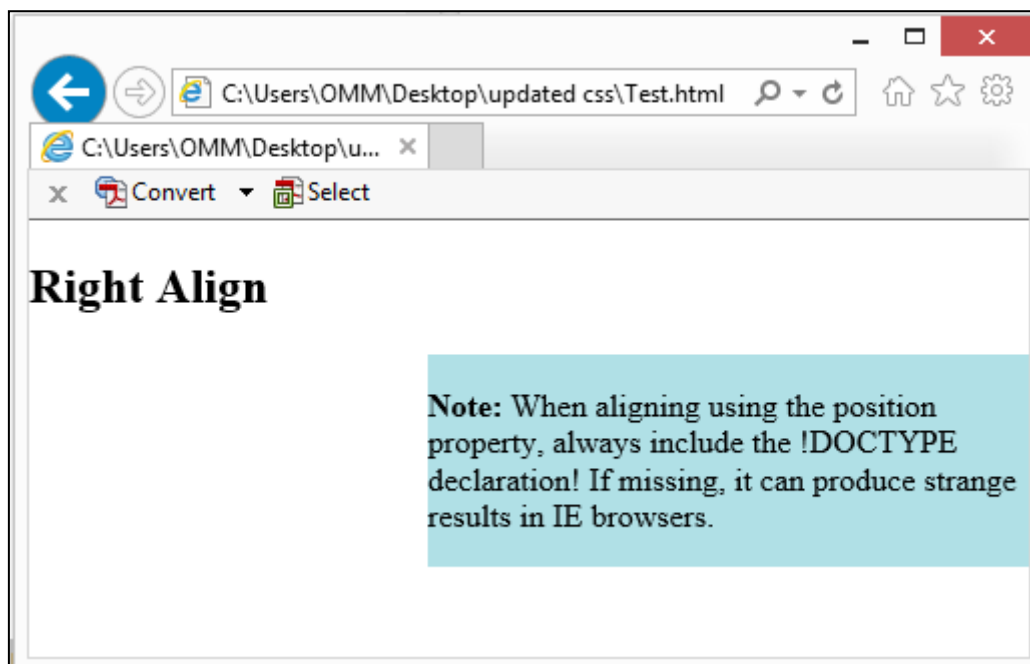
**Tip:** When aligning elements with position, always define margin and padding for the <body> element. This is to avoid visual differences in different browsers. There is also a problem with IE8 and earlier, when using position. If a container element (in our case <div class="container">) has a specified width, and the !DOCTYPE declaration is missing, IE8 and earlier versions will add a 17px margin on the right side. This seems to be space reserved for a scrollbar. So, always set the !DOCTYPE declaration when using position:

```
<!DOCTYPE html>
<html>
<head>
<style>
body
{
margin: 0;
padding: 0;
}
.container
{
position: relative;
width: 100%;
}
.right
{
position: absolute;
right: 0px;
width: 300px;
background-color: #b0e0e6;
}
</style>
</head>
<body>

<h2>Right Align</h2>

<div class="container">
<div class="right">
<p>Note: When aligning using the position property,
always include the !DOCTYPE declaration! If missing, it can
produce strange results in IE browsers.</p>
</div>
</div>
</body>
</html>
```

The above program will produce the following result



---

## 4.8 Working with Combinatory

---

A combinatory is something that explains the relationship between the selectors. A CSS selector can contain more than one simple selector. Between the simple selectors, we can include a combinatory.

There are four different combinatory in CSS:

- descendant selector (space)
- child selector (>)
- adjacent sibling selector (+)
- general sibling selector (~)

### 4.8.1 Descendant Selector

The descendant selector matches all elements that are descendants of a specified element.

The following example selects all <p> elements inside <div> elements:

Example

```
div p
{
 background-color: yellow;
}
```

### **4.8.2 Child Selector**

The child selector selects all elements that are the immediate children of a specified element.

The following example selects all <p> elements that are immediate children of a <div> element:

Example

```
<div > p {
 background-color: yellow;
}
```

### **4.8.3 Adjacent Sibling Selector**

The adjacent sibling selector selects all elements that are the adjacent siblings of a specified element. Sibling elements must have the same parent element, and "adjacent" means "immediately following". The following example selects all <p> elements that are placed immediately after <div> elements:

Example:

```
div + p {
 background-color: yellow;
}
```

### **4.8.4 General Sibling Selector**

The general sibling selector selects all elements that are siblings of a specified element. The following example selects all <p> elements that are siblings of <div> elements:

Example:

```
div ~ p {
 background-color: yellow;
}
```

---

## **4.9 Working with Pseudo-class**

---

A pseudo-class is used to define a special state of an element. For example, it can be used to:

- Style an element when a user mouses over it
- Style visited and unvisited links differently
- Style an element when it gets focus

### **Syntax**

The syntax of pseudo-classes:

selector: pseudo-class

```
{
 property: value;
}
```

#### **4.9.1 Anchor Pseudo-classes**

Links can be displayed in different ways:

Example

```
/* unvisited link */
a:link {
 color: #FF0000;
}
```

```
/* visited link */
a:visited {
 color: #00FF00;
}
```

```
/* mouse over link */
a:hover {
 color: #FF00FF;
}
```

```
/* selected link */
a:active {
 color: #0000FF;
}
```

**Note:** a: hover MUST come after a: link and a:visited in the CSS definition in order to be effective! a:active MUST come after a:hover in the CSS definition in order to be effective! Pseudo-class names are not case-sensitive.

#### **4.9.2 Pseudo-classes and CSS Classes**

Pseudo-classes can be combined with CSS classes: When you hover over the link in the example, it will change color:

Example

```
a.highlight:hover
{
 color: #ff0000;
}
```

Hover on <div>

An example of using the: hover pseudo-class on a <div> element:

Example

```
div:hover
{
 background-color: blue;
}
```

#### **4.9.3 Simple Tooltip Hover**

Hover over a <div> element to show a <p> element (like a tooltip):

Hover over me to show the <p> element.

Example

```

p
{
 display: none;
 background-color: yellow;
 padding: 20px;
}

div:hover p
{
 display: block;
}

```

#### **4.9.4 The: first-child Pseudo-class**

The: first-child pseudo-class matches a specified element that is the first child of another element.

#### **4.9.5 Match the first <p> element**

In the following example, the selector matches any <p> element that is the first child of any element:

Example

p: first-child

```

{
 color: blue;
}

```

Match the first <i> element in all <p> elements

In the following example, the selector matches the first <i> element in all <p> elements:

Example

p i:first-child

```

{
 color: blue;
}

```

Selector	Example	Example description
<u>:active</u>	a:active	Selects the active link
<u>:checked</u>	input:checked	Selects every checked <input> element
<u>:disabled</u>	input:disabled	Selects every disabled <input> element

<u>:empty</u>	p:empty	Selects every <p> element that has no children
<u>:enabled</u>	input:enabled	Selects every enabled <input> element
<u>:first-child</u>	p:first-child	Selects every <p> elements that is the first child of its parent
<u>:first-of-type</u>	p:first-of-type	Selects every <p> element that is the first <p> element of its parent
<u>:focus</u>	input:focus	Selects the <input> element that has focus
<u>:hover</u>	a:hover	Selects links on mouse over
<u>:in-range</u>	input:in-range	Selects <input> elements with a value within a specified range
<u>:invalid</u>	input:invalid	Selects all <input> elements with an invalid value
<u>:lang(language)</u>	p:lang(it)	Selects every <p> element with a lang attribute value starting with "it"
<u>:last-child</u>	p:last-child	Selects every <p> elements that is the last child of its parent
<u>:last-of-type</u>	p:last-of-type	Selects every <p> element that is the last <p> element of its parent
<u>:link</u>	a:link	Selects all unvisited links
<u>:not(selector)</u>	:not(p)	Selects every element that is not a <p> element
<u>:nth-child(n)</u>	p:nth-child(2)	Selects every <p> element that is the second child of its parent
<u>:nth-last-child(n)</u>	p:nth-last-child(2)	Selects every <p> element that is the second child of its parent, counting from the last child
<u>:nth-last-of-type(n)</u>	p:nth-last-of-type(2)	Selects every <p> element that is the second <p> element of its parent, counting from the last child
<u>:nth-of-type(n)</u>	p:nth-of-type(2)	Selects every <p> element that is the second <p> element of its parent
<u>:only-of-type</u>	p:only-of-type	Selects every <p> element that is the only <p> element of its parent
<u>:only-child</u>	p:only-child	Selects every <p> element that is the only child of its parent
<u>:optional</u>	input:optional	Selects <input> elements with no

		"required" attribute
<u>:out-of-range</u>	input:out-of-range	Selects <input> elements with a value outside a specified range
<u>:read-only</u>	input:read-only	Selects <input> elements with a "readonly" attribute specified
<u>:read-write</u>	input:read-write	Selects <input> elements with no "readonly" attribute
<u>:required</u>	input:required	Selects <input> elements with a "required" attribute specified
<u>:root</u>	root	Selects the document's root element
<u>:target</u>	#news:target	Selects the current active #news element (clicked on a URL containing that anchor name)
<u>:valid</u>	input:valid	Selects all <input> elements with a valid value
<u>:visited</u>	a:visited	Selects all visited links

---

## 4.10 Working with Pseudo-elements

---

CSS pseudo-elements are used to add special effects to some selectors. You do not need to use JavaScript or any other script to use those effects. A simple syntax of pseudo-element is as follows:

*selector: pseudo-element {property: value}*

CSS classes can also be used with the pseudo-elements:

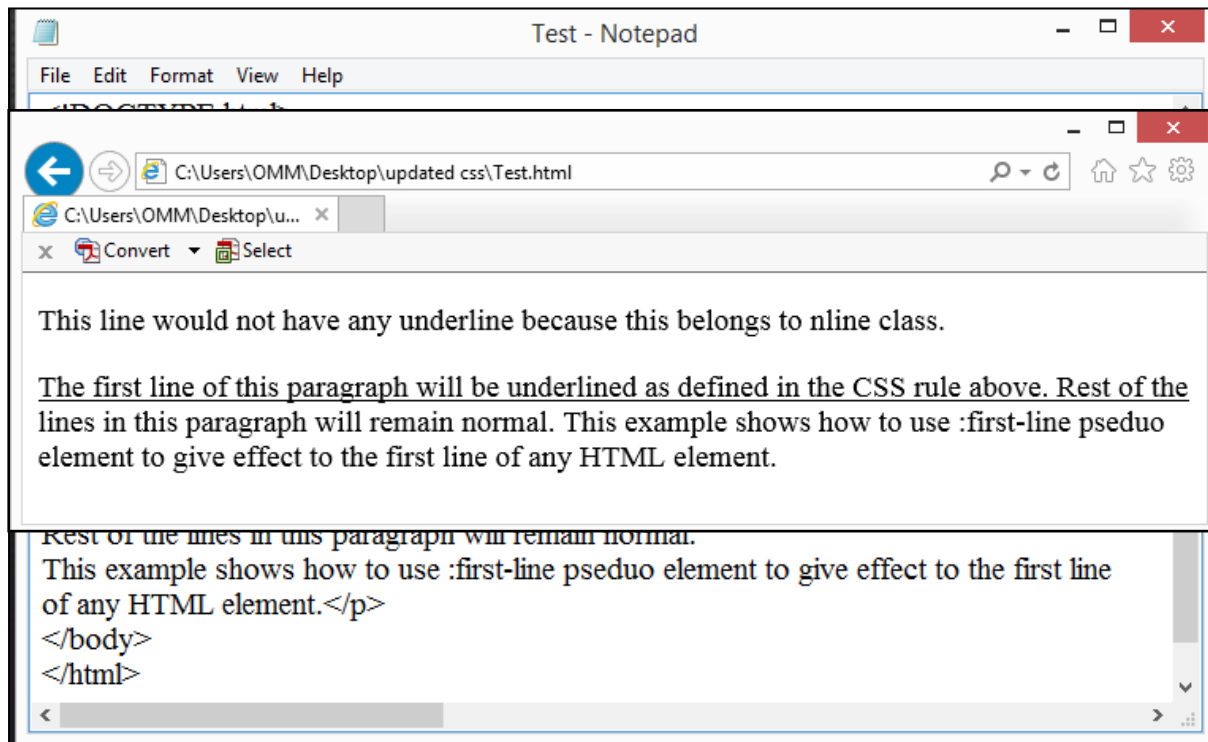
*selector.class: pseudo-element {property: value}*

The most commonly used pseudo-elements are as follows:

Value	Description
:first-line	Use this element to add special styles to the first line of the text in a selector.
:first-letter	Use this element to add special style to the first letter of the text in a selector.
:before	Use this element to insert some content before an element.
:after	Use this element to insert some content after an element.

#### **4.10.1 The: first-line pseudo-element**

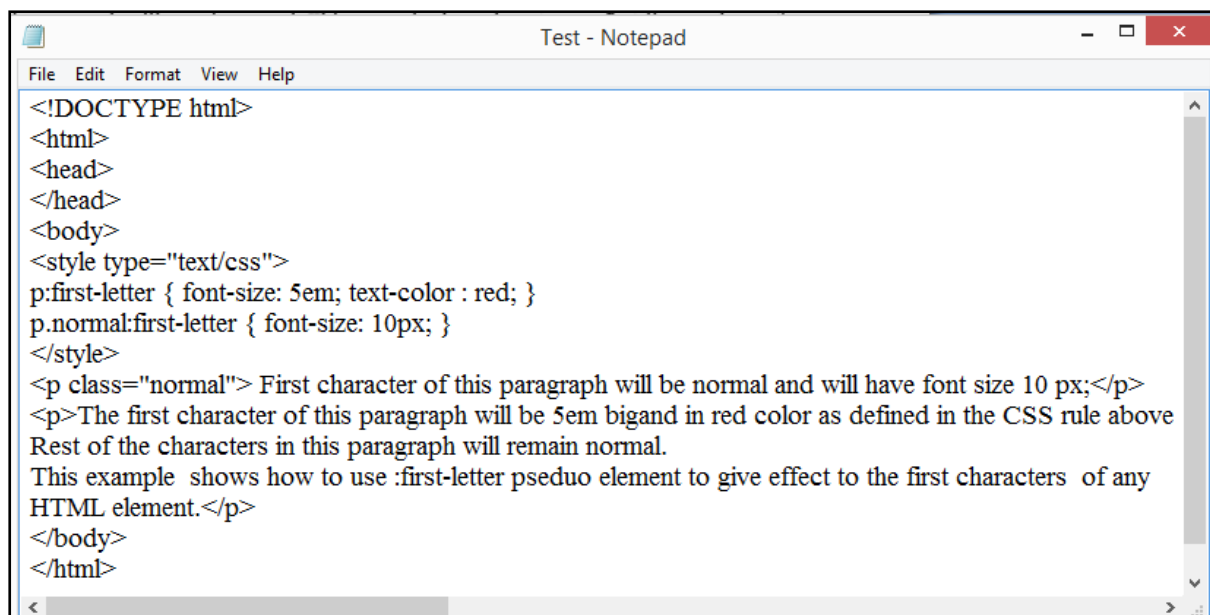
The following example demonstrates how to use the: first-line element to add special effects to the first line of elements in a document.



The above program will produce the following result :

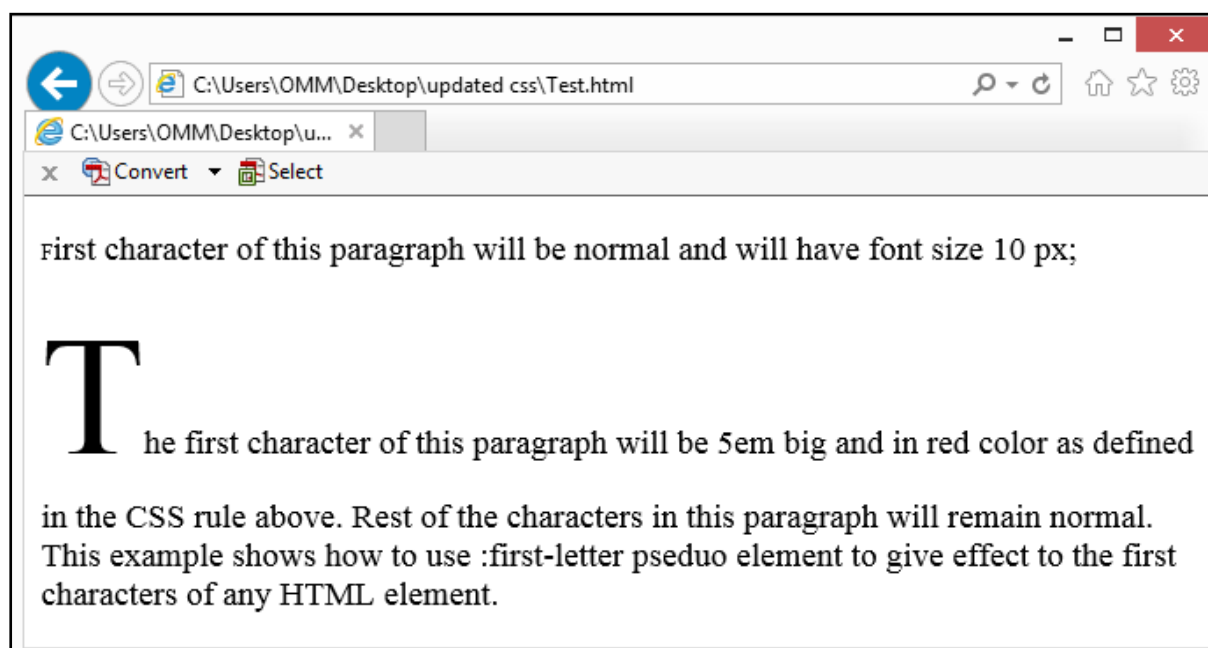
#### **4.10.2 The: first-letter pseudo-element**

The following example demonstrates how to use the: first-letter element to add special effect to the first letter of elements in the document.



```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<style type="text/css">
p:first-letter { font-size: 5em; text-color : red; }
p.normal:first-letter { font-size: 10px; }
</style>
<p class="normal"> First character of this paragraph will be normal and will have font size 10 px;</p>
<p>The first character of this paragraph will be 5em big and in red color as defined in the CSS rule above
Rest of the characters in this paragraph will remain normal.
This example shows how to use :first-letter pseduo element to give effect to the first characters of any
HTML element.</p>
</body>
</html>
```

The above program will produce the following result:



### **4.10.3 The: before pseudo-element**

The **:before pseudo-element** can be used to insert some content before the content of an element. The following example demonstrates how to use :before element to add some content before any element.

```
<style type="text/css">
p:before
{
 content: url(/images/bullet.gif)
}
</style>
<p> This line will be preceded by a bullet.</p>
<p> This line will be preceded by a bullet.</p>
<p> This line will be preceded by a bullet.</p>
```

### **4.10.4 The: after pseudo-element**

The :after pseudo-element can be used to insert some content after the content of an element. The following example demonstrates how to use: after element to add some content after any element.

```
<style type="text/css">
p:after
{
 content: url(/images/bullet.gif)
}
</style>
<p> This line will be succeeded by a bullet.</p>
<p> This line will be succeeded by a bullet.</p>
<p> This line will be succeeded by a bullet.</p>
```

### **4.10.5 The ::selection Pseudo-element**

The **selection pseudo-element** matches the portion of an element that is selected by a user. The following CSS properties can be applied to: selection: color, background, cursor, and outline. The following example makes the selected text red on a yellow background: Here is an example:

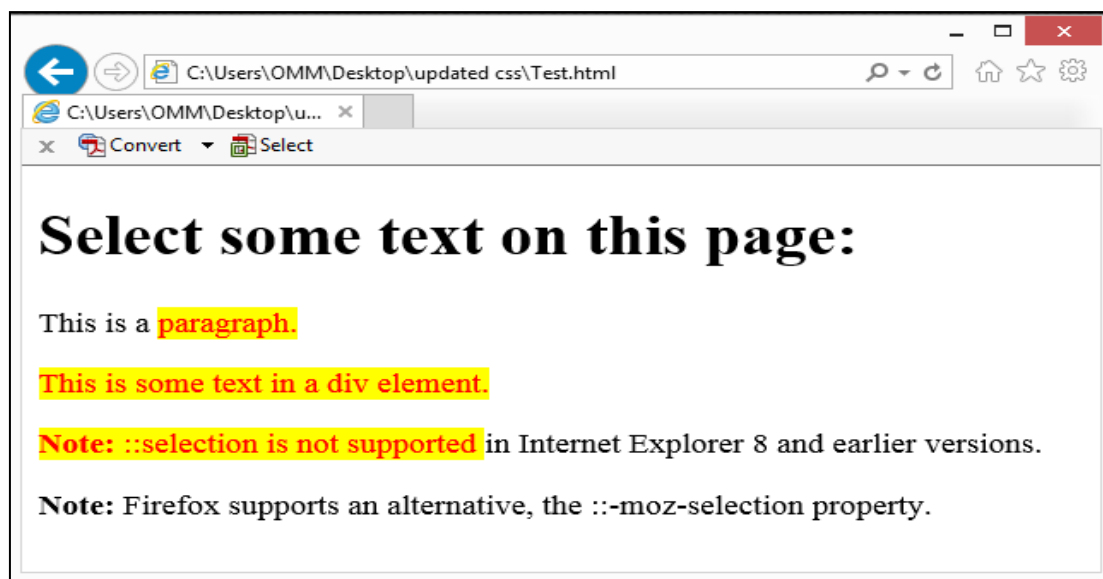
```
<!DOCTYPE html>
<html>
<head>
<style>
::-moz-selection { /* Code for Firefox */
 color: red;
 background: yellow;
```

```

}
::selection {
 color: red;
 background: yellow;
}
</style>
</head>
<body>
<h1>Select some text on this page:</h1>
<p>This is a paragraph.</p>
<div>This is some text in a div element.</div>
<p>Note: ::selection is not supported in Internet
Explorer 8 and earlier versions.</p>
<p>Note: Firefox supports an alternative, the ::-moz-
selection property.</p>
</body>
</html>

```

The above program will produce the following result:

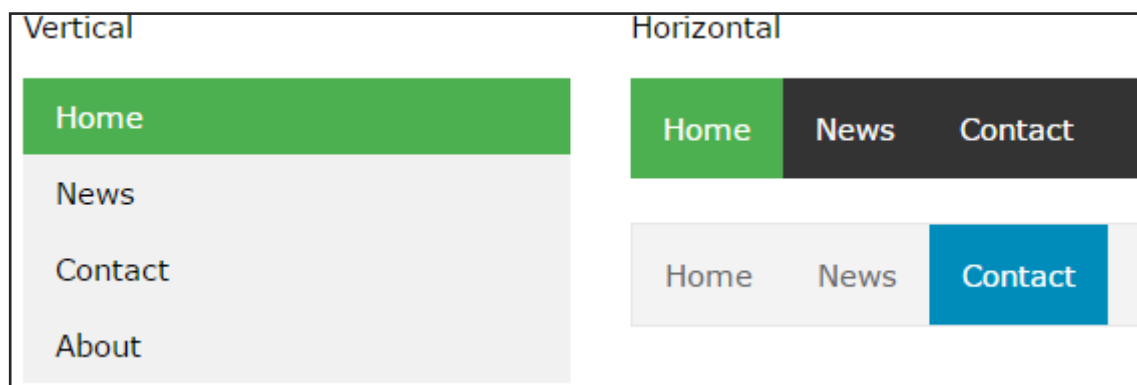


---

## 4.11 Creating a navigation bar

---

A **navigation bar** (or **navigation system**) is a section of a graphical user interface intended to aid visitors in accessing information. **Navigation bars** are implemented in file browsers, web browsers and as a design element of some web sites. A set of buttons or images in a row or column that serves as a control point to link the user to sections on a Web site. The **navigation bar** may also be a single graphic image with multiple selections.



(Example of vertical and horizontal navigation bar)

### 4.11.1 Navigation Bar = List of Links

A navigation bar needs standard HTML as a base. In our examples we will build the navigation bar from a standard HTML list. A navigation bar is basically a list of links, so using the `<ul>` and `<li>` elements makes perfect sense: Example

```

 Home
 News
 Contact
 About

```

Now let's remove the bullets and the margins and padding from the list:

```
ul
{
 list-style-type:none;
 margin:0;
 padding:0;
}
```

Example explained:

- list-style-type:none - Removes the bullets. A navigation bar does not need list markers
- Setting margins and padding to 0 to remove browser default settings

The code in the example above is the standard code used in both vertical, and horizontal navigation bars.

#### **4.11.2 Vertical Navigation Bar**

To build a vertical navigation bar we only need to style the <a> elements, in addition to the code above:

Example

```
a { display:block; width:60px; }
```

Example explained:

- display:block - Displaying the links as block elements makes the whole link area clickable (not just the text), and it allows us to specify the width
- width:60px - Block elements take up the full width available by default. We want to specify a 60 px width .

**Note:** Always specify the width for <a> elements in a vertical navigation bar. If you omit the width, IE6 can produce unexpected results. You can also set the width of <ul>, and remove the width of <a>, as they will take up the full width available when displayed as block elements. This will produce the same result as our previous example:

```
ul {
 list-style-type: none;
 margin: 0;
 padding: 0;
 width: 60px;
}

li a {
 display: block;
}
```

#### **4.11.3 Center Links & Add Borders**

Add text-align:center to <li> or <a> to center the links. Add the border property to <ul> add a border around the navbar. If you also want borders inside the navbar, add aborder-bottom to all <li> elements, except for the last one:

```
<!DOCTYPE html>
<html>
<head>
<style>
ul {
 list-style-type: none;
 margin: 0;
```

```

padding: 0;
width: 200px;
background-color: #f1f1f1;
border: 1px solid #555;
}

li a {
display: block;
color: #000;
padding: 8px 16px;
text-decoration: none;
}

li {
text-align: center;
border-bottom: 1px solid #555;
}
li:last-child {
border-bottom: none;
}

li a.active {
background-color: #4CAF50;
color: white;
}

li a:hover:not(.active) {
background-color: #555;
color: white;
}
</style>
</head>
<body>

```

## <h2>Vertical Navigation Bar</h2>

<p>In this example, we center the navigation links and add a border to the navigation bar.</p>

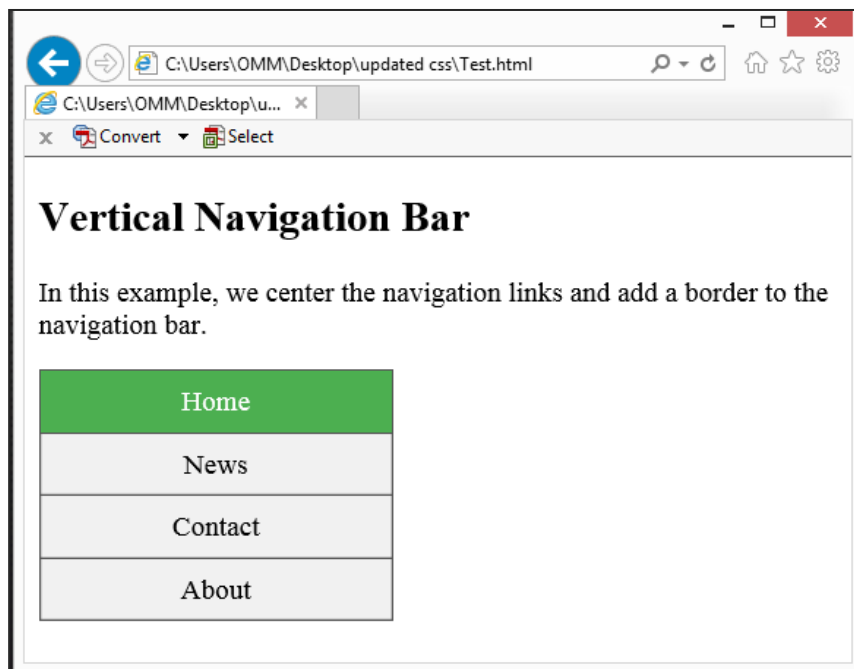
```


Home
News
Contact
About

</body>
</html>

```

The above program will produce the following result



#### **4.11.4 Horizontal Navigation Bar**

There are two ways to create a horizontal navigation bar. Using **inline** or **floating** list items. Both methods work fine, but if you want the links to be the same size, you have to use the floating method. Create a basic horizontal navigation bar with a dark background color and change the background color of the links when the user moves the mouse over them:

```
<!DOCTYPE html>
<html>
<head>
<style>
ul {
 list-style-type: none;
 margin: 0;
 padding: 0;
 overflow: hidden;
 background-color: #333;
}

li {
```

```

 float: left;
 }

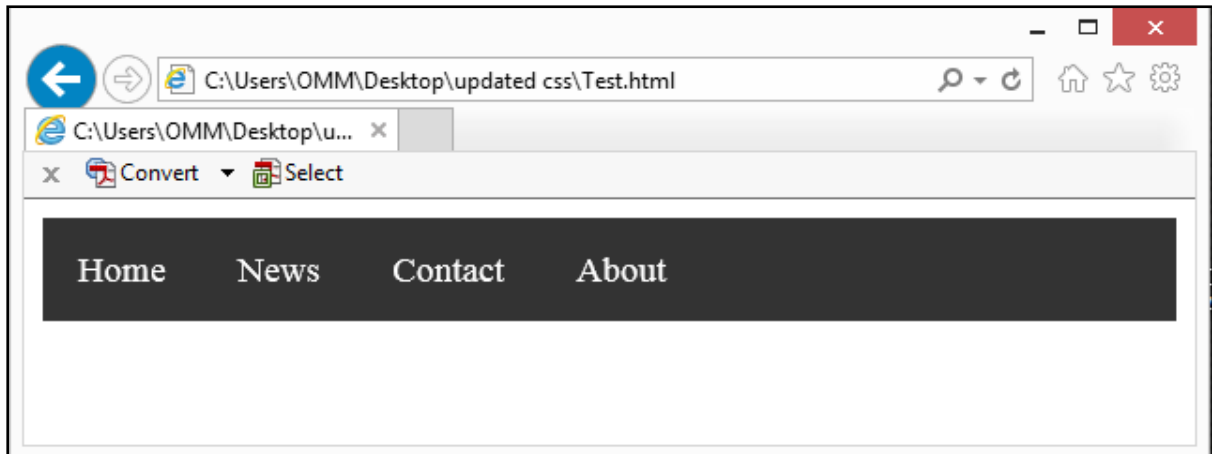
 li a {
 display: block;
 color: white;
 text-align: center;
 padding: 14px 16px;
 text-decoration: none;
 }

 li a:hover {
 background-color: #111;
 }
</style>
</head>
<body>

 Home
 News
 Contact
 About

</body>
</html>

```



---

## 4.12 Working with images

---

Images play an important role in any webpage. Though it is not recommended to include a lot of images, but it is still important to use good images wherever required. CSS plays a good role to control image display. You can set the following image properties using CSS.

The **border** property is used to set the width of an image border.

- The **height** property is used to set the height of an image.
- The **width** property is used to set the width of an image.
- The **-moz-opacity** property is used to set the opacity of an image.
- 

### 4.12.1 The Image Border Property

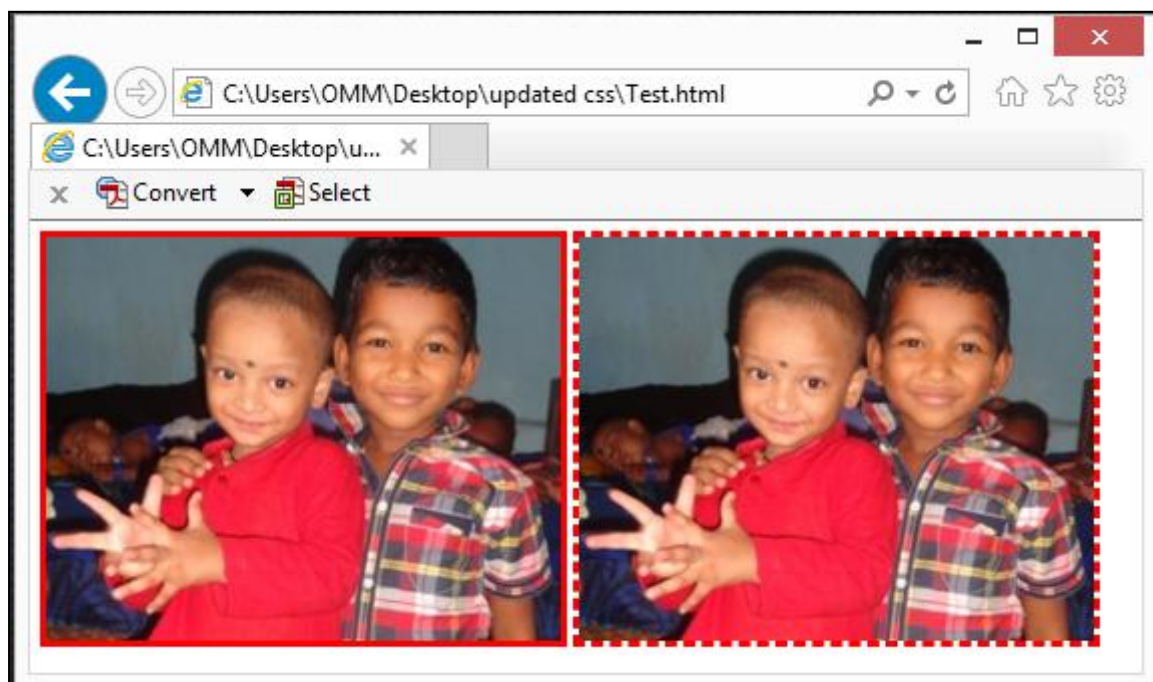
The *border* property of an image is used to set the width of an image border. This property can have a value in length or in %. A width of zero pixels means no border.

Here is an example:

```
Test - Notepad
File Edit Format View Help
<!DOCTYPE html>
<html>
<head>
</head>
<body>

</body>
</html>
```

The above program will produce the following result:



#### **4.12.2 The Image Width Property**

The *width* property of an image is used to set the width of an image. This property can have a value in length or in %. While giving value in %, it applies it in respect of the box in which an image is available.

Here is an example:

```
Test - Notepad
File Edit Format View Help
<!DOCTYPE html>
<html>
<head>
</head>
<body>

</body>
</html>
```

The above program will produce the following result:



---

## 4.13 Working with attribute selectors

---

You can also apply styles to HTML elements with particular attributes. The style rule below will match all the input elements having a type attribute with a value of *text*:

```
input[type="text"]
{
 color: #000000;
}
```

The advantage to this method is that the `<input type="submit" />` element is unaffected, and the color applied only to the desired text fields.

There are following rules applied to attribute selector.

- **p[lang]** - Selects all paragraph elements with a *lang* attribute.
- **p[lang="fr"]** - Selects all paragraph elements whose *lang* attribute has a value of exactly "fr".
- **p[lang~="fr"]** - Selects all paragraph elements whose *lang* attribute contains the word "fr".
- **p[lang|= "en"]** - Selects all paragraph elements whose *lang* attribute contains values that are exactly "en", or begin with "en-".

### 4.13.1 Multiple Style Rules

You may need to define multiple style rules for a single element. You can define these rules to combine multiple properties and corresponding values into a single block as defined in the following example:

```
h1 {
 color: #36C;
 font-weight: normal;
 letter-spacing: .4em;
 margin-bottom: 1em;
 text-transform: lowercase;
}
```

Here all the property and value pairs are separated by a **semicolon (;)**. You can keep them in a single line or multiple lines. For better readability, we keep them in separate lines.

### 4.13.2 CSS [attribute] Selector

The [attribute] selector is used to select elements with a specified attribute. The following example selects all `<a>` elements with a target attribute:

Example

```
a[target] {
 background-color: yellow;
}
```

### **4.13.3 CSS [attribute="value"] Selector**

The [attribute="value"] selector is used to select elements with a specified attribute and value.

The following example selects all <a> elements with a target="\_blank" attribute:

Example

```
a[target="_blank"] {
 background-color: yellow;
}
```

### **4.13.4 CSS [attribute~="value"] Selector**

The [attribute~="value"] selector is used to select elements with an attribute value containing a specified word. The following example selects all elements with a title attribute that contains a space-separated list of words, one of which is "flower":

Example

```
[title~="flower"] {
 border: 5px solid yellow;
}
```

The example above will match elements with title="flower", title="summer flower", and title="flower new", but not title="my-flower" or title="flowers".

### **4.13.5 CSS [attribute|= "value"] Selector**

The [attribute|= "value"] selector is used to select elements with the specified attribute starting with the specified value. The following example selects all elements with a class attribute value that begins with "top": **Note:** The value has to be a whole word, either alone, like class="top", or followed by a hyphen( - ), like class="top-text"!

Example

```
[class|= "top"] {
 background: yellow;
}
```

### **4.13.6 CSS [attribute^="value"] Selector**

The [attribute^="value"] selector is used to select elements whose attribute value begins with a specified value.

The following example selects all elements with a class attribute value that begins with "top":

**Note:** The value does not have to be a whole word!

Example

```
[class^="top"] {
 background: yellow;
}
```

#### **4.13.7 CSS [attribute\$="value"] Selector**

The [attribute\$="value"] selector is used to select elements whose attribute value ends with a specified value.

The following example selects all elements with a class attribute value that ends with "test":

**Note:** The value does not have to be a whole word!

Example

```
[class$="test"] {
 background: yellow;
}
```

#### **4.13.8 CSS [attribute\*="value"] Selector**

The [attribute\*="value"] selector is used to select elements whose attribute value contains a specified value. The following example selects all elements with a class attribute value that contains "te":

**Note:** The value does not have to be a whole word!

Example:

```
[class*="te"] {
 background: yellow;
}
```

---

### **Check your progress 2**

---

**Q1. Write the different properties of image ?**

**Answer:** \_\_\_\_\_  
\_\_\_\_\_

**Q2. What is Pseudo Class ?**

**Answer:** \_\_\_\_\_  
\_\_\_\_\_

**Q3. What is navigation bar?**

**Answer:** \_\_\_\_\_  
\_\_\_\_\_

---

#### **4.14 Let us sum up**

---

In this unit we have understood padding using CSS, setting display using CSS, setting width and max width using CSS, setting float property using CSS, inline block property, horizontal alignment in CSS, working with combinatory, pseudo-class, pseudo-elements, creating a navigation bar, working with images, attribute selectors.

---

#### **4.15 Reference**

---

1. W3Schools.com
2. Google.com
3. Tutorialpoints.com

---

#### **4.16 Check Your Progress Possible answer**

---

##### **Check your progress 1**

##### **Q1. What is Float ?**

###### **Answer**

A float is a box that is shifted to the left or right on the current line. The most interesting characteristic of a float (or "floated" or "floating" box) is that content may flow along its side (or be prohibited from doing so by the property).

##### **Q2. Write the different properties of position?**

###### **Answer**

Position property to move an HTML element anywhere in an HTML document.

- Move Left - Use a negative value for left.
- Move Right - Use a positive value for left.
- Move Up - Use a negative value for top.
- Move Down - Use a positive value for top.

##### **Q3. What is padding?**

###### **Answer**

The padding property allows you to specify how much space should appear between the content of an element and its border

##### **Q4. Write the use of Display properties?**

###### **Answer**

The display property is the most important CSS property for controlling layout. The display property specifies if/how an element is displayed

## **Check your progress 2**

### **Q1. Write the different properties of image ?**

**Answer** The **border** property is used to set the width of an image border.

The **height** property is used to set the height of an image.

The **width** property is used to set the width of an image.

### **Q2. What is Pseudo Class ?**

**Answer**

A pseudo-class is used to define a special state of an element

### **Q3. What is navigation bar?**

**Answer**

A **navigation bar** (or **navigation** system) is a section of a graphical user interface intended to aid visitors in accessing information. **Navigation bars** are implemented in file browsers, web browsers and as a design element of some web sites

## TAILWIND CSS OFFICIAL COMPLETE DICTIONARY (A TO Z UTILITIES) IN GUJARATI

આ માર્ગદર્શિકા [tailwindcss.com](https://tailwindcss.com) ના સત્તાવાર ડોક્યુમેન્ટેશન પર આધારિત છે. આમાં Tailwind CSS ના તમામ યુટિલિટી ક્લાસીસ, તેમનું માળખું, બુટસ્ટ્રેપથી તફાવત, અને તમામ પ્રોપર્ટીઝનું ઊંડાણપૂર્વકનું લીટીએ-લીટી વિવરણ સરળ ગુજરાતી ભાષામાં આપેલું છે.

### 1. FOUNDATION, HISTORY & VERSIONS (ઇતિહાસ અને પાયાના સિદ્ધાંતો)

- શેના માટે છે?: વેબસાઇટ ડિઝાઇન કરતી વખતે અલગથી મોટો કસ્ટમ CSS કોડ લખ્યા વગર, સીધા જ HTML ટેગ્સની અંદર નાના-નાના સિંગલ-પર્પઝ ક્લાસ ઉમેરીને અતિ આધુનિક ઇન્ટરફેસ બનાવવા માટે.
- સ્થાપક (Founder): Tailwind CSS ની શોધ Adam Wathan (એડમ વાથન) દ્વારા કરવામાં આવી હતી અને વર્ષ 2017 માં તેનો પ્રથમ ઓપન-સોર્સ પ્રોટોટાઇપ રિલીઝ કરાયો હતો.
- બુટસ્ટ્રેપથી તફાવત (How it differs from Bootstrap):
  - \* Bootstrap એક Component-driven framework છે. તે તમને બનેલા રેડીમેડ વિજેટ્સ આપે છે (જેમ કે `.btn` અથવા `.card`), જેના કારણે બધી વેબસાઇટ્સ એકસરખી દેખાય છે.
  - \* Tailwind એક Utility-first framework છે. આમાં કોઈ રેડીમેડ બટન કે કાર્ડ નથી હોતા. તે બિલકુલ પ્યોર CSS ની જેમ જ અણગણિત યુટિલિટીઝ (જેમ કે `pt-4`, `flex`, `bg-blue-500`) આપે છે, જેને મિક્સ કરીને તમે તમારી મરજી મુજબની અનોખી ડિઝાઇન બનાવી શકો છો.
- વર્ઝન ઇવોલ્યુશન (Version Evolution):
  - \* v1.0: બેઝિક કન્ફિગરેશન ફાઇલ અને પ્રી-ફ્લાઇટ સેટિંગ્સ લવાયા.
  - \* v2.0: ડાર્ક મોડ સપોર્ટ અને વધારાના કલર પેલેટ્સ ઉમેરાયા.
  - \* v3.0: Just-In-Time (JIT) એન્જિન ઇન-બિલ્ટ કરાયું, જે કોડ સેવ થતાં જ ઓન-ધ-ફ્લાય કસ્ટમ સીએસએસ જનરેટ કરે છે.
  - \* v4.0 (Latest): અલ્ટ્રા-ફાસ્ટ રસ્ટ-બેઝ્ડ કમ્પાઇલર અને સીએસએસ-ફર્સ્ટ કન્ફિગરેશન આર્કિટેક્ચર સાથે સંપૂર્ણ રિડિઝાઇન કરાયું.

### 2. CORE CONCEPTS & STATES (મૂળભૂત સિદ્ધાંતો અને કન્ડિશનલ ક્લાસીસ)

- Responsive Breakpoints (રિસ્પોન્સિવ સ્ક્રીન સ્ટેટ્સ):
  - \* sm: (Small - 640px અને તેથી વધુ પહોળાઈ) -> મોટા મોબાઇલ અને ટેબ્લેટ્સ.
  - \* md: (Medium - 768px અને તેથી વધુ પહોળાઈ) -> લેપટોપ સ્ક્રીન.
  - \* lg: (Large - 1024px) -> મોટા કોમ્પ્યુટર મોનિટર્સ.
  - \* xl: (Extra Large - 1280px) -> વાઇડ સ્ક્રીન ટીવી અને ડેસ્કટોપ.
  - \* 2xl: (Ultra Large - 1536px) -> સુપર વાઇડ 4K મોનિટર્સ.
  - \* ઉપયોગ: `md:text-lg` (માત્ર કોમ્પ્યુટર સ્ક્રીન પર ટેક્સ્ટ મોટું થશે, મોબાઇલમાં નોર્મલ રહેશે).
- Hover, Focus, & Active States:
  - \* hover: જ્યારે યુઝર માઉસ પોઇન્ટર એવિમેન્ટ પર લાવે ત્યારે ક્લાસ બદલવા. દા.ત. `hover:bg-black`.
  - \* focus: ઇનપુટ બોક્સ પર ક્લિક થાય ત્યારે ઇફેક્ટ આપવા. દા.ત. `focus:border-blue-500`.
  - \* active: યુઝર જ્યારે ક્લિક દબાવી રાખે તે ક્ષણે. દા.ત. `active:scale-95`.
- Dark Mode Stat:
  - \* dark: યુઝરની ઓપરેટિંગ સિસ્ટમ કે બ્રાઉઝરમાં ડાર્ક થીમ ચાલુ હોય ત્યારે જ એક્ટિવેટ થાય છે. દા.ત. `dark:bg-slate-900`.

### 3. LAYOUT UTILITY CLASSES (લેઆઉટ અને ગોઠવણની પ્રોપર્ટીઝ)

- Display (ડિસ્પ્લે મોડ):
  - \* block: એવિમેન્ટ આખી પહોળાઈ (100% width) રોકી લેશે.
  - \* inline-block: લાઇન ની અંદર રહેશે પણ માર્જિન-પેડિંગ સપોર્ટ કરશે.
  - \* inline: પ્યોર ટેક્સ્ટની જેમ વર્તશે.
  - \* flex: ફ્લેક્સબોક્સ કન્ટેનર એક્ટિવેટ કરે છે.
  - \* grid: ગ્રીડ લેઆઉટ એક્ટિવેટ કરે છે.
  - \* hidden: એવિમેન્ટને પેજ પરથી સાવ ગાયબ (Display: none) કરી દેશે.
- Aspect Ratio & Object Fit:
  - \* aspect-square: પહોળાઈ અને લંબાઈ 1:1 એટલે કે ચોરસ રાખવા.
  - \* aspect-video: 16:9 સિનેમેટિક કે યુટ્યુબ વિડીયો સાઇઝ ફિક્સ કરવા.
  - \* object-cover: ઇમેજ કે વિડિયો સ્ટ્રેચ (ખેંચાયા) વગર પોતાના કન્ટેનરમાં બરાબર ફીટ થઈ જાય.
  - \* object-contain: આખો ફોટો કપાયા વગર અંદર સેન્ટર થશે.
- Overflow:
  - \* overflow: આખો ફોટો કપાયા વગર અંદર સેન્ટર થશે.

- \* overflow-auto: જો ડેટા બોક્સની બહાર જાય, તો આપોઆપ સ્ક્રોલ બાર આવી જશે.
- \* overflow-hidden: બોક્સની બહાર નીકળતો વધારાનો ડેટા સાવ કાપી નાખશે (Crop).
- Position & Z-Index:
  - \* static: નોર્મલ ડીફ્યુમેન્ટ ફ્લો.
  - \* relative: પોતાના ઓરિજિનલ સ્થાનના સંદર્ભમાં ખસવા માટે રેપર.
  - \* absolute: પેજના કોઈ ચોક્કસ સ્થાને ફિક્સ એડેસિંગ કરવા.
  - \* fixed: પેજ ગમે એટલું સ્ક્રોલ કરો તો પણ એલિમેન્ટ સ્ક્રીન પર ત્યાં જ ચોટી રહેશે.
  - \* sticky: સ્ક્રોલ કરતા કરતા એક સ્ટેજ પર આવીને ચોટી જશે.
  - \* top-0, bottom-0, left-0, right-0: ચારેય દિશાઓના અંતર ઝીરો કરવા.
  - \* z-0, z-10, z-20, z-50, z-auto: કયો એલિમેન્ટ કયા એલિમેન્ટની ઉપર (3D Layering) દેખાશે તે સ્તર નક્કી કરે છે.

#### 4. FLEXBOX & GRID UTILITY CLASSES (આધુનિક અવાઈનમેન્ટ ટૂલ્સ)

- Flexbox Direction & Wrapping:
  - \* flex-row: અંદરની આઇટમ્સ આડી લાઇનમાં ગોઠવાશે.
  - \* flex-col: અંદરની આઇટમ્સ ઉભી લાઇન (Column) માં ગોઠવાશે.
  - \* flex-wrap: જો જગ્યા ઓછી પડે, તો આઇટમ્સ ઓટોમેટિક નવી લાઇનમાં નીચે ઉતરી જશે.
- Justify Content (આડી દિશાનું અવાઈનમેન્ટ):
  - \* justify-start: બધી વસ્તુઓ ડાબી બાજુ ભેગી થશે.
  - \* justify-end: બધી વસ્તુઓ જમણી બાજુ ભેગી થશે.
  - \* justify-center: બધી વસ્તુઓ બિલકુલ સેન્ટરમાં આવી જશે.
  - \* justify-between: બે છેડાની વસ્તુઓ ખૂણા પર જશે અને વચ્ચે સરખી જગ્યા છૂટશે.
- Align Items (ઉભી દિશાનું અવાઈનમેન્ટ):
  - \* items-start: ઉભી રીતે ટોપ પર સેટ થશે.
  - \* items-end: ઉભી રીતે બોટમ પર સેટ થશે.
  - \* items-center: ઉભી દિશામાં બરાબર વચ્ચે (Vertically Center) એલાઈન થશે.
- Grid Columns & Rows:
  - \* grid-cols-1, grid-cols-2, grid-cols-3, grid-cols-4, grid-cols-12: આખા બોક્સને આડા ૧ થી ૧૨ ભાગમાં વહેંચવા.
  - \* col-span-1, col-span-2, col-span-6, col-span-full: કોઈ એક ચાઇલ્ડ બોક્સ કેટલી કોલમની જગ્યા રોકશે તે સેટ કરવા.
- Gap (જગ્યા):
  - \* gap-1, gap-2, gap-4, gap-8: ગ્રીડ કે ફ્લેક્સ આઇટમ્સ વચ્ચે અંદર અંદર સ્પેસિંગ સેટ કરવા.

#### 5. SPACING, SIZING, & TYPOGRAPHY (જગ્યા, કદ અને અક્ષરોનું સેટિંગ)

- Padding & Margin (અંદર અને બહારની સ્પેસ):
  - \* p-\*, pt-\*, pb-\*, ps-\*, pe-\*, px-\*, py-\*: Padding (અંદરની સ્પેસ) ૦ થી ૯૬ ના રેશિયોમાં.
  - \* m-\*, mt-\*, mb-\*, ms-\*, me-\*, mx-\*, my-\*: Margin (બહારની સ્પેસ).
  - \* mx-auto: બોક્સને ડાબે-જમણેથી સરખી માર્જિન આપીને પેજની વચ્ચે સેન્ટર કરવા.
- Sizing (Width & Height):
  - \* w-0, w-4, w-10, w-64, w-96: પિક્સલ આધારિત પહોળાઈ.
  - \* w-1/2, w-1/3, w-2/3, w-full: ટકાવારી આધારિત પહોળાઈ (50%, 33%, 66%, 100%).
  - \* h-screen, w-screen: બ્રાઉઝરની આખી સ્ક્રીનની લંબાઈ અને પહોળાઈ રોકવા (100vh / 100vw).
- Typography (અક્ષરોની સ્ટાઇલ):
  - \* text-xs, text-sm, text-base, text-lg, text-xl, text-2xl, text-6xl: ફોન્ટની સાઇઝ નાનીથી લઈને સુપર મેગા મોટી કરવા.
  - \* font-light, font-normal, font-medium, font-bold, font-extrabold: અક્ષરોની જાડાઈ કંટ્રોલ કરવા.
  - \* text-left, text-center, text-right, text-justify: ટેક્સ્ટને અલાઈન કરવા.
  - \* text-black, text-white, text-gray-500, text-blue-600: અક્ષરોનો રંગ બદલવા.
  - \* uppercase, lowercase, capitalize: કેપિટલાઇઝેશન કંટ્રોલ.
  - \* tracking-tight, tracking-wide: બે અક્ષરો વચ્ચેની સ્પેસ (Letter spacing).
  - \* leading-none, leading-relaxed: બે લાઇન વચ્ચેનું અંતર (Line height).

#### 6. BACKGROUNDS, BORDERS, & EFFECTS (બેકગ્રાઉન્ડ, બોર્ડર અને ખાસ ઇફેક્ટ્સ)

- Background Utilities:
  - \* bg-transparent: બેકગ્રાઉન્ડ પારદર્શક કરવા.
  - \* bg-black, bg-white, bg-red-500, bg-indigo-900: બેકગ્રાઉન્ડના અણગણિત શેડ્સ કલર આપવા.

- \* bg-gradient-to-r, bg-gradient-to-b: બે અથવા વધુ કલરના મિશ્રણ વાળા ગ્રેડિયન્ટ આપવા.
- \* from-blue-500, to-purple-600: ગ્રેડિયન્ટ ક્યાંથી શરૂ થઈને ક્યાં પૂરો થશે તે રંગ નક્કી કરવા.
- Border Utilities:
  - \* border: બેઝિક ૧ પિક્સલની બોર્ડર આપશે.
  - \* border-0, border-2, border-4, border-8: બોર્ડરની જાડાઈ સેટ કરવા.
  - \* border-solid, border-dashed, border-dotted: બોર્ડરની ડિઝાઇન સ્ટાઇલ.
  - \* border-gray-200, border-blue-500: બોર્ડરનો કલર.
  - \* rounded-none, rounded-sm, rounded, rounded-lg, rounded-xl, rounded-3xl: ખૂણાઓને સહેજથી લઈને વધારે ગોળ કરવા.
  - \* rounded-full: ચારેય ખૂણા સાવ ગોળ કરીને અંડાકાર કે વર્તુળ (Circle) શેપ બનાવવા.
- Effects (છાયા અને ઓપેસિટી):
  - \* shadow-sm, shadow, shadow-md, shadow-lg, shadow-xl, shadow-2xl, shadow-none: ઓબ્જેક્ટની પાછળ ઢળવાથી લઈને બહુ ઘટ્ટ પડછાયો (3d Depth Effect) આપવા.
  - \* opacity-0, opacity-25, opacity-50, opacity-75, opacity-100: એલિમેન્ટની વિઝિબિલિટી પારદર્શકતા (Transparency) નક્કી કરવા.

## 7. TABLES, FILTERS, & TRANSITIONS (ટેબલ, ફિલ્ટર્સ અને એનિમેશન)

- Tables Properties:
  - \* border-collapse: ટેબલની બધી બોર્ડર્સને એકબીજામાં ઓગાળીને સિંગલ લાઇન બનાવવી.
  - \* table-auto: અંદરના ડેટાના કદ પ્રમાણે કોલમની પહોળાઈ આપોઆપ સેટ થશે.
  - \* table-fixed: અંદર ગમે એટલો મોટો ડેટા હોય, પણ કોલમની સાઇઝ કડક રીતે એક્સરખી જ રહેશે.
- Filters & Backdrop (ઇમેજ એડિટિંગ અને ગ્લાસ ઇફેક્ટ્સ):
  - \* blur-sm, blur, blur-lg: ફોટોને કે બેકગ્રાઉન્ડને ઝાંખું (Blur) કરવા.
  - \* grayscale: ફોટોને સાવ બ્લેક એન્ડ વ્હાઇટ (કાળો-ધોળો) કરવા.
  - \* brightness-50, brightness-100, brightness-150: ફોટોનો પ્રકાશ વધારવા કે ઘટાડવા.
  - \* hue-rotate-90, hue-rotate-180: ફોટોના અસલી કલર ટોનને આખા બદલી નાખવા.
  - \* backdrop-blur-sm, backdrop-blur-md, backdrop-blur-lg: આધુનિક Glassmorphism ડિઝાઇન (કાચ જેવી ઇફેક્ટ જેની પાછળનું બધું બ્લર દેખાય) બનાવવા માટે સૌથી પોપ્યુલર યુટિલિટી છે.
- Transitions & Animation (એનિમેશન કંટ્રોલ):
  - \* transition-all: કલર કે સાઇઝ બદલાતી વખતે ઝટકા સાથે બદલાવવાને બદલે સ્મૂથ એનિમેશન આપશે.
  - \* duration-75, duration-300, duration-700: એનિમેશન પૂરું થવામાં કેટલી મિલીસેકન્ડ લાગશે તે સ્પીડ નક્કી કરવા.
  - \* ease-linear, ease-in, ease-out, ease-in-out: એનિમેશનની ગતિની ગણતરી (પહેલા ધીમું પછી ફાસ્ટ, વગેરે).
  - \* animate-spin: લૉડર આઇકનને ચક્રોળની જેમ ગોળ-ગોળ ફેરવવા.
  - \* animate-ping: નોટિફિકેશન ડોટને રડારની જેમ ઝબકાવવા.
  - \* animate-pulse: આખા બોક્સને ધીમે ધીમે ઓગળતું અને બહાર આવતું (Skeleton Loading) લુક આપવા.
  - \* animate-bounce: એલિમેન્ટને દડાની જેમ ઉપર-નીચે કુદાવવા (Attention seeker).

## TAILWIND CSS COMPLETE MASTER GUIDE (A TO Z) - TAILWINDTEXT.COM CLONE

### 1. INTRODUCTION, HISTORY & FOUNDER (ટેલવિન્ડ સીએસએસ શું છે, કોણે અને કેમ બનાવ્યું?)

- શેના માટે છે?: વેબ ડેવલપમેન્ટમાં કસ્ટમ અને આધુનિક યુઝર ઇન્ટરફેસ (UI) ને અલગથી CSS ફાઇલ લખ્યા વગર, અતિશય ઝડપથી અને પૂરી ફ્લેક્સિબિલિટી સાથે ડિઝાઇન કરવા માટે ટેલવિન્ડ સીએસએસનો ઉપયોગ થાય છે.
- શું કામ કરે છે?: તે એક "Utility-First" CSS ફ્રેમવર્ક છે. આનો અર્થ એ છે કે બુટસ્ટ્રેપની જેમ આમાં તમને કોઈ બનેલા બનાવેલા `.btn` કે `.card` જેવા કમ્પોનન્ટ ક્લાસ નથી મળતા. તેના બદલે, નાના-નાના સિંગલ-પર્પઝ યુટિલિટી ક્લાસીસ (જેમ કે `pt-4` પેડિંગ માટે, `text-center` ટેક્સ્ટ માટે, `bg-blue-500` કલર માટે) મળે છે, જેને સીધા HTML ટેગમાં ભેગા કરીને તમે તમારી મરજી મુજબની કોઈપણ ડિઝાઇન તૈયાર કરી શકો છો.
- ઇતિહાસ અને શોધક (History & Founder):
  - \* ટેલવિન્ડ સીએસએસના મુખ્ય સર્જક Adam Wathan (એડમ વાથન) છે.
  - \* એડમે તેમના પોતાના સાઇડ પ્રોજેક્ટ્સ પર કામ કરતી વખતે અનુભવ્યું કે દર વખતે વારંવાર કસ્ટમ CSS લખવું અથવા બુટસ્ટ્રેપની ડિફોલ્ટ ડિઝાઇનને ઓવરરાઇટ કરવી ખૂબ કંટાળાજનક અને અઘરી હતી. આ સમસ્યાના કાયમી ઉકેલ માટે તેમણે વર્ષ 2017 માં ટેલવિન્ડ સીએસએસ ઓપન-સોર્સ પ્રોજેક્ટ તરીકે લોન્ચ કર્યો.
- વર્ઝનની ઇતિહાસ (Versions & Evolution):
  - \* Tailwind v1.0 (2019): આ વર્ઝન સાથે ફ્રેમવર્ક સ્ટેબલ થયું અને ડેવલપર્સમાં પ્રખ્યાત થયું.
  - \* Tailwind v2.0 (2020): ડાર્ક મોડ (Dark Mode) નો સપોર્ટ, નવો કલર પેલેટ અને પ્લગઇન્સ ઇન્ટિગ્રેશન વધારવામાં આવ્યું.
  - \* Tailwind v3.0 (2021): સૌથી ક્રાંતિકારી વર્ઝન. આમાં "Just-In-Time" (JIT) એન્જિન બાય-ડિફોલ્ટ એક્ટિવેટ કરવામાં આવ્યું. આ એન્જિન તમે HTML માં જે ક્લાસ લખો, ફક્ત અને ફક્ત તેટલા જ ક્લાસની સીએસએસ રિયલ-ટાઇમમાં જનરેટ કરે છે, જેના કારણે ફાઇલનું સીએસએસ ફાઇલની સાઇઝ માત્ર થોડાક કિલોબાઇટ્સ (KB) ની જ બને છે.
  - \* Tailwind v4.0 (2025/2026): નેક્સ્ટ જનરેશન પર્ફોર્મન્સ, સુપર-ફાસ્ટ કમ્પાઇલેશન (Rust એન્જિન આધારિત) અને સાદી CSS કોન્ફિગરેશન સાથેનું અત્યંત આધુનિક વર્ઝન.

### 2. HOW IT IS DIFFERENT FROM BOOTSTRAP (બુટસ્ટ્રેપથી આ કઈ રીતે અલગ છે?)

વેબ ડિઝાઇનિંગની દુનિયામાં બુટસ્ટ્રેપ અને ટેલવિન્ડ બંને રાજા છે, પણ બંનેની ફિલસૂફી તદ્દન અલગ છે:

કન્સેપ્ટ / વિશેષતા (ટેલવિન્ડ)	Bootstrap (બુટસ્ટ્રેપ)	Tailwind CSS
અભિગમ (Approach) (યુટિલિટી ફર્સ્ટ)	Component-Driven (કમ્પોનન્ટ આધારિત)	Utility-First
ડિઝાઇન ફ્લેક્સિબિલિટી તમે ધારો તેવી કસ્ટમ ડિઝાઇન.	સીમિત છે. બધી વેબસાઇટ્સ એકસરખી લાગે છે.	અમર્યાદિત છે.
CSS ફાઇલની સાઇઝ (JIT એન્જિન માત્ર વપરાયેલા ક્લાસ લે છે).	મોટી (બધા જ રેડીમેડ ક્લાસ લોડ થાય છે).	અતિશય નાની
HTML નો લુક ટેગમાં ૧૦-૧૫ યુટિલિટી ક્લાસ).	સાફ-સુથરો (માત્ર ૧ કે ૨ ક્લાસના નામ).	ભરેલો (એક જ
લર્નિંગ કર્વ જ્ઞાન હોવું જરૂરી છે.	ખૂબ જ સરળ (ક્લાસ ગોખી શકાય).	સામાન્ય CSS નું

### 3. CORE CONCEPTS (ટેલવિન્ડના પાયાના સિદ્ધાંતો)

- Utility-First Workflow: સીએસએસ ફાઇલમાં ક્લાસના નવા નામો (જેમ કે `.user-profile-card`) વિચારીને લખવાને બદલે સીધા જ HTML માં બિલ્ટ-ઇન ક્લાસ વાપરીને ડિઝાઇન સેટ કરવી.
- Responsive Design (રિસ્પોન્સિવનેસ): ટેલવિન્ડમાં કોઈપણ ક્લાસની આગળ બ્રેકપોઇન્ટ લગાવી દેવાથી તે ચોક્કસ સ્ક્રીન પર જ એક્ટિવેટ થાય છે.
  - \* `sm:` (મોબાઇલ/ટેબ્લેટ - 640px)
  - \* `md:` (મોટા ટેબ્લેટ - 768px)
  - \* `lg:` (લેપટોપ - 1024px)

- \* `x1:` (મોટા મોનિટર - 1280px)
- \* `2x1:` (સુપર વાઇડ સ્ક્રીન - 1536px)
- \* ઇ.ત. : `w-full md:w-1/2` એટલે મોબાઇલમાં 100% પહોળાઈ પણ ટેબ્લેટ કે કોમ્પ્યુટર આવતા જ પેજ આપોઆપ અડધું (50%) થઈ જશે.
- Hover, Focus, & Other States: માઉસ ઓવર અથવા ઇનપુટ સિલેક્શન વખતે ઇફેક્ટ બદલવા માટે કન્ડીશનલ પ્રિફિક્સ વપરાય છે.
  - \* `hover:` (માઉસ ઉપર જાય ત્યારે) -> `hover:bg-black`
  - \* `focus:` (ઇનપુટ બોક્સ પર ક્લિક થાય ત્યારે) -> `focus:border-blue-500`
  - \* `dark:` (જ્યારે યુઝરના ફોન કે સિસ્ટમમાં ડાર્ક મોડ ઓન હોય) -> `dark:bg-slate-900`
- Arbitrary Values (કસ્ટમ વેલ્યુઝ): જો ટેલિવિઝના ડિફોલ્ટ બોક્સમાં તમને જોઈતી ચોક્કસ સાઇઝ ન મળે, તો તમે સ્ક્રેવર બ્રેકેટ `[ ]` વાપરીને કસ્ટમ સીએસએસ પ્રોપર્ટી સીધી પાસ કરી શકો છો.
  - \* ઇ.ત. : `top-[117px]` અથવા `bg-[#fff5733]` (કસ્ટમ હેક્સ કલર કોડ માટે).

#### 4. LAYOUT PROPERTIES & CLASSES (વેબપેજનું માળખું ગોઠવવા માટે)

- Aspect Ratio (દેખાવનું પ્રમાણ):
  - \* `aspect-auto` (ઓટોમેટિક રેશિયો)
  - \* `aspect-square` (ચોરસ બોક્સ - 1:1)
  - \* `aspect-video` (વીડિયો સ્ક્રીન જેવો રેશિયો - 16:9)
- Container (કન્ટેનર):
  - \* `container` (સ્ક્રીન સાઇઝ પ્રમાણે ફિક્સ વિડ્થ સેટ કરે છે. આને સેન્ટરમાં લાવવા `mx-auto` સાથે વાપરવું પડે છે).
- Display (ડિસ્પ્લે ટાઇપ):
  - \* `block` (આખી લાઇન રોકી લે છે)
  - \* `inline-block` (લાઇનમાં પણ રહે છે અને વિડ્થ-હાઇટ પણ સેટ કરવા દે છે)
  - \* `flex` (ફ્લેક્સબોક્સ મોડ ચાલુ કરે છે)
  - \* `grid` (ગ્રીડ લેઆઉટ મોડ એક્ટિવેટ કરે છે)
  - \* `hidden` (એલિમેન્ટને પેજ પરથી સાવ ગાયબ/છુપાવી દે છે - `display: none`)
- Flexbox (ફ્લેક્સબોક્સ કંટ્રોલ):
  - \* `flex-row` (આઇટમ્સ આડી લાઇનમાં ગોઠવાશે)
  - \* `flex-col` (આઇટમ્સ ઉભી લાઇનમાં સ્ટેનની જેમ ગોઠવાશે)
  - \* `flex-wrap` (જગ્યા ન હોય તો ચાઇલ્ડ આઇટમ ઓટોમેટિક નીચેની લાઇનમાં જતી રહેશે)
  - \* `justify-start` / `justify-center` / `justify-between` (આડી ધરી પર આઇટમ્સને ડાબે, વચ્ચે કે આજુબાજુ જગ્યા છોડીને સેટ કરવા)
  - \* `align-items` માટે: `items-start` / `items-center` / `items-end` (ઉભી ધરી પર એલિમેન્ટ્સને બરાબર સેન્ટર કે ટોપ-બોટમ ગોઠવવા)
- Grid (ગ્રીડ સિસ્ટમ):
  - \* `grid-cols-1` થી `grid-cols-12` (પેજને ઉભા ૧ થી ૧૨ ખાનામાં વહેંચવા)
  - \* `gap-4` (ગ્રીડના ખાનાઓ વચ્ચે અંદરની જગ્યા/સ્પેસ સેટ કરવા)
- Floats & Clear:
  - \* `float-start` / `float-end` (ઓબ્જેક્ટને ડાબી કે જમણી બાજુ વહેવડાવવા)
  - \* `clear-both` (ફ્લોટ થયેલી આઇટમ્સની નીચેની લાઇન ક્લિયર કરવા)
- Object Fit:
  - \* `object-cover` (ઇમેજનો રેશિયો બગાડ્યા વગર આખા બોક્સમાં ફોટો બરાબર ફીટ કરે છે)
  - \* `object-contain` (આખો ફોટો દેખાય તે રીતે બોક્સની અંદર સેટ થાય છે)
- Overflow:
  - \* `overflow-hidden` (બોક્સની બહાર જતો વધારાનો ડેટા કાપી નાખે છે)
  - \* `overflow-y-scroll` (જો ડેટા ઉભી સાઇઝમાં મોટો થાય તો સ્ક્રોલ બાર આપે છે)
- Position (પોઝિશનિંગ):
  - \* `static` (નોર્મલ ફ્લો)
  - \* `relative` (પોતાની મૂળ જગ્યાના સંદર્ભમાં)
  - \* `absolute` (પેરન્ટ ઓબ્જેક્ટના ખૂણાના આધારે ગમે ત્યાં ગોઠવવા)
  - \* `fixed` (બ્રાઉઝર વિન્ડો પર ચોટાડી દેવા)
  - \* `sticky` (સ્ક્રોલ કરતા કરતા એક જગ્યાએ આવીને લોક થઈ જવા)
- Top / Right / Bottom / Left:
  - \* `top-0`, `right-4`, `bottom-10`, `left-1/2` (જગ્યા સેટ કરવા)
- Z-Index (ઉપર-નીચેના લેયર્સ):
  - \* `z-0` થી લઈને `z-50` (કઈ વસ્તુ કયા લેયરની ઉપર દેખાશે તે ક્રમ નક્કી કરવા)

#### 5. TABLES PROPERTIES & CLASSES (ટેબલ ડેટા સેટ કરવા માટે)

- Border Collapse:
  - \* `border-collapse` (ટેબલની બધી બોર્ડર્સને ભેગી કરીને સિંગલ લાઇન બનાવે છે)
  - \* `border-separate` (દરેક ખાનાની બોર્ડર અલગ રાખે છે)
- Table Layout:
  - \* `table-auto` (અંદર રહેલા અક્ષરોના કદ પ્રમાણે કોલમની પહોળાઈ ઓટોમેટિક નક્કી થાય છે)
  - \* `table-fixed` (અક્ષરો ગમે એટલા મોટા હોય, કોલમની પહોળાઈ કડક રીતે એક્સરખી જ રહે છે)
- Caption Sizing:
  - \* `caption-top` / `caption-bottom` (ટેબલના ટાઇટલ/કેપ્શન લખાણને ટેબલની ઉપર કે નીચે ગોઠવવા)

## 6. FILTERS PROPERTIES & CLASSES (ફોટા અને બોક્સ પર ગ્રાફિકલ ઇફેક્ટ્સ આપવા)

- Blur (ઝાંખું કરવું):
  - \* `blur-none` (ચોખ્ખું દેખાશે)
  - \* `blur-sm` / `blur` / `blur-md` / `blur-lg` / `blur-xl` (ઓબ્જેક્ટને બેકગ્રાઉન્ડમાં ધૂંધળો કરવા)
- Brightness (પ્રકાશ વધારવો-ઘટાડવો):
  - \* `brightness-50` (અંધારું કરવા)
  - \* `brightness-100` (નોર્મલ)
  - \* `brightness-150` (લાઈટ વધારવા)
- Contrast (રંગોની તીવ્રતા):
  - \* `contrast-50` / `contrast-125` / `contrast-200`
- Drop Shadow (વાસ્તવિક પડછાયો):
  - \* `drop-shadow-sm` / `drop-shadow-md` / `drop-shadow-lg` / `drop-shadow-xl` (પીએનજી ફોટાના કટિંગ આકાર પ્રમાણે જ પડછાયો આપે છે)
- Grayscale (બ્લેક એન્ડ વ્હાઇટ ઇફેક્ટ):
  - \* `grayscale-0` (રંગીન ફોટો)
  - \* `grayscale` (આખો ફોટો કાળો-ધોળો/જુના જમાના જેવો કરવા)
- Hue Rotate (કલર બદલવા):
  - \* `hue-rotate-15` / `hue-rotate-90` / `hue-rotate-180` (ફોટાના મૂળ રંગોને આખા બદલીને નવો શેડ આપવા)
- Invert (રંગો ઉલટાવવા - નેગેટિવ ઇફેક્ટ):
  - \* `invert-0` / `invert`
- Saturate (રંગોની ગીચતા):
  - \* `saturate-50` / `saturate-100` / `saturate-200` (રંગોને વધુ લાઈવ કે ફિક્કા કરવા)
- Sepia (બ્રાઉન ટિન્ટ ઇફેક્ટ):
  - \* `sepia-0` / `sepia` (ફોટાને જૂની ડાયરી જેવો બ્રાઉન લુક આપવા)
- Backdrop Filter (કાય જેવી પારદર્શક ઇફેક્ટ - Glassmorphism):
  - \* `backdrop-blur-md` (જ્યારે આ ક્લાસ કોઈ અર્ધ-પારદર્શક બોક્સની પાછળ લાગે, ત્યારે તેની પાછળની આખી સ્ક્રીન કાયની પાછળ હોય તેવી સુંદર ધૂંધળી દેખાય છે)

## 7. EFFECTS PROPERTIES & CLASSES (શેડો અને ઓપેસિટી કંટ્રોલ)

- Box Shadow (બોક્સનો પડછાયો):
  - \* `shadow-none` (કોઈ પડછાયો નહિ)
  - \* `shadow-sm` (ખૂબ હળવો દેખાવ)
  - \* `shadow` (નોર્મલ શેડો)
  - \* `shadow-md` / `shadow-lg` / `shadow-xl` / `shadow-2xl` (બોક્સ ઉંચું હવામાં તરતું હોય તેવો ભવ્ય 3D દેખાવ આપવા)
  - \* `shadow-inner` (બોક્સની અંદરની તરફ ખાડો હોય તેવો પડછાયો આપે છે)
- Opacity (પારદર્શકતા):
  - \* `opacity-0` (સાવ ગાયબ/અદ્રશ્ય પણ જગ્યા રોકશે)
  - \* `opacity-25` / `opacity-50` / `opacity-75` (ઝાંખું બતાવવા)
  - \* `opacity-100` (૧૦૦% ઘટ્ટ અને ચોખ્ખું દેખાશે)
- Mix Blend Mode & Background Blend Mode:
  - \* `blend-multiply` / `blend-screen` / `blend-overlay` (જ્યારે બે લેયર કે ફોટા એકબીજાની ઉપર આવે ત્યારે ફોટોશોપ સોફ્ટવેરની જેમ કલર્સ મિક્સ કરવા માટે આ પ્રોપર્ટીઝ વપરાય છે).



## TAILWIND CSS MA CUSTOM CLASSES ANE CUSTOM PROPERTIES (V4+ UPDATED)

Tailwind CSS ma potana kustom klasiyo (Custom Classes) athva kustom propurtiyo (Custom Properties / Design Tokens) kevi rite banavavi teni purn mahiti saral Gujarati ma niche mujab che.

### 1. CUSTOM DESIGN TOKENS / PROPERTIES (નવી કસ્ટમ વેલ્યુ ઉમેરવી)

- Shena mate che?: Jyare tamare Tailwind na in-built colos, spacing ke fonts sivay potani brand na khas colos ke unique size vaprvi hoy tyare.
- Shu kam kare che?: Aa lakhvathi tamare sakhsiyat mujab na nava klasiyo jema ke `bg-brand-blue` ke `text-hero-size` automatic bani jay che.

👉 Tailwind CSS v4+ ma configuration CSS file ma `@theme` direktiv (Directive) thi thay che:

```

```css
@import "tailwindcss";

@theme {
  /* Nava kustom colos banavava mate */
  --color-brand-blue: #0052cc;
  --color-brand-accent: #ff9900;

  /* Nava kustom spacing / padding / margin mate */
  --spacing-128: 32rem;

  /* Nava kustom font size mate */
  --text-hero: 4.5rem;
}
```

```

HTML ma teno upyog kevi rite karvo?:

```

```html
<!-- Automatic bani gayela nava utility classes -->
<div class="bg-brand-blue text-hero p-128">
  <h1 class="text-brand-accent">Tailwind Custom Theme!</h1>
</div>
```

```

### 2. INLINE CUSTOM VALUES (ARBITRARY VALUES - સ્કેર બ્રેકેટ રીત)

- Shena mate che?: Jyare tamare aakha project mate nahi parantu pure-pura page ma ek j jagyae koi ek ekdam vicitra ke sakhsiyat valo data ( jema ke `top-[13px]` ) vaparvo hoy tyare.
- Shu kam kare che?: Configuration file bgaadya vagar direct HTML ni andar j square brackets `[...]` no upyog karine inline kustom value muki sakay che.

👉 Udayaranon (Examples):

- Kustom Width: `w-[423px]` -> `width: 423px;`
- Kustom Background Color: `bg-[#ff007f]` -> `background-color: #ff007f;`
- Kustom Position: `top-[17.5rem]` -> `top: 17.5rem;`
- Kustom Grid Layout: `grid-cols-[100px\_1fr\_200px]`

HTML ma teno upyog kevi rite karvo?:

```

```html
<div class="w-[450px] bg-[#1a1a1a] text-[22px] p-[3.5rem]">
  Arbitrary values no upyog thai gayo!
</div>
```

```

### 3. CUSTOM UTILITY CLASSES (કસ્ટમ સીએસએસ ક્લાસ બનાવવા)

- Shena mate che?: Jyare tamare Tailwind na utilities thi bani shake teva khas CSS niyam lakhi ne potano svatantra klas banavvo hoy tyare.
- Shu kam kare che?: Tailwind ni CSS file ma sado CSS code lakhi ne kustom klas banavi sakay che,

ane temoni andar pan tailwind na modifiers jema ke `hover:`, `md:` automatic kam kare che.

👉 CSS File ma lakhvani rit:

```
```css
@import "tailwindcss";

/* Tamaro potano kustom utility class */
.btn-premium {
  background-image: linear-gradient(to right, #667eea, #764ba2);
  color: white;
  border-radius: 9999px;
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.3);
}
```
```

HTML ma teno upyog kevi rite karvo?:

```
```html
<!-- Kustom class ni sathe tailwind modifier pan kam karse -->
<button class="btn-premium px-6 py-3 transition-transform hover:scale-105">
  Premium Button
</button>
```
```

#### ----- 4. @apply DIRECTIVE NO UPYOG (ଏક્ટ્ર ԄԿԻԻ ԿԵԻԻ ԼԵԴԻ ԿԵԴԻԻ) -----

- Shena mate che?: Jyare HTML ma ek j sarkha bhu badha utility classes (jema ke `px-4 py-2 bg-blue-500 text-white rounded`) varamvar repeat thta hoy ane HTML no code bhu moto dekhato hoy tyare.  
- Shu kam kare che?: Tailwind na bhu badha utilities ne pakdi ne ek sato kustom component class banavi de che.

👉 CSS File ma `@apply` no upyog:

```
```css
@import "tailwindcss";

.card-box {
  @apply bg-white p-6 rounded-xl shadow-md border border-gray-200;
}

.card-title {
  @apply text-xl font-bold text-gray-800 mb-2;
}
```
```

HTML ma teno upyog kevi rite karvo?:

```
```html
<!-- HTML code ekdam clean thai gayo -->
<div class="card-box">
  <h2 class="card-title">Kustom Component Class</h2>
  <p class="text-gray-600">@apply no upyog karine banavelo code.</p>
</div>
```
```

=====

=====

WEB DEVELOPMENT SYLLABUS  
BEGINNER TO ADVANCED IN GUJARATI

=====

-----

🎨 1. HTML – Beginner to Advanced

-----

🟢 Beginner Level

1. HTML Introduction (HTML પરિચય)

- Definition: HTML નું પૂરું નામ HyperText Markup Language છે. તે પ્રોગ્રામિંગ લેંગ્વેજ નથી પરંતુ એક માર્કઅપ લેંગ્વેજ છે, જેનો ઉપયોગ વેબપેજનું મૂળભૂત માળખું (ફ્રેમવર્ક) તૈયાર કરવા માટે થાય છે.
- Syntax: <tagname> કન્ટેન્ટ અહીં આવશે... </tagname>
- Example: <p>આ એક પેરેગ્રાફ છે.</p>

2. HTML Document Structure (HTML દસ્તાવેજનું માળખું)

- Definition: બ્રાઉઝરને સમજાય તે રીતે HTML કોડ લખવા માટેનું પ્રમાણભૂત માળખું. આ માળખામાં મુખ્યત્વે <html>, <head> અને <body> જેવા પાયાના ટેગ્સનો સમાવેશ થાય છે.

- Example:

```
<!DOCTYPE html>
<html>
 <head>
 <title>પેજનું નામ</title>
 </head>
 <body>
 <h1>માહિતી અહીં લખાશે</h1>
 </body>
</html>
```

3. <!DOCTYPE html>

- Definition: આ કોઈ HTML ટેગ નથી, પણ બ્રાઉઝર માટેની એક સૂચના (Declaration) છે જે જણાવે છે કે આ દસ્તાવેજ HTML5 વર્ઝનમાં લખાયેલો છે.
- Syntax: <!DOCTYPE html>

4. <html>

- Definition: આને HTML દસ્તાવેજનો Root Element કહેવાય છે. વેબપેજનો તમામ કોડ આ ટેગની અંદર જ લખવામાં આવે છે.
- Attributes: lang (ભાષા નક્કી કરવા માટે, જેમ કે lang="gu" એટલે કે ગુજરાતી).
- Syntax: <html lang="en"> ... કોડ ... </html>

5. <head>

- Definition: આ ટેગની અંદર વેબપેજની ટેકનિકલ માહિતી (Metadata), પેજનું ટાઇટલ, અને CSS કે JavaScript ફાઇલોની લિંક મૂકવામાં આવે છે. આ ભાગ વેબપેજ પર યુઝરને સીધો દેખાતો નથી.
- Syntax: <head> ... ગુપ્ત માહિતીઓ ... </head>

6. <body>

- Definition: વેબપેજનો સૌથી મુખ્ય ભાગ. યુઝરને બ્રાઉઝરની સ્ક્રીન પર જે કંઈ પણ દેખાય છે (જેમ કે ફોટો, લખાણ, બટન્સ, વીડિયો), તે બધું જ આ ટેગની અંદર લખાય છે.
- Syntax: <body> ... મુખ્ય કન્ટેન્ટ ... </body>

7. Headings – h1 to h6

- Definition: વેબપેજમાં અલગ અલગ સાઇઝના મથાળા (Headings) આપવા માટે આ ટેગ વપરાય છે. <h1> સૌથી મોટું અને મહત્વનું મથાળું છે, જ્યારે <h6> સૌથી નાનું છે.
- Selectors/Properties: બ્રાઉઝર દ્વારા બાય-ડિફોલ્ટ ઘાટા (Bold) અને ચોક્કસ ફોન્ટ સાઇઝ સાથે પ્રદર્શિત થાય છે.
- Syntax: <h1>તમારું હેડિંગ</h1>
- Example:
 

```
<h1>આ મુખ્ય મથાળું છે</h1>
<h2>આ ગૌણ મથાળું છે</h2>
```

8. Paragraph – p

- Definition: વેબપેજ પર કોઈ સામાન્ય લખાણ કે ફકરો (Paragraph) દર્શાવવા માટે આ ટેગનો ઉપયોગ થાય છે. આ એક બ્લોક-લેવલ એલિમેન્ટ છે જે નવી લાઇનથી શરૂ થાય છે.

- Syntax: <p>તમારો ફકરો અહીં લખો</p>
- Example: <p>વેબ ડેવલપમેન્ટ શીખવું ખૂબ જ સરળ અને રસપ્રદ છે.</p>

### 9. Text Formatting

- Definition: લખાણને આકર્ષક બનાવવા કે ચોક્કસ અર્થ આપવા માટે વપરાતા ટેગ્સ.
- Tags & Meanings:
  - \* <b> અને <strong>: લખાણને ઘાટું (Bold) કરવા. <strong> મહત્વ દર્શાવે છે.
  - \* <i> અને <em>: લખાણને ત્રાંસું (Italic) કરવા.
  - \* <u>: લખાણની નીચે અંડરલાઇન કરવા.
- Example: આ વાક્યમાં એક શબ્દ <b>ઘાટો</b> છે અને એક <i>ત્રાંસો</i> છે.

### 10. Comments

- Definition: કોડની અંદર પોતાની સમજણ માટે લખાતી શોર્ટ નોટ્સ. આ લખાણ બ્રાઉઝરની સ્ક્રીન પર દેખાતું નથી અને કમ્પાઇલર તેને ઇગ્નોર કરે છે.
- Syntax: <!-- તમારી કમેન્ટ અહીં લખો -->
- Example:
 

```
<!-- આ બટન ફોર્મ સબમિટ કરવા માટે છે -->
<button>Submit</button>
```

### 11. Links – a

- Definition: એક વેબપેજ પરથી બીજા વેબપેજ પર જવા માટે એન્કર ટેગ <a> વપરાય છે, જેને આપણે હાઇપરલિંક કહીએ છીએ.
- Attributes:
  - \* href: જે પેજ પર જવું હોય તેનું URL એડ્રેસ.
  - \* target: જો લિંકને નવા ટેબમાં ખોલવી હોય તો target="\_blank" લખાય છે.
- Syntax: <a href="URL"> લિંકનું નામ </a>
- Example: <a href="https://www.google.com" target="\_blank">ગૂગલ ઓપન કરો</a>

### 12. Images – img

- Definition: વેબપેજ પર ફોટોગ્રાફ કે ઇમેજ મૂકવા માટે આ ટેગ વપરાય છે. આ એક Empty Tag (સેલ્ફ-ક્લોઝિંગ) છે, એટલે કે તેને બંધ કરવા માટે </img> લખવાની જરૂર નથી.
- Attributes:
  - \* src: ઇમેજ ફાઇલનો રસ્તો (Path) કે URL.
  - \* alt: જો ઇન્ટરનેટ ધીમું હોય અને ફોટો ન ખુલે, તો તેના બદલે દેખાતું લખાણ.
  - \* width / height: ફોટોની પહોળાઈ અને ઊંચાઈ સેટ કરવા.
- Syntax: 
- Example: 

### 13. Lists (યાદી)

- Definition: કોઈ પણ વસ્તુ કે મુદ્દાઓને લિસ્ટ સ્વરૂપે ગોઠવવા માટે HTML માં ત્રણ પ્રકારના લિસ્ટ ઉપલબ્ધ છે.
- Types & Syntax:
  - \* Ordered List (<ol>): ક્રમબદ્ધ યાદી (1, 2, 3 અથવા A, B, C). દરેક આઇટમ <li> (List Item) માં લખાય છે.
  - \* Unordered List (<ul>): બુલેટ પોઇન્ટ્સ વાળી યાદી.
  - \* Description List (<dl>): શબ્દ અને તેની વ્યાખ્યા દર્શાવવા માટે. જેમાં <dt> (Term) અને <dd> (Definition) વપરાય છે.
- Example:
 

```

 HTML
 CSS

```

### 14. Tables (કોષ્ટક)

- Definition: માહિતીને આડી લાઈન (Rows) અને ઊભી લાઈન (Columns) ના ટેબલ સ્વરૂપમાં વ્યવસ્થિત ગોઠવવા માટે આ ટેગ સમૂહ વપરાય છે.
- Tags:
  - \* <table>: મુખ્ય ટેબલ કન્ટેનર.
  - \* <tr>: Table Row (આડી લાઈન).
  - \* <th>: Table Header (મુખ્ય મથાળાની કોલમ - બોલ્ડ દેખાય છે).
  - \* <td>: Table Data (સામાન્ય ખાનાની માહિતી).
- Example:
 

```
<table>
 <tr>
 <th>વિષય</th>
```

```

<th>મીક્સ</th>
</tr>
<tr>
<td>HTML</td>
<td>95</td>
</tr>
</table>

```

### 15. Forms (ફોર્મ)

- Definition: યુઝર પાસેથી કોઈપણ માહિતી (જેમ કે નામ, ઇમેલ, પાસવર્ડ) એકઠી કરીને સર્વર પર મોકલવા માટે ફોર્મ ટેગનો ઉપયોગ થાય છે.
- Attributes: action (ડેટા ક્યા પેજ પર મોકલવો છે), method (GET કે POST).
- Syntax: <form action="submit.php" method="POST"> ... ફોર્મના એલિમેન્ટ્સ ... </form>

### 16. Input Types

- Definition: ફોર્મની અંદર યુઝર અલગ-અલગ રીતે ડેટા એન્ટર કરી શકે તે માટે <input> ટેગ વપરાય છે.
- Attributes:
  - \* type: ઇનપુટનો પ્રકાર નક્કી કરે છે (જેમ કે text, password, email, checkbox, radio, submit).
  - \* placeholder: બોક્સ ખાલી હોય ત્યારે અંદર દેખાતું હળવું લખાણ જે યુઝરને ગાઈડ કરે છે.
- Example:
 

```

<input type="text" placeholder="તમારું આખું નામ લખો">
<input type="password" placeholder="પાસવર્ડ">

```

### 17. Buttons

- Definition: કોઈ એક્શન પરફોર્મ કરવા કે ફોર્મ સબમિટ કરવા માટે ક્લિકેબલ બટન બનાવવા માટે વપરાય છે.
- Attributes: type="submit" (ફોર્મ સબમિટ કરવા), type="button" (સામાન્ય એક્શન માટે).
- Syntax: <button type="submit"> ક્લિક કરો </button>

### 18. Labels

- Definition: ફોર્મના ઇનપુટ બોક્સ કઈ માહિતી માટે છે તેનું નામ આપવા માટે <label> વપરાય છે.
- Attributes: for (આમાં ઇનપુટ બોક્સની id લખવામાં આવે છે, જેથી લેબલ પર ક્લિક કરવાથી પણ ઇનપુટ બોક્સ સિલેક્ટ થઈ જાય).
- Example:
 

```

<label for="user-email">ઈમેલ આઈડી:</label>
<input type="email" id="user-email">

```

### 19. textarea

- Definition: જો યુઝર પાસેથી એક કરતાં વધુ લાઇનનું લાંબુ લખાણ મેળવવું હોય (જેમ કે એડ્રેસ કે ફીડબેક મેસેજ), ત્યારે આ ટેગ વપરાય છે.
- Attributes: rows (કેટલી લાઇન ઊંચી રાખવી), cols (કેટલી પહોળાઈ રાખવી).
- Example: <textarea rows="4" cols="50" placeholder="તમારો સંદેશ અહીં લખો..."></textarea>

### 20. select / option

- Definition: યુઝરને ઘણા બધા વિકલ્પોમાંથી કોઈ એક વિકલ્પ પસંદ કરવા માટે ડ્રોપડાઉન (Dropdown List) મેનુ બનાવવા આ ટેગ વપરાય છે.
- Syntax:
 

```

<select>
 <option value="surat">સુરત</option>
 <option value="ahmedabad">અમદાવાદ</option>
</select>

```

### 21. fieldset / legend

- Definition: ફોર્મની અંદર સંબંધિત ઇનપુટ બોક્સને એક બોક્સ બોર્ડર આપીને અલગ જૂથ (Group) માં વહેંચવા માટે <fieldset> વપરાય છે, અને તે જૂથનું નામ આપવા માટે <legend> ટેગ વપરાય છે.
- Example:
 

```

<fieldset>
 <legend>લોગિન વિગત</legend>
 <input type="text" placeholder="યુઝરનેમ">
</fieldset>

```

## ● Intermediate Level

### 22. div and span

- Definition: આ બંને નોન-સેમેન્ટિક (કોઈ ખાસ અર્થ વગરના) કન્ટેનર ટેગ છે, જેનો ઉપયોગ CSS વડે સ્ટાઇલિંગ કરવા કે લેઆઉટ બનાવવા માટે થાય છે.

- Difference:

- \* **div (Division):** આ એક Block-level એલિમેન્ટ છે. તે સ્ક્રીનની આખી આડી લાઇન રોકી લે છે અને હંમેશા નવી લાઇનથી શરૂ થાય છે.

- \* **span:** આ એક Inline એલિમેન્ટ છે. તે માત્ર એટલી જ જગ્યા રોકે છે જેટલું તેની અંદર લખાણ હોય. તે નવી લાઇન લેતું નથી.

- Example:

```
<div>આ એક આખો બ્લોક છે.</div>
```

```
<p>આ ફક્તમાં એક શબ્દ લાલ કલરનો છે.</p>
```

## 23. Semantic HTML

- Definition: એવા ટેગ્સ જે માત્ર વેબપેજ બનાવતા નથી, પરંતુ બ્રાઉઝર અને સર્ચ એન્જિન (SEO) ને એ પણ સમજાવે છે કે વેબસાઇટનો કયો ભાગ કઈ માહિતી ધરાવે છે.

- Core Tags: <header>, <nav>, <main>, <section>, <article>, <aside>, <footer>

## 24. Audio & Video

- Definition: વેબસાઇટ પર થર્ડ-પાર્ટી પ્લેગઇન વગર સીધા જ ઓડિયો (મ્યુઝિક) કે વીડિયો પ્લે કરવા માટે HTML5 ના આ ટેગ્સ વપરાય છે.

- Attributes: controls (પ્લે, પોઝ અને વોલ્યુમના બટન બતાવવા), autoplay (ઓટોમેટિક પ્લે કરવા), loop (વારંવાર ચલાવવા).

- Example:

```
<video src="sample.mp4" controls width="400"></video>
```

```
<audio src="song.mp3" controls></audio>
```

## 25. Iframe

- Definition: એક ચાલુ વેબપેજની અંદર આખે આખું બીજું વેબપેજ કે ગૂગલ મેપ અથવા યુટ્યુબ વીડિયો એમ્બેડ (દર્શાવવા) કરવા માટે વપરાય છે.

- Attributes: src (જે વેબસાઇટ બતાવવી હોય તેનું એડ્રેસ), frameborder (બોર્ડર હટાવવા).

- Example: <iframe src="https://www.wikipedia.org" width="100%" height="300"></iframe>

## 26. HTML Entities

- Definition: HTML કોડિંગમાં અમુક ખાસ ચિહ્નો (જેમ કે <, >, કે વધારે સ્પેસ) સીધા ટાઇપ કરવાથી કોડિંગ બગડી શકે છે. તેથી તેને દર્શાવવા માટે ચોક્કસ કોડ વપરાય છે જેને Entities કહે છે.

- Common Entities:

- \* &lt; : Less than (<) માટે

- \* &gt; : Greater than (>) માટે

- \* &amp; : Ampersand (&) માટે

- \* &nbsp; : Non-breaking space (એક વધારાની ખાલી જગ્યા છોડવા માટે)

- Example: જો તમારે સ્ક્રીન પર <html> લખવું હોય તો: &lt;html&gt; લખવું પડે.

## 27. Meta Tags

- Definition: આ ટેગ્સ <head> ની અંદર લખાય છે જે સર્ચ એન્જિન (ગૂગલ) ને વેબસાઇટ વિશે માહિતી આપે છે કે આ સાઇટ શેના વિશે છે અને તેના કીવર્ડ્સ કયા છે.

- Attributes: name, content, charset="UTF-8" (બધી ભાષાઓના કેરેક્ટર સપોર્ટ કરવા માટે).

- Example: <meta name="description" content="સરળ ગુજરાતીમાં વેબ ડેવલપમેન્ટ કોર્સ">

## 28. Favicon

- Definition: જ્યારે તમે બ્રાઉઝરમાં કોઈ સાઇટ ઓપન કરો, ત્યારે ટેબ ઉપર વેબસાઇટના નામની ડાબી બાજુએ જે નાનો લોગો દેખાય છે, તેને ફેવિકોન કહેવાય છે.

- Syntax: <link rel="icon" type="image/x-icon" href="favicon.ico">

## 29. Form Validation

- Definition: યુઝર ફોર્મ સબમિટ કરે ત્યારે તે ખોટો ડેટા ન નાખે કે બોક્સ ખાલી ન છોડે તે બ્રાઉઝર લેવલ પર જ ચેક કરવાની રીત.

- Attributes:

- \* required: આ ઇનપુટ બોક્સ ખાલી રાખી શકાશે નહીં.

- \* min / max: સંખ્યા કે તારીખની લઘુત્તમ કે મહત્તમ મર્યાદા સેટ કરવા.

- \* pattern: રેગ્યુલર એક્સપ્રેશન (Regex) નો ઉપયોગ કરી ચોક્કસ ફોર્મેટ (જેમ કે ફોન નંબરના ૧૦ આંકડા જ હોવા જોઈએ) સેટ કરવા.

- Example:

```
<input type="text" required placeholder="આ બોક્સ ભરવું જ પડશે">
```

```
<input type="number" min="18" max="60">
```

## 30. Accessibility Basics & SEO-Friendly HTML

- Definition: Accessibility (a11y) એટલે વેબસાઇટને એવી રીતે ડિઝાઇન કરવી કે દિવ્યાંગ લોકો પણ સરળતાથી

સમજી શકે અને SEO-Friendly એટલે સાચા સેમેન્ટિક ટેગ વાપરવા જેથી ગૂગલ સર્ચમાં વેબસાઇટ ટોપ પર આવે.

## ● Advanced Level

### 31. Advanced Forms & HTML5 APIs

- Definition: આધુનિક વેબ ફીચર્સ ડાયરેક્ટ બ્રાઉઝરમાં વાપરવા માટે.
- Sub-topics: Local Storage, Session Storage (બ્રાઉઝરમાં ડેટા સાચવવા), Geolocation (યુઝરનું લોકેશન જાણવા), Drag and Drop.

### 32. Canvas & SVG

- Definition: વેબપેજ પર ગ્રાફિક્સ, એનિમેશન અને ડ્રોઇંગ બનાવવા માટે આ ટેગ્સ વપરાય છે. Canvas જાવાસ્ક્રિપ્ટથી પિક્સલ ડ્રોઇંગ કરે છે જ્યારે SVG ગમે તેટલી મોટી કરો તો પણ ફાટતી નથી.

### 33. Web Accessibility & Semantic SEO

- Definition: રોબોટ્સ અને સ્ક્રીન રીડર્સ માટે વેબસાઇટને વધુ અનુકૂળ બનાવવા aria-\* એટ્રિબ્યુટ્સનો ઉપયોગ કરવો.

### 34. Performance-Friendly & Responsive HTML Structure

- Definition: કોડને ઓપ્ટિમાઇઝ કરવો જેથી વેબસાઇટ ફાસ્ટ લોડ થાય અને મોબાઇલ કે ડેસ્કટોપ બંને સ્ક્રીન સાઇઝ પર આપોઆપ ગોઠવાઈ જાય.

## 🔧 2. CSS – Beginner to Advanced

## ● Beginner Level

### 1. CSS Introduction (CSS પરિચય)

- Definition: CSS નું પૂરું નામ Cascading Style Sheets છે. તેનો ઉપયોગ HTML વડે બનાવેલા વેબપેજની ડિઝાઇન, કલર, લેઆઉટ અને દેખાવને આકર્ષક બનાવવા માટે થાય છે.
- Syntax: selector { property: value; }
- Example: p { color: blue; }

### 2. Inline CSS

- Definition: જ્યારે કોઈ ચોક્કસ HTML ટેગની અંદર જ style એટ્રિબ્યુટનો ઉપયોગ કરીને ડિઝાઇન આપવામાં આવે, ત્યારે તેને ઇનલાઇન CSS કહેવાય છે.
- Syntax: <tagname style="property: value;"> લખાણ </tagname>
- Example: <h1 style="color: red; font-size: 20px;">આ લાલ હેડિંગ છે.</h1>

### 3. Internal CSS

- Definition: 輸入 HTML ડોક્યુમેન્ટના <head> ભાગની અંદર <style> ટેગ બનાવીને આખા પેજ માટેની ડિઝાઇન ત્યાં જ લખવામાં આવે, તેને ઇન્ટરનલ CSS કહે છે.
- Syntax: <head> <style> selector { property: value; } </style> </head>
- Example:

```
<head>
 <style>
 body { background-color: lightgray; }
 h2 { color: green; }
 </style>
</head>
```

### 4. External CSS

- Definition: ડિઝાઇન માટે એક અલગ જ ફાઇલ (જેમ કે style.css) બનાવવામાં આવે છે અને તેને HTML પેજ સાથે <link> ટેગ દ્વારા જોડવામાં આવે છે. આ સૌથી બેસ્ટ રીત છે.
- Syntax: <link rel="stylesheet" href="filename.css">
- Example: <link rel="stylesheet" href="mystyle.css">

### 5. CSS Syntax

- Definition: CSS નો કોડ લખવાના નિયમો. આમાં મુખ્ય ત્રણ ભાગ હોય છે: Selector (કોને ડિઝાઇન કરવી છે), Property (શું બદલવું છે જેમ કે કલર) અને Value (કેવો કલર આપવો છે).

### 6. Element Selector

- Definition: આ સિલેક્ટર સીધા HTML ના ટેગના નામ (જેમ કે h1, p, div) નો ઉપયોગ કરીને તે પ્રકારના બધા જ એલિમેન્ટ્સને એકસાથે સ્ટાઇલ આપે છે.
- Syntax: tagname { property: value; }
- Example: p { text-align: center; }

## 7. Class Selector

- Definition: HTML ના કોઈ પણ ટેગમાં આપેલા કસ્ટમ ક્લાસના નામ પરથી સ્ટાઇલ આપવા આ સિલેક્ટર વપરાય છે. CSS માં તેની આગળ ડોટ (.) લગાવવામાં આવે છે. એક ક્લાસ એકથી વધુ ટેગમાં વાપરી શકાય છે.
- Syntax: `.classname { property: value; }`
- Example: `.card-box { background-color: white; }`

## 8. ID Selector

- Definition: કોઈ એક ચોક્કસ યુનિક HTML એલિમેન્ટને સ્ટાઇલ આપવા માટે વપરાય છે. CSS માં તેની આગળ હેશ (#) ચિહ્ન લગાવવામાં આવે છે. આખા પેજ પર એક ID એક જ વાર આપી શકાય.
- Syntax: `#idname { property: value; }`
- Example: `#main-btn { background-color: blue; }`

## 9. Universal Selector

- Definition: આખા વેબપેજના તમામ HTML એલિમેન્ટ્સને એકસાથે સિલેક્ટ કરવા માટે સ્ટાર (\*) ચિહ્નનો ઉપયોગ થાય છે. સામાન્ય રીતે આખા પેજની ડિફોલ્ટ માર્જિન-પેડિંગ હટાવવા માટે આ વપરાય છે.
- Syntax: `* { property: value; }`
- Example: `* { margin: 0; padding: 0; }`

## 10. Group Selector

- Definition: જ્યારે ઘણા બધા અલગ-અલગ સિલેક્ટર્સને એકસરખી જ ડિઝાઇન આપવાની હોય, ત્યારે કોડ ટૂંકો કરવા માટે બધાને અલ્પવિરામ (,) થી અલગ કરીને સાથે લખી શકાય છે.
- Syntax: `selector1, selector2 { property: value; }`
- Example: `h1, h2, p { color: darkblue; }`

## 11. Colors & Background

- Definition: વેબપેજમાં લખાણનો કલર કે બેકગ્રાઉન્ડ સેટ કરવા માટેની પ્રોપર્ટીઝ.
- Properties: `color` (ટેક્સ્ટ કલર), `background-color` (પાછળનો કલર), `background-image` (બેકગ્રાઉન્ડ ફોટો).

## 12. Text Properties

- Definition: લખાણનું અવાઈનમેન્ટ, સજાવટ કે અક્ષરો વચ્ચેની જગ્યા કન્ટ્રોલ કરવા માટેની પ્રોપર્ટીઝ.
- Properties: `text-align` (left, right, center, justify), `text-decoration` (none, underline), `text-transform` (uppercase, lowercase).

## 13. Font Properties

- Definition: લખાણના અક્ષરોના મરોડ (Fonts), તેની સાઇઝ અને જાડાઈ બદલવા માટેની પ્રોપર્ટીઝ.
- Properties: `font-family` (ફોન્ટ સ્ટાઇલ), `font-size` (અક્ષરોની સાઇઝ), `font-weight` (bold, normal).

## 14. Width & Height

- Definition: કોઈપણ બોક્સ કે એલિમેન્ટની પહોળાઈ (width) અને ઊંચાઈ (Height) નક્કી કરવા માટે આ પ્રોપર્ટીઝ વપરાય છે.

## 15. Margin & Padding

- Definition: Margin એટલે બોક્સની બહારની તરફ છોડાતી ખાલી જગ્યા અને Padding એટલે બોક્સની અંદર કન્ટેન્ટ અને બોર્ડર વચ્ચે છોડાતી ખાલી જગ્યા.
- Properties: `margin-top`, `padding-left` વગેરે.

## 16. Border & Border Radius

- Definition: બોક્સની ચારેય બાજુ કિનારી (બોર્ડર) બનાવવા અને તેના ખૂણાઓને ગોળ કરવા માટે આ પ્રોપર્ટીઝ વપરાય છે.
- Example: `border: 2px solid red; border-radius: 8px;`

## 17. Box Model & box-sizing

- Definition: Box Model મુજબ દરેક એલિમેન્ટ Content, Padding, Border, અને Margin નો બનેલો છે. `box-sizing: border-box;` કરવાથી પેડિંગ ઉમેરવા છતાં બોક્સની કુલ સાઇઝ બદલાતી નથી.

## 18. Display Property

- Definition: એલિમેન્ટ સ્ક્રીન પર કઈ રીતે પ્રદર્શિત થશે તે નક્કી કરે છે.
- Values: `block` (આખી લાઇન રોકે), `inline` (જેટલી જરૂર હોય તેટલી જ જગ્યા લે), `inline-block` (લાઇનમાં રહે પણ width/height સેટ કરી શકાય), `none` (છુપાવી દે).

## 19. Overflow

- Definition: જ્યારે બોક્સની અંદરનું લખાણ બોક્સની સાઇઝ કરતાં મોટું થઈ જાય ત્યારે તેને કેવી રીતે હેન્ડલ કરવું (hidden, scroll, auto).

## ● Intermediate Level

**20. Position & Z-index**

- Definition: Position જગ્યા નક્કી કરે છે. static (ડિફોલ્ટ), relative (પોતાની જગ્યાથી ખસવા), absolute (પેરન્ટના આધારે ગોઠવવા), fixed (સ્ક્રીન પર એક જગ્યાએ ચોટી જાય), sticky (સ્ક્રીલ કરતા ચોટી જાય). z-index લેયર નક્કી કરે છે કે કયું બોક્સ ઉપર દેખાશે.

**21. Flexbox (ખૂબ જ મહત્વનું)**

- Definition: લેઆઉટને એક જ લાઇન (આડી અથવા ઊભી) માં સરળતાથી અને રિસ્પોન્સિવ રીતે ગોઠવવા માટે display: flex; વપરાય છે.

- Properties: flex-direction (row/column), justify-content (આડી સ્પેસિંગ), align-items (ઊભી સ્પેસિંગ), gap, flex-wrap.

- Example (Featured 3-Price Box layout):

```
.pricing-container { display: flex; justify-content: center; align-items: center; gap: 20px; }
.price-box.featured { transform: scale(1.1); }
```

**22. CSS Grid**

- Definition: રો અને કોલમ બંને દિશામાં જટિલ લેઆઉટ એકસાથે બનાવવા display: grid; વપરાય છે.

- Properties: grid-template-columns, grid-template-rows, gap.

**23. CSS Units & Pseudo Classes**

- Units: px, %, em, rem, vh, vw (સ્ક્રીન સાઇઝ આધારિત).

- Pseudo Classes: એલિમેન્ટની ખાસ સ્થિતિ દર્શાવે જેમ કે :hover (માઉસ ઉપર જાય ત્યારે), :focus, :active, :nth-child().

**24. Pseudo Elements, Shadow & Effects**

- Pseudo Elements: ::before અને ::after (કોડથી આગળ-પાછળ કંઈક ઉમેરવા).

- Effects: box-shadow (બોક્સ શેડો), text-shadow, opacity (પારદર્શકતા), cursor: pointer, transition (સ્મૂથ બદલાવ માટે).

**● Advanced Level****25. Transform, Animations & Media Queries**

- Transform: rotate, scale, translate.

- Animations: @keyframes દ્વારા કસ્ટમ એનિમેશન બનાવવું.

- Media Queries: Responsive વેબ ડિઝાઇન માટે જેમ કે @media (max-width: 768px) { ... } (મોબાઇલ માટે લેઆઉટ બદલવા).

**26. Modern CSS Functions & Variables**

- Functions: calc(), min(), max(), clamp().

- Variables: --main-color: #3498db; જેવા વેરીએબલ્સ બનાવી વાપરવા.

**⚡ 3. JavaScript – Beginner to Advanced****● Beginner Level****1. JavaScript Introduction & Adding to HTML**

- Definition: JS એ વેબપેજને જીવંત અને ઇન્ટરેક્ટિવ બનાવતી પ્રોગ્રામિંગ લેંગ્વેજ છે. તેને <script> ટેગ અથવા બાહ્ય .js ફાઇલ દ્વારા ઉમેરી શકાય છે.

**2. Variables (વેરીએબલ્સ)**

- Definition: ડેટા સ્ટોર કરવા માટેના કન્ટેનર. var (જૂનું, ફંક્શન સ્કોપ), let (નવું, બ્લોક સ્કોપ), const (બદલી ન શકાય તેવી કિંમત).

**3. Data Types & Operators**

- Data Types: String (લખાણ), Number (અંકડા), Boolean (true/false), Undefined, Null, BigInt, Symbol.

- Operators: Arithmetic (+, -, \*, /), Comparison (==, ===), Logical (&&, ||), Assignment (=), Ternary (? :).

**4. Conditionals & Loops**

- Conditionals: નિર્ણય લેવા માટે (if, if...else, else if, switch).

- Loops: વારંવાર કોડ રન કરવા (for, while, do...while). break લૂપ બંધ કરે છે અને continue એક સ્ટેપ સ્કીપ કરે છે.

## ● Intermediate Level

### 5. Functions & Scope

- Definition: વારંવાર વાપરી શકાય તેવો કોડ બ્લોક. Arrow Functions લખવાની ટૂંકી રીત છે (const myFunc = () => { ... }). Scope વેરીએબલ ક્યાંથી એક્સેસ થશે તે નક્કી કરે છે (Global vs Local).

### 6. Arrays & Important Methods

- Array: વિસ્તરેલ સ્વરૂપે ડેટા સાચવવો.  
- Basic Methods: push(), pop(), shift(), unshift(), slice(), splice(), includes(), indexOf().  
- Iteration Methods: map() (નવી એરે બનાવે), filter() (શરત મુજબ ગાળાણ), find(), findIndex(), reduce() (સિંગલ વેલ્યુ રિટર્ન), forEach().

### 7. Objects & Modern ES6+ Features

- Objects: કી-વેલ્યુ જોડીમાં જટિલ ડેટા સાચવવો.  
- ES6+ Features: Destructuring (ડેટા છૂટો પાડવો), Spread & Rest Operators (...), Template Literals (`\${var}`), try...catch (Error Handling).

## 🌐 DOM (Document Object Model) – Very Important

### 8. DOM Selector & Manipulation

- Definition: HTML પેજને જાવાસ્ક્રિપ્ટથી બદલવાની ઇન-બ્રિલ્ટ સિસ્ટમ.  
- Selectors: getElementById(), querySelector(), querySelectorAll().  
- Manipulation: innerHTML, innerText, textContent, style.color, classList.add(), classList.remove(), createElement(), appendChild(), remove().

### 9. Event Handling

- Definition: યુઝરની એક્શન (ક્લિક, ટાઇપ) પર કોડ રન કરવો.  
- Events: click, input, change, submit, mouseover.  
- Concepts: Event Object, Event Bubbling, Event Delegation.

## ● Advanced JavaScript

### 10. Advanced Scope, Context & OOP

- Concepts: Callbacks, Higher-Order Functions, Closures, Hoisting, Execution Context, this કીવર્ડ, call(), apply(), bind().  
- OOP: Constructor Functions, Prototypes, Classes, Inheritance, Encapsulation.

### 11. Asynchronous JS & APIs

- Tools: Promises (.then(), .catch()), async / await (સરળ અસિંક્રોનસ કોડ), Fetch API / REST API દ્વારા સર્વરથી ડેટા મેળવવો (GET, POST, PUT, DELETE મેથડ્સ).  
- Extras: Debouncing & Throttling (પરફોર્મન્સ), Local/Session Storage, Regex.

## 📦 4. Bootstrap – Beginner to Advanced

## ● Beginner Level

### 1. Bootstrap Intro, Installation & Containers

- Definition: આ રેડીમેડ CSS ક્લાસ ફ્રેમવર્ક છે. .container (વચ્ચે ફિક્સ જગ્યા) અને .container-fluid (100% પહોળાઈ) કન્ટેનર પ્રોવાઇડ કરે છે.

### 2. Grid System & Components

- Grid: row અને col-\* નો ઉપયોગ કરી ૧૨ ભાગમાં સ્ક્રીન વહેંચવી (sm, md, lg બ્રેકપોઇન્ટ્સ સાથે).  
- Basic UI: Typography, Colors (.text-primary, .bg-dark), Buttons (.btn-success), Alerts, Badges, Cards, Tables.

## ● Intermediate Level

### 3. Intermediate Components

- Components: Navbar (રિસ્પોન્સિવ મેનુ), Forms (.form-control), Input Groups, Dropdowns, Modal (પોપ-અપ વિન્ડો), Carousel (સ્લાઇડર), Accordion (FAQs માટે કોલેપ્સ વિસ્ત), Pagination, Progress Bar, Spinner.

## ● Advanced Level

### 4. Utilities & Advanced Layouts

- Utilities: Flex utilities (.d-flex, .justify-content-center), Spacing (.mt-3, .p-4), Display, Position, Sizing (.w-100).
- Application: આ બધા ક્લાસ ભેગા કરી Responsive Dashboard, E-commerce UI અને Admin Panel UI કસ્ટમાઇઝેશન સાથે બિલ્ડ કરવું.

## 5. Tailwind CSS – Beginner to Advanced

### ● Beginner Level

#### 1. Tailwind Intro, CDN & Utility-First Concept

- Definition: આ એક Utility-First ફ્રેમવર્ક છે જેમાં બનેલા કમ્પોનન્ટ નથી હોતા, પણ નાના ક્લાસ (bg-red-500, p-4) જોડીને કસ્ટમ ડિઝાઇન કરાય છે.

#### 2. Basic Utilities

- Classes: bg-blue-500, text-white, font-bold, text-lg, w-full, h-32, m-4, p-2, border, rounded-lg, shadow-md, display, flex.

### ● Intermediate Level

#### 3. Layout & States

- Classes: flex-col, justify-between, items-center, gap-4, grid, grid-cols-3, relative, absolute, z-50.
- Breakpoints & States: md:grid-cols-3 (રિસ્પોન્સિવ), hover:bg-blue-600, focus:, active:, group-hover:, transition, transform, animate-spin.
- UI Blocks: આ ક્લાસ વાપરીને કસ્ટમ Navbar, Cards, Modal, Dropdown બિલ્ડ કરવા.

### ● Advanced Level

#### 4. Configuration & Custom Architecture

- Customization: tailwind.config.js ફાઇલમાં કસ્ટમ કલર્સ, ફોન્ટ્સ અને થીમ સેટ કરવી.
- Advanced Features: Dark mode (dark:bg-gray-900), Reusable Components (@apply ઢૂલો), Tailwind Optimization, Framework integration (React / Next.js સાથે).

## BOOTSTRAP COMPLETE ARCHITECTURE & REFERENCE GUIDE

### 1. ★ BOOTSTRAP SHU CHE? (What is Bootstrap?)

- શેના માટે છે?: વેબ ડેવલપમેન્ટમાં વેબસાઈટની ડિઝાઇન એટલે કે યુઝર ઇન્ટરફેસ (UI) બનાવવાની પ્રક્રિયાને અત્યંત ઝડપી, સરળ અને મોબાઈલ-ફ્રેન્ડલી (Responsive) બનાવવા માટે આ ફ્રેમવર્ક વપરાય છે.
- શું કરે છે?: આ એક ઓપન-સોર્સ HTML, CSS અને JavaScript ફ્રેમવર્ક છે. તે ડેવલપર્સને પહેલેથી લખાયેલો, ટેસ્ટ કરેલો અને રેડીમેડ ડિઝાઇન કોડ આપે છે. આના લીધે ડેવલપરે સ્કેયથી હજારો લાઇનનો CSS કોડ લખવો નથી પડતો. આ ફ્રેમવર્કની મદદથી બટનો, નેવિગેશન બાર, ફોર્મ્સ, ટેબલો અને આખું પેજ લેઆઉટ બહુ ઓછા સમયમાં સુંદર અને મોડર્ન લુક સાથે તૈયાર થઈ જાય છે.

### 2. ★ HISTORY, FOUNDERS & KEM GOTAYU? (History & Motivation)

- કોણે શોધ્યું? (Founders): બુટસ્ટ્રેપની શોધ Twitter (હવે X) કંપનીમાં કામ કરતા બે સોફ્ટવેર એન્જિનિયર્સ Mark Otto (@mdo) અને Jacob Thornton (@fat) દ્વારા કરવામાં આવી હતી.
- કેમ શોધાયું? (Motivation & Need): Twitter કંપનીમાં જ્યારે અલગ-અલગ ડેવલપર્સ ઇન્ટરનલ ટૂલ્સ અને પ્રોજેક્ટ્સ પર કામ કરતા હતા, ત્યારે દરેક ડેવલપર પોતાની મરજી મુજબ અલગ-અલગ CSS લખતો હતો. આના કારણે વેબસાઈટના અલગ-અલગ પેજમાં ડિઝાઇનની સુસંગતતા (Consistency) જળવાતી નહોતી અને મેન્ટેનન્સ બહુ અઘરું થઈ ગયું હતું. આ મોટી આંતરિક સમસ્યાને ઉકેલવા માટે, માર્ક અને જેકોબે એક એવું કોમન ટૂલ બનાવવાનું વિચાર્યું જે આખી કંપનીના ડેવલપર્સ વાપરી શકે. શરૂઆતમાં તેનું નામ "Twitter Blueprint" રાખવામાં આવ્યું હતું. ત્યારબાદ ઓગસ્ટ 2011 માં તેને ઓપન-સોર્સ (જાહેર જનતા માટે ફ્રી) તરીકે લોન્ચ કરવામાં આવ્યું અને નામ બદલીને "Bootstrap" રાખવામાં આવ્યું.

### 3. ★ VERSION HISTORY (Evolution of Bootstrap)

- Bootstrap v1 (2011): સૌથી પહેલું વર્ઝન, જે ફક્ત Twitter ના આંતરિક ટૂલ્સને સુધારવા અને બેઝિક ગ્રીડ આપવા માટે લોન્ચ થયું હતું.
- Bootstrap v2 (2012): આ વર્ઝનમાં સૌથી મોટો ફેરફાર આવ્યો - 12-column નું responsive layout સિસ્ટમ લાવવામાં આવી, જે મોબાઈલ સ્ક્રીન પ્રમાણે ઓટોમેટિક એડજસ્ટ થતી હતી.
- Bootstrap v3 (2013): આ વર્ઝન "Mobile-First" એપ્રોચ સાથે આવ્યું. એટલે કે પહેલા મોબાઈલ સ્ક્રીન માટે ડિઝાઇન બને અને પછી ડેસ્કટોપ માટે મોટી થાય. આ વર્ઝન ઇન્સ્ટ્રીમાં સુપરહિટ રહ્યું.
- Bootstrap v4 (2018): આ વર્ઝનમાં જૂની ફ્લોટ (Float) આધારિત ગ્રીડ સિસ્ટમને સંપૂર્ણપણે હટાવીને આધુનિક Flexbox સિસ્ટમ અપનાવવામાં આવી, જેનાથી અલાઈનમેન્ટ બહુ સરળ બન્યું. સાથોસાથ Sass (CSS preprocessor) નો સપોર્ટ શરૂ થયો.
- Bootstrap v5 (2021 થી વર્તમાન સમય સુધી): આ અત્યારનું સૌથી મોડર્ન અને ફાસ્ટ વર્ઝન છે.
  - \* jQuery મુક્તિ: સૌથી મોટો ફેરફાર એ થયો કે તેમાંથી jQuery ની ડિપેન્ડન્સી સાવ હટાવી દેવામાં આવી અને તેની જગ્યાએ પ્યોર વેનીલા જાવાસ્ક્રિપ્ટ (Vanilla JS) નો ઉપયોગ શરૂ થયો, જેનાથી વેબસાઈટની સ્પીડ સુપરફાસ્ટ થઈ ગઈ.
  - \* Utility API અને કસ્ટમ CSS Variables (Variables) નો પાવરફૂલ સપોર્ટ ઉમેરવામાં આવ્યો.
  - \* ઇન્ટરનેટ એક્સપ્લોરર (IE 10/11) જેવા જૂના બ્રાઉઝર્સનો સપોર્ટ સિક્વોરિટી અને સ્પીડના કારણે બંધ કરી દેવાયો.

### 4. ★ COLUMN & GRID STRUCTURE (Deep Dive)

- શેના માટે છે?: વેબસાઈટના પેજને અલગ-અલગ ડિવાઈસ (મોબાઈલ, ટેબ્લેટ, લેપટોપ, મોનિટર) ની સ્ક્રીનની સાઈઝ પ્રમાણે ઓટોમેટિક ગોઠવવા અને વ્યવસ્થિત માળખું આપવા માટે ગ્રીડ સિસ્ટમ વપરાય છે.
- શું કરે છે?: બુટસ્ટ્રેપની ગ્રીડ સિસ્ટમ Flexbox પર આધારિત છે અને તે આખા પેજની પહોળાઈને આડી લાઈનમાં કુલ 12 કાલ્ક્યુલેશન લાગ એટલે કે 12 Columns માં વહેંચે છે.
- મુખ્ય 3 પિલર્સ (The 3 Pillars of Grid):
  1. Container (કન્ટેનર): આ આખી ગ્રીડ સિસ્ટમનું સૌથી બહારનું મુખ્ય બોક્સ છે. પેજની પ્રોપર વિડ્થ (width) અને આજુબાજુથી પેડિંગ સેટ કરવાનું કામ આ કરે છે.
  2. Row (રો/લાઈન): કન્ટેનરની અંદર હંમેશા Row આવે છે. આ એક આડી લાઈન બનાવે છે જેની અંદર જ તમે કોલમ મૂકી શકો છો. તે કોલમ્સ માટે રેપર (Wrapper) નું કામ કરે છે.
  3. Col (કોલમ): Row ની અંદર જ કોલમ આવે છે. આ અસલી બોક્સ છે જેની અંદર આપણો કન્ટેન્ટ (લખાણ કે ઇમેજ) મુકાય છે. કોલમના નંબરોનો સરવાળો હંમેશા એક Row માં 12 થવો જોઈએ (દા.ત. col-6 + col-6 = 12 એટલે કે અડધા-અડધા ભાગના બે બોક્સ).
- Screen Breakpoints (ડિવાઈસ પ્રમાણે ક્લાસ):

- \* Extra Small (xs): સ્ક્રીનની સાઈઝ < 576px (નાના મોબાઈલ) -> ક્લાસ ફોર્મેટ: `col-\*`
- \* Small (sm): સ્ક્રીનની સાઈઝ >= 576px (મોટા મોબાઈલ) -> ક્લાસ ફોર્મેટ: `col-sm-\*`
- \* Medium (md): સ્ક્રીનની સાઈઝ >= 768px (ટેબ્લેટ) -> ક્લાસ ફોર્મેટ: `col-md-\*`
- \* Large (lg): સ્ક્રીનની સાઈઝ >= 992px (લેપટોપ) -> ક્લાસ ફોર્મેટ: `col-lg-\*`
- \* Extra Large (xl): સ્ક્રીનની સાઈઝ >= 1200px (મોટા કમ્પ્યુટર મોનિટર) -> ક્લાસ ફોર્મેટ: `col-xl-\*`
- \* Extra Extra Large (xxl): સ્ક્રીનની સાઈઝ >= 1400px (ખૂબ મોટી સ્ક્રીન) -> ક્લાસ ફોર્મેટ: `col-xxl-\*`

## 5. ★ PREDEFINED CLASSES REFERENCE (A-Z Categorized)

બુટસ્ટ્રેપમાં પહેલેથી બનેલા હજારો ક્લાસીસ છે. અહીં તમામ મહત્વના ક્લાસીસનું કેટેગરી વાઈઝ ઊંડાણપૂર્વકનું વિસ્તર આપેલું છે:

### A. LAYOUT CLASSES (વેઆઉટ અને બોક્સ સેટિંગ્સ)

- `container` : ફિક્સ પહોળાઈ વાળું કેન્દ્રિત (Centered) બોક્સ બનાવે છે જે સ્ક્રીન પ્રમાણે સ્ટેપ બાય સ્ટેપ નાનું-મોટું થાય છે.
- `container-fluid` : સ્ક્રીનની પૂરેપૂરી પહોળાઈ (100% Width) રોકી લેતું ફ્લુ-સ્ક્રીન બોક્સ બનાવે છે.
- `row` : કોલમ્સને એક આડી લાઈનમાં રાખવા માટે ફ્લેક્સ કન્ટેનર એકિટવેટ કરે છે.
- `col-[breakpoint]-[number]` : ચોક્કસ સ્ક્રીન પર બોક્સની પહોળાઈ (1 થી 12 ની વચ્ચે) નક્કી કરે છે. (દા.ત. `col-md-4` એટલે ટેબ્લેટ કે તેથી મોટી સ્ક્રીન પર ત્રીજા ભાગનું બોક્સ).
- `col-auto` : કન્ટેન્ટની અંદર જેટલું લખાણ હશે, તેટલી જ જગ્યા આ કોલમ રોકશે, વધારાની નહીં.

### B. TYPOGRAPHY CLASSES (લખાણ અને ફોન્ટ્સ સેટિંગ્સ)

- `h1` થી `h6` : કોઈપણ સામાન્ય ટેગ (જેમ કે <p> કે <span>) ને પણ H1 થી H6 જેવા હેડિંગ્સ જેવડા મોટા અને બોલ્ડ લુકમાં ફેરવી નાખે છે.
- `display-1` થી `display-6` : સામાન્ય હેડિંગ કરતાં પણ અતિશય મોટા, પાતળા અને આકર્ષક કિંગ-સાઈઝ ફોન્ટ્સ બનાવવા માટે વપરાય છે (બેનર કે હેડર માટે).
- `lead` : પેરેગ્રાફના અક્ષરોને નોર્મલ કરતાં થોડા મોટા અને સોફ્ટ બનાવે છે જેથી લખાણ પેજ પર મુખ્ય હાઈલાઈટ જેવું દેખાય.
- `text-start` : અક્ષરોને ડાબી બાજુ (Left-align) ગોઠવે છે.
- `text-center` : અક્ષરોને સ્ક્રીનની બરોબર વચ્ચે (Center-align) ગોઠવે છે.
- `text-end` : અક્ષરોને જમણી બાજુ (Right-align) ગોઠવે છે.
- `text-uppercase` : બધા જ અક્ષરોને કેપિટલ (Capital LETTERS) માં ફેરવે છે.
- `text-lowercase` : બધા જ અક્ષરોને નાની એબીસીડી (Small letters) માં ફેરવે છે.
- `text-capitalize` : દરેક શબ્દના માત્ર પહેલા અક્ષરને કેપિટલ બનાવે છે.
- `text-muted` : અક્ષરોનો કલર આછો ગ્રે (Faded gray) કરી નાખે છે જેથી ગૌણ માહિતી અલગ તારવી શકાય.
- `font-weight-bold` અથવા `fw-bold` : ફોન્ટને એકદમ ઘાટા (Bold) કરે છે.
- `fw-normal` : અક્ષરોને સામાન્ય થીકનેસ આપે છે.
- `fw-light` : અક્ષરોને એકદમ પાતળા (Light) લુકમાં ફેરવે છે.

### C. COLOR CLASSES (બેકગ્રાઉન્ડ અને લખાણના કલર્સ)

બુટસ્ટ્રેપમાં કલરોના કસ્ટમ નામ રાખેલા છે જેને થીમ કલર્સ કહેવાય છે:

- `text-primary` / `bg-primary` : થીમનો મુખ્ય બ્લુ (વાદળી) કલર ટેક્સ્ટ / બેકગ્રાઉન્ડ પર લગાવે છે.
- `text-secondary` / `bg-secondary` : ગ્રે (રાખોડી) કલર ઇફેક્ટ આપે છે.
- `text-success` / `bg-success` : લીલો (ગ્રીન) કલર કરે છે (મોટેભાગે સક્સેસ કે સેફ મેસેજ માટે).
- `text-danger` / `bg-danger` : લાલ (રેડ) કલર કરે છે (એરર, ચેતવણી કે ડિલીટ બટન માટે).
- `text-warning` / `bg-warning` : પીળો (યલો) કલર કરે છે (અવર્ટ કે વોર્નિંગ મેસેજ માટે).
- `text-info` / `bg-info` : આછો વાદળી (સ્કાય બ્લુ) કલર કરે છે (માહિતી દર્શાવવા માટે).
- `text-light` / `bg-light` : એકદમ આછો સફેદ/ગ્રે કલર સેટ કરે છે.
- `text-dark` / `bg-dark` : એકદમ ઘાટો કાળો/ગ્રે કલર લગાવે છે.
- `text-white` / `bg-white` : ખોર સફેદ કલર આપે છે.

### D. SPACING CLASSES (માર્જિન અને પેડિંગ - અત્યંત મહત્વનું)

આ ક્લાસીસ `[property][sides]-[size]` ના ફોર્મ્યુલા પર કામ કરે છે.

પ્રોપર્ટી: `m` = Margin (બહારની જગ્યા), `p` = Padding (અંદરની જગ્યા).

સાઈડ્સ: `t` = Top, `b` = Bottom, `s` = Start (Left), `e` = End (Right), `x` = ડાબી અને જમણી બંને બાજુ, `y` = ઉપર અને નીચે બંને બાજુ.

સાઈઝ: `0` થી `5` સુધી (0 એટલે સાવ બંધ, 5 એટલે સૌથી વધારે સ્પેસ), અને `auto` (ઓટોમેટિક સેન્ટર માટે).

- `mt-3` : બોક્સની ઉપરની તરફ (Top) મીડિયમ માર્જિન સ્પેસ ઉમેરે છે.
- `mb-5` : બોક્સની નીચેની તરફ (Bottom) સૌથી મોટી માર્જિન સ્પેસ છોડે છે.
- `p-4` : બોક્સની અંદરની બાજુ ચારેય તરફ (ચોરસ) સરખી પેડિંગ જગ્યા બનાવે છે.
- `px-2` : ડાબી અને જમણી બંને બાજુ આછી પેડિંગ સ્પેસ આપે છે.
- `py-3` : ઉપર અને નીચે બંને બાજુ પ્રોપર અંદરની જગ્યા સેટ કરે છે.

- `mx-auto` : જો બોક્સની વિડ્થ ફિક્સ હોય, તો તેને સ્ક્રીનની બરોબર વચ્ચે (Horizontal Center) લાવી દે છે.

#### E. BUTTON CLASSES (બટનની અદ્યતન ડિઝાઇન)

- `btn` : બટનની બેઝિક થીમ એક્ટિવેટ કરે છે (બોર્ડર હટાવવી, કર્સર પોઇન્ટર કરવું, સ્મૂથ ટ્રાન્ઝિશન આપવું).
- `btn-primary` : સુંદર સોલિડ બ્લુ કલરનું પ્રાથમિક બટન બનાવે છે.
- `btn-success` : લીલા કલરનું સફળતા બટન સેટ કરે છે.
- `btn-danger` : લાલ કલરનું ડેન્જર બટન બનાવે છે.
- `btn-outline-primary` : બટનની અંદર કોઈ કલર નથી હોતો, ફક્ત બ્લુ બોર્ડર હોય છે. યુઝર જ્યારે તેના પર માઉસ લઈ જાય (Hover) ત્યારે જ તે આખું બ્લુ થાય છે.
- `btn-lg` : મોટા સાઈઝનું સ્પેશિયલ બટન બનાવે છે.
- `btn-sm` : નાના સાઈઝનું ક્લુટ બટન ક્રિએટ કરે છે.

#### F. FLEXBOX CLASSES (ડાયનેમિક અલાઇનમેન્ટ ટૂલ્સ)

- `d-flex` : એલિમેન્ટને Flexbox કન્ટેનરમાં ફેરવે છે, જેથી યાદબેંધ એલિમેન્ટ્સને આડા-ઊભા ગમે તેમ ગોઠવી શકાય.
- `justify-content-start` : ફ્લેક્સની બધી આઈટમ્સને લાઈનની શરૂઆતમાં (ડાબી બાજુ) ભેગી કરે છે.
- `justify-content-center` : બધી આઈટમ્સને આડી લાઈનમાં સ્ક્રીનની બરોબર વચ્ચે સેન્ટર કરે છે.
- `justify-content-end` : બધી આઈટમ્સને લાઈનના છેડે (જમણી બાજુ) ધકેલી દે છે.
- `justify-content-between` : પહેલી અને છેલ્લી આઈટમ ખૂણામાં જતી રહે છે અને બાકીની આઈટમ્સ વચ્ચે સરખી જગ્યા (Equal Space) ઓટોમેટિક સેટ થઈ જાય છે.
- `align-items-center` : ઊભી લાઈનમાં (Vertical direction) બધી આઈટમ્સને બરોબર વચ્ચે ગોઠવે છે.
- `flex-column` : આઈટમ્સને આડી રાખવાને બદલે એકની નીચે એક ઊભી લાઈન (Vertical stack) માં ગોઠવી દે છે.

#### G. COMPONENTS CLASSES (રેડીમેડ યુઝર ઇન્ટરફેસ એલિમેન્ટ્સ)

- `card` : એક મસ્ત બોર્ડર, શેડો અને વ્હાઇટ બેકગ્રાઉન્ડ વાળું પ્રોડક્ટ કે પ્રોફાઇલ બોક્સ (બોક્સ) બનાવે છે.
- `card-body` : કાર્ડની અંદર પ્રોપર પેડિંગ સ્પેસ સાથેનો કન્ટેન્ટ એરિયા નક્કી કરે છે.
- `card-title` : કાર્ડની અંદરના હેડિંગને પ્રોપર અક્ષર આપે છે.
- `navbar` : વેબસાઇટનો મેઇન નેવિગેશન હેડર બાર એક્ટિવેટ કરે છે.
- `navbar-expand-lg` : મોટી સ્ક્રીન પર આખો મેનૂ બાર ખુલ્લો બતાવશે, પણ મોબાઇલ સ્ક્રીન થતાં જ તે મેનૂ ઓટોમેટિક સંકોચાઈને ત્રણ લીટી વાળા ટોગલ બટન (Hamburger Menu) માં ફેરવાઈ જશે.
- `alert` : યુઝરને કોઈ નોટિફિકેશન આપવા માટેનું પ્લેન બોક્સ બનાવે છે.
- `alert-danger` : લાલ કલરનું અલર્ટ બોક્સ (દા.ત. "લોગિન નિષ્ફળ ગયું!").
- `badge` : ટેક્સ્ટની બાજુમાં નાનું ગોળ કલરફુલ બોક્સ બનાવે છે (દા.ત. ન્યૂ નોટિફિકેશન કાઉન્ટ બતાવવા માટે).
- `table` : સામાન્ય HTML ટેબલને એકદમ ક્લીન, મોર્ડન અને પ્રોપર સ્પેસિંગ વાળા પ્રોફેશનલ ડેટા ટેબલમાં બદલી નાખે છે.
- `table-striped` : ટેબલની આડી લાઈનોમાં એક છોડીને એક લાઈન આછા ગ્રે કલરની (Zebra crossing style) થાય છે જેથી ડેટા વાંચવામાં બહુ સરળતા રહે છે.
- `shadow` : બોક્સની પાછળ સુંદર અને સોફ્ટ પડછાયો (Drop Shadow) ઉમેરે છે, જેનાથી બોક્સ પેજ પરથી બહાર ઉપસી આવ્યું હોય તેવો પ્રીમિયમ લુક મળે છે.
- `rounded` : બોક્સના તીક્ષ્ણ ખૂણાઓને સહેજ ગોળ (Border-radius) બનાવે છે.
- `rounded-circle` : સ્ક્રેર (ચોરસ) ઇમેજ કે બોક્સને સાવ ગોળ મટોળ કુક્કડા જેવું ઇંડું અથવા પરફેક્ટ સર્કલમાં ફેરવી નાખે છે (પ્રોફાઇલ પિક્ચર માટે).

=====

# COMPLETE WEB DEVELOPMENT MASTER GUIDE: HTML, CSS & JAVASCRIPT A TO Z CONCEPT-BY-CONCEPT DETAILED GUIDE IN GUJARATI

## SECTION: HTML - Fundamentals to Advanced

### 👉 CONCEPT #1: HTML introduction

#### 🔴 શેના માટે છે અને શું કરે છે?:

HTML (HyperText Markup Language) એ વેબ પેજનું મૂળ માળખું અને સ્ટ્રક્ચર બનાવવા માટે વપરાતી સ્ટાન્ડર્ડ માર્કઅપ લેંગ્વેજ છે. તે બ્રાઉઝરને જણાવે છે કે કન્ટેન્ટ હેડિંગ, પેરેગ્રાફ કે લિંક તરીકે કેવી રીતે ડિસ્પ્લે કરવું.

### 👉 CONCEPT #2: HTML Editors

#### 🔴 શેના માટે છે અને શું કરે છે?:

HTML કોડ લખવા માટે નોટપેડ અથવા એડવાન્સ એડિટર્સ (જેમ કે VS Code કે Sublime Text) વપરાય છે. આ એડિટર્સ ઓટો-કમ્પ્લીશન અને સિન્ટેક્સ હાઇલાઇટિંગ આપીને કોડિંગ ખૂબ ઝડપી બનાવે છે.

### 👉 CONCEPT #3: HTML Elements & Tags

#### 🔴 શેના માટે છે અને શું કરે છે?:

HTML ટેગ્સ (જેમ કે `<p>`, `<h1>`) હંમેશા ઓપનિંગ અને ક્લોઝિંગ જોડીમાં આવે છે. ટેગ્સની વચ્ચે જે કન્ટેન્ટ લખાય તેને ભેગું કરીને એક આખો HTML Element કહેવાય છે જે વેબપેજનો એક બ્લોક બને છે.

### 👉 CONCEPT #4: HTML Attributes

#### 🔴 શેના માટે છે અને શું કરે છે?:

Attributes હંમેશા ઓપનિંગ ટેગની અંદર લખાય છે જે એલિમેન્ટને વધારાની માહિતી અથવા પ્રોપર્ટી પૂરી પાડે છે, જેમ કે લિંક માટે `href` અથવા ઇમેજ માટે `src`.

### 👉 CONCEPT #5: HTML Headings (h1 to h6)

#### 🔴 શેના માટે છે અને શું કરે છે?:

વેબપેજ પર હેડિંગ્સ આપવા માટે `<h1>` થી `<h6>` સુધીના ટેગ્સ વપરાય છે. `<h1>` સૌથી મોટું અને મુખ્ય હેડિંગ દર્શાવે છે, જ્યારે `<h6>` સૌથી નાનું અને ઓછી પ્રાયોરિટી ધરાવતું સબ-હેડિંગ બતાવે છે.

### 👉 CONCEPT #6: HTML Paragraphs

#### 🔴 શેના માટે છે અને શું કરે છે?:

`<p>` ટેગનો ઉપયોગ ટેક્સ્ટને પેરેગ્રાફ ફોર્મેટમાં ગોઠવવા માટે થાય છે. બ્રાઉઝર ઓટોમેટિકલી દરેક પેરેગ્રાફની ઉપર અને નીચે થોડી ખાલી જગ્યા (મર્જિન) ઉમેરી દે છે.

### 👉 CONCEPT #7: HTML Styles (Inline)

#### 🔴 શેના માટે છે અને શું કરે છે?:

Inline Style દ્વારા કોઈપણ પર્ટીક્યુલર HTML ટેગની અંદર જ `style` એટ્રિબ્યુટ વાપરીને તેનો કલર, ફોન્ટ કે સાઈઝ બદલી શકાય છે, જો કે મોટા પ્રોજેક્ટ્સમાં આ રીત યોગ્ય નથી ગણાતી.

### 👉 CONCEPT #8: HTML Text Formatting

#### 🔴 શેના માટે છે અને શું કરે છે?:

ટેક્સ્ટને આકર્ષક બનાવવા માટે `<b>` (bold), `<i>` (italic), `<strong>` (મહત્વનું), `<em>` (ભારપૂર્વકનું), `<sub>` (subscript), અને `<sup>` (superscript) જેવા ફોર્મેટિંગ ટેગ્સનો ઉપયોગ થાય છે.

## 👉 CONCEPT #9: HTML Quotations

### ✦ શેના માટે છે અને શું કરે છે?:

બીજી વેબસાઇટ પરથી લીધેલા લાંબા લખાણને દર્શાવવા માટે `<blockquote>` વપરાય છે (જે ઇન્ડેન્ટેશન ઉમેરે છે) અને ટૂંકા વાક્યોને ડબલ કોટ્સમાં બતાવવા માટે `<q>` ટેગ વપરાય છે.

## 👉 CONCEPT #10: HTML Comments

### ✦ શેના માટે છે અને શું કરે છે?:

HTML કોડમાં ડેવલપરની સમજણ માટે નોંધ લખવા `<!-- Comment -->` સિન્ટેક્સ વપરાય છે. આ કમેન્ટ્સ બ્રાઉઝરની સ્ક્રીન પર દેખાતી નથી પણ સોર્સ કોડમાં જોઈ શકાય છે.

## 👉 CONCEPT #11: HTML Colors (RGB, HEX, HSL)

### ✦ શેના માટે છે અને શું કરે છે?:

વેબપેજ પર કલર્સ સેટ કરવા માટે ક્યાં તો કલરનું નામ, અથવા RGB (Red, Green, Blue કિંમતો), HEX (#FFFFFF જેવો હેક્સાડેસિમલ કોડ), અથવા HSL (Hue, Saturation, Lightness) ફોર્મેટ વપરાય છે.

## 👉 CONCEPT #12: HTML CSS Integration

### ✦ શેના માટે છે અને શું કરે છે?:

HTML ફાઇલમાં CSS ને જોડવા માટે ત્રણ રીતો છે: Inline (ટેગની અંદર), Internal (`<head>` માં `<style>` ટેગ દ્વારા), અને External (અલગ `.css` ફાઇલ બનાવીને `<link>` ટેગ વડે જોડવી).

## 👉 CONCEPT #13: HTML Links & Hyperlinks

### ✦ શેના માટે છે અને શું કરે છે?:

`<a>` (Anchor) ટેગનો ઉપયોગ કરીને એક પેજ પરથી બીજા પેજ પર જવા માટે હાઇપરલિંક બનાવાય છે. આમાં `href` એટ્રિબ્યુટમાં ટાર્ગેટ વેબસાઇટનું URL આપવાનું હોય છે.

## 👉 CONCEPT #14: HTML Images & Map

### ✦ શેના માટે છે અને શું કરે છે?:

`<img>` ટેગ દ્વારા વેબપેજ પર ફોટા મૂકી શકાય છે જેમાં `alt` એટ્રિબ્યુટ ફોટો લોડ ન થાય તો વૈકલ્પિક ટેક્સ્ટ બતાવે છે. Image Map (`<map>`) દ્વારા એક જ ફોટાના અલગ-અલગ ભાગો પર અલગ લિંક્સ સેટ કરી શકાય છે.

## 👉 CONCEPT #15: HTML Tables

### ✦ શેના માટે છે અને શું કરે છે?:

ડેટાને રો (આડી લાઇન) અને કોલમ (ઊભી લાઇન) માં ગોઠવવા માટે `<table>` ટેગ વપરાય છે. આમાં `<tr>` (table row), `<th>` (table heading), અને `<td>` (table data) ના કોમ્બિનેશનથી ટેબલ બને છે.

## 👉 CONCEPT #16: HTML Lists (Ordered & Unordered)

### ✦ શેના માટે છે અને શું કરે છે?:

માહિતીનું વિસ્તર બનાવવા માટે `<ol>` (Ordered List - જે નંબર કે એબીસીડી ક્રમ આપે છે) અને `<ul>` (Unordered List - જે બુલેટ પોઇન્ટ્સ આપે છે) ટેગ્સ વપરાય છે. વિસ્તરની આઇટમ `<li>` ટેગમાં લખાય છે.

## 👉 CONCEPT #17: HTML Block & Inline Elements

### ✦ શેના માટે છે અને શું કરે છે?:

Block Elements (જેમ કે `<div>`, `<p>`) સ્ક્રીનની આખી પહોળાઈ રોકે છે અને નવી લાઇનથી શરૂ થાય છે. Inline Elements (જેમ કે `<span>`, `<a>`) માત્ર પોતાની સાઈઝ જેટલી જ જગ્યા રોકે છે અને નવી લાઇન લેતા નથી.

## 👉 CONCEPT #18: HTML Div & Span

✦ શેના માટે છે અને શું કરે છે?:

<div> એકઠ્ઠ વ્લોક-લેવલ કન્ટેનર છે જે મોટા સેક્શનનું ગ્રુપિંગ કરવા વપરાય છે. જ્યારે <span> એક ઇનલાઇન કન્ટેનર છે જે પેરેગ્રાફની અંદરના નાના ટેક્સ્ટ ભાગને ટાર્ગેટ કરીને સ્ટાઇલ કરવા વપરાય છે.

## 👉 CONCEPT #19: HTML Classes & Ids

✦ શેના માટે છે અને શું કરે છે?:

Class (`. `) નો ઉપયોગ એક સરખી સ્ટાઇલ મલ્ટિપલ એલિમેન્ટ્સને આપવા માટે થાય છે. જ્યારે ID (`#`) આખા પેજમાં માત્ર એક જ યુનિક એલિમેન્ટને ઓળખવા અને તેના પર જાવાસ્ક્રિપ્ટ કે સીએસએસ લગાવવા વપરાય છે.

## 👉 CONCEPT #20: HTML Iframes

✦ શેના માટે છે અને શું કરે છે?:

<iframe> (Inline Frame) નો ઉપયોગ તમારા ચાલુ વેબપેજની અંદર જ દુનિયાની બીજી કોઈ વેબસાઇટનું આખું પેજ કે યુટ્યુબ વીડિયો એક નાના બોક્સ તરીકે એમ્બેડ (પ્રદર્શિત) કરવા માટે થાય છે.

## 👉 CONCEPT #21: HTML Form Elements & Inputs

✦ શેના માટે છે અને શું કરે છે?:

<form> ટેગ યુઝર પાસેથી ડેટા કલેક્ટ કરવાનું પ્લેટફોર્મ છે. તેની અંદર <input type='text' | 'password' | 'email'>, <textarea>, અને <select> ડ્રોપડાઉન જેવા એલિમેન્ટ્સ યુઝર ઇનપુટ માટે વપરાય છે.

## 👉 CONCEPT #22: HTML5 Canvas & SVG

✦ શેના માટે છે અને શું કરે છે?:

<canvas> ટેગ જાવાસ્ક્રિપ્ટ વડે ડાયનેમિક ગેમ ગ્રાફિક્સ દોરવા વપરાય છે, જ્યારે SVG (Scalable Vector Graphics) ગણિત આધારિત વેક્ટર આકારો દોરવા માટે છે જે ક્યારેય ઝૂમ કરવા પર ફાટતા નથી.

## 👉 CONCEPT #23: HTML Media (Audio & Video)

✦ શેના માટે છે અને શું કરે છે?:

કોઈપણ પ્લગઇન વગર પેજ પર સીધા જ ઓડિયો ફાઇલો ચલાવવા માટે <audio> ટેગ અને પ્લે/પોઝ કંટ્રોલ સાથે મુવી ચલાવવા માટે <video> ટેગનો ઉપયોગ થાય છે.

## 👉 CONCEPT #24: HTML Web Storage API

✦ શેના માટે છે અને શું કરે છે?:

બ્રાઉઝરની અંદર જ યુઝરનો પ્રાઇવેટ સેશન ડેટા ઓનલાઇન કે ઓફલાઇન સાવ સેફલી લોકલી સેવ રાખવા માટે `localStorage` અને `sessionStorage` ની સુવિધા પ્રોવાઇડ કરે છે.

## ■ SECTION: CSS - Styling from Basics to Grid/Flexbox

## 👉 CONCEPT #25: CSS Introduction & Syntax

✦ શેના માટે છે અને શું કરે છે?:

CSS (Cascading Style Sheets) સીદા HTML પેજને બ્યુટીફુલ લુક, કલર, ફોન્ટ અને ડિઝાઇન આપવા વપરાય છે. તેમાં એક Selector (ટેગ પકડનાર) અને Declaration (`property: value;`) હોય છે.

## 👉 CONCEPT #26: CSS Box Model (Imp)

✦ શેના માટે છે અને શું કરે છે?:

બ્રાઉઝર દરેક એલિમેન્ટને બોક્સ ગણે છે. આ મોડેલમાં મુખ્ય ચાર સ્તરો હોય છે: અસલી કન્ટેન્ટ (Content), કન્ટેન્ટની

બહારનું પેડિંગ (Padding), બોર્ડર (Border), અને સાવ બહારની મોકળી જગ્યા માર્જિન (Margin).

#### 👉 CONCEPT #27: CSS Margins & Padding

✦ શેના માટે છે અને શું કરે છે?:

Margin એ બોર્ડરની બહાર બે અલગ-અલગ બોક્સ વચ્ચે જગ્યા જાળવવા વપરાય છે. જ્યારે Padding બોર્ડરની અંદર કન્ટેન્ટ અને બોર્ડરની દીવાલ વચ્ચે સેફ સ્પેસ રાખવા માટે વપરાય છે.

#### 👉 CONCEPT #28: CSS Display Property

✦ શેના માટે છે અને શું કરે છે?:

એલિમેન્ટનું બિહેવિયર બદલવા માટે આ વપરાય છે. `display: block;` આખી લાઇન રોકે છે, `display: inline;` ફક્ત કન્ટેન્ટ જેટલી જગ્યા રોકે છે, અને `display: none;` એલિમેન્ટને આખેઆખો ગાયબ કરે છે.

#### 👉 CONCEPT #29: CSS Position Property

✦ શેના માટે છે અને શું કરે છે?:

એલિમેન્ટને પેજ પર ક્યાં ગોઠવવો તે નક્કી કરે છે. આમાં `static` (નોર્મલ), `relative` (પોતાની જગ્યાથી ખસવા), `absolute` (પેરેન્ટ બોક્સના આધારે), અને `fixed/sticky` (સ્ક્રોલ કરવા છતાં ચોટી રહેવા) કિંમતો આવે છે.

#### 👉 CONCEPT #30: CSS Z-index & Overflow

✦ શેના માટે છે અને શું કરે છે?:

`z-index` દ્વારા ઓવરલેપ થતા એલિમેન્ટ્સમાં કયો ઉપર દેખાશે તે 3D સ્તરે નક્કી થાય છે. `overflow: hidden/scroll;` બોક્સની બહાર નીકળી જતા લખાણને કાપવા કે સ્ક્રોલબાર આપવા વપરાય છે.

#### 👉 CONCEPT #31: CSS Pseudo-classes & Elements

✦ શેના માટે છે અને શું કરે છે?:

Pseudo-classes (`:hover`) એલિમેન્ટ પર માઉસ જાય તેવી ખાસ સ્થિતિ સ્ટાઇલ કરે છે. Pseudo-elements (`::before`/`::after`) કોડિંગ દ્વારા એલિમેન્ટની આગળ કે પાછળ નવો આભાસી ડેટા ઉમેરે છે.

#### 👉 CONCEPT #32: CSS Media Queries

✦ શેના માટે છે અને શું કરે છે?:

વેબસાઇટને મોબાઇલ અને ટેબ્લેટ ફ્રેન્ડલી (Responsive) બનાવવા માટે `@media` નો ઉપયોગ કરીને અલગ-અલગ સ્ક્રીન પહોળાઈ માટે અલગ-અલગ ડિઝાઇન અને કલર સ્કેમ સેટ કરી શકાય છે.

#### 👉 CONCEPT #33: CSS Flexbox (Detailed)

✦ શેના માટે છે અને શું કરે છે?:

આ વન-ડાયમેન્શનલ (આડી અથવા ઊભી એક જ લાઇન) લેઆઉટ સિસ્ટમ છે. `display: flex;` કરવાથી યાઇલ્ડ બોક્સ આપોઆપ ગોઠવાઈ જાય છે અને તેમની વચ્ચે ગેપ કે સેન્ટર ગોઠવવું રમકડાં જેવું સરળ બને છે.

#### 👉 CONCEPT #34: CSS Grid (Detailed)

✦ શેના માટે છે અને શું કરે છે?:

આ ટુ-ડાયમેન્શનલ (રો અને કોલમ બંને એકસાથે) એડવાન્સ લેઆઉટ સિસ્ટમ છે. `display: grid;` વડે આખા પેજના જટિલ ખાના પાડીને પ્રોફેશનલ મેગેઝીન જેવું વેબ લેઆઉટ ડિઝાઇન કરી શકાય છે.

### SECTION: JavaScript - Core Fundamentals to Master Advanced

#### 👉 CONCEPT #35: JS Variables (var, let, const)

✦ શેના માટે છે અને શું કરે છે?:

મેમરીમાં ડેટા સ્ટોર કરવા માટે વપરાય છે. `var` જૂનો ફંક્શન-સ્કોપ કીવર્ડ છે. `let` નવો અને મર્યાદિત બ્લોક-સ્કોપ વેરીએબલ છે જેની કિંમત બદલી શકાય છે, અને `const` અચળ ફિક્સ વેલ્યુ સાચવે છે.

#### 👉 CONCEPT #36: JS Data Types & Null vs Undefined

✦ શેના માટે છે અને શું કરે છે?:

ડેટાના પ્રકારો જેમ કે પ્રિમિટિવ (string, number, boolean) અને રેફરન્સ (object, array). `null` જાણીજોઈને ખાલી રાખેલો ઓબ્જેક્ટ છે, જ્યારે `undefined` એટલે વેરીએબલ તો છે પણ કોઈ વેલ્યુ અપાઈ નથી.

#### 👉 CONCEPT #37: JS "use strict" & Operators

✦ શેના માટે છે અને શું કરે છે?:

`"use strict";` કોડમાં થતી નાની ભૂલોને ચલાવી લેવાના બદલે સીધી એરર આપે છે. ગણતરી માટે ઓપરેટર્સ વપરાય છે, જેમાં `===` કિંમતની સાથે ડેટાનો પ્રકાર (Type) પણ સખ્તાઈથી મેચ કરે છે.

#### 👉 CONCEPT #38: JS Control Structures & Loops

✦ શેના માટે છે અને શું કરે છે?:

`if-else` અને `switch` શરત આધારે કોડનો રસ્તો નક્કી કરે છે. `for`, `while` લૂપ્સ કોડ રીપીટ કરે છે. `for...in` ઓબ્જેક્ટની અંદર ફરે છે અને `for...of` એરેના એલિમેન્ટ્સ બહાર કાઢે છે.

#### 👉 CONCEPT #39: JS Functions & Arrow Functions

✦ શેના માટે છે અને શું કરે છે?:

કોડના રીયુઝેબલ ટુકડા છે. રેગ્યુલર ફંક્શન હોઇસ્ટ થાય છે. Arrow Function `() => {}` મોડર્ન ટૂંકી સિન્ટેક્સ આપે છે, જેનું પોતાનું કોઈ `this` હોતું નથી, તે પેરન્ટ સ્કોપનું `this` વાપરે છે.

#### 👉 CONCEPT #40: JS Closures & Scope Chain

✦ શેના માટે છે અને શું કરે છે?:

Closure એટલે અંદરનું યાઇલ્ડ ફંક્શન પેરન્ટ ફંક્શન બંધ થઈ ગયા પછી પણ તેની મેમરી અને વેરીએબલ્સ યાદ રાખે છે. Scope Chain એટલે કરન્ટ ફંક્શનમાં વેરીએબલ ન મળે તો પેરન્ટ સ્કોપમાં શોધવાની સીડી.

#### 👉 CONCEPT #41: JS Event Loop (Micro vs Macrotasks)

✦ શેના માટે છે અને શું કરે છે?:

જાવાસ્ક્રિપ્ટ બેકગ્રાઉન્ડ મલ્ટીટાસ્કિંગ આનાથી હેન્ડલ કરે છે. ઇવેન્ટ લૂપ કોલ સ્ટેક ખાલી થતાં જ પહેલા હાઇ પ્રાયોરિટી વાળા માઇક્રોટાસ્ક (Promises) ક્લિયર કરે છે અને પછી જ મેક્રોટાસ્ક (setTimeout) ઉપાડે છે.

#### 👉 CONCEPT #42: JS Memoisation & Currying

✦ શેના માટે છે અને શું કરે છે?:

Memoisation ભારે ગણતરીના રિઝલ્ટને કેશ મેમરીમાં સ્ટોર કરીને સ્પીડ વધારે છે. Currying એટલે મલ્ટીપલ પેરામીટર વાળા ફંક્શનને તોડીને સિંગલ-સિંગલ પેરામીટર વાળા ફંક્શન્સની કનેક્ટેડ ચેઇન બનાવવી.

#### 👉 CONCEPT #43: JS Debouncing & Throttling

✦ શેના માટે છે અને શું કરે છે?:

Debouncing યુઝર ટાઇપિંગ બંધ કરે તેના થોડા સમય પછી જ ફંક્શન રન કરે છે (જેમ કે સર્ચબાર). Throttling ગમે એટલી ઇવેન્ટ થાય છતાં પણ રન થવાની રેટ ફિક્સ કરે છે (જેમ કે દર 300ms માં એક જ વાર ચાલવું).

#### 👉 CONCEPT #44: JS OOPs & Prototypes

✦ શેના માટે છે અને શું કરે છે?:

`class` અને `objects` દ્વારા રિયલ પ્રોજેક્ટ આર્કિટેક્ચર બનાવાય છે. `#` થી પ્રાઇવેટ ડેટા એન્કેપ્સ્યુલેટ થાય છે. Prototypes દ્વારા એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટની પ્રોપર્ટીઝ વારસામાં વાપરી શકે છે.

-----

#### 👉 CONCEPT #45: JS Call, Apply, and Bind

-----

✦ શેના માટે છે અને શું કરે છે?:

ફંક્શનની અંદર રહેલા `this` નો કંટ્રોલ મેન્યુઅલી બીજા ઓબ્જેક્ટ પર શિફ્ટ કરવા. `call` અને `apply` તરત રન થાય છે (એપ્પાય આર્ગ્યુમેન્ટ્સ એરેમાં લે છે), જ્યારે `bind` ભવિષ્ય માટે નવું ફંક્શન આપે છે.

-----

#### 👉 CONCEPT #46: JS Promises, Async/Await & Fetch API

-----

✦ શેના માટે છે અને શું કરે છે?:

એસિંક્રોનસ ઓપરેશન્સ મેનેજ કરવા પ્રોમિસ વપરાય છે. `async/await` તેને એકદમ સરળ લાઈન-બાય-લાઈન લુક આપે છે. `fetch()` દ્વારા સર્વરની API પરથી અસમયે લાઈવ ડેટા ખેંચીને લાવી શકાય છે.

-----

#### 👉 CONCEPT #47: JS DOM Manipulation

-----

✦ શેના માટે છે અને શું કરે છે?:

HTML ડોક્યુમેન્ટને જાવાસ્ક્રિપ્ટથી લાઈવ મોડિફાય કરવા માટે. `querySelector()` થી ટેગ પકડીને, `addEventListener()` થી ક્લિક ડિટેક્ટ કરીને, પેજ પર ડાયનેમિકલી તત્વો ઉમેરી કે બદલી શકાય છે.

-----

=====

END OF MASTER GUIDE

=====